

	МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
	федеральное государственное бюджетное образовательное учреждение высшего образования
	«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ

Кафедра Информационные системы

УТВЕРЖДАЮ

К ЗАЩИТЕ ДОПУСТИТЬ

Директор предприятия

Зав. кафедрой

_____ / Р. Р. Мансуров /
подпись инициалы, фамилия

_____ / А. А. Романов /
подпись инициалы, фамилия

«_____» _____ 2026 г.

«_____» _____ 2026 г.

БАКАЛАВРСКАЯ РАБОТА

Вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

Тема Разработка мобильного приложения проверки даты изготовления продукта на Flutter

Студент _____ / Т. А. Кувшинов /
подпись инициалы, фамилия

Обозначение ВКР БР – УлГТУ – 09.03.04 – 22/2523 – 2026 Группа ПИБд-41
для технических направлений подготовки (специальностей)

Направление подготовки (специальность) 09.03.04 Программная инженерия
код, наименование
(профиль «Программная инженерия»)

Руководитель ВКР _____ / О. Ф. Власенко /
подпись, дата инициалы, фамилия

Нормоконтроль _____ / А. А. Романов /
подпись, дата инициалы, фамилия

Ульяновск
2026 г.

	МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
	федеральное государственное бюджетное образовательное учреждение высшего образования
	«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ

Кафедра Информационные системы

Направление подготовки (специальность) 09.03.04 Программная инженерия
(профиль «Программная инженерия»)

УТВЕРЖДАЮ

Зав. кафедрой

_____ / А. А. Романов /
подпись инициалы, фамилия

«___» _____ 2026 г.

ЗАДАНИЕ

на БАКАЛАВРСКУЮ РАБОТУ

указать вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

студенту Кувшинову Тимуру Александровичу курса 4 группы ПИБд-41
фамилия, имя, отчество

Тема ВКР Разработка мобильного приложения проверки даты изготовления продукта на Flutter

утверждена приказом по УлГТУ от «11» февраля 2026 г. № 337

Срок сдачи обучающимся законченной ВКР _____

Исходные данные к ВКР Разработать мобильное приложение проверки даты изготовления продукта на Flutter, обеспечивающее получение изображения с камеры мобильного устройства, автоматический поиск области с датой изготовления на упаковке продукта с использованием обученной модели детекции, распознавание текстового значения даты с использованием обученной RCNN-модели, отображение результата пользователю, добавление продукта в личный кабинет, отправку сообщений об ошибках с пользовательской разметкой, а также взаимодействие с серверной частью, административным приложением,

базой данных и сервисом обучения моделей.

Содержание расчетно-пояснительной записки (перечень подлежащих разработке вопросов) Анализ предметной области и формирование требований к мобильному приложению проверки даты изготовления продукта на Flutter; проект автоматизации процесса проверки даты изготовления продукта; реализация проекта мобильного приложения и административной подсистемы; оценка эффективности проекта мобильного приложения проверки даты изготовления продукта на Flutter.

Перечень графического материала (с точным указанием обязательных чертежей)

IDEF0-модели «КАК ЕСТЬ» и «КАК ДОЛЖНО БЫТЬ» с декомпозициями; диаграмма вариантов использования; ER-модель и схема данных; блок-схемы обработки изображения и инференса модели; диаграммы классов мобильного приложения, серверной части, административного приложения и моделей данных; диаграмма компонентов; диаграмма развертывания; диаграммы состояний мобильного приложения, серверной части и административного приложения; экранные формы контрольного примера; материалы тестирования ПО.

Календарный график работы над ВКР на весь период (с указанием сроков выполнения и содержания отдельных этапов)

№ этапа	Содержание этапа	Срок выполнения
1	Анализ предметной области и требований	13.02.2026 – 28.02.2026
2	Проектирование программного обеспечения	01.03.2026 – 14.03.2026
3	Разработка программного обеспечения	15.03.2026 – 05.04.2026
4	Обучение и интеграция моделей, тестирование и корректировка программного обеспечения	06.04.2026 – 10.04.2026
5	Оформление ПЗ и подготовка к защите	11.04.2026 – 15.04.2026

Дата выдачи задания «__»_____ 20__ г.

Руководитель _____
должность, учёная степень, ученое звание

_____ / О. Ф. Власенко /
подпись инициалы, фамилия

Задание принял к исполнению

_____ / Т. А. Кувшинов /
подпись обучающегося инициалы, фамилия

РЕФЕРАТ

Пояснительная записка содержит материалы выпускной квалификационной работы на тему «Разработка мобильного приложения проверки даты изготовления продукта на Flutter». Работа посвящена созданию программного комплекса, который помогает пользователю быстро найти на упаковке продукта область с датой изготовления и использовать результат в мобильном сценарии проверки.

В работе рассмотрены предметная область, проблемы ручного поиска даты, требования к программной системе, проектное решение, реализация мобильного и серверного контуров, результаты тестирования и экономическая эффективность внедрения. Разрабатываемая система включает мобильное приложение на Flutter, backend API, административное приложение, базу данных PostgreSQL и сервис обучения модели компьютерного зрения.

Ключевые слова: мобильное приложение, Flutter, компьютерное зрение, YOLO, TensorFlow Lite, дата изготовления, проверка продукта, backend API, PostgreSQL, тестирование, экономическая эффективность.

Объект исследования – процесс проверки даты изготовления продукта при помощи мобильного устройства.

Предмет исследования – методы и программные средства мобильной разработки, компьютерного зрения, хранения пользовательских данных и сопровождения модели детекции.

Цель работы – повысить удобство и надежность проверки даты изготовления продукта за счет разработки мобильного приложения на Flutter, выполняющего автоматическую локализацию области даты на изображении.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ ИСПОЛЬЗУЕМЫХ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СОКРАЩЕНИЙ И ТЕРМИНОВ	10
ВВЕДЕНИЕ.....	11
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РАЗРАБАТЫВАЕМОЙ СИСТЕМЕ.....	13
1.1 Описание автоматизации поиска даты изготовления и срока годности на упаковке товара	13
1.1.1 Описание объекта исследования при автоматизации поиска даты на упаковке товара	13
1.1.2 Описание информационных процессов поиска даты изготовления и срока годности на упаковке товара	14
1.2 Описание проблем автоматизации поиска даты изготовления и срока годности на упаковке товара и анализ решений.....	17
1.2.1 Анализ проблем автоматизации информационных процессов поиска даты на упаковке товара	17
1.2.2 Анализ существующих решений по автоматизации поиска даты изготовления и срока годности на упаковке товара.....	18
1.2.3 Формирование предложений по решению проблем автоматизации поиска даты на упаковке товара	19
1.3 Постановка задачи на разработку мобильного приложения для поиска даты изготовления на продукте	21
1.3.1 Цели и задачи проекта. Место и окружение разрабатываемой системы.....	21
1.3.2 Построение и обоснование модели разрабатываемой системы.....	21

1.3.3 Спецификация функциональных требований к мобильному приложению для поиска даты изготовления на продукте	24
1.3.4 Спецификация нефункциональных требований к мобильному приложению для поиска даты изготовления на продукте	25
1.4 Календарно-ресурсное планирование проекта по разработке мобильного приложения для поиска даты изготовления на продукте.....	26
1.4.1 Модель жизненного цикла разработки мобильного приложения для поиска даты изготовления на продукте	26
1.4.2 Анализ ограничений и рисков проекта разработки мобильного приложения	27
1.4.3 Календарный план разработки мобильного приложения для поиска даты изготовления на продукте	28
2 ПРОЕКТ АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ МОБИЛЬНОГО ПРИЛОЖЕНИЯ ПРОВЕРКИ ДАТЫ ИЗГОТОВЛЕНИЯ ПРОДУКТА НА FLUTTER	30
2.1 Функциональная структура.....	30
2.2 Информационное обеспечение	31
2.2.1 Инфологическая модель	31
2.2.2 Схема данных и реквизитный состав таблиц.....	32
2.2.3 Экранные формы первичных документов.....	34
2.2.4 Классификаторы и нормативно-справочная информация.....	36
2.2.5 Выходные экранные формы и документы	37
2.3 Алгоритмическое обеспечение.....	37
2.3.1 Алгоритм обнаружения области даты на мобильном устройстве.....	37

2.3.2 Алгоритм формирования новой модели на основе сообщений об ошибке	39
2.3.3 Формулы расчёта показателей качества.....	41
2.4 Программное обеспечение	42
2.4.1 Описание используемых технологий, фреймворков и библиотек.	42
2.4.2 Структура программного обеспечения.....	43
2.4.3 Спецификации программных модулей.....	46
2.4.4 Структура диалога с пользователем	47
2.5 Обеспечение информационной безопасности	49
2.6 Техническое обеспечение	50
2.6.1 Схема развёртывания.....	50
2.6.2 Технические характеристики.....	51
2.7 Технологическое обеспечение.....	51
2.8 Организационное обеспечение	53
2.9 Выводы по проектному разделу	53
3 РЕАЛИЗАЦИЯ ПРОЕКТА.....	54
3.1 Контрольный пример работы программного комплекса.....	54
3.2 Реализованные технологии и основные сценарии	57
3.3 Тестирование программного обеспечения	58
4 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА.....	60
4.1 Анализ качества проектного решения	60
4.2 Расчет экономических результатов от внедрения	60
4.2.1 Расчет трудоемкости разработки программного обеспечения	60

4.2.1.1 Модель композиции приложения.....	61
4.2.1.2 Метод, основанный на экспертных оценках	62
4.2.1.3 Выводы по расчету трудоемкости.....	62
4.2.2 Анализ затрат на ресурсное обеспечение.....	63
4.2.2.1 Расчет затрат на разработку программного продукта	63
4.2.2.2 Расчет стоимости эксплуатации оборудования	64
4.2.3 Анализ качественных и количественных факторов воздействия проекта.....	64
4.2.3.1 Расчет экономических результатов от внедрения	64
4.2.3.2 Расчет сроков окупаемости проекта	65
4.3 Выводы по оценке эффективности проекта	65
ЗАКЛЮЧЕНИЕ	67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	68
ПРИЛОЖЕНИЕ А. ПРОТОКОЛ ТЕСТИРОВАНИЯ ПРОГРАММНОГО КОМПЛЕКСА.....	71
А.1.1 Цели, объект и границы тестирования.....	71
А.1.2 Виды и техники тестирования	71
А.1.3 Матрица трассируемости требований.....	72
А.1.4 Критерии завершения тестирования	73
А.2 Протокол тестирования	73
А.2.1 Функциональное тестирование.....	73
А.2.2 Автоматизированные unit-тесты training_service.....	79
А.2.3 Полуавтоматизированные интеграционные HTTP contract-тесты ...	80

А.2.4 Нагрузочное и производительное тестирование	82
А.2.5 Тестирование удобства использования и пользовательского интерфейса	83
А.2.6 Пилотное пользовательское тестирование и анализ обратной связи	84
А.2.7 Сводные результаты тестирования	86
А.3 Выводы по протоколу тестирования.....	87
ПРИЛОЖЕНИЕ Б. ЛИСТИНГ UNIT-ТЕСТОВ	89
ПРИЛОЖЕНИЕ В. ЖУРНАЛЫ ВЫПОЛНЕННЫХ АВТОМАТИЗИРОВАННЫХ ПРОВЕРОК	90
ПРИЛОЖЕНИЕ Г. ЛИСТИНГ ПОЛУАВТОМАТИЗИРОВАННЫХ HTTP CONTRACT-ТЕСТОВ	91
ПРИЛОЖЕНИЕ Д. ЛИСТИНГ НАГРУЗОЧНОГО СЦЕНАРИЯ.....	93
ПРИЛОЖЕНИЕ Е Листинг кода информационной системы.....	94

ПЕРЕЧЕНЬ ИСПОЛЬЗУЕМЫХ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СОКРАЩЕНИЙ И ТЕРМИНОВ

Сокращение	Расшифровка
API	Application Programming Interface, программный интерфейс приложения
БД	база данных
ВКР	выпускная квалификационная работа
ИИ	искусственный интеллект
JSON	JavaScript Object Notation, формат обмена данными
REST	архитектурный стиль построения HTTP API
OCR	Optical Character Recognition, оптическое распознавание символов
TFLite	TensorFlow Lite, формат и среда выполнения мобильных моделей
YOLO	You Only Look Once, семейство моделей детекции объектов
IoU	Intersection over Union, мера пересечения предсказанной и эталонной рамки
mAP	mean Average Precision, средняя точность детекции
bbox	bounding box, ограничивающая рамка объекта
backend	серверная часть программной системы
training_service	сервис обучения и экспорта модели детекции

ВВЕДЕНИЕ

Темой выпускной квалификационной работы является «Разработка мобильного приложения проверки даты изготовления продукта на Flutter». Разрабатываемая система объединяет мобильный клиент, серверный API, административное приложение и сервис обучения модели компьютерного зрения.

Актуальность работы обусловлена тем, что дата изготовления продукта часто наносится мелким шрифтом, на сгибах упаковки, в зонах бликов или недостаточной контрастности. Пользователь тратит время на визуальный поиск маркировки и может ошибиться при проверке. Для торговых точек такие ошибки связаны с риском просрочки товаров и дополнительными трудозатратами персонала.

Существующие приложения учета сроков годности обычно ориентированы на инвентаризацию и ручной ввод даты после сканирования штрих-кода. Решения, использующие OCR, не всегда устойчиво работают без предварительного выделения области с датой, поэтому в работе акцент сделан на локализацию области даты на изображении [5; 6].

Целью работы является повышение удобства и надежности проверки даты изготовления продукта за счет разработки мобильного приложения на Flutter, которое автоматически выделяет на изображении область с датой и поддерживает пользовательский, административный и обучающий контуры.

Для достижения цели решаются задачи анализа предметной области, формирования требований, проектирования архитектуры, разработки мобильного и серверного контуров, организации хранения пользовательских данных, реализации механизма обратной связи, дообучения YOLO-модели и проверки качества программного комплекса [7; 8; 11].

Объектом исследования является процесс проверки даты изготовления продукта при помощи мобильного устройства. Предметом исследования являются

методы и программные средства мобильной разработки, компьютерного зрения, хранения пользовательских данных, обработки обратной связи и сопровождения версий модели детекции.

Методическую и нормативную основу оформления пояснительной записки составляют требования к структуре ВКР и правила оформления текстовых документов, библиографического описания, сокращений и единиц измерения [1-4; 21; 22]. Технологическая основа проекта включает Flutter, Dart, ASP.NET Core, PostgreSQL, TensorFlow Lite, OpenCV, FastAPI, PyTorch, Ultralytics YOLO, JWT, BCrypt и требования к мобильной безопасности [7-20].

Практическая значимость работы заключается в создании программного комплекса, который может использоваться как цифровой помощник при бытовой проверке продуктов и как средство ускорения контроля скоропортящихся товаров. Наличие механизма накопления пользовательских примеров позволяет постепенно улучшать качество модели без изменения базовых сценариев приложения.

Пояснительная записка включает введение, четыре раздела основной части, заключение, список использованных источников и приложения. В приложениях приведены протокол тестирования и листинги проверочных сценариев.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РАЗРАБАТЫВАЕМОЙ СИСТЕМЕ

1.1 Описание автоматизации поиска даты изготовления и срока годности на упаковке товара

1.1.1 Описание объекта исследования при автоматизации поиска даты на упаковке товара

Объектом исследования является процесс поиска даты изготовления и срока годности на упаковке товаров при помощи мобильного устройства и программных средств компьютерного зрения. В реальной практике пользователь сталкивается с тем, что необходимая дата наносится на упаковку в различных форматах, может располагаться на сгибах, швах, боковых поверхностях, крышках или в зонах с низкой контрастностью. В результате даже простая задача визуального поиска даты часто требует существенных временных затрат и не всегда завершается успешно.

С прикладной точки зрения рассматриваемая область находится на пересечении задач мобильной разработки, компьютерного зрения, хранения пользовательских данных и управления версиями моделей машинного обучения. Разрабатываемая система должна объединять пользовательский сценарий сканирования упаковки, серверный контур хранения и обработки данных, административный контур сопровождения, а также контур обучения и дообучения модели на новых примерах.

В отличие от традиционных систем складского учёта, где срок годности обычно вводится вручную после сканирования штрих-кода, в данной работе объект автоматизации связан именно с задачей поиска визуально представленной даты на изображении упаковки. Это делает предметную область ближе к прикладным системам интеллектуального анализа изображений, чем к классическим системам учёта запасов.

Круг пользователей системы включает три основные группы. Первая группа – конечные пользователи, то есть покупатели и бытовые пользователи, которым требуется быстро найти дату на упаковке перед покупкой или при домашнем хранении продуктов. Вторая группа – сотрудники торговой точки или иного хозяйствующего субъекта, заинтересованные в снижении риска просрочки и в ускорении операций контроля. Третья группа – администраторы и разработчики системы, которые сопровождают пользовательские обращения, анализируют ошибки и управляют процессом дообучения модели.

Для выбранной предметной области важны не только функциональные возможности, но и условия эксплуатации. Работа мобильного клиента должна происходить в разных условиях освещения, при наличии бликов, поворота упаковки, частичного размытия изображения и различий в способе нанесения даты. Следовательно, предмет исследования включает не только программную реализацию, но и обеспечение достаточной устойчивости детекции в реальных пользовательских сценариях.

1.1.2 Описание информационных процессов поиска даты изготовления и срока годности на упаковке товара

В существующем неавтоматизированном варианте пользовательский процесс поиска даты начинается с визуального осмотра упаковки. Пользователь поворачивает продукт, пытается найти место нанесения даты, оценивает читаемость символов, а затем вручную интерпретирует найденную информацию. Если упаковка имеет сложную форму или дата нанесена неочевидно, процесс повторяется несколько раз. При этом значительная часть времени тратится именно на локализацию области, в которой размещена дата, а не на её интерпретацию.

Прикладной процесс, автоматизируемый в рамках проекта, включает более формализованную последовательность действий. Пользователь открывает мобильное приложение, при необходимости авторизуется, направляет камеру на

упаковку и получает визуальную подсветку области, где, по оценке модели, находится дата. При неудовлетворительном результате пользователь может сообщить об ошибке, приложив видеоданные и уточнение по правильному расположению области. Далее эти данные используются в административном и обучающем контуре системы.

Серверный контур прикладных процессов охватывает регистрацию и авторизацию пользователей, ведение списка продуктов на хранении, приём сообщений об ошибках, хранение файловых артефактов, просмотр жалоб администратором и организацию дообучения модели. Таким образом, прикладные процессы включают не только взаимодействие пользователя с камерой, но и последующее накопление данных, анализ качества и выпуск новых версий модели.

Таким образом, исследуемая система объединяет несколько взаимосвязанных информационных процессов: пользовательское сканирование, локальную детекцию на устройстве, сбор обратной связи, серверное хранение данных, административную обработку жалоб и обучение модели на накопленных примерах. В контексте первой главы ВКР эти процессы составляют основу для описания модели «КАК ЕСТЬ», представленной на рисунке 1.1, и её детализации, представленной на рисунке 1.2, а также для последующего построения модели «КАК ДОЛЖНО БЫТЬ».

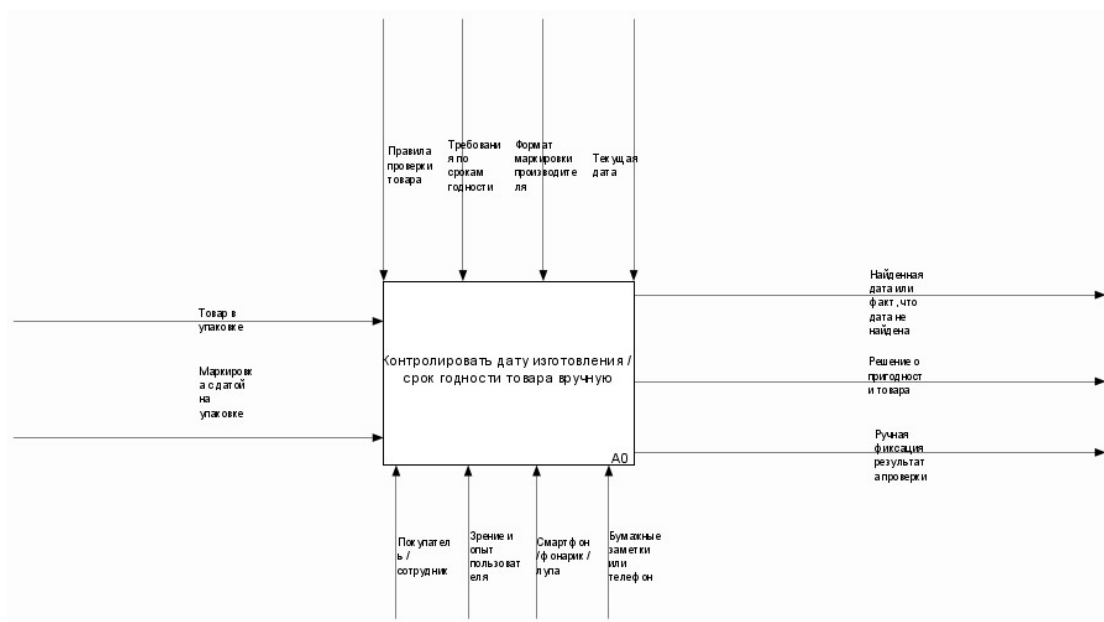


Рисунок 1.1 – IDEF0-модель «КАК ЕСТЬ»

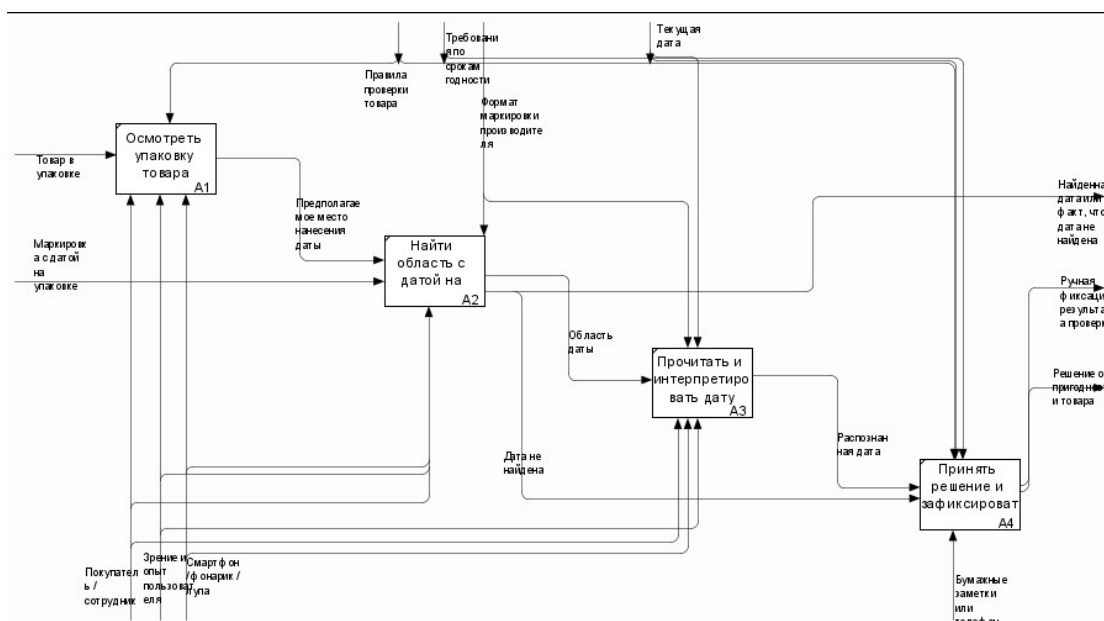


Рисунок 1.2 – Детализация IDEF0-модели «КАК ЕСТЬ»

1.2 Описание проблем автоматизации поиска даты изготовления и срока годности на упаковке товара и анализ решений

1.2.1 Анализ проблем автоматизации информационных процессов поиска даты на упаковке товара

Основная проблема рассматриваемой предметной области состоит в том, что существующий процесс поиска даты на упаковке во многом зависит от человеческого зрения, внимательности и опыта. На практике это означает нестабильность результата: один и тот же товар может быть быстро проверен одним пользователем и вызвать существенные затруднения у другого. В случае неудачи пользователь либо тратит дополнительное время на поиск даты, либо вовсе отказывается от точной проверки.

Для торговой точки и сотрудников магазина проблема имеет ещё и экономический аспект. Ошибки при контроле сроков годности могут приводить к просрочке товаров, дополнительным трудозатратам на ручные проверки и снижению эффективности работы персонала. Для конечного потребителя проблема выражается в неудобстве выбора товара и в риске приобретения продукта с истекающим сроком годности.

Попытка решать задачу исключительно средствами OCR также сталкивается с ограничениями. Для надёжного распознавания требуется сначала выделить нужную область изображения. Если подавать на OCR весь кадр, то возрастает влияние постороннего текста, декоративных элементов упаковки, штрих-кодов и других шумовых факторов. Следовательно, в данной предметной области ключевой задачей становится не только чтение даты, но прежде всего локализация области, где она расположена.

Ещё одной существенной проблемой является разрыв между пользовательским сценарием и процессом совершенствования модели. Даже если система ошибается, в традиционных приложениях эта ошибка либо не

фиксируется, либо не превращается в обучающий пример. В результате качество решения улучшается медленно и не использует реальные пользовательские данные как источник развития модели.

1.2.2 Анализ существующих решений по автоматизации поиска даты изготовления и срока годности на упаковке товара

Анализ аналогов показал, что существующие программные решения можно условно разделить на две группы. Первая группа – приложения учёта сроков годности и домашнего инвентаря. Такие системы обычно позволяют сканировать штрих-код, вручную вводить дату и получать напоминания об истечении срока хранения. В качестве характерного примера можно привести приложение ВЕЕР – Expiry Date Tracking, ориентированное прежде всего на инвентаризацию и уведомления, а не на автоматический поиск области даты на изображении [2].

Вторая группа – приложения, пытающиеся определить дату средствами компьютерного зрения и OCR. Они ближе к рассматриваемой задаче, однако часто ориентированы именно на распознавание текста, а не на устойчивую локализацию области с датой в сложных условиях съёмки. Примером такого решения является приложение Zuzanka, позиционируемое как средство поиска срока годности на упаковке, однако его практическая применимость во многом зависит от качества изображения и условий съёмки [3].

Проведённое сравнение показало, что разрабатываемая система занимает промежуточную, но в то же время слабо занятую нишу. С одной стороны, она наследует полезные возможности приложений учёта, такие как хранение списка продуктов и пользовательские уведомления о проблемных ситуациях. С другой стороны, она ориентирована именно на задачу поиска области даты на изображении и на последующее улучшение качества модели по пользовательской обратной связи.

Научные работы по распознаванию сроков годности и производственных маркировок также подтверждают актуальность выбранной темы, но чаще решают более узкую задачу чтения символов или даты после предварительного выделения области. Например, в исследованиях по use-by-date recognition используются связки FCN/CRNN или специализированные сверточные сети для распознавания даты на изображениях упаковок, а обобщённые фреймворки распознавания дат делают акцент на парсинге форматов DMU и повышении точности чтения [23–25].

Ограничение таких подходов для рассматриваемой ВКР состоит в том, что они в основном ориентированы на экспериментальное распознавание даты или промышленные сценарии контроля, а не на полный пользовательский контур мобильного приложения: локальную подсветку области даты на устройстве, режим увеличения, сохранение продукта, отправку ошибочных примеров и последующее дообучение модели. Поэтому разрабатываемая система рассматривается не как замена OCR-метода, а как прикладной программный комплекс, объединяющий детекцию, распознавание текста даты и цикл улучшения модели по пользовательским данным.

Стратегия развития подобных решений на современном этапе связана с переходом от чисто серверных сценариев обработки к гибридной архитектуре. В такой архитектуре быстрая и частая детекция выполняется непосредственно на устройстве пользователя, а более тяжёлые задачи хранения данных, агрегации статистики и дообучения модели выносятся в серверный контур. Данный подход позволяет совместить оперативность пользовательского взаимодействия и возможность централизованного сопровождения модели.

1.2.3 Формирование предложений по решению проблем автоматизации поиска даты на упаковке товара

С учётом выявленных недостатков существующих решений предлагается разрабатывать систему как распределённый программный комплекс, включающий

мобильный клиент, серверное API, административное приложение и отдельный сервис обучения модели. Такая архитектура позволяет разделить пользовательские, административные и вычислительно затратные процессы, сохраняя при этом их логическую связанность.

В качестве базового метода компьютерного зрения целесообразно использовать детектор объектов семейства YOLO. Такой выбор обусловлен тем, что задача локализации области даты по своей сути является задачей обнаружения объекта на изображении, а архитектуры семейства YOLO хорошо зарекомендовали себя в задачах детекции при приемлемом балансе между качеством и производительностью [4].

Технологически целесообразно использовать следующую связку: Flutter для мобильного клиента, ASP.NET Core для серверного API, PostgreSQL в качестве СУБД, отдельный Python/FastAPI сервис для обучения модели, TensorFlow Lite для исполнения мобильной модели и OpenCV для обработки изображений на этапе подготовки данных и экспериментов [5-9]. Такой стек соответствует текущему состоянию проекта и обеспечивает разделение задач по слоям системы.

Предлагаемая система должна разрабатываться с учётом принципа замкнутого цикла улучшения качества: пользовательский сценарий создаёт данные, данные пополняют датасет, датасет используется для дообучения модели, а обновлённая модель снова поступает пользователю. Именно наличие такого контура делает проект не просто приложением-сканером, а развивающейся интеллектуальной системой.

1.3 Постановка задачи на разработку мобильного приложения для поиска даты изготовления на продукте

1.3.1 Цели и задачи проекта. Место и окружение разрабатываемой системы

Целью проекта является повышение удобства и надёжности контроля сроков годности товаров за счёт разработки программной системы, способной автоматически находить на изображении область, содержащую дату изготовления или срок годности, и использовать полученные данные в пользовательском и административном контуре.

Для достижения указанной цели в проекте решаются следующие задачи: разработка мобильного клиента с локальной детекцией области даты; реализация серверного API для авторизации, хранения данных и приёма обратной связи; разработка административного приложения для сопровождения пользователей и обработки сообщений об ошибках; организация контура подготовки датасета и дообучения модели; интеграция механизма публикации и обновления мобильной версии модели.

Разрабатываемая система не является изолированной утилитой с единственной функцией детекции. Она представляет собой самостоятельный программный комплекс, включающий клиентский, серверный, административный и обучающий контуры. Следовательно, место системы в предметной области определяется как средство автоматизации пользовательского поиска даты на упаковке и последующего сопровождения жизненного цикла модели детекции.

1.3.2 Построение и обоснование модели разрабатываемой системы

Целевая модель функционирования системы предполагает переход от полностью ручного поиска даты к автоматизированному процессу, в котором основные действия пользователя поддерживаются программными средствами. В модели «КАК ДОЛЖНО БЫТЬ» пользователь выполняет съёмку упаковки через

мобильное приложение, система локально определяет предполагаемую область даты и подсвечивает её на экране, а при обнаружении ошибки предоставляет возможность отправки обратной связи в серверный контур. Административная подсистема обеспечивает просмотр обращений, а обучающий контур – накопление и использование новых примеров для повышения качества модели.

Обоснование такой модели связано с необходимостью сократить долю ручных операций, уменьшить время поиска даты и обеспечить систематическое улучшение качества детекции на основе реальных пользовательских данных. Целевая организация процессов отражается в IDEF0-модели «КАК ДОЛЖНО БЫТЬ».

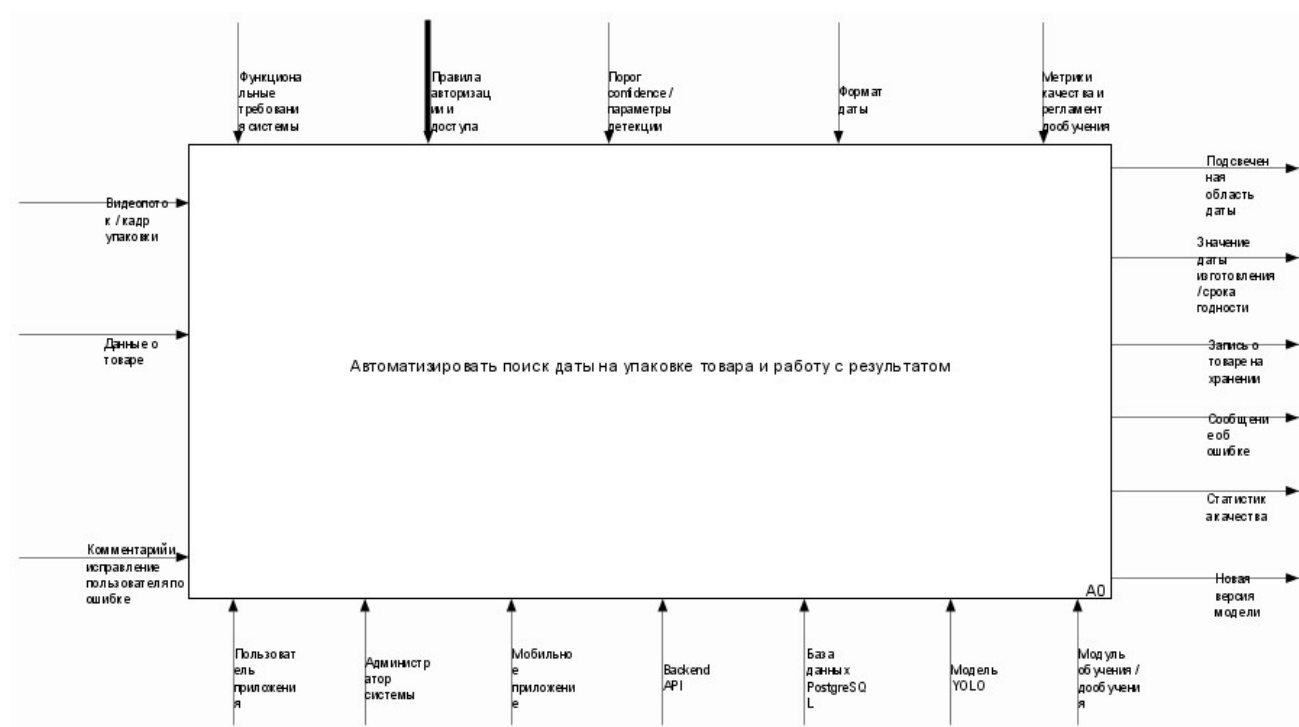


Рисунок 1.3 – IDEF0-модель «КАК ДОЛЖНО БЫТЬ»

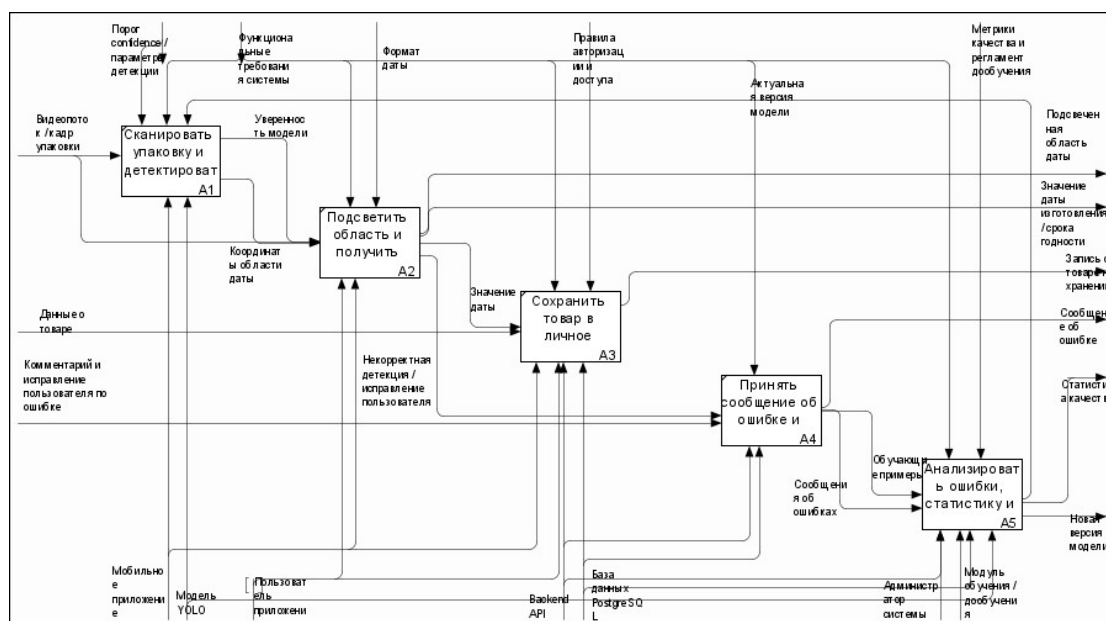


Рисунок 1.4 – Детализация IDEF0-модели «КАК ДОЛЖНО БЫТЬ»

На рисунке 1.4 представлена детализация функциональной модели «КАК ДОЛЖНО БЫТЬ». Модель показывает целевой сквозной процесс: получение кадра упаковки, локализацию области даты, извлечение текстового значения, сохранение результата в личном кабинете, регистрацию ошибки и передачу накопленных примеров в контур дообучения модели.

Входами процесса являются кадр или видеопоток упаковки, сведения о товаре и пользовательский комментарий по ошибке. Управляющими воздействиями выступают параметры детекции, формат даты, правила авторизации, актуальная версия модели и регламент дообучения. Механизмами выполнения являются мобильное приложение, модель YOLO/TFLite, backend API, PostgreSQL, административное приложение и сервис обучения.

Декомпозиция целевого процесса включает пять функций: сканирование и детекцию области даты; подсветку области и получение значения даты; сохранение товара в личном кабинете; прием сообщения об ошибке; анализ ошибок, статистики и выпуск новой версии модели. Такая модель сокращает

ручной поиск даты и одновременно формирует данные для последующего улучшения интеллектуального модуля.

1.3.3 Спецификация функциональных требований к мобильному приложению для поиска даты изготовления на продукте

Функциональные требования к системе определяются исходя из целевого пользовательского сценария и логики сопровождения модели. Базовой функцией выступает сканирование упаковки с запуском детекции области даты на видеопотоке или отдельном кадре. Результатом выполнения этой функции является визуальная подсветка предполагаемой области даты и возможность дальнейшего использования результата в пользовательском интерфейсе.

Отдельный блок функций связан с безопасностью доступа: регистрация пользователя, авторизация, проверка прав доступа и хранение токена сессии. Дополнительный блок относится к сбору обратной связи: пользователь должен иметь возможность отправить сообщение об ошибке, сопроводив его видеоданными или корректной областью даты. Серверная часть должна обеспечивать хранение таких обращений, а административная – их просмотр и обработку.

В ранее выполненном анализе функциональные требования были детализированы в виде перечня функций F1–F15, охватывающих пользовательские, административные и обучающие процессы. В их число входят сканирование упаковки, подсветка найденной области, ручной ввод даты, помещение продукта на хранение, просмотр и удаление продуктов, отправка сообщений об ошибке, просмотр жалоб и пользователей администратором, блокировка пользователей, анализ статистики качества и запуск дообучения модели.

Регламент выполнения указанных функций предполагает разделение ответственности между компонентами системы. Быстрая детекция должна

выполняться локально на устройстве, что минимизирует задержку при взаимодействии пользователя с камерой. Хранение пользовательских данных, обработка жалоб, публикация версий модели и административные операции выполняются в серверном контуре. Подготовка датасета, обучение и экспорт мобильной модели выносятся в отдельный обучающий сервис.

1.3.4 Спецификация нефункциональных требований к мобильному приложению для поиска даты изготовления на продукте

Нефункциональные требования определяют качественные характеристики разрабатываемой системы и во многом задают границы применимости технических решений. Прежде всего к системе предъявляются требования по производительности: пользовательское сканирование должно обеспечивать приемлемое время отклика, а локальный инференс модели не должен делать работу интерфейса некомфортной. В практическом плане это означает необходимость оптимизации модели и ограничения вычислительной нагрузки на мобильное устройство [4; 8].

Следующая группа требований относится к программно-технической среде. Целевой клиентский вариант предполагает использование смартфона на Android или iOS с камерой не ниже среднего уровня и достаточным объёмом оперативной памяти. Серверная часть должна размещаться в среде, поддерживающей ASP.NET Core, а база данных должна обеспечивать надёжную работу с транзакциями, связями между сущностями и расширяемыми пользовательскими настройками. Для этих целей в проекте используется PostgreSQL, в том числе благодаря поддержке JSON-полей и развитым средствам работы с индексами и ограничениями [6; 7].

С точки зрения надёжности система должна корректно обрабатывать ситуацию отсутствия детекции, ошибки доступа к камере, сетевые сбои, недоступность сервера и некорректные пользовательские вводы. Это означает

необходимость предусмотреть сценарии отказоустойчивого поведения и понятные пользователю сообщения об ошибках. Кроме того, сервис обучения должен быть способен завершать длительные операции с фиксацией промежуточного состояния и метрик качества.

Особое значение имеют требования информационной безопасности. Пользовательские данные, включая учётные записи, токены доступа и сообщения об ошибках, должны обрабатываться только в рамках разграничения прав доступа. Административные функции должны быть доступны только авторизованным администраторам. При передаче данных между мобильным клиентом, админкой и серверной частью должен использоваться защищённый сетевой обмен. Видеофайлы и иные файловые артефакты должны храниться отдельно с метаданными БД и использоваться только для целей, предусмотренных логикой системы.

Для пользовательского интерфейса важны эргономические требования. Основной экран сканирования должен быть визуально простым, не перегруженным элементами управления и понятным при первом использовании. Крупные элементы управления и контрастная визуализация подсветки области даты особенно важны с учётом того, что одной из целевых категорий пользователей выступают люди, испытывающие затруднения при визуальном поиске даты на упаковке.

1.4 Календарно-ресурсное планирование проекта по разработке мобильного приложения для поиска даты изготовления на продукте

1.4.1 Модель жизненного цикла разработки мобильного приложения для поиска даты изготовления на продукте

Для рассматриваемого проекта наиболее целесообразно использовать итерационную модель жизненного цикла с элементами прототипирования. Такой выбор объясняется тем, что разработка системы тесно связана с

экспериментальной оценкой различных подходов к решению задачи: OCR, аналитического поиска области даты и нейросетевой детекции. На ранних этапах проекта требуется последовательно проверять гипотезы, отбрасывать неэффективные решения и уточнять состав требований.

Итерационный подход позволяет на каждом цикле уточнять требования, дорабатывать архитектуру и оценивать результат. Первый цикл ориентирован на аналитическую проработку, выбор подхода и создание прототипов. Второй цикл связан с подготовкой данных и обучением модели. Третий цикл – с интеграцией модели в мобильное приложение и серверный контур. Последующие циклы направлены на стабилизацию, сбор обратной связи и улучшение качества решения.

1.4.2 Анализ ограничений и рисков проекта разработки мобильного приложения

На реализацию проекта влияет ряд ограничений. Во-первых, качество детекции напрямую зависит от разнообразия и объёма обучающего датасета. Во-вторых, мобильное устройство накладывает ограничения на размер модели, объём памяти и допустимую длительность инференса. В-третьих, серверный контур должен согласованно работать с файловыми артефактами, базой данных и административными операциями, что повышает требования к согласованности архитектуры.

К числу ключевых рисков относятся недостаточное качество первичной выборки, переобучение модели на ограниченном наборе примеров, рост времени обработки на мобильном устройстве, нестабильность сетевого взаимодействия между клиентом и сервером, а также организационные риски, связанные с увеличением трудоёмкости проекта. Для снижения указанных рисков целесообразно использовать поэтапную проверку гипотез, регулярное

тестирование, накопление реальных пользовательских примеров и разделение проекта на логически завершённые итерации.

1.4.3 Календарный план разработки мобильного приложения для поиска даты изготовления на продукте

Календарно-ресурсное планирование проекта опирается на уже выполненный перечень работ по аналитике, проектированию, подготовке данных, обучению модели, мобильной разработке, разработке бэкенда и документации. Аналитический блок включал формирование и уточнение темы, проверку реализуемости, анализ аналогов и методов решения. Технический блок включал разработку инструмента разметки, подготовку обучающих данных, дообучение модели, а также проектирование архитектуры будущей программной системы.

Следует отметить, что приведённый план носит оценочный характер и может уточняться по мере развития проекта. Наиболее ресурсоёмкими этапами являются обучение модели, оптимизация инференса на мобильном устройстве, реализация серверного API и интеграция полного цикла обратной связи пользователя с контуром дообучения. Тем не менее уже выполненный объём работ подтверждает практическую реализуемость проекта и достаточную проработанность первой стадии ВКР.

Таблица 1.1 – Календарный план разработки мобильного приложения проверки даты изготовления продукта на Flutter

Этап	Содержание работ	Трудоемкость, ч
Аналитика и постановка задачи	Уточнение темы, проверка реализуемости, анализ аналогов и методов решения	55
Экспериментальная проверка подходов	Подготовка первичных данных, прототипы OCR, аналитического поиска области даты и YOLO-детекции	36
Проектирование и документация	Формирование требований, архитектурное проектирование, диаграммы, описание результатов	66
Мобильное приложение и офлайн-инференс	Сценарии сканирования, подсветки, сохранения продукта, отправки ошибки, TFLite-инференс и оптимизация	124

Серверная, административная и data-часть	Backend API, схема БД, административные функции, статистика и обработка сообщений об ошибках	118
ML-контур и интеграция модели	Подготовка датасета, обучение, оценка качества, экспорт мобильной модели и обновление версии	92
Тестирование	Проверка отказов, сценариев мобильного клиента, backend API, административного приложения и обучающего контура	10
Итого	Общая трудоемкость разработки программного комплекса	501

В таблице 1.1 приведена укрупнённая календарно-ресурсная оценка проекта. Детальный перечень работ использован как исходная декомпозиция трудоемкости при экономическом обосновании в разделе 4.

2 ПРОЕКТ АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ МОБИЛЬНОГО ПРИЛОЖЕНИЯ ПРОВЕРКИ ДАТЫ ИЗГОТОВЛЕНИЯ ПРОДУКТА НА FLUTTER

2.1 Функциональная структура

Функциональная структура разрабатываемой системы определяется составом пользовательских, серверных, административных и обучающих функций, реализуемых в рамках мобильного приложения, backend API, приложения администратора и сервиса обучения модели. Для формального представления функциональной структуры используется диаграмма вариантов использования, подготовленная в среде Visual Paradigm.

В системе выделяются три основные группы участников: неавторизованный пользователь, авторизованный пользователь мобильного приложения и администратор системы. Неавторизованный пользователь может выполнять регистрацию и авторизацию. После входа в систему пользователь получает доступ к персонализированным функциям: сохранению продуктов в личный кабинет, просмотру истории сохранённых продуктов, отправке сообщения об ошибке и обновлению локальной версии модели искусственного интеллекта. Администратор системы работает с сообщениями об ошибках и запускает обучение новой версии модели на основе утверждённых примеров.

Функциональная модель системы включает следующие ключевые варианты использования: сканирование упаковки с локальной детекцией области даты, просмотр статуса работы модели, открытие личного кабинета, ручное добавление продукта с указанием срока годности, запись видеоролика с ошибочным примером, извлечение кадров и формирование YOLO-разметки, отправка датасета на сервер, просмотр административного списка сообщений об ошибках, детальный просмотр кадров и рамок, запуск задачи обучения, контроль состояния задачи и публикация новой мобильной модели.

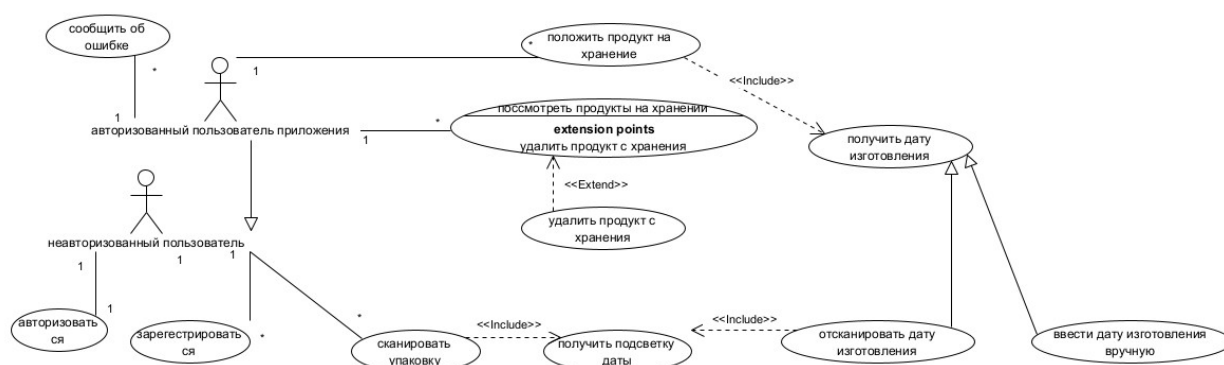


Рисунок 2.1 – Диаграмма вариантов использования системы

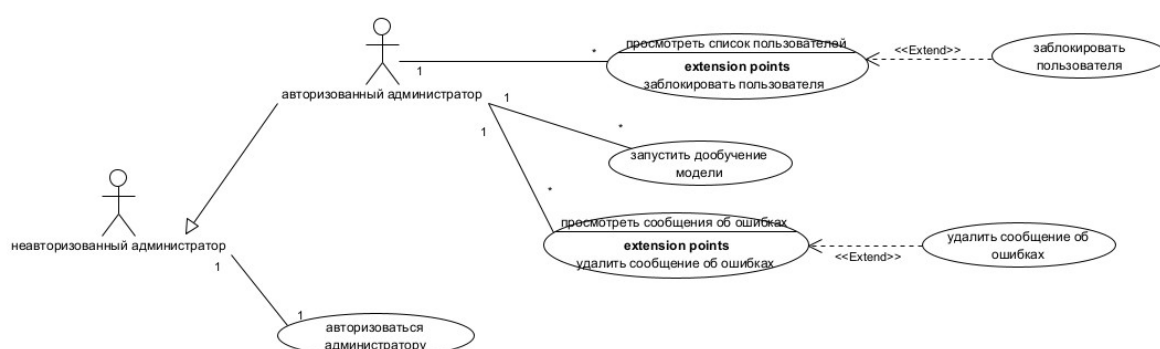


Рисунок 2.2 – Диаграмма вариантов использования системы административного модуля

2.2 Информационное обеспечение

2.2.1 Инфологическая модель

Информационное обеспечение системы построено на реляционной модели данных и реализуется в СУБД PostgreSQL. Инфологическая модель отражает основные сущности предметной области и связи между ними. В текущем состоянии проекта в базе данных используются шесть базовых сущностей: пользователь, продукт, продукт на хранении, сообщение об ошибке, видеопример и версия модели.

Сущность «Пользователь» хранит сведения об учётной записи: адрес электронной почты, хэш пароля, признак блокировки и дополнительные настройки в формате JSON. Сущность «Продукт» выступает справочником

товаров и содержит наименование, производителя и штрих-код. Сущность «Продукт на хранении» связывает пользователя и продукт и дополнительно хранит дату изготовления, срок годности и дату добавления записи в личный кабинет.

Сущность «Сообщение об ошибке» предназначена для хранения пользовательских обращений, связанных с некорректной детекцией даты. Каждое сообщение связано с пользователем и при необходимости с версией модели, на которой была обнаружена ошибка. Сущность «Видеопример» хранит путь к каталогу с исходным датасетом, сформированным на мобильном устройстве и загруженным на сервер. Сущность «Версия модели» фиксирует дату обучения, метрики качества и пути к артефактам обучения, включая исходные веса best.pt и мобильную TFLite-модель.

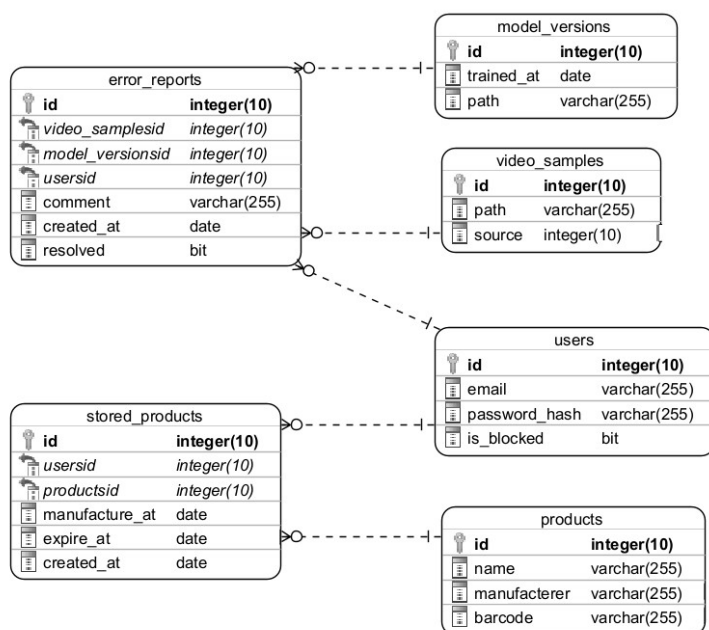


Рисунок 2.3 – ER-модель базы данных системы

2.2.2 Схема данных и реквизитный состав таблиц

Логическая схема базы данных соответствует выделенным сущностям и обеспечивает поддержку основных сценариев работы системы: регистрацию

пользователя, ведение личного кабинета, накопление обучающих примеров и сопровождение жизненного цикла модели. Связи между таблицами реализуются на основе первичных и внешних ключей.

Один пользователь может иметь множество записей о продуктах на хранении и множество сообщений об ошибках. Один видеопример связан с загруженным датасетом конкретного обращения. Одна версия модели может использоваться для связи со многими сообщениями об ошибках и позволяет анализировать качество конкретной опубликованной версии.

Таблица 2.1 – Реквизитный состав таблицы Users

Поле	Тип	Назначение
Id	int	Уникальный идентификатор пользователя
Email	varchar(256)	Адрес электронной почты
PasswordHash	varchar(256)	Хэш пароля
IsBlocked	bool	Признак блокировки учётной записи
SettingsJson	text	Дополнительные настройки клиента в JSON
CreatedAt	timestamp	Дата и время регистрации

Таблица 2.2 – Реквизитный состав таблицы Products

Поле	Тип	Назначение
Id	int	Уникальный идентификатор продукта
Name	varchar(256)	Наименование продукта
Manufacturer	varchar(256)	Производитель
Barcode	varchar(64)	Штрих-код товара

Таблица 2.3 – Реквизитный состав таблицы StoredProducts

Поле	Тип	Назначение
Id	int	Уникальный идентификатор записи
UserId	int, FK	Ссылка на пользователя
ProductId	int, FK	Ссылка на продукт
ManufactureAt	timestamp, null	Дата изготовления
ExpiryAt	timestamp, null	Срок годности
CreatedAt	timestamp	Дата добавления в личный кабинет

Таблица 2.4 – Реквизитный состав таблицы VideoSamples

Поле	Тип	Назначение
Id	int	Уникальный идентификатор видеопримера
VideoPath	varchar(1024)	Путь к каталогу с датасетом на сервере
Source	varchar(64)	Источник примера, например mobile_dataset

Таблица 2.5 – Реквизитный состав таблицы ErrorReports

Поле	Тип	Назначение
Id	int	Уникальный идентификатор сообщения
UserId	int, FK	Ссылка на автора сообщения
VideoId	int, FK, null	Ссылка на каталог датасета
ModelVersionId	int, FK, null	Версия модели, на которой зафиксирована ошибка
Comment	varchar(2000), null	Комментарий пользователя
CreatedAt	timestamp	Дата создания обращения
Resolved	bool	Признак обработки
Approved	bool	Признак включения примера в обучающий датасет
FramesCount	int	Количество извлечённых и размеченных кадров

Таблица 2.6 – Реквизитный состав таблицы ModelVersions

Поле	Тип	Назначение
Id	int	Уникальный идентификатор версии модели
TrainedAt	timestamp	Дата завершения обучения
MetricsJson	text	Метрики качества в формате JSON
ExternalJobId	varchar(128)	Идентификатор задачи в training service
BaseModel	varchar(256)	Базовая модель для дообучения
BestWeightsPath	varchar(2048)	Путь к файлу best.pt
MobileModelPath	varchar(2048)	Путь к мобильному артефакту
MobileFormat	varchar(32)	Формат мобильной модели, например tflite

2.2.3 Экранные формы первичных документов

В разрабатываемой системе первичными документами являются изображения упаковок и этикеток товаров, поступающие в мобильное приложение

с камеры устройства. На таких изображениях дата изготовления или срок годности выступает основным информативным реквизитом, подлежащим обнаружению, а остальные элементы упаковки – наименование товара, состав, штрихкод, сведения о производителе и декоративные элементы – образуют визуальный фон.

Дата на упаковке может наноситься различными способами: типографской печатью, струйной маркировкой, штампом, тиснением или лазерной гравировкой. Кроме того, она может располагаться на основной этикетке, боковой поверхности, крышке, шве упаковки или термоусадочной плёнке. Это усложняет автоматический поиск даты и обосновывает необходимость применения методов компьютерного зрения.

На рисунке 2.4 представлен пример этикетки товара с отчётливо читаемой датой изготовления. Такой вариант отражает благоприятные условия работы системы: символы различимы, область даты контрастна и не перекрыта бликами или деформациями материала.



Рисунок 2.4 – Пример этикетки товара с отчётливо читаемой датой изготовления

На рисунке 2.5 представлен пример маркировки, нанесённой в сложных условиях наблюдения. Подобные изображения отражают реальные условия эксплуатации, при которых дата может быть слабо контрастной, частично искажённой или расположенной на неоднородном фоне.



Рисунок 2.5 – Пример маркировки товара с датой, нанесённой в сложных условиях наблюдения

Таким образом, изображения упаковок и этикеток являются исходным источником данных для мобильного приложения, служат входом для алгоритмов детекции и формируют основу для последующего пользовательского и серверного сценария обработки информации.

2.2.4 Классификаторы и нормативно-справочная информация

В текущей реализации система использует ограниченный, но явно заданный набор классификаторов и справочных значений. В мобильной части направление и наличие детекции задаются через индекс класса модели, где класс 0 соответствует области даты. В серверной и обучающей частях используются фиксированные значения статусов задач обучения: `queued`, `running`, `completed`, `failed` и `canceled`.

Для источника поступления датасета в сущности VideoSample используется значение `mobile_dataset`. Для формата публикуемой мобильной модели используется значение `tflite`.

Кроме того, в системе используется параметр `device`, задающий устройство запуска обучения. В пользовательском интерфейсе администратора допустимы значения `auto`, `cpu` и конкретный индекс GPU. В модуле `training service` реализована интерпретация этих значений и разрешение фактического устройства выполнения.

2.2.5 Выходные экранные формы и документы

К выходным экранным формам системы относятся: список продуктов на хранении в личном кабинете, список административных обращений, форма детального просмотра сообщения об ошибке, список задач обучения и детальная форма задачи обучения. Эти формы обеспечивают пользователя и администратора результатной информацией, необходимой для принятия решений.

Выходными документами системы являются ZIP-архив датасета, содержащий изображения, YOLO-метки и метаданные; файл `best.pt`, получаемый по результатам обучения; мобильный TFLite-артефакт, предназначенный для использования на устройстве; а также JSON-представление метрик обучения, сохраняемое в сущности `ModelVersion` и выводимое в административном интерфейсе и на экране настроек мобильного приложения.

2.3 Алгоритмическое обеспечение

2.3.1 Алгоритм обнаружения области даты на мобильном устройстве

Текущий алгоритм обнаружения реализован в классе `TfliteDateDetector` и фактически выполняется в отдельном изоляте, что позволяет не блокировать интерфейс мобильного приложения. На вход алгоритма поступают кадры камеры в формате YUV420 либо BGRA8888. Сначала выполняется преобразование цветового пространства в RGB. Затем, с учётом ориентации сенсора камеры,

выполняется поворот изображения, а для фронтальной камеры при необходимости – зеркальное отражение.

После приведения ориентации выполняется операция letterbox: исходный кадр масштабируется с сохранением пропорций до квадратного входа модели, а свободные области заполняются нейтральным цветом. Далее строится входной тензор для TFLite-интерпретатора. В зависимости от типа модели формируется либо float32-тензор с нормализацией каналов в диапазон от 0 до 1, либо int8-тензор с квантованием по параметрам scale и zero point.

Следующим этапом является запуск инференса. После получения выходного тензора выполняется постобработка: из всех строк выхода выбираются только те, у которых confidence не ниже 0,05. Далее координаты рамки переводятся из координат модели в координаты исходного изображения с учётом коэффициента масштабирования и внутренних отступов letterbox. Полученные детекции сортируются по убыванию уверенности, после чего в пользовательский интерфейс передаются верхние результаты для визуального отображения.

Дополнительно алгоритм собирает статистику производительности по этапам обработки кадра: ожидание в очереди, YUV→RGB, поворот, resize/letterbox, построение тензора, инференс и постобработка. Эти измерения выводятся пользователю и позволяют оценивать узкие места мобильного контура.

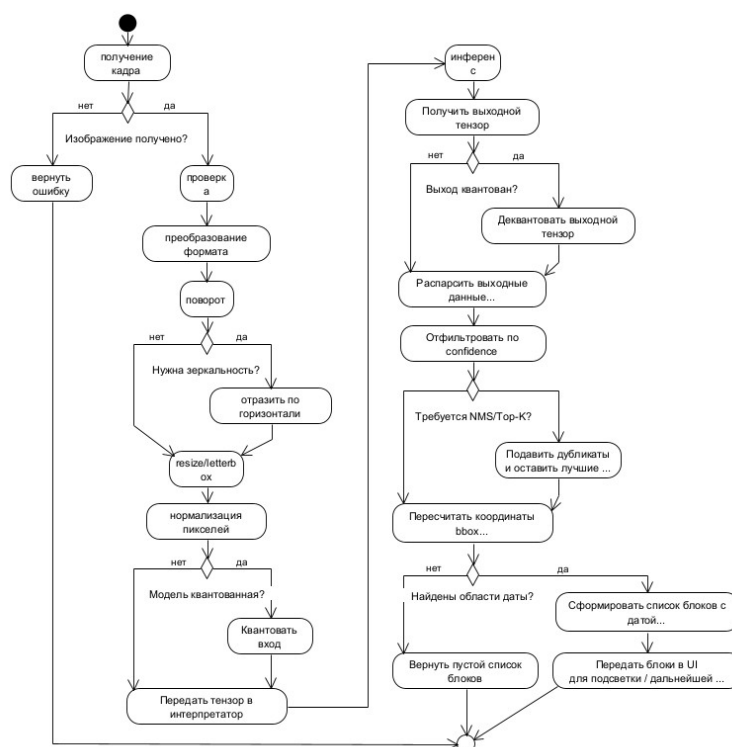


Рисунок 2.6 – Блок-схема алгоритма обнаружения области даты на мобильном устройстве

2.3.2 Алгоритм формирования новой модели на основе сообщений об ошибке

Второй ключевой алгоритм описывает замкнутый цикл улучшения качества модели на основе пользовательской обратной связи. Пользователь в мобильном приложении открывает экран сообщения об ошибке, записывает видеоролик продолжительностью до тридцати секунд, после чего приложение с помощью FFmpeg извлекает кадры с частотой шесть кадров в секунду. Кадры сохраняются в исходных пропорциях без квадратного ра, что уменьшает искажение обучающих примеров. Далее пользователь размечает область даты на кадрах, а приложение рассчитывает YOLO-параметры рамки, формирует изображения, файлы меток и meta.json, упаковывает их в ZIP-архив и отправляет на backend API.

На сервере ErrorReportsDatasetController сохраняет и распаковывает архив, проверяет наличие папок images и labels, а также контролирует наличие хотя бы

одного изображения. После валидации создаются записи VideoSample и ErrorReport, причём новое обращение получает признак Approved = false. Затем администратор просматривает сообщение об ошибке, проверяет кадры и рамки и либо подтверждает обращение для включения в обучающий датасет, либо удаляет его.

При запуске обучения AdminTrainingController обращается к TrainingDatasetPackager, который собирает временный датасет из seed-датасета и всех approved-обращений, копирует пары изображений и YOLO-меток и формирует dataset.yaml. Датасет архивируется и передаётся в training service, где создаётся задача обучения, запускается YOLO-модель, выполняются обучение, валидация и экспорт мобильной модели в формат TFLite. После завершения сохраняются артефакты, метрики и сведения о задаче, backend API синхронизирует результат с таблицей ModelVersions, а мобильное приложение может скачать и применить новую версию модели через экран настроек.

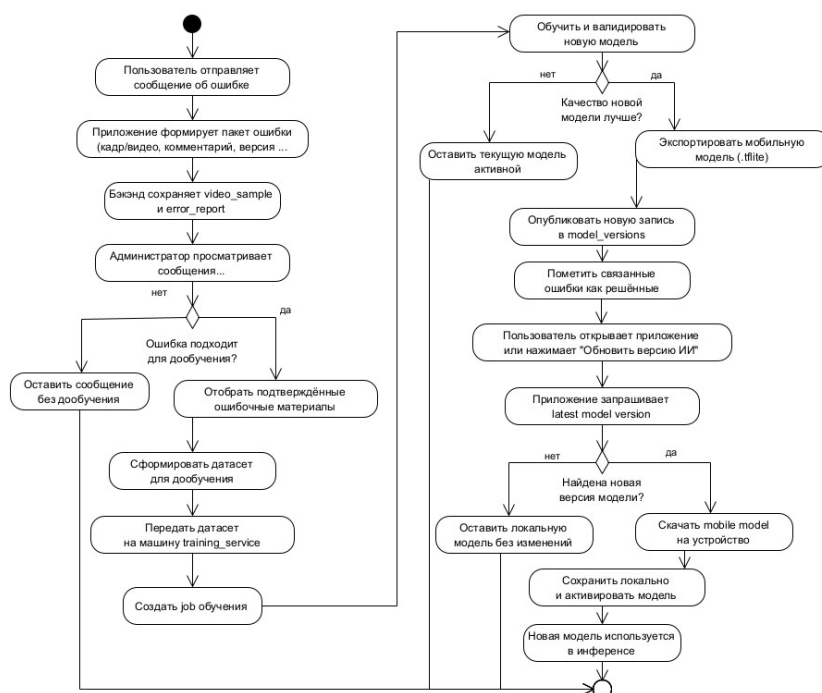


Рисунок 2.7 – Блок-схема процесса от получения сообщения об ошибке до публикации новой модели

2.3.3 Формулы расчёта показателей качества

Для оценки качества детекции в обучающем контуре используется стандартный набор метрик обнаружения объектов: IoU, precision, recall, AP, mAP50, mAP75 и mAP50–95. В текущем проекте рассчитанные значения сохраняются в составе метрик версии модели и используются при сравнении результатов обучения.

Базовой величиной является метрика IoU, определяющая степень пересечения предсказанной и эталонной ограничивающих рамок:

$$IoU = \frac{S(B_p \cap B_t)}{S(B_p \cup B_t)} \quad (2.1)$$

где B_p – предсказанная моделью ограничивающая рамка; B_t – эталонная рамка из разметки; $S(B_p \cap B_t)$ – площадь пересечения рамок; $S(B_p \cup B_t)$ – площадь их объединения.

Для оценки корректности найденных объектов используются показатели точности и полноты. Точность показывает, какая доля найденных моделью объектов действительно относится к целевому классу:

$$Precision = \frac{TP}{TP + FP} \quad (2.2)$$

где TP – число истинно-положительных детекций; FP – число ложноположительных детекций.

Полнота показывает, какая доля объектов целевого класса была обнаружена моделью:

$$Recall = \frac{TP}{TP + FN} \quad (2.3)$$

где FN – число пропущенных объектов.

Для обобщённой оценки качества детекции используется показатель AP, который определяется как площадь под кривой зависимости precision от recall:

$$AP = \int_0^1 Precision(Recall) dRecall \quad (2.4)$$

Метрика mAP представляет собой среднее значение AP. Для одной категории объектов, соответствующей области даты, значение mAP фактически совпадает с AP при выбранном пороге IoU. В обучающем контуре дополнительно используются разновидности этой метрики:

$$mAP50 = AP_{IoU=0.50} \quad (2.5)$$

$$mAP75 = AP_{IoU=0.75} \quad (2.6)$$

$$mAP50-95 = \frac{1}{10} \sum_{t=0.50}^{0.95} AP_t, \quad t = 0.50, 0.55, \dots, 0.95 \quad (2.7)$$

Метрика mAP50 отражает качество обнаружения при пороге IoU 0,50, mAP75 – при более строгом пороге 0,75, а mAP50–95 усредняет качество модели по нескольким порогам IoU от 0,50 до 0,95 с шагом 0,05. В интерфейсе training_service дополнительно хранится показатель fitness – интегральный служебный показатель, возвращаемый библиотекой обучения и используемый для ранжирования качества полученной модели.

2.4 Программное обеспечение

2.4.1 Описание используемых технологий, фреймворков и библиотек

Программное обеспечение системы построено как совокупность четырёх связанных модулей: мобильного приложения на Flutter, серверного API на ASP.NET Core, административного десктопного приложения на Windows Forms и отдельного сервиса обучения на FastAPI. Такое разбиение позволяет отделить пользовательский сценарий сканирования от хранения данных, администрирования и ресурсоёмких задач обучения.

Таблица 2.7 – Технологический стек системы

Подсистема	Технологии	Назначение
Мобильное приложение	Flutter, Dart, camera, tfliite_flutter, image, ffmpeg_kit_flutter_new	Сканирование, локальная детекция, разметка и отправка датасета

Подсистема	Технологии	Назначение
Backend API	.NET 8, ASP.NET Core Web API, Entity Framework Core, PostgreSQL, BCrypt, JWT	Авторизация, личный кабинет, приём обращений, связь с training service
Административное приложение	C#, Windows Forms	Просмотр обращений, запуск и мониторинг обучения
Сервис обучения	Python, FastAPI, Ultralytics YOLO, Torch, YAML, psutil	Формирование задач, обучение, валидация и экспорт мобильной модели
Хранилище данных	PostgreSQL	Пользователи, продукты, обращения, версии модели

2.4.2 Структура программного обеспечения

Программное обеспечение разрабатываемой системы построено по модульному принципу. Для обеспечения наглядности и удобства анализа структура классов представлена не одной общей диаграммой, а несколькими диаграммами, каждая из которых отражает отдельный контур системы: общий слой предметных моделей, мобильное приложение, серверную часть и административное приложение. Такое разбиение позволяет показать не только состав классов, но и распределение ответственности между подсистемами, а также их место в общей архитектуре программного решения.

Структура программного обеспечения раскрывается через четыре диаграммы классов. Общая модель сущностей представлена на рисунке 2.8 и включает пользователей, продукты, записи хранения, сессии сканирования, результаты детекции, сообщения об ошибках и версии модели. Эти классы задают единый доменный слой, используемый мобильным клиентом, серверной частью и административным приложением.

Диаграмма классов мобильного приложения на рисунке 2.9 показывает разделение Flutter-клиента на экранные формы, сервисы логики, репозитории и компонент локального инференса. В отдельные классы вынесены авторизация, работа с личным кабинетом, сканирование, отправка ошибочных примеров, синхронизация модели и распознавание даты.

Серверная часть на рисунке 2.10 построена как многослойный backend API: контроллеры принимают HTTP-запросы, сервисы реализуют бизнес-логику, репозитории работают с базой данных, а вспомогательные классы отвечают за токены, конфигурацию, хранение файлов и обмен с training_service.

Административное приложение на рисунке 2.11 представляет WinForms-клиент сопровождения. Оно содержит формы входа, главного меню, просмотра сообщений об ошибках, управления обучением и версиями модели. Такое разделение упрощает сопровождение системы и позволяет независимо развивать пользовательский, серверный, административный и обучающий контуры.

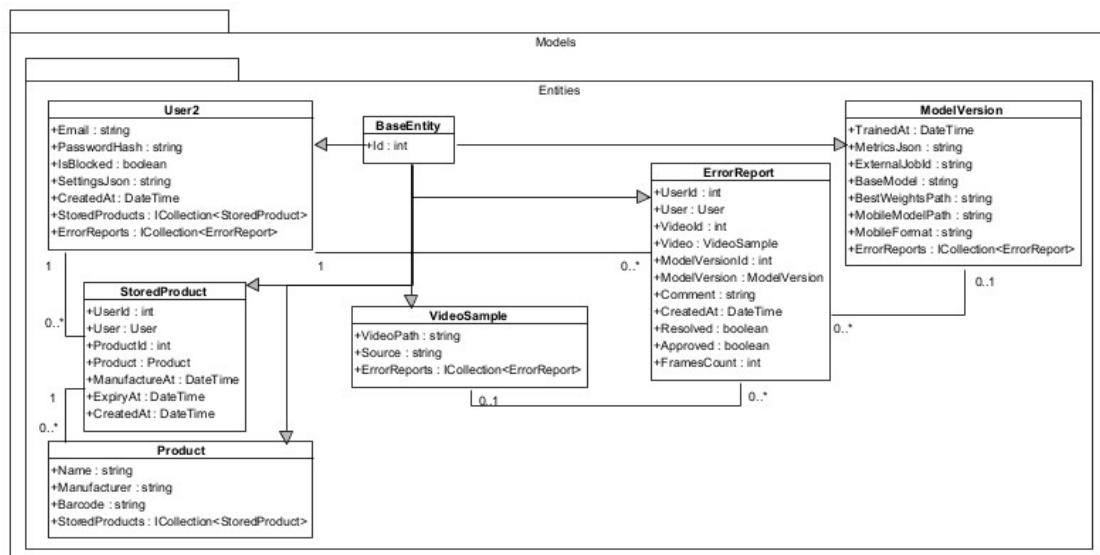


Рисунок 2.8 – Диаграмма классов общих моделей и сущностей

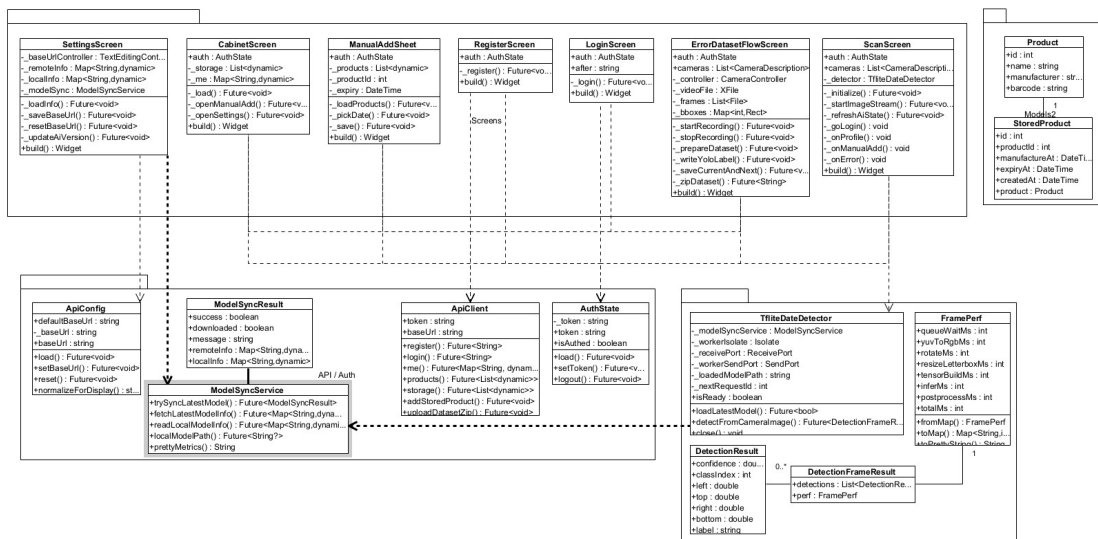


Рисунок 2.9 – Диаграмма классов мобильного приложения

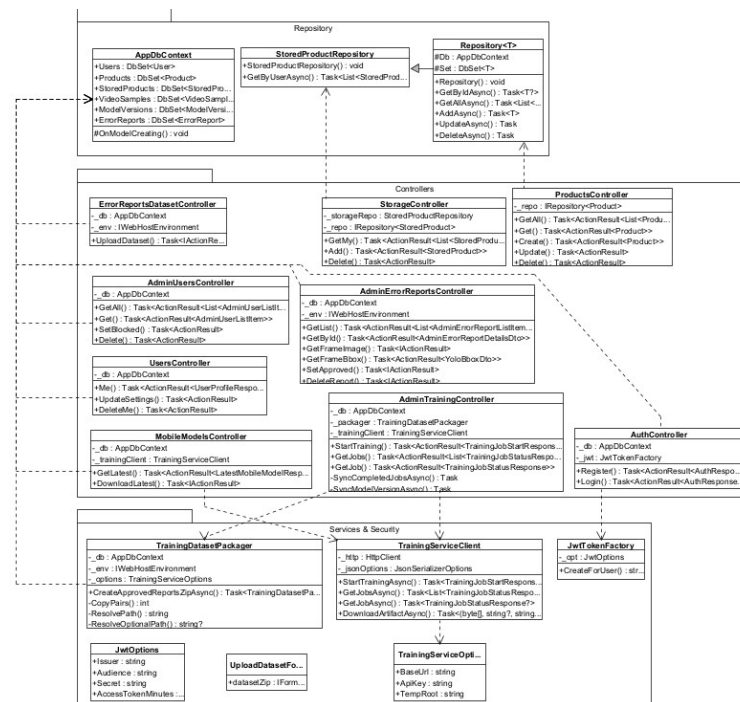


Рисунок 2.10 – Диаграмма классов backend API

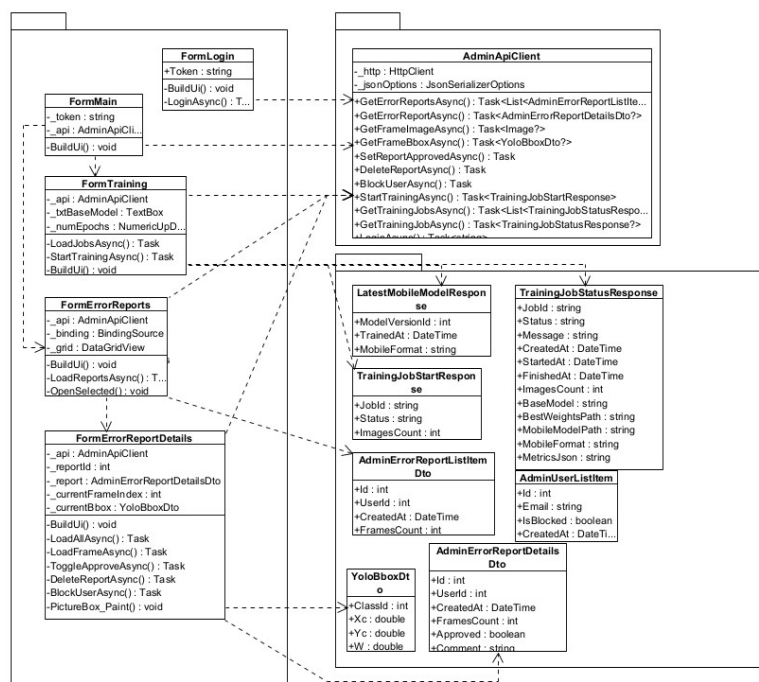


Рисунок 2.11 – Диаграмма классов административного приложения

2.4.3 Спецификации программных модулей

Диаграмма компонентов, представленная на рисунке 2.12, отражает состав основных программных модулей системы и связи между ними. Мобильное приложение выполняет сканирование упаковки, локальную детекцию, распознавание даты, работу с личным кабинетом и отправку обратной связи. Backend API обеспечивает авторизацию, хранение продуктов, прием сообщений об ошибках, публикацию версий модели и обмен данными с административным приложением.

Административное приложение предоставляет оператору функции просмотра обращений, утверждения или удаления размеченных примеров, резервного копирования, запуска обучения и просмотра метрик версий модели. Сервис обучения выделен в отдельный контур: он получает задачу от backend API, формирует датасет, запускает обучение YOLO, сохраняет метрики и экспортирует мобильный tflite-файл.

База данных PostgreSQL хранит учетные записи, продукты, пользовательские записи хранения, сообщения об ошибках, сведения о видеопримерах и версиях моделей. Файловые артефакты, включая изображения, датасеты и веса модели, хранятся отдельно, а в базе фиксируются пути и метаданные. Такое разделение снижает нагрузку на СУБД и упрощает резервное копирование.

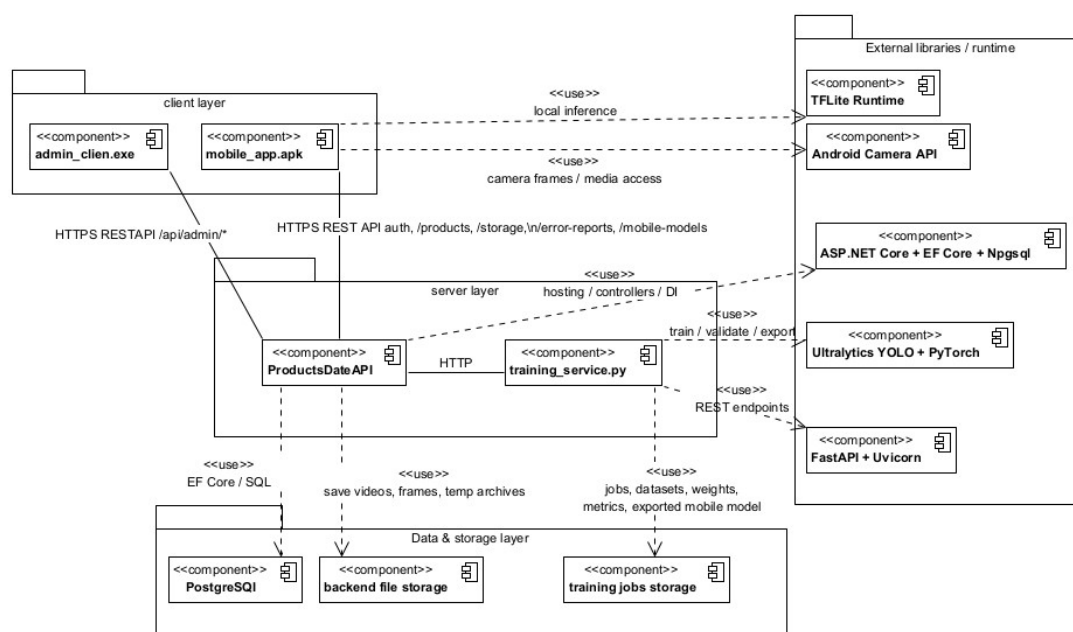


Рисунок 2.12 – Диаграмма компонентов программной системы

2.4.4 Структура диалога с пользователем

Структура диалога с пользователем в разрабатываемой системе определяется составом экранов мобильного клиента и административного приложения, а также переходами между ними в рамках основных сценариев работы. Диаграммы состояний, представленные на рисунках 2.13 и 2.14, отражают логику пользовательского взаимодействия с системой и показывают, в какой последовательности пользователь и администратор переходят между экранами интерфейса.

Диаграмма состояний мобильного приложения на рисунке 2.13 отражает, что центральным экраном является сканирование. Из него пользователь переходит к авторизации, личному кабинету, добавлению продукта, настройкам и сообщению об ошибке. Сценарий обратной связи включает запись кадров, ручную разметку области даты и отправку архива на сервер.

Таким образом, структура диалога построена вокруг двух центральных процессов: пользовательского сканирования упаковки и административного сопровождения качества модели. Краткое описание переходов между экранами мобильного клиента приведено в таблице 2.8.

Рисунок 2.13 – Диаграмма состояний мобильного приложения



Рисунок 2.14 – Диаграмма состояний административного приложения

Таблица 2.8 – Переходы между экранами мобильного приложения

Текущий экран	Доступные переходы	Основные действия
Главный экран сканирования	Личный кабинет, ручное добавление, сообщение об ошибке, настройки, авторизация	Просмотр камеры, отображение детекции, запуск прикладных сценариев
Авторизация	Регистрация, настройки, возврат к исходному сценарию	Ввод email и пароля, получение токена
Регистрация	Авторизация	Создание учётной записи
Личный кабинет	Настройки, ручное добавление, выход	Просмотр профиля и списка продуктов
Ручное добавление	Возврат к кабинету или главному экрану	Ввод названия продукта и даты срока годности
Настройки	Возврат к предыдущему экрану	Изменение BaseUrl, обновление модели ИИ
Сообщение об ошибке	Возврат на главный экран	Запись видео, разметка, отправка датасета

2.5 Обеспечение информационной безопасности

Вопросы информационной безопасности в проекте связаны с защитой учётных данных пользователей, ограничением доступа к персональным данным и контролем административных действий. В backend API для пользовательского контура применяется JWT-аутентификация, а в административном приложении используется тот же токен доступа для обращения к административным эндпоинтам.

Пароли в базе данных не хранятся в открытом виде и обрабатываются с использованием BCrypt. Доступ к персональным данным и к личному кабинету ограничивается идентификатором текущего пользователя, извлекаемым из токена. В административном контуре реализованы действия по блокировке пользователя, удалению обращения и включению либо исключению примера из обучающего датасета.

Дополнительно безопасность обеспечивается валидацией входных данных, контролем структуры загружаемого ZIP-архива, ограничением размера запроса для загрузки датасета и изоляцией сервиса обучения в отдельном процессе. Видеофайлы не сохраняются внутри базы данных: в БД хранятся только ссылки на каталоги с данными и служебные метаданные.

2.6 Техническое обеспечение

2.6.1 Схема развёртывания

Система реализована как распределённый программный комплекс с мобильным клиентом, сервером приложений, сервером базы данных и выделенным сервисом обучения. Мобильное приложение запускается на смартфоне пользователя и содержит встроенный модуль локальной TFLite-детекции. Backend API и training service могут быть размещены на одном сервере разработки либо на разных узлах. PostgreSQL используется как централизованное хранилище данных.

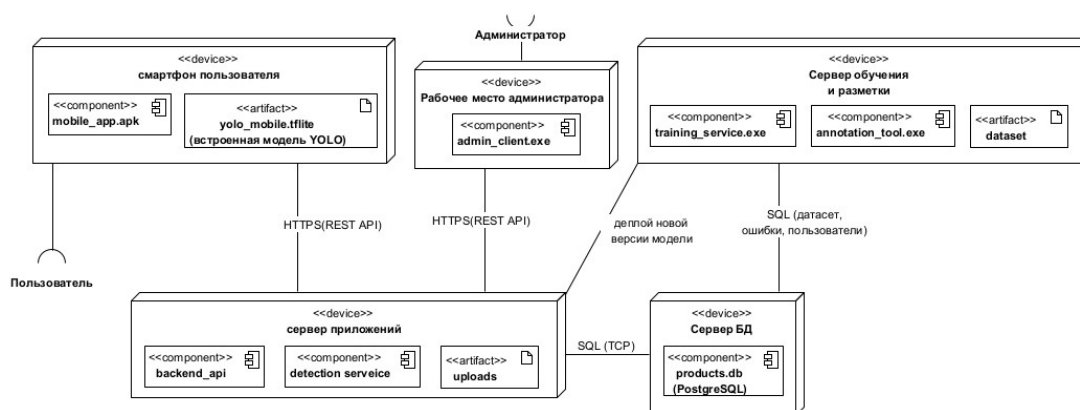


Рисунок 2.15 – Диаграмма развёртывания системы

2.6.2 Технические характеристики

Для корректного функционирования разрабатываемой системы требуется минимальный набор технических средств, обеспечивающий выполнение пользовательских, серверных, административных и обучающих операций. Основная вычислительная нагрузка в пользовательском контуре приходится на мобильное устройство, на котором выполняется локальная детекция области даты, тогда как серверный контур отвечает за хранение данных, авторизацию, обработку обращений и взаимодействие с сервисом обучения. Отдельные требования предъявляются к обучающему модулю, поскольку задачи дообучения модели и экспорта мобильного артефакта являются наиболее ресурсоёмкими. Минимальные технические требования к основным компонентам системы приведены в таблице 2.9.

Таблица 2.9 – Минимальные технические требования

Компонент	Минимальные требования	Назначение
Смартфон пользователя	Android 10+, камера, 3 ГБ ОЗУ	Локальная детекция и отправка обучающих примеров
Сервер backend API	.NET 8, 4 ГБ ОЗУ, сетевой доступ к PostgreSQL	Авторизация, хранение данных, обмен с training service
Сервер базы данных	PostgreSQL, 20+ ГБ диска	Хранение пользователей, продуктов, обращений, версий модели
Сервис обучения	Python 3.10+, CUDA-совместимая GPU желательна	Дообучение YOLO и экспорт TFLite
Рабочее место администратора	Windows, .NET Desktop Runtime	Контроль обращений и задач обучения

2.7 Технологическое обеспечение

Технологическое обеспечение системы отражает внутреннюю логику работы серверного контура, который связывает мобильное приложение, административное приложение, базу данных и сервис обучения модели. В данном пункте рассматривается состояние backend API как центрального программного модуля, через который проходят пользовательские, административные и

обучающие процессы. При этом серверный контур обеспечивает цикличность движения данных в системе: пользовательские обращения и сообщения об ошибках накапливаются в базе данных, затем используются для формирования обучающего набора, после чего результаты дообучения возвращаются в эксплуатационный контур в виде новой версии модели.

Серверная часть проходит этап инициализации, в ходе которого загружается конфигурация, устанавливается соединение с базой данных и подготавливаются необходимые зависимости приложения. После успешной инициализации сервер переходит в рабочее состояние, включающее вложенные состояния ожидания запросов, обработки API-запросов и запуска обучения модели. При возникновении необработанного исключения сервер переходит в состояние критической ошибки, из которого возможны повторная инициализация или остановка.

Данная модель показывает, что обучение модели рассматривается как отдельный технологический режим работы сервера, не совпадающий с обычной обработкой пользовательских запросов. Одновременно диаграмма подчёркивает цикличность данных в системе: результаты пользовательской эксплуатации поступают в серверный контур, преобразуются в обучающие данные, используются для формирования новой модели и затем возвращаются в рабочий контур в виде обновлённой версии для мобильного приложения.



Рисунок 2.16 – Диаграмма состояния backend-контур и технологического процесса обработки данных

2.8 Организационное обеспечение

Внедрение проектируемой системы не требует изменения организационной структуры предметной области, однако изменяет состав функций пользователей и администратора. Пользователь получает инструмент автоматизированного поиска даты и механизм отправки корректирующих примеров. Администратор получает средства модерации обращений и управления жизненным циклом модели. Разработчик или технический специалист сопровождает серверную часть и сервис обучения, контролирует публикацию новых версий модели и корректность инфраструктуры.

Таблица 2.10 – Распределение функций по ролям

Роль	Функции
Пользователь мобильного приложения	Сканирование упаковки, просмотр статуса ИИ, ручное добавление продукта, отправка сообщения об ошибке
Администратор	Просмотр сообщений об ошибках, проверка кадров и рамок, подтверждение примеров, запуск и контроль обучения
Технический специалист / разработчик	Сопровождение backend API, training service, базы данных и процесса публикации модели

2.9 Выводы по проектному разделу

В ходе проектирования сформировано проектное решение программной системы автоматического поиска даты изготовления и срока годности на упаковке товара. Рассмотрены основные подсистемы: мобильное приложение на Flutter, backend API, административное приложение, база данных PostgreSQL и сервис обучения модели.

В разделе описаны функциональная структура, информационное обеспечение, алгоритмы обнаружения и дообучения, программная архитектура, экранные формы, меры информационной безопасности и техническое обеспечение. Полученная модель служит основой для реализации, тестирования и дальнейшего развития программного комплекса.

3 РЕАЛИЗАЦИЯ ПРОЕКТА

3.1 Контрольный пример работы программного комплекса

Контрольный пример демонстрирует полный пользовательский сценарий работы мобильного приложения на Flutter: запуск экрана сканирования, получение кадра с камеры, локальный инференс TFLite-модели, подсветку найденной области даты, режим лупы для уточнения изображения, распознавание текстового значения даты и добавление продукта в личный кабинет пользователя.

В актуальном состоянии программный комплекс разделен на четыре контура: мобильное приложение, backend API, административное приложение и training_service. Мобильный клиент обеспечивает пользовательские сценарии сканирования и личного кабинета, а также сбор обратной связи через запись кадров и ручную разметку правильной области даты, что показано на рисунке 3.2. Состав контуров приведен в таблице 3.1.

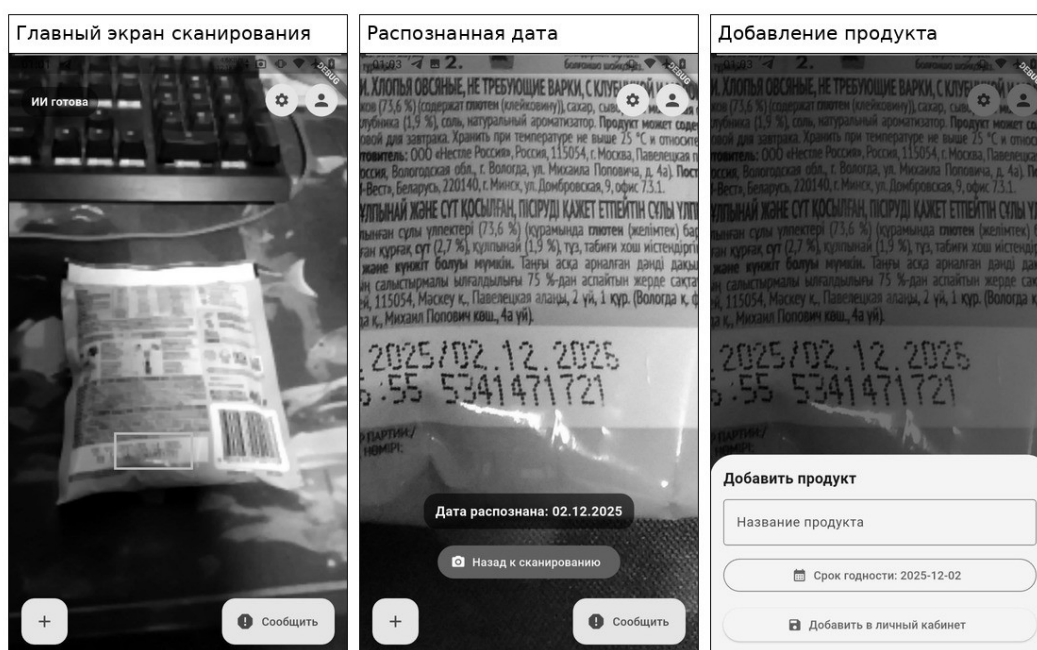


Рисунок 3.1 – Контрольный пример мобильного сценария сканирования, распознавания даты и добавления продукта

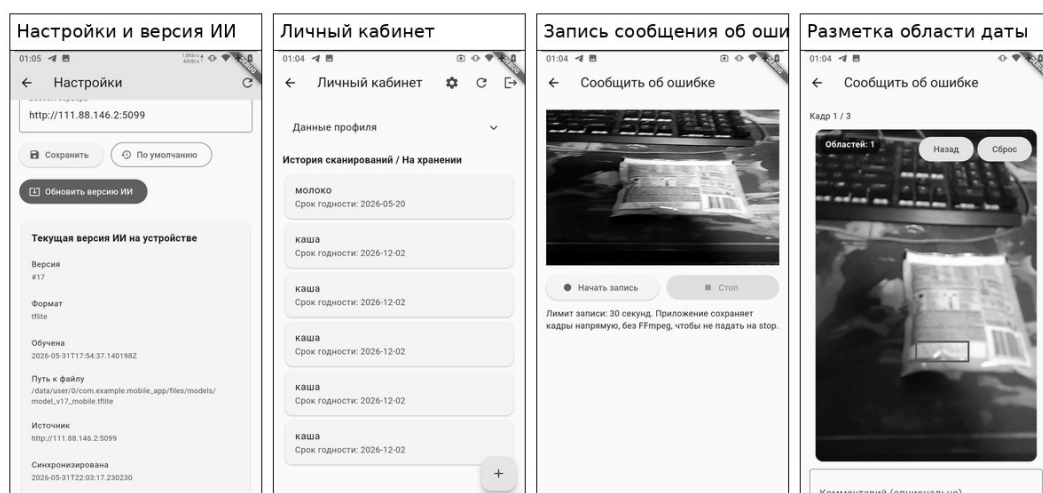


Рисунок 3.2 – Настройки, личный кабинет и сценарий отправки сообщения об ошибке в мобильном приложении

Административное приложение используется для сопровождения данных, полученных от пользователей: просмотра сообщений об ошибках, открытия отдельных примеров с разметкой, запуска резервного копирования, управления задачами обучения и сопровождения опубликованных версий модели. Основные экранные формы административного приложения представлены на рисунках 3.3–3.6.

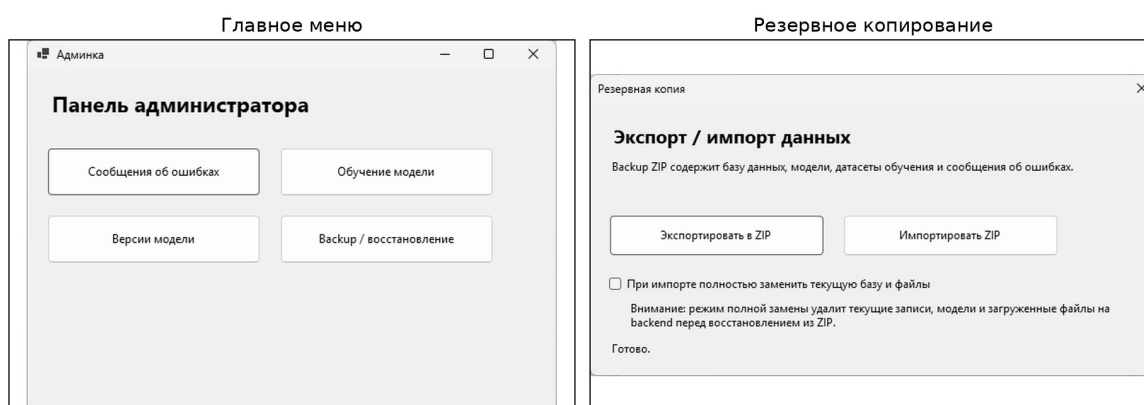


Рисунок 3.3 – Главное меню административного приложения и экран резервного копирования

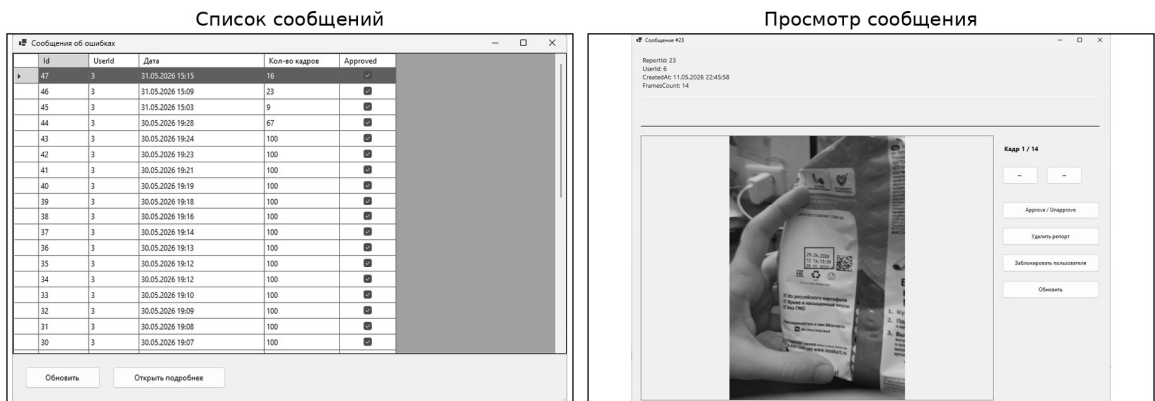


Рисунок 3.4 – Список сообщений об ошибках и просмотр отдельного сообщения

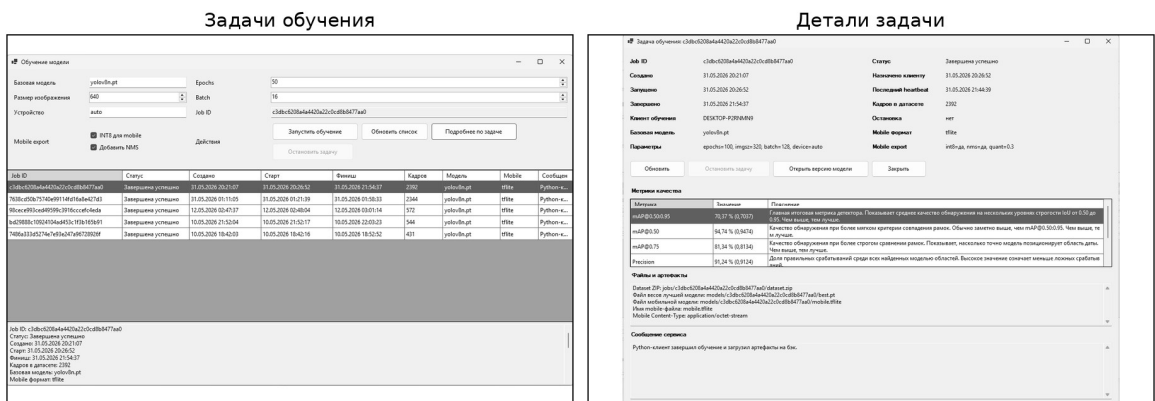


Рисунок 3.5 – Экран задач обучения и подробная информация о задаче обучения

МОДЕЛИ

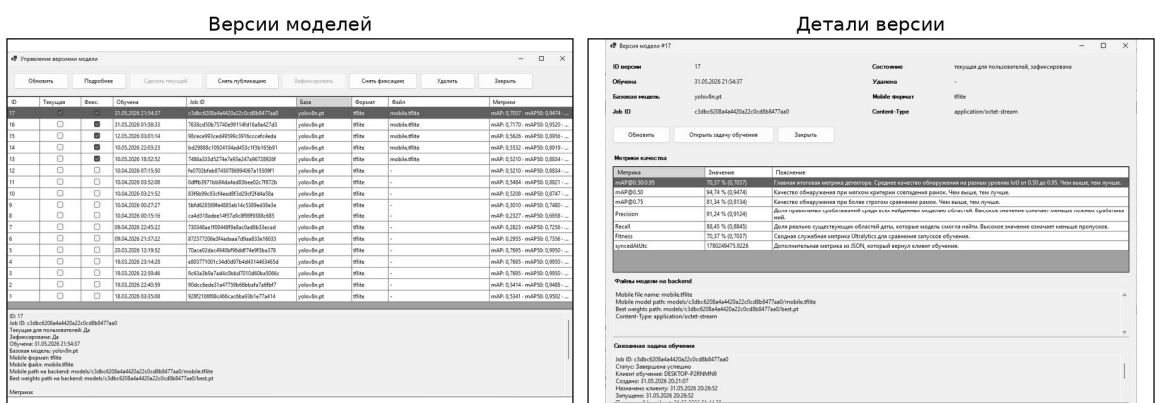


Рисунок 3.6 – Управление версиями моделей и сведения об опубликованной версии

Таблица 3.1 – Состав актуального программного комплекса

Подсистема	Технологическая основа	Реализуемая роль
mobile_app	Flutter, Dart, camera, tflite_flutter, image	Экран сканирования, авторизация, личный кабинет, обновление модели, поток отправки ошибочного датасета и ручная разметка рамок
netSolution	ASP.NET Core API, Windows Forms AdminApp, Entity Framework Core, PostgreSQL	Авторизация, личный кабинет, прием датасетов ошибок, административные действия, задачи обучения, мобильные модели
training_service	Python, requests, torch, Ultralytics YOLO, PyYAML	Получение задач обучения от backend, подготовка датасета, запуск YOLO, валидация, экспорт модели, загрузка артефактов

3.2 Реализованные технологии и основные сценарии

Мобильное приложение выполняет локальную детекцию области даты с использованием TFLite-модели, после чего предоставляет пользователю визуальную подсветку найденной области. Для повышения удобства проверки реализован режим лупы, позволяющий приблизить выделенный фрагмент упаковки, и дополнительный этап распознавания текста даты. Для этого в рамках проекта была обучена RCNN-модель, которая используется после выделения области даты и преобразует изображение фрагмента в текстовое значение даты, пригодное для последующего сохранения продукта.

Основные реализованные сценарии включают пользовательское сканирование упаковки, работу с личным кабинетом, отправку обратной связи, администрирование обращений и обучение модели. При ошибочной детекции пользователь записывает кадры, размечает область даты, после чего мобильное приложение формирует архив и отправляет его на сервер.

Таблица 3.2 – Основные сценарии текущей реализации

Сценарий	Описание
Пользовательское сканирование	Экран сканирования получает кадры камеры, подготавливает вход модели, выполняет инференс и возвращает найденные области даты
Личный кабинет	Пользователь добавляет, просматривает и удаляет продукты на хранении

Сценарий	Описание
Обратная связь	Приложение записывает кадры, пользователь размечает область даты, архив отправляется на API как обучающий пример
Администрирование	Администратор просматривает обращения, пользователей, задачи обучения и опубликованные версии модели
Обучение модели	Датасет формируется из одобренных примеров, training_service обучает YOLO-модель и экспортирует мобильный артефакт

Реализация перечисленных сценариев выполнена в виде программного комплекса, включающего мобильное приложение, backend API, административное приложение и сервис обучения модели. Основные фрагменты исходного кода информационной системы, отражающие реализацию указанных компонентов и их взаимодействие, приведены в приложении Е

3.3 Тестирование программного обеспечения

Тестирование выполнено по требованиям F1–F15 и N1–N8. Проверки охватывают мобильный клиент, backend API, административное приложение и обучающий контур. В основной части приведены сводные результаты, а полный протокол тестирования вынесен в приложение А.

В план включены функциональное системное тестирование, автоматизированные unit-тесты training_service, интеграционные HTTP contract-тесты, нагрузочные проверки, экспертное UI-тестирование и пилотное пользовательское тестирование. Такой набор проверок позволяет сопоставить каждое требование с одним или несколькими видами испытаний.

Таблица 3.3 – Сводные результаты тестирования

Вид тестирования	Покрытие	Результат	Ссылка на приложение
Функциональное системное	F1-F15, N2, N3, N6, N8	Основные пользовательские и административные сценарии пройдены	Приложение А
Unit-тесты training_service	F15, N4	Проверены функции подготовки датасета, обработки статусов и артефактов	Приложение Б

Вид тестирования	Покрытие	Результат	Ссылка на приложение
HTTP contract-тесты	F6, F8-F15, N4-N6	Проверены контракты backend/training client и ограничения доступа	Приложение Г
Нагрузочное и производительное	F1, F9, F15, N1, N4	Подтверждена работоспособность операций подготовки датасета и локального инференса	Приложение Д
Пользовательское тестирование	F9, N3, N7	Получены замечания по удобству, часть дефектов устранена	Приложение А

По итогам тестирования подтверждена работоспособность основных функций приложения и выявлены направления дальнейшей доработки: расширение пользовательской геймификации, развитие рейтинга разметчиков и повышение устойчивости мобильного сканирования в сложных условиях съемки. Полные таблицы тест-кейсов, журналы запусков и листинги приведены в приложениях А–Д.

4 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА

4.1 Анализ качества проектного решения

Качество проектного решения оценивается по соответствию разработанного программного комплекса заявленным атрибутам качества: производительности, офлайн-доступности детекции, надежности пользовательских сценариев, целостности данных, информационной безопасности и удобству интерфейса.

Производительность обеспечивается переносом инференса модели на мобильное устройство и выполнением обработки кадров без обязательного обращения к серверу. Надежность пользовательских сценариев повышается за счет обработки ошибок камеры, сетевых сбоев и некорректных архивов пользовательской обратной связи. Целостность данных поддерживается разделением метаданных в PostgreSQL и файловых артефактов в серверном хранилище.

Информационная безопасность достигается разграничением пользовательских и административных функций, применением авторизации и контролем прав доступа к административным операциям. Удобство интерфейса повышено за счет режима увеличения найденной области даты, автоматического отображения распознанного значения и передачи этой даты в форму добавления продукта. Пилотное пользовательское тестирование подтвердило, что пользователи смогли выполнять основные сценарии без дополнительного обучения, а выявленные замечания были использованы для доработки сценария отправки ошибочных примеров.

4.2 Расчет экономических результатов от внедрения

4.2.1 Расчет трудоемкости разработки программного обеспечения

Для оценки трудоемкости разрабатываемого программного комплекса используются модель композиции приложения и метод, основанный на

экспертных оценках. Первый подход позволяет получить раннюю укрупненную оценку по количеству экранов, отчетов и программных компонентов, второй – опереться на фактическую декомпозицию работ, выполненную в ходе преддипломной практики.

4.2.1.1 Модель композиции приложения

Модель композиции приложения применяется на раннем этапе проектирования и ориентирована на объектные указатели. Для рассматриваемого проекта в качестве объектных указателей выделяются экраны мобильного приложения и административного интерфейса, отчетные формы статистики и программные компоненты, реализуемые на языке общего назначения.

Распределение по типам и весам приведено в таблице.

Таблица 4.1 – Оценка объектных указателей проекта

Тип объекта	Количество	Вес	Итого
Простые экраны	3	1	3
Средние экраны	4	2	8
Сложные экраны	2	3	6
Средние отчеты / статистические формы	2	5	10
Сложные отчеты	1	8	8
3GL-компоненты	2	10	20

Общее количество объектных указателей составляет: $OP = 3 + 8 + 6 + 10 + 8 + 20 = 55$.

С учетом повторного использования типовых интерфейсных решений и инфраструктурных заготовок примем уровень повторного использования 15 %. Тогда количество новых объектных указателей составит: $NOP = OP \times (100 - 15) / 100 = 46,75$. Для учебного проекта при номинальной зрелости среды разработки принимается $PROD = 13$.

Трудоемкость разработки по модели композиции приложения определяется по формуле: $T = NOP / PROD = 46,75 / 13 = 3,60$ чел.-мес.

Оценочная длительность разработки составит: $t_{разр} = 2,5 * T^{0,35} = 2,5 * 3,60^{0,35} = 3,91$ мес.

4.2.1.2 Метод, основанный на экспертных оценках

Методы, основанные на экспертных оценках, применяются в тех случаях, когда отсутствует достаточная статистика по аналогичным проектам, но имеется возможность оценить состав и сложность работ на основе профессионального опыта. Для разрабатываемого программного комплекса в качестве экспертной базы используется детальная декомпозиция работ, подготовленная в рамках преддипломной практики. Она охватывает аналитические, проектные, мобильные, серверные, административные и ML-задачи, поэтому может служить основанием для итоговой оценки трудоемкости.

Таблица 4.2 – Экспертная оценка трудоемкости работ

Блок работ	Трудоемкость, ч
Аналитика и исследование	87
Проектирование и документация	66
Мобильное приложение и офлайн-инференс	124
Серверная часть и БД	72
Административное приложение и статистика	46
ML-пайплайн и инструменты разметки	84
Интеграция и тестирование	22
Итого	501

По результатам экспертной оценки суммарная трудоемкость составляет: $T = 501$ чел.-ч.

В пересчете на человеко-месяцы при 160 часах в месяц получаем: $T_{мес} = 501 / 160 = 3,13$ чел.-мес.

Оценочная длительность разработки составит: $t_{разр} = 2,5 * T^{0,35} = 2,5 * 3,13^{0,35} = 3,73$ мес.

4.2.1.3 Выводы по расчету трудоемкости

Таблица 4.3 – Сравнение результатов оценки трудоемкости

Метод оценки	Трудоемкость, чел.-мес.	Оценочная длительность, мес.
Модель композиции приложения	3,60	3,91
Экспертные оценки	3,13	3,73

Оценки, полученные двумя методами, находятся в близком диапазоне: модель композиции приложения дает 3,60 чел.-мес., а экспертная оценка – 3,13 чел.-мес. Это подтверждает, что для пилотной версии программного комплекса выбранный объем работ соответствует программному продукту средней сложности.

Для дальнейших экономических расчетов принимается экспертная трудоемкость 501 час (3,13 чел.-мес.), поскольку она основана на детальной декомпозиции фактических работ по проекту и наиболее полно отражает реальный состав задач, выполненных в рамках разработки программного комплекса.

4.2.2 Анализ затрат на ресурсное обеспечение

4.2.2.1 Расчет затрат на разработку программного продукта

Расчет затрат на разработку выполняется по трудоемкости, принятой по экспертной оценке. Предполагается, что проект выполняется одним разработчиком, использующим собственное оборудование и бесплатные инструменты разработки. Величина заработной платы разработчика принимается равной 40 000 руб. в месяц, что соответствует текущей заработной плате студента.

$$\text{ЗФОТР} = 40\,000 * (501 / 160) = 125\,250,00 \text{ руб.}$$

$$\text{ЗОВФ} = \text{ЗФОТР} * 0,302 = 125\,250,00 * 0,302 = 37\,825,50 \text{ руб.}$$

$$\text{ЗСПП} = 0 \text{ руб.}; \text{ЗХОН} = 0 \text{ руб.}$$

$$\text{РН} = 0,60 * \text{ЗФОТР} = 0,60 * 125\,250,00 = 75\,150,00 \text{ руб.}$$

$$\text{КРПР} = \text{ЗФОТР} + \text{ЗОВФ} + \text{ЗСПП} + \text{ЗХОН} + \text{РН} = 238\,225,50 \text{ руб.}$$

Итоговая стоимость разработки пилотной версии программного продукта составляет 238 225,50 руб. Данная сумма отражает прямые затраты на труд

разработчика и накладные расходы, но не включает возможные будущие затраты на промышленное масштабирование.

4.2.2.2 Расчет стоимости эксплуатации оборудования

Для пилотной эксплуатации системы требуется облачная инфраструктура для размещения backend API, хранения базы данных и резервных копий пользовательских материалов. Примем следующую ежемесячную структуру расходов: виртуальный сервер – 2 000 руб./мес., файловое хранилище и резервное копирование – 1 000 руб./мес. Тогда стоимость эксплуатации оборудования и инфраструктуры составит 3 000 руб. в месяц.

$$З_{\text{экспл}} = 2\,000 + 1\,000 = 3\,000 \text{ руб./мес.}$$

4.2.3 Анализ качественных и количественных факторов воздействия проекта

Внедрение системы влияет на бизнес-процессы, связанные с контролем скоропортящихся товаров. Качественный эффект проявляется в снижении количества ошибок при поиске маркировки, ускорении спорных проверок, уменьшении зависимости от опыта конкретного сотрудника и накоплении цифровой истории проблемных упаковок. Количественный эффект выражается в сокращении времени типовых операций и уменьшении затрат на контроль товара.

4.2.3.1 Расчет экономических результатов от внедрения

Рассмотрим пилотный сценарий использования в отделе магазина, где два сотрудника регулярно проверяют партии скоропортящихся товаров и ведут цифровой список продуктов на хранении. Предполагается, что за счет автоматического поиска даты и ускоренного ввода данных каждый сотрудник экономит около 40 часов в месяц, то есть суммарно 80 часов на подразделение.

Таблица 4.4 – Экономический эффект от внедрения системы

Операция	До внедрения, ч/мес.	После внедрения, ч/мес.
Поиск даты на новой партии товара	28	12

Операция	До внедрения, ч/мес.	После внедрения, ч/мес.
Плановая проверка продукции на полке	22	10
Повторная проверка спорных упаковок	14	6
Формирование и ведение списка хранения	8	4
Итого на одного сотрудника	72	32

Экономия времени на одного сотрудника составляет $72 - 32 = 40$ ч/мес. Для двух сотрудников суммарная экономия составляет 80 ч/мес.

$$Сч = 68\,000 / 176 = 386,36 \text{ руб./ч.}$$

$$Эмес = 80 * 386,36 = 30\,909,09 \text{ руб./мес.}$$

$$Эгод = 30\,909,09 * 12 = 370\,909,09 \text{ руб./год.}$$

Таким образом, прямой экономический эффект от внедрения составляет около 30 909,09 руб. в месяц без учета эффекта от уменьшения списаний и повышения лояльности пользователей.

4.2.3.2 Расчет сроков окупаемости проекта

Срок окупаемости определим как отношение затрат на разработку к чистому ежемесячному эффекту, уменьшенному на стоимость эксплуатации инфраструктуры.

$$\text{Ток} = \text{КРПР} / (\text{Эмес} - \text{Зэкспл}) = 238\,225,50 / (30\,909,09 - 3000) = 8,54 \text{ мес.}$$

Следовательно, при принятых исходных данных проект окупается примерно за 8,54 месяца, то есть менее чем за учебный год эксплуатации. При использовании системы в нескольких торговых точках срок окупаемости будет дополнительно сокращаться.

4.3 Выводы по оценке эффективности проекта

В разделе выполнено экономическое обоснование выпускной квалификационной работы по разработке мобильного приложения проверки даты изготовления продукта на Flutter. На основе проектной документации определены

бизнес-причины возникновения проекта, цели, назначение, основные требования к программному продукту, пользователи и ограничения его применения.

Трудоемкость разработки оценена двумя методами: по модели композиции приложения и по методу, основанному на экспертных оценках. По модели композиции приложения получено 3,60 чел.-мес., по экспертной оценке – 3,13 чел.-мес. Для расчета стоимости проекта принята экспертная трудоемкость, основанная на детальной декомпозиции фактических работ по проекту.

Проведенный расчет показал, что при принятых исходных данных затраты на разработку составляют 238 225,50 руб., ежемесячный экономический эффект – 30 909,09 руб., а срок окупаемости проекта – 8,54 месяца. Это подтверждает экономическую целесообразность разработки системы и ее практическую значимость для задач контроля сроков годности товаров.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы на тему «Разработка мобильного приложения проверки даты изготовления продукта на Flutter» выполнен анализ предметной области, выявлены проблемы ручного поиска даты изготовления на упаковке продукта и обоснована необходимость автоматизации данного процесса с использованием мобильного приложения и методов компьютерного зрения.

Сформированы функциональные и нефункциональные требования к системе, разработаны модели информационных процессов, проектная архитектура, информационное, алгоритмическое, программное, техническое и организационное обеспечение программного комплекса. В проекте определены роли пользователя, администратора и обучающего контура, а также описан замкнутый цикл улучшения качества модели на основе пользовательской обратной связи.

Реализация проекта включает мобильное приложение на Flutter, backend API, административное приложение, базу данных PostgreSQL и сервис обучения модели YOLO. Проведенное тестирование подтвердило работоспособность основных пользовательских, административных и обучающих сценариев. Полный протокол тестирования, журналы запусков и листинги приведены в приложениях А–Д.

Экономическое обоснование показало целесообразность пилотного внедрения программного продукта за счет сокращения времени поиска даты и снижения трудозатрат при проверке продуктов. Дальнейшее развитие проекта целесообразно направить на повышение точности модели, расширение датасета, добавление OCR-модуля для чтения значения даты и развитие пользовательских механизмов мотивации разметчиков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – Текст : непосредственный.
2. ГОСТ Р 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. – Москва : Стандартинформ, 2021. – Текст : непосредственный.
3. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2018. – Текст : непосредственный.
4. Оформление выпускной квалификационной работы бакалавра : методические указания / сост. Н. В. Корунова, Е. Н. Эгов. – Ульяновск : УлГТУ, 2024. – 54 с. – Текст : электронный.
5. BEEP – Expiry Date Tracking [Электронный ресурс] // Google Play. – Режим доступа: <https://play.google.com/store/apps/details?id=com.bgpworks.beeper>. – Дата обращения: 09.04.2026.
6. Zuzanka [Электронный ресурс] // App Store. – Режим доступа: <https://apps.apple.com/ru/app/zuzanka/id6444279925>. – Дата обращения: 09.04.2026.
7. Ultralytics Docs [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/>. – Дата обращения: 09.04.2026.
8. Flutter documentation. Take a picture using the camera [Электронный ресурс]. – Режим доступа: <https://docs.flutter.dev/cookbook/plugins/picture-using-camera>. – Дата обращения: 09.04.2026.

9. Microsoft Learn. Create web APIs with ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/aspnet/core/web-api/>. – Дата обращения: 09.04.2026.
10. PostgreSQL Documentation. JSON Types [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/datatype-json.html>. – Дата обращения: 09.04.2026.
11. TensorFlow Lite / LiteRT. On-device Inference [Электронный ресурс]. – Режим доступа: <https://www.tensorflow.org/lite/guide/inference>. – Дата обращения: 09.04.2026.
12. OpenCV Documentation. Introduction [Электронный ресурс]. – Режим доступа: <https://docs.opencv.org/4.x/d1/dfb/intro.html>. – Дата обращения: 09.04.2026.
13. Dart documentation [Электронный ресурс]. – Режим доступа: <https://dart.dev/guides>. – Дата обращения: 09.04.2026.
14. Entity Framework Core documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/ef/core/>. – Дата обращения: 09.04.2026.
15. BCrypt.Net documentation [Электронный ресурс]. – Режим доступа: <https://github.com/BcryptNet/bcrypt.net>. – Дата обращения: 09.04.2026.
16. JWT Introduction [Электронный ресурс]. – Режим доступа: <https://jwt.io/introduction>. – Дата обращения: 09.04.2026.
17. FastAPI documentation [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/>. – Дата обращения: 09.04.2026.
18. PyTorch documentation [Электронный ресурс]. – Режим доступа: <https://pytorch.org/docs/stable/index.html>. – Дата обращения: 09.04.2026.
19. YAML Ain't Markup Language (YAML) Version 1.2 [Электронный ресурс]. – Режим доступа: <https://yaml.org/spec/1.2/spec.html>. – Дата обращения: 09.04.2026.

20. OWASP Mobile Application Security [Электронный ресурс]. – Режим доступа: <https://owasp.org/www-project-mobile-top-10/>. – Дата обращения: 09.04.2026.
21. ГОСТ Р 7.0.12-2011. Библиографическая запись. Сокращение слов и словосочетаний на русском языке. Общие требования и правила. – Москва : Стандартинформ, 2012. – Текст : непосредственный.
22. ГОСТ 8.417-2002. Государственная система обеспечения единства измерений. Единицы величин. – Москва : Стандартинформ, 2003. – Текст : непосредственный.
23. Gong, L. A novel unified deep neural networks methodology for use by date recognition in retail food package image / L. Gong, M. Thota, M. Yu [et al.] // Signal, Image and Video Processing. – 2021. – Vol. 15. – P. 449–457. – DOI: 10.1007/s11760-020-01764-7. – Текст : электронный.
24. Seker, A. C. A generalized framework for recognition of expiration dates on product packages using fully convolutional networks / A. C. Seker, S. C. Ahn // Expert Systems with Applications. – 2022. – Vol. 203. – Article 117310. – DOI: 10.1016/j.eswa.2022.117310. – Текст : электронный.
25. Koponen, J. Novel Deep Learning Application: Recognizing Inconsistent Characters on Pharmaceutical Packaging / J. Koponen, K. Haataja, P. Toivanen // F1000Research. – 2024. – Vol. 12. – Article 427. – DOI: 10.12688/f1000research.131775.2. – Текст : электронный.

ПРИЛОЖЕНИЕ А. ПРОТОКОЛ ТЕСТИРОВАНИЯ ПРОГРАММНОГО КОМПЛЕКСА

А.1.1 Цели, объект и границы тестирования

Цель тестирования – подтвердить соответствие системы функциональным требованиям F1–F15 и нефункциональным требованиям N1–N8, выявить риски реализации и зафиксировать результаты.

Объект	Программный комплекс поиска даты на упаковке: mobile_app, ProductsDateAPI/AdminApp, training_service.
Предмет	Корректность пользовательских сценариев, API-процессов, обучающего контура, устойчивость к ошибкам, безопасность и производительность отдельных операций.
Основание	Функциональные и нефункциональные требования, проектная архитектура и текущее состояние репозитория.
Границы	Автоматизированные прогоны охватывают Python training_service и его HTTP-контракт. Для .NET и Flutter в протокол включены ручные системные сценарии, пользовательские проверки и анализ итогового поведения приложения.

А.1.2 Виды и техники тестирования

Для уровня «отлично» в плане предусмотрено шесть видов тестирования. Они подобраны так, чтобы закрыть функциональные требования F1–F15 и нефункциональные требования N1–N8, а также сочетать ручные, автоматизированные, полуавтоматизированные и пользовательские испытания. Внутри функционального и интеграционного тестирования дополнительно используются негативные сценарии, классы эквивалентности и граничные значения.

Таблица А.1 – Применяемые виды и техники тестирования

Код	Вид испытаний	Метод	Техника	Покрытие
T1	Функциональное системное тестирование	Ручное, «черный ящик»	Формальная спецификация, позитивные/негативные сценарии, эквивалентное	F1-F15, N2, N3, N6, N8

Код	Вид испытаний	Метод	Техника	Покрытие
			разбиение и граничные значения	
T2	Модульное тестирование	Автоматизированное, «белый ящик»	Специализированное тестирование функций training_service	F15, N4
T3	Интеграционное и контрактное тестирование	Полуавтоматизированное, «серый ящик»	HTTP-контракты training client/backend, проверка статусов и ограничений доступа	F6, F8-F15, N4, N5, N6
T4	Нагрузочное и производительное тестирование	Полуавтоматизированное	Операционный профиль: подготовка датасета 100/1000/5000 пар; контроль времени инференса на мобильном устройстве	F1, F9, F15, N1, N4
T5	Тестирование удобства использования и UI	Ручное экспертное	Проверка понятности экранов и пользовательских потоков по чек-листу	F1, F3-F5, F9, N7
T6	Пилотное пользовательское тестирование и анализ обратной связи	Ручное исследовательское	Бета-тестирование на сторонних пользователях, фиксация дефектов и предложений по развитию	F9, N3, N7

А.1.3 Матрица трассируемости требований

Матрица показывает, что каждое заявленное функциональное и нефункциональное требование связано как минимум с одним видом испытаний. Для критичных требований применены два и более способа проверки.

Таблица А.2 – Матрица трассируемости требований и тестов

Требование	Содержание	Тесты	Виды проверки
F1	Сканирование	TC-F01, TC-F15, PERF-01	T1, T4, T5
F2	Подсветка bbox	TC-F01, UI-01	T1, T5
F3	Дата: автомат/ручной ввод	TC-F02	T1, T5
F4	Добавление продукта	TC-F03	T1, T5
F5	Просмотр продуктов	TC-F04	T1, T5
F6	Удаление продукта	TC-F05, CT-03	T1, T3
F7	Регистрация	TC-F06	T1
F8	Авторизация	TC-F07, CT-03	T1, T3
F9	Отправка ошибочного датасета	TC-F08, TC-F17, BETA-03, LOAD-01	T1, T4, T6
F10	Просмотр обращений	TC-F09, CT-03	T1, T3
F11	Модерация/удаление обращений	TC-F10, CT-03	T1, T3

Требование	Содержание	Тесты	Виды проверки
F12	Список пользователей	TC-F11, CT-03	T1, T3
F13	Блокировка пользователя	TC-F12, CT-03	T1, T3
F14	Метрики и версии модели	TC-F13	T1, T3
F15	Обучение/дообучение	TC-F14, UT-01...UT-06, CT-01...CT-03, LOAD-01	T1, T2, T3, T4
N1	Производительность сканирования	PERF-01	T4
N2	Офлайн-детекция	TC-F15	T1
N3	Ошибки камеры/сети	TC-F15, TC-F16, BETA-03	T1, T6
N4	Надежность обучающего контура	UT-01...UT-06, CT-01...CT-02, LOAD-01	T2, T3, T4
N5	Безопасность и разграничение доступа	CT-03	T3
N6	Целостность пользовательских и файловых данных	TC-F03...TC-F14, CT-01...CT-03	T1, T3
N7	Удобство пользовательского интерфейса	UI-01...UI-05, BETA-01...BETA-04	T5, T6
N8	Совместимость целевой среды	TC-F18	T1

А.1.4 Критерии завершения тестирования

все функциональные требования F1–F15 должны иметь отражение в тестовом регистре;

для автоматизированных и полуавтоматизированных проверок должны быть приведены результаты запуска;

обнаруженные несоответствия должны быть оформлены как дефекты с указанием причины и рекомендации;

нефункциональные требования должны быть сопоставлены с отдельными тестами или ограничениями текущей проверки.

А.2 Протокол тестирования

А.2.1 Функциональное тестирование

Функциональное тестирование оформлено как системный ручной контроль по спецификации. В него включены позитивные сценарии, негативные сценарии, классы эквивалентности и граничные значения для дат, учетных данных и ZIP-архивов пользовательской обратной связи. Такой состав позволяет в рамках

одного вида испытаний проверить основные пользовательские функции, офлайн-детекцию, реакцию на отсутствие разрешений камеры и корректность обработки ошибочных входных данных.

Таблица А.3 – Реестр функциональных тест-кейсов

Кейс	Треб.	Сценарий	Шаги	Ожидаемый результат	Результат
TC-F01	F1-F2	Сканирование упаковки и подсветка даты	1) Открыть экран сканирования. 2) Разрешить доступ к камере. 3) Навести камеру на упаковку с датой.	Отображается видеопоток; при детекции рисуется рамка и confidence.	Пройдено по ручному контрольному прогону.
TC-F02	F3	Ручной ввод даты	1) Открыть сценарий добавления продукта. 2) Выбрать ручной ввод даты. 3) Ввести корректную дату; затем повторить с некорректной.	Корректная дата принимается; некорректная не должна сохраняться без сообщения об ошибке.	Пройдено по ручному прогону: некорректный формат даты не возможен.
TC-F03	F4	Добавление продукта на хранение	1) Авторизоваться. 2) Ввести название продукта и даты. 3) Нажать «Сохранить».	Создается StoredProduct, запись появляется в личном кабинете.	Пройдено по ручному прогону.
TC-F04	F5	Просмотр списка продуктов	1) Авторизоваться. 2) Открыть кабинет. 3) Просмотреть список сохраненных товаров.	Отображается список пользователя либо корректное пустое состояние.	Пройдено по ручному прогону.
TC-F05	F6	Удаление продукта с хранения	1) В списке выбрать собственную запись. 2) Нажать удаление. 3) Обновить список.	Запись удаляется, список обновляется; удаление чужой записи сервер должен запретить.	Пройдено по ручному прогону.
TC-F06	F7	Регистрация пользователя	1) Открыть регистрацию. 2) Ввести новый email/пароль. 3) Повторить с уже существующим email.	Новый пользователь создается; повторный email отклоняется.	Пройдено по ручному прогону.
TC-F07	F8	Авторизация пользователя	1) Ввести корректные учетные данные. 2) Повторить с неверным паролем. 3) Проверить	Корректный вход выдает токен; неверный пароль/блокировка приводят к отказу.	Пройдено по ручному прогону.

Кейс	Треб.	Сценарий	Шаги	Ожидаемый результат	Результат
			заблокированного пользователя.		
TC-F08	F9	Отправка сообщения об ошибке	1) Авторизоваться. 2) Записать кадры ошибочного примера. 3) Разметить область даты. 4) Сформировать и отправить архив.	Backend принимает ZIP, проверяет images/labels, создает ErrorReport и VideoSample.	Пройдено по ручному сценарию;
TC-F09	F10	Просмотр сообщений об ошибках в админке	1) Выполнить вход в админ-приложение. 2) Открыть список обращений. 3) Открыть детали одного обращения.	Отображаются обращения, кадры и координаты разметки.	Пройдено по ручному сценарию.
TC-F10	F11	Модерация сообщения об ошибке	1) Открыть обращение. 2) Выбрать «Одобрить» либо «Удалить». 3) Обновить список.	Статус Approved изменяется либо обращение удаляется.	Пройдено по ручному сценарию.
TC-F11	F12	Просмотр пользователей администратором	1) Открыть раздел пользователей. 2) Просмотреть список. 3) Открыть карточку пользователя.	Показываются email, ID, статус блокировки и дата создания.	Пройдено по ручному сценарию.
TC-F12	F13	Блокировка пользователя	1) Выбрать пользователя. 2) Изменить IsBlocked. 3) Попробовать войти под заблокированным пользователем.	Флаг блокировки сохраняется; вход заблокированного пользователя отклоняется.	Пройдено по ручному прогону.
TC-F13	F14	Просмотр метрик и версий модели	1) Открыть раздел обучения/версий модели. 2) Выбрать завершенную задачу. 3) Просмотреть metricsJson.	Показываются mAP, precision, recall и сведения о версии модели.	Пройдено по ручному прогону.
TC-F14	F15	Запуск и контроль дообучения	1) Одобрить обращения. 2) Запустить обучение. 3) Проверить статус queued/running. 4) Получить артефакты и опубликовать модель.	Создается TrainingJob, клиент забирает задачу, загружает best.pt и мобильную модель, версия модели доступна для публикации.	Пройдено по ручному прогону.
TC-F15	N2, N3	Офлайн-детекция без доступа к сети	1) Отключить Wi-Fi и мобильные данные. 2) Открыть	Сканирование и локальная подсветка области	Пройдено по ручному прогону.

Кейс	Треб.	Сценарий	Шаги	Ожидаемый результат	Результат
			экран сканирования. 3) Навести камеру на упаковку с видимой датой.	даты остаются доступными; отправка серверных действий переносится до появления сети.	
TC-F16	N3	Отказ в доступе к камере	1) Запретить приложению доступ к камере в настройках ОС. 2) Запустить экран сканирования.	Приложение не завершается аварийно, показывает понятное сообщение о необходимости разрешения камеры.	Пройдено по ручному прогону.
TC-F17	F9, N6	Отправка некорректного ZIP-архива датасета	1) Авторизоваться. 2) Отправить архив без папки labels либо без изображений. 3) Проверить ответ API.	Сервер отклоняет архив и возвращает сообщение об ошибке без создания некорректной записи.	Пройдено: некорректный ZIP-архив отклоняется сервером.
TC-F18	N8	Проверка совместимости целевой среды	1) Запустить мобильное приложение на Android 10+ или iOS 14+. 2) Открыть экран сканирования и личный кабинет. 3) Зафиксировать корректный запуск.	Основные экраны открываются без ошибок на целевой версии ОС.	Пройдено на целевой мобильной среде; основные экраны открываются без ошибок.

Для наглядного подтверждения результатов функционального тестирования после таблицы 4.1 предусмотрено размещение экранных снимков ключевых сценариев.

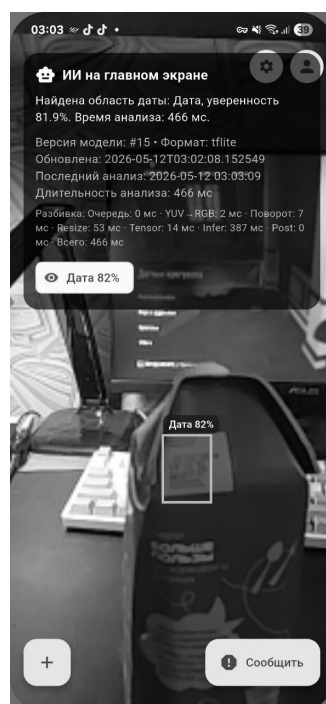


Рисунок А.1 – Результат ТС-F01: локальная детекция области даты

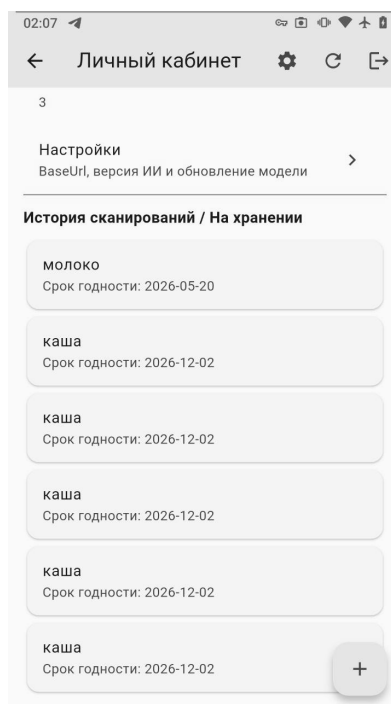


Рисунок А.2 – Результаты ТС-F03 и ТС-F04: сохранение и просмотр продукта

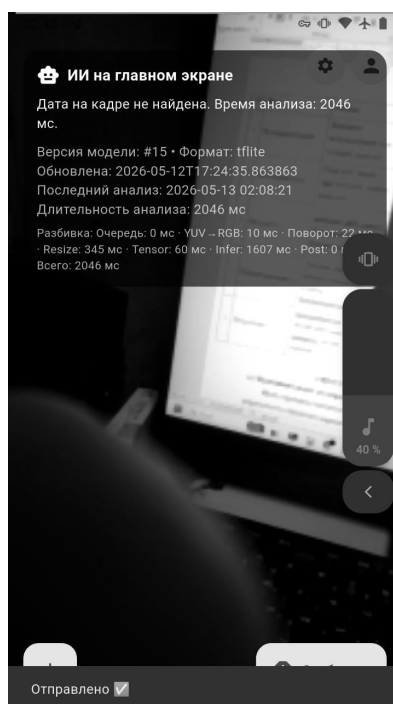


Рисунок А.3 – Результат ТС-F08: формирование и отправка ошибочного примера

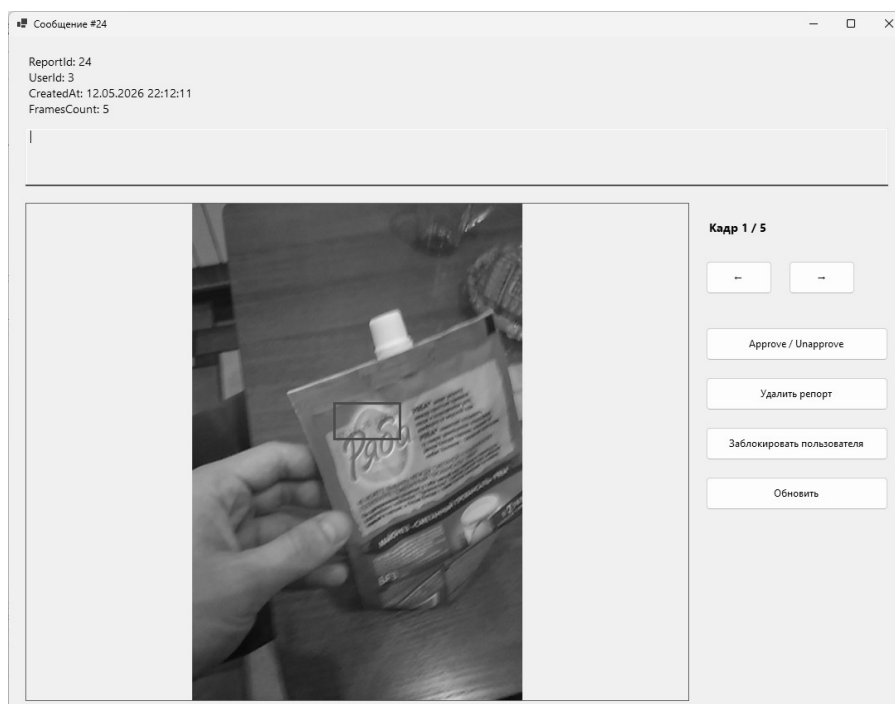


Рисунок А.4 – Результаты ТС-F09 и ТС-F10: обработка сообщения об ошибке администратором

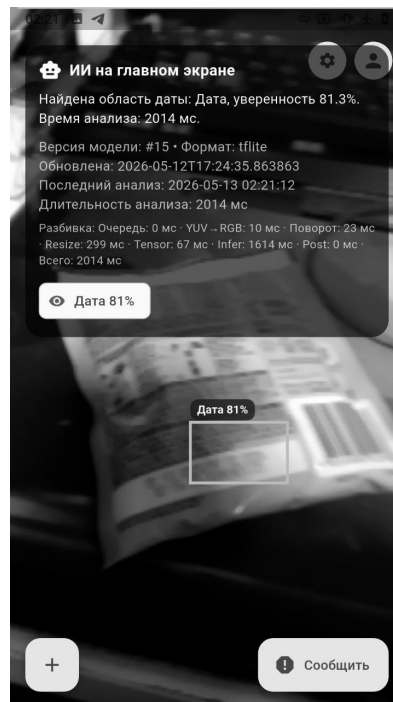


Рисунок А.5 – Результат ТС-F15: офлайн-детекция на мобильном устройстве

А.2.2 Автоматизированные unit-тесты training_service

В ходе подготовки отчета выполнен автоматизированный прогон шести unit-тестов на Python. Тесты запускаются стандартным модулем unittest и проверяют служебные функции сервиса обучения без запуска тяжелой модели YOLO.

Таблица А.4 – Результаты unit-тестирования Python training_service

Код	Тестируемый модуль	Цель	Результат
UT-01	TrainConfig.from_job	Проверка корректного чтения параметров обучения.	OK
UT-02	normalize_dataset_layout	Копирование вложенных папок images/labels в корень датасета.	OK
UT-03	normalize_dataset_yaml	Создание dataset.yaml с path/train/val/nc/names.	OK
UT-04	count_images	Подсчет jpg/jpeg/png и игнорирование иных расширений.	OK
UT-05	resolve_train_device	Обработка режимов auto и cpu.	OK
UT-06	extract_metrics	Преобразование метрик в JSON-представление.	OK

Листинг 4.1 – Вывод запуска автоматизированных unit-тестов

```

test_count_images_ignores_non_images
(__main__.TrainingServiceHelperTests.test_count_images_ignores_non_images) ... ok
test_extract_metrics_maps_detection_metrics
(__main__.TrainingServiceHelperTests.test_extract_metrics_maps_detection_metrics) ... ok
test_normalize_dataset_layout_copies_nested_images_and_labels
(__main__.TrainingServiceHelperTests.test_normalize_dataset_layout_copies_nested_images_and_labels) ... ok
test_normalize_dataset_yaml_creates_required_fields
(__main__.TrainingServiceHelperTests.test_normalize_dataset_yaml_creates_required_fields) ... ok
test_resolve_train_device_auto_and_cpu
(__main__.TrainingServiceHelperTests.test_resolve_train_device_auto_and_cpu) ... ok
test_train_config_default_and_override
(__main__.TrainingServiceHelperTests.test_train_config_default_and_override) ... ok
-----
Ran 6 tests in 0.038s OK

```

Фактический результат автоматизированного unit-тестирования зафиксирован на рисунке 4.6.

```

Python unittest: training_service
test_count_images_ignores_non_images (__main__.TrainingServiceHelperTests.test_count_images_ignores_non_images) ... ok
test_extract_metrics_maps_detection_metrics (__main__.TrainingServiceHelperTests.test_extract_metrics_maps_detection_metrics) ... ok
test_normalize_dataset_layout_copies_nested_images_and_labels (__main__.TrainingServiceHelperTests.test_normalize_dataset_layout_copies_nested_images_and_labels) ... ok
test_normalize_dataset_yaml_creates_required_fields (__main__.TrainingServiceHelperTests.test_normalize_dataset_yaml_creates_required_fields) ... ok
test_resolve_train_device_auto_and_cpu (__main__.TrainingServiceHelperTests.test_resolve_train_device_auto_and_cpu) ... ok
test_train_config_default_and_override (__main__.TrainingServiceHelperTests.test_train_config_default_and_override) ... ok
-----
Ran 6 tests in 0.038s
OK

```

Рисунок А.6 – Скриншот успешного запуска unit-тестов training_service

Итог прогона: 6 тестов выполнены успешно за 0,038 с. Это подтверждает корректность вспомогательной логики подготовки и нормализации обучающего датасета, а также формирования метрик, что связано с требованиями F15 и N4.

А.2.3 Полуавтоматизированные интеграционные HTTP contract-тесты

Для полуавтоматизированной проверки взаимодействия training_service с backend-контрактом использован локальный HTTP stub. Такой подход соответствует тестированию «серого ящика»: проверяется корректность формируемых HTTP-вызовов, обработка 204 No Content, реакция клиента на отмену задачи и разграничение доступа к чувствительным административным маршрутам. Именно в рамках данного вида испытаний контролируется нефункциональное требование безопасности N5.

Таблица А.5 – Результаты полуавтоматизированных интеграционных HTTP contract-тестов

Код	Контракт	Проверяемое поведение	Результат
СТ-01	GET /api/training-client/jobs/next -> 204	process_next_job() должен вернуть False при отсутствии queued-задач.	ОК
СТ-02	GET canceled job + POST status	ensure_not_canceled() должен отправить статус canceled и завершить выполнение RuntimeError.	ОК
СТ-03	GET/POST /api/admin/* под токеном обычного пользователя	Сервер должен вернуть 401/403, а не административные данные или успешное изменение состояния.	Выявлен дефект D-01: требуется усиление ролевого ограничения административных маршрутов.

Листинг 4.2 – Вывод запуска HTTP contract-тестов

```
test_ensure_not_canceled_reports_canceled_status
(__main__.TrainingClientHttpContractTests.test_ensure_not_canceled_reports_canceled_status) ... ok
test_process_next_job_handles_204_no_content
(__main__.TrainingClientHttpContractTests.test_process_next_job_handles_204_no_content) ... ok
-----
Ran 2 tests in 0.519s OK
```

Фактический результат полуавтоматизированных HTTP contract-тестов представлен на рисунке 4.7.

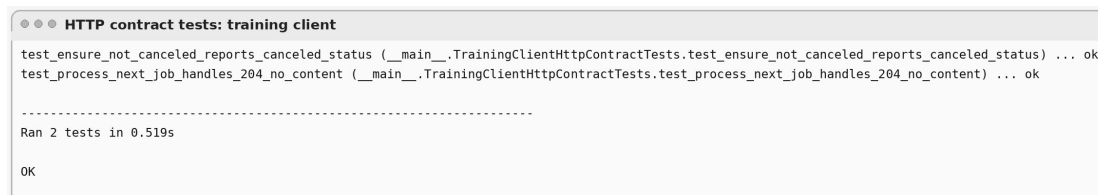


Рисунок А.7 – Скриншот успешного запуска HTTP contract-тестов

Итог автоматизированного прогона по обучающему клиенту: 2 контрактных теста выполнены успешно за 0,519 с. Дополнительно выполнен сценарий СТ-03 по разграничению доступа к административным маршрутам, в ходе которого зафиксирован дефект D-01.

Дефект D-01: обычный авторизованный пользователь не должен получать доступ к административным endpoint. Результат проверки оформлен как отдельное замечание к реализации механизма ролевого разграничения и включен в перечень доработок.

А.2.4 Нагрузочное и производительное тестирование

Нагрузочное и производительное тестирование включает два объекта.

Первый объект – подготовка датасета в training_service: создание ZIP-архива с парами images/labels, распаковка, нормализация структуры, генерация dataset.yaml и подсчет изображений. Для него использован операционный профиль с 100, 1000 и 5000 парами файлов. Второй объект – мобильный инференс на экране сканирования: фактическое время обработки кадра зафиксировано через метрики производительности и сопоставлено с требованием N1.

Таблица А.6 – Результаты нагрузочного и производительного тестирования

Код / объем	Размер / условия	Контрольный показатель	Время, мс	Статус
LOAD-01 / 100 пар	31,56 КБ	100 изображений	376,74	OK
LOAD-02 / 1000 пар	315,45 КБ	1000 изображений	3943,94	OK
LOAD-03 / 5000 пар	1577,17 КБ	5000 изображений	15612,47	OK
PERF-01 / 30 с сканирования	Целевой смартфон, открытый ScanScreen	Среднее и/или максимальное время обработки кадра	466	OK

Листинг 4.3 – Вывод нагрузочного сценария подготовки датасета

```
pairs; size_kb; images_found; elapsed_ms; status 100; 31.56; 100; 376.74; OK 1000; 315.45; 1000; 3943.94; OK 5000; 1577.17; 5000; 15612.47; OK
```

Фактический вывод нагрузочного сценария подготовки датасета представлен на рисунке 4.8.



Рисунок А.8 – Скриншот результата нагрузочного тестирования датасета

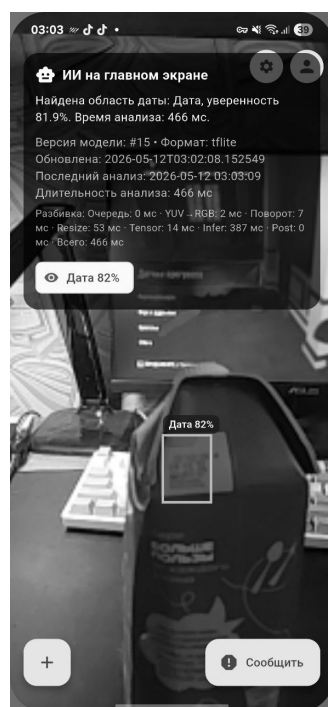


Рисунок А.9 – Скриншот результата нагрузочного тестирования PERF-01

Все три набора датасета обработаны корректно: число найденных изображений совпало с размером набора. Время обработки закономерно растёт с увеличением объема входных данных. По сценарию PERF-01 среднее время мобильной обработки кадра составило 466 мс, что не превышает целевой ориентир ранней постановки – до 1 с на кадр.

А.2.5 Тестирование удобства использования и пользовательского интерфейса

Цель тестирования удобства использования – оценить понятность ключевых пользовательских потоков мобильного клиента: сканирования упаковки, регистрации, личного кабинета, отправки ошибочного примера и обновления модели. Для проверки были привлечены пять независимых пользователей, которым предложено пройти сценарии UI-01...UI-05 без подробных инструкций.

Объект проверки – экранные сценарии и команды пользователя. Метод – ручная экспертная проверка по чек-листу как «черный ящик». Техника – исследовательское UI/usability-тестирование с фиксацией результата выполнения

заданий и последующей оценкой по шкале от 0 до 10 по критериям интуитивности, скорости, отсутствия ошибок и общей удовлетворенности.

Таблица А.7 – Сценарии тестирования удобства использования и пользовательского интерфейса

Таблица А.8 – Результаты пользовательских оценок интерфейса

Код	Проверяемый экран / сценарий	Критерий удобства	Результат
UI-01	Экран сканирования	Понятность основного действия, читаемость рамки детекции, заметность статуса модели.	Пользователи быстро поняли назначение экрана; замечаний по рамке детекции не выявлено.
UI-02	Авторизация и регистрация	Понятность полей ввода, наличие сообщений об ошибке и маршрута возврата.	Поля ввода и сообщения об ошибках признаны понятными.
UI-03	Личный кабинет продуктов	Читаемость списка, доступность добавления и удаления записи, ясность пустого состояния.	Работа со списком продуктов не вызвала затруднений.
UI-04	Поток сообщения об ошибке	Последовательность записи видео, разметки области даты и отправки архива.	Выявлена потребность в кнопке «Пропустить кадр» при разметке; доработка выполнена.
UI-05	Настройки и обновление модели	Понятность статуса версии модели и действия обновления.	Статус версии модели и действие обновления воспринимаются однозначно.

Пользователь	Интуитивность	Скорость	Отсутствие ошибок	Общая удовлетворенность	Итог
Tester_1	8	9	8	9	8,5
Tester_2	9	8	9	9	8,75
Tester_3	8	8	8	8	8,0
Tester_4	9	9	8	9	8,75
Tester_5	8	9	9	9	8,75
Средняя оценка	8,4	8,6	8,4	8,8	8,6

А.2.6 Пилотное пользовательское тестирование и анализ обратной связи

Дополнительно к формализованной оценке удобства интерфейса проведено пилотное пользовательское тестирование. Его целью стала проверка поведения приложения в более свободном сценарии использования, а также сбор замечаний

и предложений, возникающих у пользователей при самостоятельной работе с функциями разметки и отправки сообщений об ошибке.

Объектом испытаний выступили поток ручной разметки кадров, отправка пользовательского сообщения об ошибке и общее восприятие приложения. Метод – исследовательское пользовательское тестирование, близкое к бета-тестированию, с фиксацией фактических наблюдений, предложений и выявленных дефектов.

Таблица А.9 – Результаты пилотного пользовательского тестирования и обработки обратной связи

Код	Наблюдение или предложение	Категория	Принятое решение	Статус
БЕТА-01	При покадровой разметке пользователю требовалась возможность исключить неинформативный кадр.	Улучшение удобства интерфейса	Добавлена кнопка «Пропустить кадр» в сценарий разметки.	Исправлено
БЕТА-02	Предложено добавить рейтинг разметчиков и систему достижений.	Предложение по развитию	Идея принята как направление дальнейшего развития пользовательской мотивации.	Запланировано
БЕТА-03	У одного пользователя некорректно отправлялись сообщения об ошибке.	Функциональный дефект	Логика отправки сообщения об ошибке скорректирована, после чего сценарий повторно проверен.	Исправлено
БЕТА-04	Пользователи в целом положительно оценили понятность основных сценариев и полезность функции обратной связи.	Общая оценка	Результат учтен при итоговой оценке удобства решения.	Подтверждено

Итоги пилотного пользовательского тестирования показывают, что обратная связь позволяет выявлять не только интерфейсные затруднения, но и реальные функциональные дефекты. По результатам испытаний была реализована кнопка «Пропустить кадр», исправлена некорректная отправка сообщений об ошибке, а предложение о рейтинге разметчиков и достижениях принято в план дальнейшего развития.

А.2.7 Сводные результаты тестирования

Сводная таблица объединяет шесть видов испытаний: функциональные, модульные, интеграционные контрактные, нагрузочные и производительные, UI/usability-проверки, а также пилотное пользовательское тестирование с анализом обратной связи. Такая структура позволяет оценить полноту покрытия требований и одновременно зафиксировать результаты реального взаимодействия пользователей с приложением.

Таблица А.10 – Сводка результатов тестирования

Направление	Объем	Итог
T1. Функциональное системное тестирование	18 сценариев TC-F01...TC-F18	Реестр охватывает пользовательские, административные и негативные сценарии; для ключевых кейсов предусмотрены экранные подтверждения на рисунках 4.1-4.5.
T2. Модульное тестирование	6 unit-тестов training_service	6/6 ОК; вспомогательная логика подготовки датасета и метрик работает корректно.
T3. Интеграционное и контрактное тестирование	2 HTTP contract-теста и сценарий СТ-03	СТ-01...СТ-02 – ОК; СТ-03 позволил оформить дефект D-01 по ролевому ограничению административных маршрутов.
T4. Нагрузочное и производительное тестирование	3 нагрузочных профиля датасета и PERF-01	LOAD-01...LOAD-03 – ОК; среднее время мобильной обработки кадра по PERF-01 составило 466 мс.
T5. Тестирование удобства использования и UI	5 пользовательских сценариев, 5 оценочных анкет	Средняя итоговая оценка интерфейса составила 8,6 из 10.
T6. Пилотное пользовательское тестирование	4 зафиксированных наблюдения БЕТА-01...БЕТА-04	Две доработки выполнены, одно предложение принято в план развития, общая пользовательская оценка признана положительной.

Общий вывод по тестированию: сформирован план, включающий шесть взаимодополняющих видов испытаний, и подготовлен протокол, сопоставленный со всеми заявленными функциональными и нефункциональными требованиями. Проведенные проверки подтвердили работоспособность основных сценариев мобильного приложения, обучающего контура и пользовательского взаимодействия. Отдельно зафиксированы выявленный дефект разграничения административного доступа D-01, выполненное исправление отправки сообщений

об ошибке и реализованное улучшение потока разметки в виде кнопки «Пропустить кадр».

А.3 Выводы по протоколу тестирования

В ходе разработки были систематизированы постановка задачи, проектное решение и текущее состояние программного комплекса автоматического поиска даты изготовления и срока годности на упаковке товара. В отчете отражены функциональные требования F1–F15, нефункциональные требования качества и архитектура, а также подтверждены результаты тестирования основных пользовательских и административных сценариев.

Особое внимание уделено разделу тестирования. План испытаний включает шесть видов проверок: функциональное системное тестирование, автоматизированные unit-тесты, полуавтоматизированные интеграционные HTTP contract-тесты, нагрузочное и производительное тестирование, оценку удобства интерфейса, а также пилотное пользовательское тестирование с анализом обратной связи.

Результаты испытаний показали, что основные пользовательские и административные сценарии реализованы и поддаются формальному контролю. Модульные тесты `training_service` и HTTP contract-тесты завершились успешно, нагрузочный сценарий подтвердил корректность подготовки датасета при увеличении объема входных данных, а показатель мобильной обработки кадра соответствует заданному ориентиру. Средняя пользовательская оценка интерфейса составила 8,6 из 10.

Дополнительное пользовательское тестирование позволило получить практическую обратную связь. По его итогам в сценарий ручной разметки добавлена кнопка «Пропустить кадр», исправлена некорректная отправка сообщений об ошибке, а предложение о введении рейтинга разметчиков и достижений включено в направления дальнейшего развития. Также в протоколе

выделен дефект D-01, связанный с усилением ролевого ограничения административных маршрутов.

Таким образом, результаты тестирования подтверждают соответствие выполненной работы теме выпускной квалификационной работы и показывают готовность программного комплекса к дальнейшему развитию, уточнению качества модели и расширению пользовательских функций.

ПРИЛОЖЕНИЕ Б. ЛИСТИНГ UNIT-ТЕСТОВ

Листинг А.1 – Unit-тесты вспомогательных функций training_service

```
import sys import types import tempfile from pathlib import Path import unittest # training_service/app.py imports
ultralitics.YOLO, but helper tests do not need the library. fake_ultralitics = types.ModuleType("ultralitics")
fake_ultralitics.YOLO = object sys.modules.setdefault("ultralitics", fake_ultralitics) TRAINING_SERVICE_DIR =
Path('/mnt/data/_work/training_service').resolve() sys.path.insert(0, str(TRAINING_SERVICE_DIR)) import app # noqa:
E402 class TrainingServiceHelperTests(unittest.TestCase): def test_train_config_default_and_override(self): cfg =
app.TrainConfig.from_job({ 'baseModel': 'custom.pt', 'epochs': 12, 'imgSize': 512, 'batch': 4, 'device': 'cpu', 'exportInt8':
False, 'exportNms': False, 'mobileFormat': 'tflite', 'quantizationFraction': 0.2, }) self.assertEqual(cfg.base_model,
'custom.pt') self.assertEqual(cfg.epochs, 12) self.assertEqual(cfg.imgsz, 512) self.assertEqual(cfg.batch, 4)
self.assertEqual(cfg.device, 'cpu') self.assertFalse(cfg.export_int8) self.assertFalse(cfg.export_nms)
self.assertAlmostEqual(cfg.quantization_fraction, 0.2) def
test_normalize_dataset_layout_copies_nested_images_and_labels(self): with tempfile.TemporaryDirectory() as td: root =
Path(td) nested = root / 'nested' / 'dataset' (nested / 'images').mkdir(parents=True) (nested / 'labels').mkdir(parents=True)
(nested / 'images' / 'frame_001.jpg').write_bytes(b'jpg') (nested / 'labels' / 'frame_001.txt').write_text('0 0.5 0.5 0.1 0.1',
encoding='utf-8') app.normalize_dataset_layout(root) self.assertTrue((root / 'images' / 'frame_001.jpg').exists())
self.assertTrue((root / 'labels' / 'frame_001.txt').exists()) def test_normalize_dataset_yaml_creates_required_fields(self):
with tempfile.TemporaryDirectory() as td: root = Path(td) app.normalize_dataset_yaml(root) yaml_text = (root /
'dataset.yaml').read_text(encoding='utf-8') self.assertIn('path:', yaml_text) self.assertIn('train: images', yaml_text)
self.assertIn('val: images', yaml_text) self.assertIn('nc: 1', yaml_text) self.assertIn('expiry_date', yaml_text) def
test_count_images_ignores_non_images(self): with tempfile.TemporaryDirectory() as td: root = Path(td) for name in
['a.jpg', 'b.jpeg', 'c.png', 'd.txt', 'e.webp']: (root / name).write_bytes(b'x') self.assertEqual(app.count_images(root), 3) def
test_resolve_train_device_auto_and_cpu(self): self.assertIsNone(app.resolve_train_device('auto'))
self.assertEqual(app.resolve_train_device('cpu'), 'cpu') def test_extract_metrics_maps_detection_metrics(self): class Box:
map = 0.31 map50 = 0.55 map75 = 0.41 mp = 0.72 mr = 0.68 class Metrics: box = Box() fitness = 0.42 result =
app.extract_metrics(Metrics()) self.assertAlmostEqual(result['mAP50_95'], 0.31) self.assertAlmostEqual(result['mAP50'],
0.55) self.assertAlmostEqual(result['mAP75'], 0.41) self.assertAlmostEqual(result['precision'], 0.72)
self.assertAlmostEqual(result['recall'], 0.68) self.assertAlmostEqual(result['fitness'], 0.42) self.assertIn('syncedAtUtc',
result) if __name__ == '__main__': unittest.main(verbosity=2)
```

ПРИЛОЖЕНИЕ В. ЖУРНАЛЫ ВЫПОЛНЕННЫХ АВТОМАТИЗИРОВАННЫХ ПРОВЕРОК

Листинг Б.1 – Журнал unit-тестов

```
test_count_images_ignores_non_images
(__main__.TrainingServiceHelperTests.test_count_images_ignores_non_images) ... ok
test_extract_metrics_maps_detection_metrics
(__main__.TrainingServiceHelperTests.test_extract_metrics_maps_detection_metrics) ... ok
test_normalize_dataset_layout_copies_nested_images_and_labels
(__main__.TrainingServiceHelperTests.test_normalize_dataset_layout_copies_nested_images_and_labels) ... ok
test_normalize_dataset_yaml_creates_required_fields
(__main__.TrainingServiceHelperTests.test_normalize_dataset_yaml_creates_required_fields) ... ok
test_resolve_train_device_auto_and_cpu (__main__.TrainingServiceHelperTests.test_resolve_train_device_auto_and_cpu)
... ok test_train_config_default_and_override
(__main__.TrainingServiceHelperTests.test_train_config_default_and_override) ... ok
-----
----- Ran 6 tests in 0.038s OK
```

Листинг Б.2 – Журнал HTTP contract-тестов

```
test_ensure_not_canceled_reports_canceled_status
(__main__.TrainingClientHttpContractTests.test_ensure_not_canceled_reports_canceled_status) ... ok
test_process_next_job_handles_204_no_content
(__main__.TrainingClientHttpContractTests.test_process_next_job_handles_204_no_content) ... ok
-----
----- Ran 2 tests in 0.519s OK
```

Листинг Б.3 – Журнал нагрузочного сценария

```
pairs; size_kb; images_found; elapsed_ms; status 100; 31.56; 100; 376.74; OK 1000; 315.45; 1000; 3943.94; OK
5000; 1577.17; 5000; 15612.47; OK
```

ПРИЛОЖЕНИЕ Г. ЛИСТИНГ ПОЛУАВТОМАТИЗИРОВАННЫХ HTTP CONTRACT-ТЕСТОВ

Листинг В.1 – HTTP contract-тесты training_service

```
import sys
import types
import threading
import unittest
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import urlparse

# Stub heavy optional dependency before importing training_service/app.py.
fake_ultralitics = types.ModuleType('ultralitics')
fake_ultralitics.YOLO = object
sys.modules.setdefault('ultralitics', fake_ultralitics)

TRAINING_SERVICE_DIR = Path('/mnt/data/_work/training_service').resolve()
sys.path.insert(0, str(TRAINING_SERVICE_DIR))
import app # noqa: E402

class StubState:
    status_payloads: list[dict] = []

class Handler(BaseHTTPRequestHandler):
    def log_message(self, fmt, *args):
        return

    def _send_json(self, code: int, body: str):
        encoded = body.encode('utf-8')
        self.send_response(code)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(encoded)))
        self.end_headers()
        self.wfile.write(encoded)

    def do_GET(self): # noqa: N802
        parsed = urlparse(self.path)
        if parsed.path == '/api/training-client/jobs/next':
            self.send_response(204)
            self.end_headers()
            return
        if parsed.path == '/api/training-client/jobs/JOB-1':
            self._send_json(200, '{"jobId":"JOB-1","status":"canceled","cancellationRequested":true}')
            return
        self._send_json(404, '{"error":"not found"}')

    def do_POST(self): # noqa: N802
        parsed = urlparse(self.path)
        if parsed.path == '/api/training-client/jobs/JOB-1/status':
```

```

length = int(self.headers.get('Content-Length', '0'))
payload = self.rfile.read(length).decode('utf-8')
StubState.status_payloads.append({'path': parsed.path, 'body': payload})
self._send_json(200, '{"ok":true}')
return
self._send_json(404, '{"error":"not found"}')

class TrainingClientHttpContractTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        StubState.status_payloads = []
        cls.server = ThreadingHTTPServer(('127.0.0.1', 0), Handler)
        cls.thread = threading.Thread(target=cls.server.serve_forever, daemon=True)
        cls.thread.start()
        cls.old_url = app.BACKEND_URL
        app.BACKEND_URL = f'http://127.0.0.1:{cls.server.server_address[1]}'

    @classmethod
    def tearDownClass(cls):
        app.BACKEND_URL = cls.old_url
        cls.server.shutdown()
        cls.server.server_close()
        cls.thread.join(timeout=5)

    def test_process_next_job_handles_204_no_content(self):
        processed = app.process_next_job()
        self.assertFalse(processed)

    def test_ensure_not_canceled_reports_canceled_status(self):
        session = app.make_session()
        with self.assertRaises(RuntimeError):
            app.ensure_not_canceled(session, 'JOB-1')
        self.assertTrue(StubState.status_payloads)
        status_call = StubState.status_payloads[-1]
        self.assertEqual(status_call['path'], '/api/training-client/jobs/JOB-1/status')
        self.assertIn("status": "canceled", status_call['body'])

if __name__ == '__main__':
    unittest.main(verbosity=2)

```


ПРИЛОЖЕНИЕ Д. ЛИСТИНГ НАГРУЗОЧНОГО СЦЕНАРИЯ

Листинг Г.1 – Сценарий нагрузочного тестирования подготовки датасета

```
from __future__ import annotations import io import shutil import sys import tempfile import time import types
from pathlib import Path from zipfile import ZIP_DEFLATED, ZipFile # Stub heavy optional dependency before importing
training_service/app.py. fake_ultralytics = types.ModuleType('ultralytics') fake_ultralytics.YOLO = object
sys.modules.setdefault('ultralytics', fake_ultralytics) TRAINING_SERVICE_DIR =
Path('/mnt/data/_work/training_service').resolve() sys.path.insert(0, str(TRAINING_SERVICE_DIR)) import app # noqa:
E402 def make_zip(path: Path, pairs: int) -> float: with ZipFile(path, 'w', compression=ZIP_DEFLATED) as zf: for i in
range(pairs): zf.writestr(f'nested/dataset/images/frame_{i:05d}.jpg', b'jpeg-bytes')
zf.writestr(f'nested/dataset/labels/frame_{i:05d}.txt', '0 0.5 0.5 0.1 0.1') return path.stat().st_size / 1024 def run_case(pairs:
int) -> tuple[int, float, int, float, str]: with tempfile.TemporaryDirectory() as td: root = Path(td) archive = root / 'dataset.zip'
size_kb = make_zip(archive, pairs) target = root / 'dataset' target.mkdir() started = time.perf_counter() with ZipFile(archive,
'r') as zf: zf.extractall(target) app.normalize_dataset_layout(target) app.normalize_dataset_yaml(target) images =
app.count_images(target / 'images') elapsed_ms = (time.perf_counter() - started) * 1000 status = 'OK' if images == pairs
else 'FAIL' return pairs, size_kb, images, elapsed_ms, status if __name__ == '__main__': print('pairs; size_kb;
images_found; elapsed_ms; status') for pairs in (100, 1000, 5000): result = run_case(pairs) print(f'{result[0]};
{result[1]:.2f}; {result[2]}; {result[3]:.2f}; {result[4]}')
```

ПРИЛОЖЕНИЕ Е Листинг кода информационной системы

Листинг Д.1 – Листинг кода мобильного приложения

```
# Сборка кода мобильного приложения
Папка: D:\diplom\mobile_app
Дата сборки: 2026-06-05
18:17:19
Количество файлов: 36

## Включённые файлы

```text
analysis_options.yaml
android/app/build.gradle.kts
android/app/src/main/AndroidManifest.xml
android/app/src/main/kotlin/com/example/mobile_app/MainActivity.kt
android/app/src/main/res/values/styles.xml
android/build.gradle.kts
android/gradle.properties
android/settings.gradle.kts
ios/Runner.xcodeproj/project.pbxproj
ios/Runner/AppDelegate.swift
ios/Runner/Assets.xcassets/LaunchImage.imageset/README.md
ios/Runner/Info.plist
ios/Runner/Runner-Bridging-Header.h
ios/Runner/SceneDelegate.swift
lib/ai/date_ocr_service.dart
lib/ai/date_reader_tflite.dart
lib/ai/detection_result.dart
lib/ai/tflite_date_detector.dart
lib/api/api_client.dart
lib/api/api_config.dart
lib/api/model_sync_service.dart
lib/auth/auth_state.dart
lib/main.dart
lib/models/product.dart
lib/models/stored_product.dart
lib/screens/cabinet_screen.dart

lib/screens/error_dataset_flow_screen.dart
lib/screens/login_screen.dart
lib/screens/register_screen.dart
lib/screens/scan_screen.dart
lib/screens/settings_screen.dart
lib/storage/file_key_value_store.dart

lib/widgets/manual_add_sheet.dart

pubspec.yaml
README.md
test/widget_test.dart
```

## Содержимое файлов

=====
Файл: analysis_options.yaml
=====

# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in the UI of Dart-enabled
# IDEs
(https://dart.dev/tools#ides-and-editors).
The analyzer can also be
# invoked from the command
line by running `flutter analyze`.

# The following line activates a set of recommended lints for Flutter apps,
# packages, and plugins
designed to encourage good coding
practices.
include:
package:flutter_lints/flutter.yaml

linter:
# The lint rules applied to this
project can be customized in the
# section below to disable rules
from the `package:flutter_lints/flutter.yaml`
# included above or to enable
additional rules. A list of all available lints
# and their documentation is
published at https://dart.dev/lints.
#
# Instead of disabling a lint rule
for the entire project in the
# section below, it can also be
suppressed for a single line of code
# or a specific dart file by using
the `// ignore: name_of_lint` and
# `// ignore_for_file:
name_of_lint` syntax on the line or in the
file
# producing the lint.
rules:
# avoid_print: false #
Uncomment to disable the `avoid_print`
rule
# prefer_single_quotes: true #
Uncomment to enable the
`prefer_single_quotes` rule
# Additional information about
this file can be found at
#
https://dart.dev/guides/language/analysis-
options
=====
```

```

=====
=====
Файл:
android/app/build.gradle.kts
=====
=====
=====
=====

```

```

plugins {
    id("com.android.application")
    id("kotlin-android")

    // The Flutter Gradle Plugin
    must be applied after the Android and
    Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-
plugin")
}

android {
    namespace =
"com.example.mobile_app"
    compileSdk =
flutter.compileSdkVersion
    ndkVersion =
flutter.ndkVersion

    compileOptions {
        sourceCompatibility =
JavaVersion.VERSION_17
        targetCompatibility =
JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget =
JavaVersion.VERSION_17.toString()
    }

    defaultConfig {
        // TODO: Specify your own
        unique Application ID
        (https://developer.android.com/studio/build/
application-id.html).
        applicationId =
"com.example.mobile_app"

```

```

// You can update the
following values to match your application
needs.

// For more information,
see: https://flutter.dev/to/review-gradle-
config.

minSdk = 26
targetSdk =
flutter.targetSdkVersion
versionCode =
flutter.versionCode
versionName =
flutter.versionName
    }

buildTypes {
    release {
        // TODO: Add your own
        signing config for the release build.
        // Signing with the debug
        keys for now, so `flutter run --release`
        works.
        signingConfig =
signingConfigs.getByName("debug")
    }
}

flutter {
    source = "../.."
}

=====
=====
=====
Файл:
android/app/src/main/AndroidManifest.xml
=====
=====
=====
=====
<manifest
xmlns:android="http://schemas.android.co
m/apk/res/android">

```

```

<uses-permission
android:name="android.permission.INTER
NET" />

<uses-permission
android:name="android.permission.CAME
RA" />

<application

android:label="mobile_app"

android:name="${applicationName}"

android:icon="@mipmap/ic_launcher"

android:usesCleartextTraffic="true">
    <activity

        android:name=".MainActivity"
        android:exported="true"

        android:launchMode="singleTop"
        android:taskAffinity=""

        android:theme="@style/LaunchTheme"

        android:configChanges="orientation|keyboa
rdHidden|keyboard|screenSize|smallestScre
enSize|locale|layoutDirection|fontScale|scre
enLayout|density|uiMode"

        android:hardwareAccelerated="true"

        android:windowSoftInputMode="adjustRes
ize">

        <!-- Specifies an Android
        theme to apply to this Activity as soon as
        the Android process
        has started. This theme is visible to the user
        while the Flutter UI
        initializes. After that, this theme continues
        to determine the
        Window background behind the Flutter UI.
        -->

        <meta-data

        android:name="io.flutter.embedding.androi
d.NormalTheme"

```

```

android:resource="@style/NormalTheme"
    />
    <intent-filter>
        <action
android:name="android.intent.action.MAIN
"/>

        <category
android:name="android.intent.category.LA
UNCHER"/>

```

```

        </intent-filter>
    </activity>
    <!-- Don't delete the meta-
data below.

```

```

        This is used by the
Flutter tool to generate
GeneratedPluginRegistrant.java -->
    <meta-data

```

```

android:name="flutterEmbedding"
    android:value="2" />
</application>
<!-- Required to query
activities that can process text, see:

```

<https://developer.android.com/training/package-visibility> and

https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT.

```

        In particular, this is used
by the Flutter engine in
io.flutter.plugin.text.ProcessTextPlugin. -->

```

```

    <queries>
    <intent>
        <action
android:name="android.intent.action.PROC
ESS_TEXT"/>
        <data
android:mimeType="text/plain"/>
    </intent>
    </queries>
</manifest>

```

```

=====
Файл:
android/app/src/main/kotlin/com/example/
mobile_app/MainActivity.kt
=====
=====
=====
=====

```

```

package
com.example.mobile_app

import
io.flutter.embedding.android.FlutterActivity

class MainActivity :
FlutterActivity()

```

```

=====
Файл:
android/app/src/main/res/values/styles.xml
=====
=====
=====
=====

```

```

<?xml version="1.0"
encoding="utf-8"?>
<resources>
    <!-- Theme applied to the
Android Window while the process is
starting when the OS's Dark Mode setting is
off -->

    <style name="LaunchTheme"
parent="@android:style/Theme.Light.NoTit
leBar">

        <!-- Show a splash screen
on the activity. Automatically removed
when

        the Flutter engine draws
its first frame -->

    <item
name="android:windowBackground">@dra
wable/launch_background</item>

```

```

</style>
<!-- Theme applied to the
Android Window as soon as the process has
started.

```

This theme determines the color of the Android Window while your Flutter UI initializes, as well as behind your Flutter UI while its running.

This Theme is only used starting with V2 of Flutter's Android embedding. -->

```

<style name="NormalTheme"
parent="@android:style/Theme.Light.NoTit
leBar">

```

```

    <item
name="android:windowBackground">?andr
oid:colorBackground</item>
</style>
</resources>

```

```

=====
=====
=====
=====
Файл: android/build.gradle.kts
=====
=====
=====
=====

```

```

allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

val newBuildDir: Directory =
rootProject.layout.buildDirectory
    .dir("../build")
    .get()
rootProject.layout.buildDirectory
    .value(newBuildDir)

subprojects {

```

```

        val newSubprojectBuildDir:
Directory = newBuildDir.dir(project.name)

project.layout.buildDirectory.value(newSub
projectBuildDir)
    }
    subprojects {

project.evaluationDependsOn(":app")
    }

tasks.register<Delete>("clean") {

delete(rootProject.layout.buildDirectory)
    }

=====
=====
=====
=====

Файл: android/gradle.properties
=====
=====
=====

org.gradle.jvmargs=-Xmx8G -
XX:MaxMetaspaceSize=4G -
XX:ReservedCodeCacheSize=512m -
XX:+HeapDumpOnOutOfMemoryError
    android.useAndroidX=true
    # Отключаем incremental
Kotlin compilation: на Windows часто
падает с

    # "this and base files have
different roots" и может собрать битый
release APK.

    kotlin.incremental=false
    kotlin.incremental.useClasspathS
napshot=false

=====
=====
=====

Файл:
android/settings.gradle.kts

```

```

=====
=====
=====

pluginManagement {
    val flutterSdkPath =
        run {
            val properties =
java.util.Properties()

file("local.properties").inputStream().use {
    properties.load(it) }

    val flutterSdkPath =
properties.getProperty("flutter.sdk")
        require(flutterSdkPath !=
null) { "flutter.sdk not set in
local.properties" }

        flutterSdkPath
    }

includeBuild("$flutterSdkPath/packages/flu
tter_tools/gradle")

repositories {
    google()
    mavenCentral()
    gradlePluginPortal()
    }
    dependencyResolutionManagem
ent {

repositoriesMode.set(RepositoriesMode.PR
EFER_SETTINGS)

repositories {
    google()
    mavenCentral()
    maven(url =
"https://storage.googleapis.com/download.fl
utter.io")

    maven(url =
"https://jitpack.io")
        // НЕ добавляй
maven.arthenica.com
    }
}

```

```

plugins {
    id("dev.flutter.flutter-plugin-
loader") version "1.0.0"

    id("com.android.application")
version "8.11.1" apply false

id("org.jetbrains.kotlin.android") version
"2.2.20" apply false
    }

include(":app")

=====
=====
=====

Файл:
ios/Runner.xcodeproj/project.pbxproj
=====
=====
=====

// !${UTF8*}$!
{
    archiveVersion = 1;
    classes = {
    };
    objectVersion = 54;
    objects = {

/* Begin PBXBuildFile section
*/

1498D2341E8E89220040F4C2
/* GeneratedPluginRegistrant.m in Sources
*/ = {isa = PBXBuildFile; fileRef =
1498D2331E8E89220040F4C2 /*
GeneratedPluginRegistrant.m */; };

331C808B294A63AB00263BE5
/* RunnerTests.swift in Sources */ = {isa =
PBXBuildFile; fileRef =
331C807B294A618700263BE5 /*
RunnerTests.swift */; };

3B3967161E833CAA004F5970
/* AppFrameworkInfo.plist in Resources */

```

```
= {isa = PBXBuildFile; fileRef =
3B3967151E833CAA004F5970 /*
AppFrameworkInfo.plist */; };
```

```
74858FAF1ED2DC5600515810
/* AppDelegate.swift in Sources */ = {isa =
PBXBuildFile; fileRef =
74858FAE1ED2DC5600515810 /*
AppDelegate.swift */; };
```

```
7884E8682EC3CC0700C636F2
/* SceneDelegate.swift in Sources */ = {isa
= PBXBuildFile; fileRef =
7884E8672EC3CC0400C636F2 /*
SceneDelegate.swift */; };
```

```
97C146FC1CF9000F007C117D
/* Main.storyboard in Resources */ = {isa =
PBXBuildFile; fileRef =
97C146FA1CF9000F007C117D /*
Main.storyboard */; };
```

```
97C146FE1CF9000F007C117D
/* Assets.xcassets in Resources */ = {isa =
PBXBuildFile; fileRef =
97C146FD1CF9000F007C117D /*
Assets.xcassets */; };
```

```
97C147011CF9000F007C117D
/* LaunchScreen.storyboard in Resources */
= {isa = PBXBuildFile; fileRef =
97C146FF1CF9000F007C117D /*
LaunchScreen.storyboard */; };
```

```
/* End PBXBuildFile section */

/* Begin
PBXContainerItemProxy section */
```

```
331C8085294A63A400263BE5
/* PBXContainerItemProxy */ = {
isa =
PBXContainerItemProxy;
```

```
containerPortal =
97C146E61CF9000F007C117D /* Project
object */;
```

```
proxyType = 1;
```

```
remoteGlobalIDString =
97C146ED1CF9000F007C117D;
```

```
remoteInfo = Runner;
};
/* End PBXContainerItemProxy
section */
```

```
/* Begin
PBXCopyFilesBuildPhase section */
```

```
9705A1C41CF9048500538489
/* Embed Frameworks */ = {
isa =
PBXCopyFilesBuildPhase;
```

```
buildActionMask =
2147483647;
dstPath =
"";
```

```
dstSubfolderSpec = 10;
files = (
);
name =
"Embed Frameworks";
```

```
runOnlyForDeploymentPostproc
essing = 0;
};
/* End
PBXCopyFilesBuildPhase section */
```

```
/* Begin PBXFileReference
section */
```

```
1498D2321E8E86230040F4C2
/* GeneratedPluginRegistrant.h */ = {isa =
PBXFileReference; lastKnownFileType =
sourcecode.c.h; path =
GeneratedPluginRegistrant.h; sourceTree =
"<group>"; };
```

```
1498D2331E8E89220040F4C2
/* GeneratedPluginRegistrant.m */ = {isa =
PBXFileReference; fileEncoding = 4;
lastKnownFileType = sourcecode.c.objc;
```

```
path = GeneratedPluginRegistrant.m;
sourceTree = "<group>"; };
```

```
331C807B294A618700263BE5
/* RunnerTests.swift */ = {isa =
PBXFileReference; lastKnownFileType =
sourcecode.swift; path = RunnerTests.swift;
sourceTree = "<group>"; };
```

```
331C8081294A63A400263BE5
/* RunnerTests.xctest */ = {isa =
PBXFileReference; explicitFileType =
wrapper.cfbundle; includeInIndex = 0; path
= RunnerTests.xctest; sourceTree =
BUILT_PRODUCTS_DIR; };
```

```
3B3967151E833CAA004F5970
/* AppFrameworkInfo.plist */ = {isa =
PBXFileReference; fileEncoding = 4;
lastKnownFileType = text.plist.xml; name
= AppFrameworkInfo.plist; path =
Flutter/AppFrameworkInfo.plist;
sourceTree = "<group>"; };
```

```
74858FAD1ED2DC5600515810
/* Runner-Bridging-Header.h */ = {isa =
PBXFileReference; lastKnownFileType =
sourcecode.c.h; path = "Runner-Bridging-
Header.h"; sourceTree = "<group>"; };
```

```
74858FAE1ED2DC5600515810
/* AppDelegate.swift */ = {isa =
PBXFileReference; fileEncoding = 4;
lastKnownFileType = sourcecode.swift;
path = AppDelegate.swift; sourceTree =
"<group>"; };
```

```
7884E8672EC3CC0400C636F2
/* SceneDelegate.swift */ = {isa =
PBXFileReference; lastKnownFileType =
sourcecode.swift; path =
SceneDelegate.swift; sourceTree =
"<group>"; };
```

```
7AFA3C8E1D35360C0083082
E /* Release.xcconfig */ = {isa =
PBXFileReference; lastKnownFileType =
text.xcconfig; name = Release.xcconfig;
```

```
path = Flutter/Release.xcconfig; sourceTree = "<group>"; };
```

```
9740EEB21CF90195004384FC
/* Debug.xcconfig */ = {isa = PBXFileReference; fileEncoding = 4; lastKnownFileType = text.xcconfig; name = Debug.xcconfig; path = Flutter/Debug.xcconfig; sourceTree = "<group>"; };
```

```
9740EEB31CF90195004384FC
/* Generated.xcconfig */ = {isa = PBXFileReference; fileEncoding = 4; lastKnownFileType = text.xcconfig; name = Generated.xcconfig; path = Flutter/Generated.xcconfig; sourceTree = "<group>"; };
```

```
97C146EE1CF9000F007C117D
/* Runner.app */ = {isa = PBXFileReference; explicitFileType = wrapper.application; includeInIndex = 0; path = Runner.app; sourceTree = BUILT_PRODUCTS_DIR; };
```

```
97C146FB1CF9000F007C117D
/* Base */ = {isa = PBXFileReference; lastKnownFileType = file.storyboard; name = Base; path = Base.lproj/Main.storyboard; sourceTree = "<group>"; };
```

```
97C146FD1CF9000F007C117D
/* Assets.xcassets */ = {isa = PBXFileReference; lastKnownFileType = folder.assetcatalog; path = Assets.xcassets; sourceTree = "<group>"; };
```

```
97C147001CF9000F007C117D
/* Base */ = {isa = PBXFileReference; lastKnownFileType = file.storyboard; name = Base; path = Base.lproj/LaunchScreen.storyboard; sourceTree = "<group>"; };
```

```
97C147021CF9000F007C117D
/* Info.plist */ = {isa = PBXFileReference;
```

```
lastKnownFileType = text.plist.xml; path = Info.plist; sourceTree = "<group>"; };
```

```
/* End PBXFileReference
section */
/* Begin
PBXFrameworksBuildPhase section */
97C146EB1CF9000F007C117D
/* Frameworks */ = {
```

```
isa = PBXFrameworksBuildPhase; buildActionMask = 2147483647; files = (
```

```
runOnlyForDeploymentPostprocessing = 0;
*/ End
PBXFrameworksBuildPhase section */
```

```
/* Begin PBXGroup section */
```

```
331C8082294A63A400263BE5
/* RunnerTests */ = {
isa = PBXGroup;
children = (
```

```
331C807B294A618700263BE5
/* RunnerTests.swift */ = {
path = RunnerTests;
```

```
sourceTree = "<group>";
};
```

```
9740EEB11CF90186004384FC
/* Flutter */ = {
isa = PBXGroup;
children = (
```

```
3B3967151E833CAA004F5970
/* AppFrameworkInfo.plist */ = {
```

```
9740EEB21CF90195004384FC
/* Debug.xcconfig */ = {
```

```
7AFA3C8E1D35360C0083082
E /* Release.xcconfig */ = {
```

```
9740EEB31CF90195004384FC
/* Generated.xcconfig */ = {
name = Flutter;
sourceTree = "<group>";
};
```

```
97C146E51CF9000F007C117D
= {
isa = PBXGroup;
children = (
```

```
9740EEB11CF90186004384FC
/* Flutter */ = {
97C146F01CF9000F007C117D
/* Runner */ = {
```

```
97C146EF1CF9000F007C117D
/* Products */ = {
```

```
331C8082294A63A400263BE5
/* RunnerTests */ = {
```

```
sourceTree = "<group>";
};
```

```
97C146EF1CF9000F007C117D
/* Products */ = {
isa = PBXGroup;
children = (
```

```

97C146EE1CF9000F007C117D
/* Runner.app */,

331C8081294A63A400263BE5
/* RunnerTests.xctest */,

);
name =

Products;

sourceTree = "<group>";
};

97C146F01CF9000F007C117D
/* Runner */ = {
isa =
PBXGroup;
children =
(
97C146FA1CF9000F007C117D
/* Main.storyboard */,
97C146FD1CF9000F007C117D
/* Assets.xcassets */,
97C146FF1CF9000F007C117D
/* LaunchScreen.storyboard */,
97C147021CF9000F007C117D
/* Info.plist */,
1498D2321E8E86230040F4C2
/* GeneratedPluginRegistrant.h */,
1498D2331E8E89220040F4C2
/* GeneratedPluginRegistrant.m */,
74858FAE1ED2DC5600515810
/* AppDelegate.swift */,
7884E8672EC3CC0400C636F2
/* SceneDelegate.swift */,
74858FAD1ED2DC5600515810
/* Runner-Bridging-Header.h */,
);

path =
};

97C146ED1CF9000F007C117D
/* Runner */ = {
isa =
PBXNativeTarget;
buildConfigurationList =
97C147051CF9000F007C117D /* Build
configuration list for PBXNativeTarget
"Runner" */;
buildPhases = (
9740EEB61CF901F6004384FC
/* Run Script */,
97C146EA1CF9000F007C117D
/* Sources */,
97C146EB1CF9000F007C117D
/* Frameworks */,
97C146EC1CF9000F007C117D
/* Resources */,
9705A1C41CF9048500538489
/* Embed Frameworks */,
3B06AD1E1E4923F5004D2608
/* Thin Binary */,
);
buildRules = (
);
dependencies = (
331C8086294A63A400263BE5
/* PBXTargetDependency */,
);
name =
RunnerTests;
productName = RunnerTests;
productReference =
331C8081294A63A400263BE5 /*
RunnerTests.xctest */;
productType =
"com.apple.product-type.bundle.unit-test";
};

97C146ED1CF9000F007C117D
/* Runner */ = {
isa =
PBXNativeTarget;
buildConfigurationList =
97C147051CF9000F007C117D /* Build
configuration list for PBXNativeTarget
"Runner" */;
buildPhases = (
9740EEB61CF901F6004384FC
/* Run Script */,
97C146EA1CF9000F007C117D
/* Sources */,
97C146EB1CF9000F007C117D
/* Frameworks */,
97C146EC1CF9000F007C117D
/* Resources */,
9705A1C41CF9048500538489
/* Embed Frameworks */,
3B06AD1E1E4923F5004D2608
/* Thin Binary */,
);
buildRules = (
);
dependencies = (
);
name =
Runner;
productName = Runner;
productReference =
97C146EE1CF9000F007C117D /*
Runner.app */;

```



```

        productType =
";com.apple.product-type.application";
    };
    /* End PBXNativeTarget section
*/

    /* Begin PBXProject section */

    97C146E61CF9000F007C117D
/* Project object */ = {
    isa =
PBXProject;
    attributes
= {
        BuildIndependentTargetsInParallel = YES;
        LastUpgradeCheck = 1510;
        ORGANIZATIONNAME = "";
        TargetAttributes = {
            331C8080294A63A400263BE5
= {
            productRefGroup =
97C146E51CF9000F007C117D;
            CreatedOnToolsVersion = 14.0;
            TestTargetID =
97C146ED1CF9000F007C117D;
        };
        97C146ED1CF9000F007C117D
= {
        CreatedOnToolsVersion = 7.3.1;
        LastSwiftMigration = 1100;
    };
    };
    buildConfigurationList =
97C146E91CF9000F007C117D /* Build
configuration list for PBXProject "Runner"
*/;
    compatibilityVersion = "Xcode
9.3";
    developmentRegion = en;
    hasScannedForEncodings = 0;
    knownRegions = (
        en,
        Base,
    );
    mainGroup =
97C146E51CF9000F007C117D;
    productRefGroup =
97C146EF1CF9000F007C117D /*
Products */;
    projectDirPath = "";
    projectRoot = "";
    targets = (
        97C146ED1CF9000F007C117D
/* Runner */;
        331C8080294A63A400263BE5
/* RunnerTests */;
    );
    /* End PBXProject section */

    /* Begin
PBXResourcesBuildPhase section */

        331C807F294A63A400263BE5
/* Resources */ = {
    isa =
PBXResourcesBuildPhase;
    buildActionMask =
2147483647;
    files = (
        97C146EC1CF9000F007C117D
/* Resources */ = {
    isa =
PBXResourcesBuildPhase;
    buildActionMask =
2147483647;
    files = (
        97C147011CF9000F007C117D
/* LaunchScreen.storyboard in Resources
*/;
        3B3967161E833CAA004F5970
/* AppFrameworkInfo.plist in Resources */;
        97C146FE1CF9000F007C117D
/* Assets.xcassets in Resources */;
        97C146FC1CF9000F007C117D
/* Main.storyboard in Resources */;
    );
    runOnlyForDeploymentPostproc
essing = 0;
    };
    /* End
PBXResourcesBuildPhase section */

    /* Begin
PBXShellScriptBuildPhase section */

```

```

    3B06AD1E1E4923F5004D2608
/* Thin Binary */ = {
    isa =
PBXShellScriptBuildPhase;
    outputPaths = (
    );
    alwaysOutOfDate = 1;
    buildActionMask =
2147483647;
    files = (
    );
    inputPaths
= (
    "${TARGET_BUILD_DIR}/${I
NFOPLIST_PATH}",
    );
    name =
"Thin Binary";
    outputPaths = (
    );
    runOnlyForDeploymentPostproc
essing = 0;
    shellPath
= /bin/sh;
    shellScript = "/bin/sh
\"$FLUTTER_ROOT/packages/flutter_tool
s/bin/xcode_backend.sh\" build";
    );
    9740EEB61CF901F6004384FC
/* Run Script */ = {
    isa =
PBXShellScriptBuildPhase;
    alwaysOutOfDate = 1;
    buildActionMask =
2147483647;
    files = (
    );
    inputPaths
= (
    );
    name =
74858FAF1ED2DC5600515810
/* AppDelegate.swift in Sources */,
    1498D2341E8E89220040F4C2
/* GeneratedPluginRegistrant.m in Sources
*/,
    7884E8682EC3CC0700C636F2
/* SceneDelegate.swift in Sources */,
    );
    runOnlyForDeploymentPostproc
essing = 0;
    );
    /* End PBXSourcesBuildPhase
section */
    /* Begin PBXTargetDependency
section */
    331C8086294A63A400263BE5
/* PBXTargetDependency */ = {
    isa =
PBXTargetDependency;
    target =
97C146ED1CF9000F007C117D /* Runner
*/;
    targetProxy =
331C8085294A63A400263BE5 /*
PBXContainerItemProxy */;
    );
    /* End PBXTargetDependency
section */
    /* Begin PBXVariantGroup
section */
    97C146FA1CF9000F007C117D
/* Main.storyboard */ = {
    isa =
PBXVariantGroup;
    children =
(
    97C146FB1CF9000F007C117D
/* Base */,

```

| | | |
|---|---|---|
|); | | |
| name = | CLANG_CXX_LIBRARY = | CLANG_WARN_OBJC_ROOT |
| Main.storyboard; | "libc++"; | _CLASS = YES_ERROR; |
| | | |
| sourceTree = "<group>"; | CLANG_ENABLE_MODULES | CLANG_WARN_RANGE_LO |
| }; | = YES; | OP_ANALYSIS = YES; |
| | | |
| 97C146FF1CF9000F007C117D | CLANG_ENABLE_OBJC_AR | CLANG_WARN_STRICT_PR |
| /* LaunchScreen.storyboard */ = { | C = YES; | OTOTYPES = YES; |
| isa = | | |
| PBXVariantGroup; | CLANG_WARN_BLOCK_CA | CLANG_WARN_SUSPICIOUS |
| children = | PTURE_AUTORELEASING = YES; | _MOVE = YES; |
| (| | |
| | CLANG_WARN_BOOL_CON | CLANG_WARN_UNREACHA |
| 97C147001CF9000F007C117D | VERSION = YES; | BLE_CODE = YES; |
| /* Base */, | | |
| | | |
|); | CLANG_WARN_COMMA = | CLANG_WARN__DUPLICAT |
| name = | YES; | E_METHOD_MATCH = YES; |
| LaunchScreen.storyboard; | | |
| | | |
| sourceTree = "<group>"; | CLANG_WARN_CONSTANT | "CODE_SIGN_IDENTITY[sdk |
| }; | _CONVERSION = YES; | =iphoneos*]" = "iPhone Developer"; |
| /* End PBXVariantGroup | | |
| section */ | CLANG_WARN_DEPRECAT | COPY_PHASE_STRIP = NO; |
| | ED_OBJC_IMPLEMENTATIONS = YES; | |
| | | |
| /* Begin XCBuildConfiguration | CLANG_WARN_DIRECT_OB | DEBUG_INFORMATION_FO |
| section */ | JC_ISA_USAGE = YES_ERROR; | RMAT = "dwarf-with-dsym"; |
| | | |
| | | |
| 249021D3217E4FDB00AE95B | CLANG_WARN_EMPTY_BO | ENABLE_NS_ASSERTIONS = |
| 9 /* Profile */ = { | DY = YES; | NO; |
| | | |
| isa = | | |
| XCBuildConfiguration; | CLANG_WARN_ENUM_CON | ENABLE_STRICT_OBJC_MS |
| | VERSION = YES; | GSEND = YES; |
| | | |
| buildSettings = { | | |
| | CLANG_WARN_INFINITE_R | ENABLE_USER_SCRIPT_SA |
| ALWAYS_SEARCH_USER_P | ECURSION = YES; | NDBOXING = NO; |
| ATHS = NO; | | |
| | | |
| ASSETCATALOG_COMPILE | CLANG_WARN_INT_CONVE | GCC_C_LANGUAGE_STAND |
| R_GENERATE_SWIFT_ASSET_SYMBOL_EXTENSIONS = YES; | RSION = YES; | ARD = gnu99; |
| | | |
| CLANG_ANALYZER_NONNULL = YES; | CLANG_WARN_NON_LITERAL_NULL_CONVERSION = YES; | GCC_NO_COMMON_BLOCKS = YES; |
| | | |
| | CLANG_WARN_OBJC_IMPLICIT_RETAIN_SELF = YES; | GCC_WARN_64_TO_32_BIT_CONVERSION = YES; |
| | | |
| CLANG_CXX_LANGUAGE_STANDARD = "gnu++0x"; | CLANG_WARN_OBJC_LITERAL_CONVERSION = YES; | GCC_WARN_ABOUT_RETURN_TYPE = YES_ERROR; |

```

GCC_WARN_UNDECLARED_SELECTOR = YES;

```

```

GCC_WARN_UNINITIALIZE_D_AUTOS = YES_AGGRESSIVE;

```

```

GCC_WARN_UNUSED_FUNCTION = YES;

```

```

GCC_WARN_UNUSED_VARIABLE = YES;

```

```

IPHONEOS_DEPLOYMENT_TARGET = 13.0;

```

```

MTL_ENABLE_DEBUG_INFRO = NO;

```

```

SDKROOT = iphoneos;

```

```

SUPPORTED_PLATFORMS = iphoneos;

```

```

TARGETED_DEVICE_FAMILY = "1,2";

```

```

VALIDATE_PRODUCT = YES;

```

```

    };
    name = Profile;
};

```

```

249021D4217E4FDB00AE95B9 /* Profile */ = {

```

```

    isa = XCBuildConfiguration;

```

```

    baseConfigurationReference = 7AFA3C8E1D35360C0083082E /* Release.xcconfig */;

```

```

    buildSettings = {

```

```

    ASSETCATALOG_COMPILE_R_APPICON_NAME = AppIcon;

```

```

CLANG_ENABLE_MODULES = YES;

```

```

CURRENT_PROJECT_VERSION = "$(FLUTTER_BUILD_NUMBER)";

```

```

ENABLE_BITCODE = NO;

```

```

INFOPLIST_FILE = Runner/Info.plist;

```

```

LD_RUNPATH_SEARCH_PATHS = (

```

```

    "$(inherited)",

```

```

    "@executable_path/Frameworks",

```

```

);

```

```

PRODUCT_BUNDLE_IDENTIFIER = com.example.mobileApp;

```

```

PRODUCT_NAME = "$(TARGET_NAME)";

```

```

SWIFT_OBJC_BRIDGING_HEADER = "Runner/Runner-Bridging-Header.h";

```

```

SWIFT_VERSION = 5.0;

```

```

VERSIONING_SYSTEM = "apple-generic";

```

```

    };
    name = Profile;
};

```

```

331C8088294A63A400263BE5 /* Debug */ = {

```

```

    isa = XCBuildConfiguration;

```

```

    buildSettings = {

```

```

BUNDLE_LOADER = "$(TEST_HOST)";

```

```

CODE_SIGN_STYLE = Automatic;

```

```

CURRENT_PROJECT_VERSION = 1;

```

```

GENERATE_INFOPLIST_FILE = YES;

```

```

MARKETING_VERSION = 1.0;

```

```

PRODUCT_BUNDLE_IDENTIFIER = com.example.mobileApp.RunnerTests;

```

```

PRODUCT_NAME = "$(TARGET_NAME)";

```

```

SWIFT_ACTIVE_COMPILATION_CONDITIONS = DEBUG;

```

```

SWIFT_OPTIMIZATION_LEVEL = "-Onone";

```

```

SWIFT_VERSION = 5.0;

```

```

TEST_HOST = "$(BUILT_PRODUCTS_DIR)/Runner.app/${BUNDLE_EXECUTABLE_FOLDER_PATH}/Runner";

```

```

    };
    name = Debug;
};

```

```

331C8089294A63A400263BE5 /* Release */ = {

```

```

    isa = XCBuildConfiguration;

```

```

    buildSettings = {

```

| | | |
|--|--|---|
| BUNDLE_LOADER =
"\$(TEST_HOST)"; | CURRENT_PROJECT_VERSION = 1; | CLANG_CXX_LIBRARY =
"libc++"; |
| CODE_SIGN_STYLE =
Automatic; | GENERATE_INFOPLIST_FILE = YES; | CLANG_ENABLE_MODULES = YES; |
| CURRENT_PROJECT_VERSION = 1; | MARKETING_VERSION = 1.0; | CLANG_ENABLE_OBJC_ARC = YES; |
| GENERATE_INFOPLIST_FILE = YES; | PRODUCT_BUNDLE_IDENTIFIER =
com.example.mobileApp.RunnerTests; | CLANG_WARN_BLOCK_CAPTURE_AUTORELEASING = YES; |
| MARKETING_VERSION = 1.0; | PRODUCT_NAME =
"\$(TARGET_NAME)"; | CLANG_WARN_BOOL_CONVERSION = YES; |
| PRODUCT_BUNDLE_IDENTIFIER =
com.example.mobileApp.RunnerTests; | SWIFT_VERSION = 5.0; | CLANG_WARN_COMMA = YES; |
| PRODUCT_NAME =
"\$(TARGET_NAME)"; | TEST_HOST =
"\$(BUILT_PRODUCTS_DIR)/Runner.app/
\$(BUNDLE_EXECUTABLE_FOLDER_PATH)/Runner"; | CLANG_WARN_CONSTANT_CONVERSION = YES; |
| SWIFT_VERSION = 5.0; | };
name =
Profile; | CLANG_WARN_DEPRECATED_OBJC_IMPLEMENTATIONS = YES; |
| TEST_HOST =
"\$(BUILT_PRODUCTS_DIR)/Runner.app/
\$(BUNDLE_EXECUTABLE_FOLDER_PATH)/Runner";
};
name =
Release; | 97C147031CF9000F007C117D
/* Debug */ = {
isa =
XCBuildConfiguration; | CLANG_WARN_DIRECT_OBJC_ISA_USAGE = YES_ERROR; |
| 331C808A294A63A400263BE5
/* Profile */ = {
isa =
XCBuildConfiguration; | buildSettings = {
ALWAYS_SEARCH_USER_PATHS = NO; | CLANG_WARN_EMPTY_BODY = YES; |
| buildSettings = {
BUNDLE_LOADER =
"\$(TEST_HOST)"; | ASSETCATALOG_COMPILER_GENERATE_SWIFT_ASSET_SYMBOL_EXTENSIONS = YES;
CLANG_ANALYZER_NONNULL = YES; | CLANG_WARN_ENUM_CONVERSION = YES; |
| CODE_SIGN_STYLE =
Automatic; | CLANG_CXX_LANGUAGE_STANDARD = "gnu++0x"; | CLANG_WARN_INFINITE_RECURSION = YES; |
| | | CLANG_WARN_INT_CONVERSION = YES; |
| | | CLANG_WARN_NON_LITERAL_NULL_CONVERSION = YES; |
| | | CLANG_WARN_OBJC_IMPLICIT_RETAIN_SELF = YES; |
| | | CLANG_WARN_OBJC_LITERAL_CONVERSION = YES; |

| | | |
|---|--|--|
| <pre> CLANG_WARN_OBJC_ROOT _CLASS = YES_ERROR; CLANG_WARN_RANGE_LO OP_ANALYSIS = YES; CLANG_WARN_STRICT_PR OTOTYPES = YES; CLANG_WARN_SUSPICIOUS _MOVE = YES; CLANG_WARN_UNREACHA BLE_CODE = YES; CLANG_WARN__DUPLICAT E_METHOD_MATCH = YES; "CODE_SIGN_IDENTITY[sdk =iphoneos*]" = "iPhone Developer"; COPY_PHASE_STRIP = NO; DEBUG_INFORMATION_FO RMAT = dwarf; ENABLE_STRICT_OBJC_MS GSEND = YES; ENABLE_TESTABILITY = YES; ENABLE_USER_SCRIPT_SA NDBOXING = NO; GCC_C_LANGUAGE_STAND ARD = gnu99; GCC_DYNAMIC_NO_PIC = NO; GCC_NO_COMMON_BLOCK S = YES; GCC_OPTIMIZATION_LEVE L = 0; </pre> | <pre> GCC_PREPROCESSOR_DEFI NITIONS = ("DEBUG=1", "\$(inherited)",); GCC_WARN_64_TO_32_BIT_ CONVERSION = YES; GCC_WARN_ABOUT_RETU RN_TYPE = YES_ERROR; GCC_WARN_UNDECLARED _SELECTOR = YES; GCC_WARN_UNINITIALIZE D_AUTOS = YES_AGGRESSIVE; GCC_WARN_UNUSED_FUN CTION = YES; GCC_WARN_UNUSED_VARI ABLE = YES; IPHONEOS_DEPLOYMENT_ TARGET = 13.0; MTL_ENABLE_DEBUG_INF O = YES; ONLY_ACTIVE_ARCH = YES; SDKROOT = iphoneos; TARGETED_DEVICE_FAMIL Y = "1,2"; }; name = Debug; }; 97C147041CF9000F007C117D /* Release */ = { </pre> | <pre> isa = XCBuildConfiguration; buildSettings = { ALWAYS_SEARCH_USER_P ATHS = NO; ASSETCATALOG_COMPILE R_GENERATE_SWIFT_ASSET_SYMBOL_ EXTENSIONS = YES; CLANG_ANALYZER_NONN ULL = YES; CLANG_CXX_LANGUAGE_S TANDARD = "gnu++0x"; CLANG_CXX_LIBRARY = "libc++"; CLANG_ENABLE_MODULES = YES; CLANG_ENABLE_OBJC_AR C = YES; CLANG_WARN_BLOCK_CA PTURE_AUTORELEASING = YES; CLANG_WARN_BOOL_CON VERSION = YES; CLANG_WARN_COMMA = YES; CLANG_WARN_CONSTANT _CONVERSION = YES; CLANG_WARN_DEPRECAT ED_OBJC_IMPLEMENTATIONS = YES; CLANG_WARN_DIRECT_OB JC_ISA_USAGE = YES_ERROR; CLANG_WARN_EMPTY_BO DY = YES; </pre> |
|---|--|--|

CLANG_WARN_ENUM_CON
VERSION = YES;

CLANG_WARN_INFINITE_R
ECURSION = YES;

CLANG_WARN_INT_CONVE
RSION = YES;

CLANG_WARN_NON_LITERAL
NULL_CONVERSION = YES;

CLANG_WARN_OBJC_IMPLI
CIT_RETAIN_SELF = YES;

CLANG_WARN_OBJC_LITERAL
CONVERSION = YES;

CLANG_WARN_OBJC_ROOT
_CLASS = YES_ERROR;

CLANG_WARN_RANGE_LO
OP_ANALYSIS = YES;

CLANG_WARN_STRICT_PR
OTOTYPES = YES;

CLANG_WARN_SUSPICIOUS
_MOVE = YES;

CLANG_WARN_UNREACHA
BLE_CODE = YES;

CLANG_WARN__DUPLICAT
E_METHOD_MATCH = YES;

"CODE_SIGN_IDENTITY[sdk
=iphoneos*]" = "iPhone Developer";

COPY_PHASE_STRIP = NO;

DEBUG_INFORMATION_FO
RMAT = "dwarf-with-dsym";

ENABLE_NS_ASSERTIONS =
NO;

ENABLE_STRICT_OBJC_MS
GSEND = YES;

ENABLE_USER_SCRIPT_SA
NDBOXING = NO;

GCC_C_LANGUAGE_STAND
ARD = gnu99;

GCC_NO_COMMON_BLOCK
S = YES;

GCC_WARN_64_TO_32_BIT_
CONVERSION = YES;

GCC_WARN_ABOUT_RETU
RN_TYPE = YES_ERROR;

GCC_WARN_UNDECLARED
_SELECTOR = YES;

GCC_WARN_UNINITIALIZE
D_AUTOS = YES_AGGRESSIVE;

GCC_WARN_UNUSED_FUN
CTION = YES;

GCC_WARN_UNUSED_VARI
ABLE = YES;

IPHONEOS_DEPLOYMENT_
TARGET = 13.0;

MTL_ENABLE_DEBUG_INF
O = NO;

SDKROOT = iphoneos;

SUPPORTED_PLATFORMS =
iphoneos;

SWIFT_COMPILATION_MOD
E = wholemodule;

SWIFT_OPTIMIZATION_LEV
EL = "-O";

TARGETED_DEVICE_FAMIL
Y = "1,2";

VALIDATE_PRODUCT =
YES;

};
name =
Release;
};

97C147061CF9000F007C117D
/* Debug */ = {
isa =
XCBuildConfiguration;

baseConfigurationReference =
9740EEB21CF90195004384FC /*
Debug.xcconfig */;

buildSettings = {

ASSETCATALOG_COMPILE
R_APPICON_NAME = AppIcon;

CLANG_ENABLE_MODULES
= YES;

CURRENT_PROJECT_VERSI
ON = "\$(FLUTTER_BUILD_NUMBER)";

ENABLE_BITCODE = NO;

INFOPLIST_FILE =
Runner/Info.plist;

LD_RUNPATH_SEARCH_PA
THS = (

"\$(inherited)",

"@executable_path/Frameworks
",

);

```

PRODUCT_BUNDLE_IDENTIFIER = com.example.mobileApp;

PRODUCT_NAME = "$(TARGET_NAME)";

SWIFT_OBJC_BRIDGING_HEADER = "Runner/Runner-Bridging-Header.h";

SWIFT_OPTIMIZATION_LEVEL = "-Onone";

SWIFT_VERSION = 5.0;

VERSIONING_SYSTEM = "apple-generic";

baseConfigurationReference = 7AFA3C8E1D35360C0083082E /* Release.xcconfig */;

buildSettings = {

ASSETCATALOG_COMPILE_ASSET_NAME = AppIcon;

CLANG_ENABLE_MODULES = YES;

CURRENT_PROJECT_VERSION = "$(FLUTTER_BUILD_NUMBER)";

ENABLE_BITCODE = NO;

INFOPLIST_FILE = Runner/Info.plist;

```

```

LD_RUNPATH_SEARCH_PATHS = "$(inherited)", "@executable_path/Frameworks";

PRODUCT_BUNDLE_IDENTIFIER = com.example.mobileApp;

PRODUCT_NAME = "$(TARGET_NAME)";

SWIFT_OBJC_BRIDGING_HEADER = "Runner/Runner-Bridging-Header.h";

SWIFT_VERSION = 5.0;

VERSIONING_SYSTEM = "apple-generic";

name = Release;

/* End XCBuildConfiguration section */

/* Begin XCConfigurationList section */

331C8087294A63A400263BE5 /* Build configuration list for PBXNativeTarget "RunnerTests" */ = {

isa = XCConfigurationList;

buildConfigurations = (

331C8088294A63A400263BE5 /* Debug */,

```

```

331C8089294A63A400263BE5 /* Release */,

331C808A294A63A400263BE5 /* Profile */,

);

defaultConfigurationIsVisible = 0;

defaultConfigurationName = Release;

};

97C146E91CF9000F007C117D /* Build configuration list for PBXProject "Runner" */ = {

isa = XCConfigurationList;

buildConfigurations = (

97C147031CF9000F007C117D /* Debug */,

97C147041CF9000F007C117D /* Release */,

249021D3217E4FDB00AE95B9 /* Profile */,

);

defaultConfigurationIsVisible = 0;

defaultConfigurationName = Release;

};

97C147051CF9000F007C117D /* Build configuration list for PBXNativeTarget "Runner" */ = {

isa = XCConfigurationList;

buildConfigurations = (

```



```

97C147061CF9000F007C117D
/* Debug */,

97C147071CF9000F007C117D
/* Release */,

249021D4217E4FDB00AE95B
9 /* Profile */,

);

defaultConfigurationIsVisible =
0;

defaultConfigurationName =
Release;

};
/* End XCConfigurationList
section */
};
rootObject =
97C146E61CF9000F007C117D /* Project
object */;
}

=====
=====
=====
=====

Файл:
ios/Runner/AppDelegate.swift

=====
=====
=====
=====

import Flutter
import UIKit

@main
@objc class AppDelegate:
FlutterAppDelegate,
FlutterImplicitEngineDelegate {
override func application(
_ application: UIApplication,

didFinishLaunchingWithOptions

```

```

launchOptions:
[UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
return
super.application(application,
didFinishLaunchingWithOptions:
launchOptions)
}

func
didInitializeImplicitFlutterEngine(_
engineBridge: FlutterImplicitEngineBridge)
{

GeneratedPluginRegistrant.register(with:
engineBridge.pluginRegistry)
}
}

=====
=====
=====
=====

Файл:
ios/Runner/Assets.xcassets/LaunchImage.i
mageset/README.md

=====
=====
=====
=====

# Launch Screen Assets

You can customize the launch
screen with your own desired assets by
replacing the image files in this directory.

You can also do it by opening
your Flutter project's Xcode project with
`open ios/Runner.xcworkspace`, selecting
`Runner/Assets.xcassets` in the Project
Navigator and dropping in the desired
images.

=====
=====
=====
=====

```

```

Файл: ios/Runner/Info.plist
=====
=====
=====
=====

<?xml version="1.0"
encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//
Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-
1.0.dtd">
<plist version="1.0">
<dict>
<key>CADisableMinimumFram
eDurationOnPhone</key>
<true/>
<key>CFBundleDevelopmentRe
gion</key>
<string>$(DEVELOPMENT_L
ANGUAGE)</string>
<key>CFBundleDisplayName</
key>
<string>Mobile App</string>
<key>CFBundleExecutable</ke
y>
<string>$(EXECUTABLE_NA
ME)</string>
<key>CFBundleIdentifier</key>
<string>$(PRODUCT_BUNDL
E_IDENTIFIER)</string>
<key>CFBundleInfoDictionary
Version</key>
<string>6.0</string>
<key>CFBundleName</key>
<string>mobile_app</string>
<key>CFBundlePackageType</
key>
<string>APPL</string>
<key>CFBundleShortVersionStr
ing</key>
<string>$(FLUTTER_BUILD_
NAME)</string>
<key>CFBundleSignature</key>
>
<string>????</string>
<key>CFBundleVersion</key>

```

```

        <string>$(FLUTTER_BUILD_
NUMBER)</string>
        <key>LSRequiresIPhoneOS</ke
y>
        <true/>
        <key>UIApplicationSceneManif
est</key>
        <dict>

            <key>UIApplicationSupportsM
ultipleScenes</key>
                <false/>

            <key>UISceneConfigurations</
key>
                <dict>

                    <key>UIWindowSceneSessionR
oleApplication</key>
                        <array>

                            <dict>

                                <key>UISceneClassName</key>
                                    >

                                <string>UIWindowScene</strin
g>

                                <key>UISceneConfigurationNa
me</key>
                                    <string>flutter</string>

                                <key>UISceneDelegateClassNa
me</key>
                                    <string>$(PRODUCT_MODUL
E_NAME).SceneDelegate</string>

```

```

        <key>UISceneStoryboardFile</
key>
            <string>Main</string>
        </dict>
    </array>
</dict>
<key>UIApplicationSupportsInd
irectInputEvents</key>
    <true/>
<key>UILaunchStoryboardNam
e</key>
    <string>LaunchScreen</string>
<key>UIMainStoryboardFile</k
ey>
    <string>Main</string>
<key>UISupportedInterfaceOrie
ntations</key>
    <array>

        <string>UIInterfaceOrientationP
ortrait</string>

        <string>UIInterfaceOrientationL
andscapeLeft</string>

        <string>UIInterfaceOrientationL
andscapeRight</string>
    </array>
<key>UISupportedInterfaceOrie
ntations~ipad</key>
    <array>

        <string>UIInterfaceOrientationP
ortrait</string>

        <string>UIInterfaceOrientationP
ortraitUpsideDown</string>

        <string>UIInterfaceOrientationL
andscapeLeft</string>

        <string>UIInterfaceOrientationL
andscapeRight</string>
    </array>

```

```

    </dict>
</plist>

=====

=====

=====
Файл: ios/Runner/Runner-
Bridging-Header.h
=====

=====

=====
#import
"GeneratedPluginRegistrant.h"
=====

=====
Файл:
ios/Runner/SceneDelegate.swift
=====

=====

import Flutter
import UIKit

class SceneDelegate:
FlutterSceneDelegate {

}

=====

=====

=====
Файл:
lib/ai/date_ocr_service.dart
=====

=====

=====

import 'dart:io';

```

```

import 'dart:math' as math;

import
'package:google_mlkit_text_recognition/go
ogle_mlkit_text_recognition.dart';
import
'package:image/image.dart' as img;

class DateOcrResult {
  final DateTime? date;
  final String rawText;
  final String? matchedText;

  const DateOcrResult({
    required this.date,
    required this.rawText,
    this.matchedText,
  });

  bool get hasDate => date !=
null;
}

class DateOcrService {
  final TextRecognizer
_recognizer = TextRecognizer(script:
TextRecognitionScript.latin);

  Future<DateOcrResult>
recognizeDateFromImagePath(String
imagePath) async {
    final variantFiles = await
_prepareOcrVariants(imagePath);
    final rawParts = <String>[];

    try {
      for (final variant in
variantFiles) {
        try {
          final inputImage =
InputImage.fromFilePath(variant.path);
          final recognizedText =
await
_recognizer.processImage(inputImage);
          final text =
recognizedText.text.trim();
          if (text.isNotEmpty) {
            rawParts.add(text);

```

```

        }
      } catch (_) {
        // Один неудачный
вариант предобработки не должен
ломать OCR целиком.
      }
    } finally {
      for (final variant in
variantFiles) {
        if (!variant.deleteAfterUse)
continue;

        try {
          final file =
File(variant.path);
          if (await file.exists())
await file.delete();
        } catch (_) {}
      }
    }

    final rawText =
rawParts.join('\n');
    final parsed =
DateTextParser.parse(rawText);

    // Для отладки: если дата
глазами видна, но не распознана, здесь
будет понятно,
    // что именно вернул ML Kit
после всех вариантов предобработки.
    // ignore: avoid_print
    print("[OCR] raw="$rawText"
matched="$${parsed?.matchedText}"
date="$${parsed?.date}");

    return DateOcrResult(
      date: parsed?.date,
      rawText: rawText,
      matchedText:
parsed?.matchedText,
    );
  }
}

Future<List<_OcrVariantFile>>
_prepareOcrVariants(String imagePath)
async {

```

```

    final result =
<_OcrVariantFile>[_OcrVariantFile(image
Path, deleteAfterUse: false)];

    final bytes = await
File(imagePath).readAsBytes();
    final decoded =
img.decodeImage(bytes);
    if (decoded == null) return
result;

    final oriented =
img.bakeOrientation(decoded);
    final scaled =
_scaleForOcr(oriented);
    final paddedOriginal =
_addWhitePadding(scaled, 28);

    final gray =
img.grayscale(scaled);
    final contrast =
img.adjustColor(gray, contrast: 1.75,
brightness: 0.04, saturation: 0.0);
    final paddedContrast =
_addWhitePadding(contrast, 28);

    final binary =
_binarizeDarkText(scaled);
    final paddedBinary =
_addWhitePadding(binary, 28);

    final dilatedLight =
_dilateDarkPixels(binary, radiusX: 1,
radiusY: 1);
    final paddedDilatedLight =
_addWhitePadding(dilatedLight, 28);

    // Для точно-матричной
печати полезно слегка соединять точки
по горизонтали.
    final dilatedWide =
_dilateDarkPixels(binary, radiusX: 2,
radiusY: 1);
    final paddedDilatedWide =
_addWhitePadding(dilatedWide, 28);

```

```

        final inverted =
img.invert(img.Image.from(paddedDilatedL
ight));

        result.add(await
_writeVariant(imagePath, 'ocr_scaled',
paddedOriginal));
        result.add(await
_writeVariant(imagePath, 'ocr_contrast',
paddedContrast));
        result.add(await
_writeVariant(imagePath, 'ocr_binary',
paddedBinary));
        result.add(await
_writeVariant(imagePath, 'ocr_dilate',
paddedDilatedLight));
        result.add(await
_writeVariant(imagePath, 'ocr_dilate_wide',
paddedDilatedWide));
        result.add(await
_writeVariant(imagePath, 'ocr_invert',
inverted));

        return result;
    }

    img.Image
_scaleForOcr(img.Image source) {
        if (source.width <= 0 ||
source.height <= 0) return source;

        const targetMinHeight = 260;
        const targetMaxWidth = 2200;

        var scale = targetMinHeight /
source.height;
        if (scale < 1.0) scale = 1.0;

        var width = (source.width *
scale).round();
        var height = (source.height *
scale).round();

        if (width > targetMaxWidth) {
            final downScale =
targetMaxWidth / width;
            width = targetMaxWidth;

```

```

            height = math.max(1, (height
* downScale).round());
        }

        if (width == source.width &&
height == source.height) {
            return
img.Image.from(source);
        }

        return img.copyResize(
            source,
            width: width,
            height: height,
            interpolation:
img.Interpolation.cubic,
        );
    }

    img.Image
_addWhitePadding(img.Image source, int
padding) {
        final out = img.Image(width:
source.width + padding * 2, height:
source.height + padding * 2);
        img.fill(out, color:
img.ColorRgb8(255, 255, 255));

        for (var y = 0; y <
source.height; y++) {
            for (var x = 0; x <
source.width; x++) {
                out.setPixel(x + padding, y
+ padding, source.getPixel(x, y));
            }
        }

        return out;
    }

    img.Image
_binarizeDarkText(img.Image source) {
        final gray =
img.grayscale(img.Image.from(source));
        final threshold =
_otsuThreshold(gray);
        final out = img.Image(width:
gray.width, height: gray.height);

```

```

        for (var y = 0; y < gray.height;
y++) {
            for (var x = 0; x <
gray.width; x++) {
                final p = gray.getPixel(x,
y);
                final value = p.r.toInt() <=
threshold ? 0 : 255;
                out.setPixelRgb(x, y, value,
value, value);
            }
        }

        return out;
    }

    int _otsuThreshold(img.Image
gray) {
        final hist =
List<int>.filled(256, 0);
        var total = 0;

        for (var y = 0; y < gray.height;
y++) {
            for (var x = 0; x <
gray.width; x++) {
                final v = gray.getPixel(x,
y).r.round().clamp(0, 255);
                hist[v]++;
                total++;
            }
        }

        var sum = 0.0;
        for (var i = 0; i < 256; i++) {
            sum += i * hist[i];
        }

        var sumB = 0.0;
        var wB = 0;
        var maxBetween = -1.0;
        var threshold = 128;

        for (var i = 0; i < 256; i++) {
            wB += hist[i];
            if (wB == 0) continue;

```

```

        final wF = total - wB;
        if (wF == 0) break;

        sumB += i * hist[i];
        final mB = sumB / wB;
        final mF = (sum - sumB) /
wF;

        final between = wB * wF *
math.pow(mB - mF, 2).toDouble();

        if (between > maxBetween)
{
            maxBetween = between;
            threshold = i;
        }
    }
}

```

// На цветных этикетках
Otsu иногда делает порог слишком
светлым и теряет точки.

// Немного смещаем порог
вниз, чтобы оставить только тёмные
элементы маркировки.

```

        return (threshold -
10).clamp(50, 210);
    }

```

```

        img.Image
        _dilateDarkPixels(img.Image binary,
{required int radiusX, required int
radiusY}) {

            final out = img.Image(width:
binary.width, height: binary.height);

            img.fill(out, color:
img.ColorRgb8(255, 255, 255));

            for (var y = 0; y <
binary.height; y++) {
                for (var x = 0; x <
binary.width; x++) {
                    var hasDarkNeighbor =
false;

```

```

                    for (var dy = -radiusY; dy
<= radiusY && !hasDarkNeighbor; dy++) {
                        final yy = y + dy;
                        if (yy < 0 || yy >=
binary.height) continue;

```

```

                        for (var dx = -radiusX; dx
<= radiusX; dx++) {
                            final xx = x + dx;
                            if (xx < 0 || xx >=
binary.width) continue;

                            if (binary.getPixel(xx,
yy).r < 128) {
                                hasDarkNeighbor =
true;
                                break;
                            }
                        }
                    }

                    final value =
hasDarkNeighbor ? 0 : 255;
                    out.setPixelRgb(x, y, value,
value, value);
                }
            }

            return out;
        }

```

```

        Future<_OcrVariantFile>
        _writeVariant(String originalPath, String
suffix, img.Image image) async {

            final path =
'${originalPath}_${suffix}.png';
            final file = File(path);
            await
file.writeAsBytes(img.encodePng(image),
flush: true);

            return _OcrVariantFile(path,
deleteAfterUse: true);
        }

        void close() {
            _recognizer.close();
        }
    }

```

```

class _OcrVariantFile {
    final String path;
    final bool deleteAfterUse;

```

```

    const
    _OcrVariantFile(this.path, {required
this.deleteAfterUse});
}

```

```

class ParsedDateText {
    final DateTime date;
    final String matchedText;

```

```

    const
    ParsedDateText({required this.date,
required this.matchedText});
}

```

```

class DateTextParser {
    static ParsedDateText?

```

```

    parse(String raw) {
        final normalized =
_normalize(raw);
        final candidates =
<ParsedDateText>[];

```

```

        // yyyy.mm.dd / dd.mm.yyyy /
yy.mm.dd / dd.mm.yy

```

```

        final separated = RegExp(
            r'(?<!d)(\d{1,4})s*[\.\-
\\]s*(\d{1,2})s*[\.\-\\]s*(\d{1,4})(?!d)',
            caseSensitive: false,
        );
        for (final match in
separated.allMatches(normalized)) {
            final parsed =
_parseThreeParts(
                match.group(1)!,
                match.group(2)!,
                match.group(3)!,
                match.group(0)!,
            );
            if (parsed != null)
candidates.add(parsed);
        }
    }

```

// Частый случай после
OCR: лишний префикс партии перед
датой, например 25/02.12.2025.

```

        final trailingDate = RegExp(

```

```

        r'(?![^\d])(?:\d{1,4}[.\-
        /\])?\d{1,2})s*[.\-/\]s*(\d{1,2})s*[.\-
        /\]s*((?:20)?\d{2})(?!d)',
        caseSensitive: false,
    );
    for (final match in
    trailingDate.allMatches(normalized)) {
        final parsed = _makeDate(
            _normalizeYear(int.tryParse(match.group(3)
            )),
            int.tryParse(match.group(2)!),
            int.tryParse(match.group(1)!),
            match.group(0)!);
        if (parsed != null)
            candidates.add(parsed);
    }

    // dd mm yyyy / yyyy mm dd
    after OCR inserts spaces instead of
    separators.
    final spaced =
    RegExp(r'(?<!\d)(\d{1,4})s+(\d{1,2})\s+(\d
    {1,4})(?!d)');
    for (final match in
    spaced.allMatches(normalized)) {
        final parsed =
        _parseThreeParts(
            match.group(1)!,
            match.group(2)!,
            match.group(3)!,
            match.group(0)!);
        if (parsed != null)
            candidates.add(parsed);
    }

    // yyyyymmdd / ddmmyyyy
    compact stamps.
    final compact8 =
    RegExp(r'(?<!\d)(\d{8})(?!d)');
    for (final match in
    compact8.allMatches(normalized)) {
        final token =
        match.group(1)!;

```

```

        final parsedYearFirst =
        _makeDate(
            int.tryParse(token.substring(0, 4)),
            int.tryParse(token.substring(4, 6)),
            int.tryParse(token.substring(6, 8)),
            match.group(0)!);
        if (parsedYearFirst != null)
            candidates.add(parsedYearFirst);

        final parsedDayFirst =
        _makeDate(
            int.tryParse(token.substring(4, 8)),
            int.tryParse(token.substring(2, 4)),
            int.tryParse(token.substring(0, 2)),
            match.group(0)!);
        if (parsedDayFirst != null)
            candidates.add(parsedDayFirst);
    }

    // mm.yyyy / mm/yyyy. Для
    срока годности без дня берём последний
    день месяца.
    final monthYear =
    RegExp(r'(?<!\d)(\d{1,2})s*[.\-
    /\]s*((?:20)?\d{2})(?!d)');
    for (final match in
    monthYear.allMatches(normalized)) {
        final month =
        int.tryParse(match.group(1)!);
        final year =
        _normalizeYear(int.tryParse(match.group(2)
        1));
        if (year == null || month ==
        null || month < 1 || month > 12) continue;
        final day = DateTime(year,
        month + 1, 0).day;
        final parsed =
        _makeDate(year, month, day,
        match.group(0)!);

```

```

        if (parsed != null)
            candidates.add(parsed);
    }

    if (candidates.isEmpty) return
    null;

    candidates.sort((a, b) =>
    _scoreDate(b,
    normalized).compareTo(_scoreDate(a,
    normalized)));
    return candidates.first;
}

static String _normalize(String
raw) {
    return raw
        .toUpperCase()
        .replaceAll('O', '0')
        .replaceAll('Q', '0')
        .replaceAll('D', '0')
        .replaceAll('T', '1')
        .replaceAll('L', '1')
        .replaceAll('|', '1')
        .replaceAll('S', '5')
        .replaceAll('Б', '6')
        .replaceAll('3', '3')
        .replaceAll('В', '8')
        .replaceAll('В', '8')
        .replaceAll('—', '-')
        .replaceAll('-', '-')
        .replaceAll(',', ',')
        .replaceAll(':', ':')
        .replaceAll(RegExp(r'^0-
        9A-ZA-Я.\-/\s'], ' ')
        .replaceAll(RegExp(r'^s+'), '
        ');
    .trim();
}

static ParsedDateText?
_parseThreeParts(String aRaw, String
bRaw, String cRaw, String matchedText) {
    final a = int.tryParse(aRaw);
    final b = int.tryParse(bRaw);
    final c = int.tryParse(cRaw);

```

```

        if (a == null || b == null || c ==
null) return null;

        // YYYY-MM-DD
        if (aRaw.length == 4 || a > 31)
{
            return
_makeDate(_normalizeYear(a), b, c,
matchedText);
        }

        // DD-MM-YYYY
        if (cRaw.length == 4 || c > 31)
{
            return
_makeDate(_normalizeYear(c), b, a,
matchedText);
        }

        // YY-MM-DD, если первый
блок явно похож на год.
        if (a > 31 || aRaw.length == 2
&& b <= 12 && c <= 31 && a >= 20) {
            return
_makeDate(_normalizeYear(a), b, c,
matchedText);
        }

        // DD-MM-YY. Для
русскоязычных упаковок это самый
частый вариант.
        return
_makeDate(_normalizeYear(c), b, a,
matchedText);
    }

    static int? _normalizeYear(int?
year) {
        if (year == null) return null;
        if (year < 0) return null;
        if (year < 100) {
            return year >= 70 ? 1900 +
year : 2000 + year;
        }
        return year;
    }

```

```

    static ParsedDateText?
_makeDate(int? year, int? month, int? day,
String matchedText) {
        if (year == null || month ==
null || day == null) return null;
        if (year < 2000 || year > 2099)
return null;
        if (month < 1 || month > 12)
return null;
        if (day < 1 || day > 31) return
null;

        final date = DateTime(year,
month, day);
        if (date.year != year ||
date.month != month || date.day != day)
return null;
        return ParsedDateText(date:
date, matchedText: matchedText.trim());
    }

    static double
_scoreDate(ParsedDateText candidate,
String fullText) {
        var score = 0.0;
        final match =
candidate.matchedText.toUpperCase();
        final idx =
fullText.indexOf(match);
        final before = idx <= 0 ? " " :
fullText.substring(math.max(0, idx - 20),
idx);
        final after = idx < 0 ? " " :
fullText.substring(idx,
math.min(fullText.length, idx +
match.length + 20));
        final context = '$before
$after';

        if
(RegExp(r'EXP|EXPIR|BEST|BEFORE|ГО
ДЕН|ГОДН|СРОК|ДО').hasMatch(context)
) score += 5;
        if
(RegExp(r'PROD|MFG|ИЗГОТ|ПРОИЗВ').
hasMatch(context)) score += 2;
        if (candidate.date.year >=
2020) score += 1;

```

```

        if
(candidate.date.isAfter(DateTime.now().sub
tract(const Duration(days: 365 * 3)))) score
+= 1;
        if
(candidate.matchedText.contains(RegExp(r'
20\d{2}')))) score += 2;
        if
(candidate.matchedText.length >= 8) score
+= 0.5;
        return score;
    }
}

=====
=====
=====
=====
Файл:
lib/ai/date_reader_tflite.dart
=====
=====
=====
=====

import 'dart:convert';
import 'dart:io';
import 'dart:math' as math;
import 'dart:typed_data';

import
'package:flutter/services.dart' show
rootBundle;

import
'package:image/image.dart' as img;

import
'package:tflite_flutter/tflite_flutter.dart';

class DateReaderResult {
    const DateReaderResult({
        required this.rawText,
        required this.normalizedText,
        required this.confidence,
    });

    final String rawText;
    final String normalizedText;
    final double confidence;

```

```

        bool get looksLikeDate {
            final s = normalizedText;
            return RegExp(r'^\d{1,4}[/-
\d{1,2}([./-]\d{1,4})?$',s).hasMatch(s) ||

RegExp(r'^\d{6,8}$').hasMatch(s);
        }
    }

    class DateReaderTflite {
        DateReaderTflite._();

        static final DateReaderTflite
instance = DateReaderTflite._();

        Interpreter? _interpreter;
        String _alphabet =
'0123456789.-/';
        int _blankIndex = 13;
        int _inputWidth = 256;
        int _inputHeight = 48;
        int _timeSteps = 64;
        int _numClasses = 14;

        Future<void> load({
            String modelAsset =
'assets/ai/date_reader.tflite',
            String metaAsset =
'assets/ai/date_reader_meta.json',
        }) async {
            if (_interpreter != null) return;

            final metaRaw = await
rootBundle.loadString(metaAsset);
            final meta =
jsonDecode(metaRaw) as Map<String,
dynamic>;

            _alphabet = (meta['alphabet']
?? _alphabet).toString();
            _blankIndex =
(meta['blank_index'] as num?)?.toInt() ??
_alphabet.length;
            _inputWidth =
(meta['input_width'] as num?)?.toInt() ??
_inputWidth;

            _inputHeight =
(meta['input_height'] as num?)?.toInt() ??
_inputHeight;
            _timeSteps =
(meta['output_time_steps'] as num?)?.toInt()
?? _timeSteps;
            _numClasses =
(meta['num_classes'] as num?)?.toInt() ??
(_alphabet.length + 1);

            final options =
InterpreterOptions().threads = 2;
            _interpreter = await
Interpreter.fromAsset(modelAsset, options:
options);

            final outShape =
_interpreter!.getOutputTensor(0).shape;
            if (outShape.length == 3) {
                _timeSteps = outShape[1];
                _numClasses = outShape[2];
            }
        }

        Future<DateReaderResult?>
recognizeFile(File imageFile) async {
            await load();
            final bytes = await
imageFile.readAsBytes();
            return recognizeBytes(bytes);
        }

        Future<DateReaderResult?>
recognizeBytes(Uint8List bytes) async {
            await load();
            final decoded =
img.decodeImage(bytes);
            if (decoded == null) return
null;

            final input =
_preprocess(decoded);
            final output = List.generate(
1,
                (_) => List.generate(
                    _timeSteps,
                    (_) =>
List<double>.filled(_numClasses, 0.0),

                growable: false,
            ),
            growable: false,
        );
        _interpreter!.run(input,
output);

        final decodedText =
_ctcGreedyDecode(output[0]);
        final normalized =
_normalizeDateText(decodedText);
        final confidence =
_confidence(output[0]);

        if (normalized.isEmpty) return
null;

        return DateReaderResult(
            rawText: decodedText,
            normalizedText: normalized,
            confidence: confidence,
        );
    }

    List<List<List<List<double>>>>>
_preprocess(img.Image source) {
        final gray =
img.grayscale(source);
        final w = gray.width;
        final h = gray.height;
        final scale =
math.min(_inputWidth / math.max(1, w),
_inputHeight / math.max(1, h));
        final nw = math.max(1, (w *
scale).floor());
        final nh = math.max(1, (h *
scale).floor());

        final resized =
img.copyResize(
            gray,
            width: nw,
            height: nh,
            interpolation:
img.Interpolation.linear,
        );
    }

```



```

        final canvas =
img.Image(width: _inputWidth, height:
_inputHeight);
        img.fill(canvas, color:
img.ColorRgb8(255, 255, 255));
        final yOffset = ((_inputHeight
- nh) / 2).floor();
        img.compositeImage(canvas,
resized, dstX: 0, dstY: yOffset);

return List.generate(
1,
(_) => List.generate(
_inputHeight,
(y) => List.generate(
_inputWidth,
(x) {
final p =
canvas.getPixel(x, y);
return <double>[p.r /
255.0];
},
growable: false,
),
growable: false,
),
growable: false,
);
}

```

```

String
_cgcGreedyDecode(List<List<double>>
logits) {
final out = StringBuffer();
int? prev;

for (final step in logits) {
var bestIndex = 0;
var bestValue = step[0];
for (var i = 1; i < step.length;
i++) {
if (step[i] > bestValue) {
bestValue = step[i];
bestIndex = i;
}
}
}
}

```

```

if (bestIndex != prev &&
bestIndex != _blankIndex && bestIndex >=
0 && bestIndex < _alphabet.length) {
out.write(_alphabet[bestIndex]);
}
prev = bestIndex;
}

return out.toString();
}

double
_confidence(List<List<double>> logits) {
var sum = 0.0;
var count = 0;
int? prev;

for (final step in logits) {
var bestIndex = 0;
var bestValue = step[0];
for (var i = 1; i < step.length;
i++) {
if (step[i] > bestValue) {
bestValue = step[i];
bestIndex = i;
}
}

if (bestIndex != prev &&
bestIndex != _blankIndex) {
sum += _softmaxMax(step,
bestValue);
count++;
}
prev = bestIndex;
}

return count == 0 ? 0.0 : sum /
count;
}

double
_softmaxMax(List<double> values, double
maxValue) {
var denom = 0.0;
for (final v in values) {

```

```

denom += math.exp(v -
maxValue);
}

return denom <= 0 ? 0.0 : 1.0 /
denom;
}

String
_normalizeDateText(String text) {
var s = text.trim();
s = s.replaceAll(' ', '');
s = s.replaceAll('|', '/');
s = s.replaceAll("\\", '/');
s = s.replaceAll(RegExp(r'[^\d-
9./-]'), "");
s =
s.replaceAll(RegExp(r'[.]{2,}'), '.');
s =
s.replaceAll(RegExp(r'[.]{2,}'), '/');
s = s.replaceAll(RegExp(r'[-
]{2,}'), '-');
return s;
}

void dispose() {
_interpreter?.close();
_interpreter = null;
}
}

=====
=====
=====
=====
Файл: lib/ai/detection_result.dart
=====
=====
=====

class DetectionResult {
final double confidence;
final int classIndex;
final double left;
final double top;
final double right;
final double bottom;
}

```

```

const DetectionResult({
  required this.confidence,
  required this.classIndex,
  required this.left,
  required this.top,
  required this.right,
  required this.bottom,
});

String get label => classIndex
== 0 ? 'Датa': 'Класс $classIndex';
}

class FramePerf {
  final int queueWaitMs;
  final int yuvToRgbMs;
  final int rotateMs;
  final int resizeLetterboxMs;
  final int tensorBuildMs;
  final int inferMs;
  final int postprocessMs;
  final int totalMs;

  const FramePerf({
    this.queueWaitMs = 0,
    this.yuvToRgbMs = 0,
    this.rotateMs = 0,
    this.resizeLetterboxMs = 0,
    this.tensorBuildMs = 0,
    this.inferMs = 0,
    this.postprocessMs = 0,
    this.totalMs = 0,
  });

  factory
  FramePerf.fromMap(Map<dynamic,
dynamic>? json) {
    if (json == null) return const
  FramePerf();

    return FramePerf(
      queueWaitMs:
(json['queueWaitMs'] ?? 0) as int,
      yuvToRgbMs:
(json['yuvToRgbMs'] ?? 0) as int,
      rotateMs: (json['rotateMs'] ??
0) as int,
      resizeLetterboxMs:
(json['resizeLetterboxMs'] ?? 0) as int,

      tensorBuildMs:
(json['tensorBuildMs'] ?? 0) as int,
      inferMs: (json['inferMs'] ??
0) as int,
      postprocessMs:
(json['postprocessMs'] ?? 0) as int,
      totalMs: (json['totalMs'] ??
0) as int,
    );
  }

  Map<String, int> toMap() => {
    'queueWaitMs':
queueWaitMs,
    'yuvToRgbMs':
yuvToRgbMs,
    'rotateMs': rotateMs,
    'resizeLetterboxMs':
resizeLetterboxMs,
    'tensorBuildMs':
tensorBuildMs,
    'inferMs': inferMs,
    'postprocessMs':
postprocessMs,
    'totalMs': totalMs,
  };

  String toPrettyString() {
    return 'Очередь:
${queueWaitMs} мс · '
    'YUV→RGB:
${yuvToRgbMs} мс · '
    'Поворот: ${rotateMs} мс ·
',
    'Resize:
${resizeLetterboxMs} мс · '
    'Tensor: ${tensorBuildMs}
мс · '
    'Infer: ${inferMs} мс · '
    'Post: ${postprocessMs} мс
· '
    'Bcero: ${totalMs} мс';
  }

  class DetectionFrameResult {
    final List<DetectionResult>
detections;

    final FramePerf perf;

    const DetectionFrameResult({
      required this.detections,
      required this.perf,
    });
  }
}

=====
Файл:
lib/ai/tflite_date_detector.dart
=====
import 'dart:async';
import 'dart:io';
import 'dart:isolate';
import 'dart:developer' as
developer;
import 'dart:math' as math;
import 'dart:typed_data';

import
'package:camera/camera.dart';
import
'package:image/image.dart' as img;
import
'package:tflite_flutter/tflite_flutter.dart';

import
'../api/model_sync_service.dart';
import 'detection_result.dart';

class TfliteDateDetector {
  final ModelSyncService
_modelSyncService;

  Isolate? _workerIsolate;
  ReceivePort? _receivePort;
  SendPort? _workerSendPort;
  String? _loadedModelPath;
  String?
_loadedModelSignature;

```

```

        int _nextRequestId = 1;
        final Map<int,
Completer<DetectionFrameResult>>
        _pendingRequests = {};

        TfliteDateDetector({ModelSyncService?
modelSyncService})
            : _modelSyncService =
modelSyncService ?? ModelSyncService();

        bool get isReady =>
        _workerSendPort != null;

        Future<bool>
loadLatestModel() async {
            final modelPath = await
        _modelSyncService.localModelPath();
            if (modelPath == null) {
                close();
                return false;
            }

            final file = File(modelPath);
            if (!await file.exists()) {
                return false;
            }

            final modelSignature = await
        _fileSignature(file);
            if (_loadedModelPath ==
modelPath &&
                _loadedModelSignature ==
modelSignature &&
                _workerSendPort != null) {
                return true;
            }

            close();

            final receivePort =
ReceivePort();
            final readyPortCompleter =
Completer<SendPort>();
            final initCompleter =
Completer<bool>();

            receivePort.listen((message) {

```

```

                if (message is Map) {
                    final type = message['type'];
                    if (type == 'ready' &&
message['sendPort'] is SendPort) {
                        if
(!readyPortCompleter.isCompleted) {
                            readyPortCompleter.complete(message['sen
dPort'] as SendPort);
                        }
                        return;
                    }

                    if (type == 'inited') {
                        final ok = message['ok']
== true;
                        if
(!initCompleter.isCompleted) {
                            if (ok) {
                                initCompleter.complete(true);
                            } else {
                                initCompleter.completeError(
message['error']?.toString() ?? 'Не удалось
инициализировать worker ИИ.'
                                );
                            }
                        }
                        return;
                    }

                    if (type == 'result') {
                        final requestId =
message['requestId'] as int?;
                        if (requestId == null)
return;

                        final completer =
        _pendingRequests.remove(requestId);
                        if (completer == null ||
completer.isCompleted) return;

                        if (message['ok'] == true)
{
                            final raw =
(message['detections'] as List?) ?? const [];
                            final detections = raw

```

```

                                .whereType<Map>()
                                .map(
                                    (item) =>
DetectionResult(
                                        confidence:
(item['confidence'] as num).toDouble(),
                                        classIndex:
(item['classIndex'] as num).toInt(),
                                        left: (item['left'] as
num).toDouble(),
                                        top: (item['top'] as
num).toDouble(),
                                        right: (item['right']
as num).toDouble(),
                                        bottom:
(item['bottom'] as num).toDouble(),
                                    ),
                                )
                                .toList(growable:
false);

                                final perf =
FramePerf.fromMap(message['perf'] as
Map?);
                                developer.log('AI PERF:
${perf.toPrettyString()}', name: 'AI');

                                completer.complete(DetectionFrameResult(
detections: detections, perf: perf));
                                } else {
                                    completer.completeError(message['error']?.t
oString() ?? 'Ошибка worker ИИ.');
```

```

        _loadedModelPath =
modelPath;
        _loadedModelSignature =
modelSignature;

        workerSendPort.send({
            'type': 'init',
            'modelPath': modelPath,
            'threads': 2,
        });

        try {
            return await
initCompleter.future.timeout(const
Duration(seconds: 20));
        } catch (_) {
            close();
            rethrow;
        }
    }

    Future<DetectionFrameResult>
detectFromCameraImage(
        CameraImage cameraImage, {
        required int rotationDegrees,
        bool mirrorHorizontally =
false,
    }) async {
        final workerSendPort =
_workerSendPort;
        if (workerSendPort == null) {
            return const
DetectionFrameResult(detections: [], perf:
FramePerf());
        }

        final requestId =
_nextRequestId++;
        final completer =
Completer<DetectionFrameResult>();
        _pendingRequests[requestId]
= completer;

        final planes =
cameraImage.planes
            .map(
                (plane) => <String,
Object?>{

```

```

            'bytes':
TransferableTypedData.fromList([plane.by
tes]),
            'bytesPerRow':
plane.bytesPerRow,
            'bytesPerPixel':
plane.bytesPerPixel,
        },
    )
        .toList(growable: false);

        workerSendPort.send({
            'type': 'detect',
            'requestId': requestId,
            'width': cameraImage.width,
            'height': cameraImage.height,
            'formatGroup':
cameraImage.format.group.index,
            'rotationDegrees':
rotationDegrees,
            'mirrorHorizontally':
mirrorHorizontally,
            'planes': planes,
            'submittedAtMicros':
DateTime.now().microsecondsSinceEpoch,
        });

        try {
            return await
completer.future.timeout(const
Duration(seconds: 12));
        } finally {
            _pendingRequests.remove(requestId);
        }
    }

    Future<DetectionFrameResult>
detectFromImageFile(
        String imagePath, {
        DetectionResult?
focusRegion,
        double focusPadding = 1.6,
        double confidenceThreshold =
0.50,
        int maxDetections = 3,

```

```

        bool strictGeometryFilters =
true,
    }) async {
        final workerSendPort =
_workerSendPort;
        if (workerSendPort == null) {
            return const
DetectionFrameResult(detections: [], perf:
FramePerf());
        }

        final requestId =
_nextRequestId++;
        final completer =
Completer<DetectionFrameResult>();
        _pendingRequests[requestId]
= completer;

        workerSendPort.send({
            'type': 'detectFile',
            'requestId': requestId,
            'imagePath': imagePath,
            'focusPadding':
focusPadding,
            'confidenceThreshold':
confidenceThreshold,
            'maxDetections':
maxDetections,
            'strictGeometryFilters':
strictGeometryFilters,
            if (focusRegion != null)
                'focus': {
                    'left': focusRegion.left,
                    'top': focusRegion.top,
                    'right': focusRegion.right,
                    'bottom':
focusRegion.bottom,
                },
            'submittedAtMicros':
DateTime.now().microsecondsSinceEpoch,
        });

        try {
            return await
completer.future.timeout(const
Duration(seconds: 20));
        } finally {

```

```

        _pendingRequests.remove(requestId);
    }
}

void close() {
    for (final completer in
        _pendingRequests.values) {
        if (!completer.isCompleted)
        {
            completer.complete(const
DetectionFrameResult(detections: [], perf:
FramePerf()));
        }
    }
    _pendingRequests.clear();

    _workerSendPort?.send({'type': 'dispose'});
    _workerSendPort = null;
    _receivePort?.close();
    _receivePort = null;
    _workerIsolate?.kill(priority:
Isolate.immediate);
    _workerIsolate = null;
    _loadedModelPath = null;
    _loadedModelSignature =
null;
}

Future<String>
_fileSignature(File file) async {
    final stat = await file.stat();
    return
'$ {file.path} | $ {stat.size} | $ {stat.modified.mi
crosecondsSinceEpoch}';
}

void
_detectorWorkerEntry(SendPort
mainSendPort) async {
    final port = ReceivePort();
    mainSendPort.send({'type':
'ready', 'sendPort': port.sendPort});

    Interpreter? interpreter;

    TensorType inputType =
TensorType.float32;
    TensorType outputType =
TensorType.float32;
    int inputSize = 640;
    double inputScale = 1.0;
    int inputZeroPoint = 0;
    double outputScale = 1.0;
    int outputZeroPoint = 0;

    await for (final message in
port) {
        if (message is! Map) continue;
        final type = message['type'];

        if (type == 'dispose') {
            interpreter?.close();
            port.close();
            break;
        }

        if (type == 'init') {
            try {
                interpreter?.close();
                final modelPath =
message['modelPath']?.toString();
                if (modelPath == null ||
modelPath.isEmpty) {
                    throw StateError('Путь к
модели не передан. ');
                }

                final options =
InterpreterOptions().threads =
(message['threads'] as int?) ?? 2;
                final created =
Interpreter.fromFile(File(modelPath),
options: options);
                created.allocateTensors();

                final inputTensor =
created.getInputTensor(0);
                final outputTensor =
created.getOutputTensor(0);

                inputType =
inputTensor.type;

                outputType =
outputTensor.type;
                inputSize =
inputTensor.shape.length >= 3 ?
inputTensor.shape[1] : 640;

                final inputParams =
inputTensor.params;
                inputScale =
inputParams.scale == 0 ? 1.0 :
inputParams.scale;
                inputZeroPoint =
inputParams.zeroPoint;

                final outputParams =
outputTensor.params;
                outputScale =
outputParams.scale == 0 ? 1.0 :
outputParams.scale;
                outputZeroPoint =
outputParams.zeroPoint;

                developer.log(
'TFLite model loaded. '
'inputShape=$ {inputTensor.shape},
inputType=$inputType, '
'inputScale=$inputScale,
inputZeroPoint=$inputZeroPoint; '
'outputShape=$ {outputTensor.shape},
outputType=$outputType, '
'outputScale=$outputScale,
outputZeroPoint=$outputZeroPoint',
name: 'AI',
);

                interpreter = created;
                mainSendPort.send({'type':
'init', 'ok': true});
            } catch (e, st) {
                mainSendPort.send({'type':
'init', 'ok': false, 'error': '$e\n$st'});
            }
            continue;
        }
    }
}

```

```

        if (type == 'detect') {
            final requestId =
message['requestId'] as int?;
            if (requestId == null)
continue;

            try {
                final activeInterpreter =
interpreter;
                if (activeInterpreter == null)
{
                    throw
StateError('Interpreter ещё не
инициализирован.');
```

```

                planes: planes
                    .map(
                        (plane) =>
WorkerPlane(
                            bytes: (plane['bytes']
as
TransferableTypedData).materialize().asUin
t8List(),
                            bytesPerRow:
plane['bytesPerRow'] as int,
                            bytesPerPixel:
plane['bytesPerPixel'] as int?,
                        ),
                    )
                    .toList(growable: false),
            );

            final yuvSw =
Stopwatch()..start();
            final converted =
_cameraImageToImage(cameraImage);
            yuvSw.stop();
            perf['yuvToRgbMs'] =
yuvSw.elapsedMilliseconds;
            if (converted == null) {
                mainSendPort.send({
                    'type': 'result',
                    'requestId': requestId,
                    'ok': true,
                    'detections': const
<Map<String, Object>>[],
                });
                continue;
            }

            final rotateSw =
Stopwatch()..start();
            var oriented =
_applyRotation(converted,
rotationDegrees);
            rotateSw.stop();
            perf['rotateMs'] =
rotateSw.elapsedMilliseconds;
            if (mirrorHorizontally) {
                oriented =
img.flipHorizontal(oriented);
            }
        }
    }
}

```

```

        final resizeSw =
Stopwatch()..start();
        final prep =
_letterbox(oriented, inputSize);
        resizeSw.stop();
        perf['resizeLetterboxMs'] =
resizeSw.elapsedMilliseconds;
        final outputShape =
activeInterpreter.getOutputTensor(0).shape;

        List<Map<String, Object>>
detections;
        final tensorSw =
Stopwatch()..start();

        final inputData =
_isQuantizedTensor(inputType)
            ?
_buildQuantizedInput(prepare.image,
inputSize, inputScale, inputZeroPoint,
inputType)
            :
_buildFloatInput(prepare.image, inputSize);
        tensorSw.stop();
        perf['tensorBuildMs'] =
tensorSw.elapsedMilliseconds;

        final output =
_isQuantizedTensor(outputType)
            ?
_createOutputContainerInt(outputShape,
outputZeroPoint)
            :
_createOutputContainerFloat(outputShape);

        final inferSw =
Stopwatch()..start();

        activeInterpreter.run(inputData, output);
        inferSw.stop();
        perf['inferMs'] =
inferSw.elapsedMilliseconds;

        final postSw =
Stopwatch()..start();
        detections =
_isQuantizedTensor(outputType)
            ? _parseOutputInt(

```

```

        output,
        prep,
        inputSize,
        outputScale,
        outputZeroPoint,
        confidenceThreshold:
0.50,
        maxDetections: 3,
        strictGeometryFilters:
true,
    )
    : _parseOutputFloat(
        output,
        prep,
        inputSize,
        confidenceThreshold:
0.50,
        maxDetections: 3,
        strictGeometryFilters:
true,
    );
    postSw.stop();
    perf['postprocessMs'] =
postSw.elapsedMilliseconds;

    totalSw.stop();
    perf['totalMs'] =
totalSw.elapsedMilliseconds;

    mainSendPort.send({
        'type': 'result',
        'requestId': requestId,
        'ok': true,
        'detections': detections,
        'perf': perf,
    });
} catch (e, st) {
    mainSendPort.send({
        'type': 'result',
        'requestId': requestId,
        'ok': false,
        'error': '$e\n$st',
    });
}
}

if (type == 'detectFile') {

        final requestId =
message['requestId'] as int?;
        if (requestId == null)
            continue;

        try {
            final activeInterpreter =
interpreter;
            if (activeInterpreter == null)
            {
                throw
                StateError('Interpreter ещё не
инициализирован.');
```

```

        final inferSw =
Stopwatch()..start();

activeInterpreter.run(inputData, output);
        inferSw.stop();
        perf['inferMs'] =
inferSw.elapsedMilliseconds;

        final postSw =
Stopwatch()..start();

        final confidenceThreshold =
(message['confidenceThreshold'] as
num?)?.toDouble() ?? 0.50;

        final maxDetections =
(message['maxDetections'] as int?) ?? 3;

        final strictGeometryFilters
= message['strictGeometryFilters'] != false;

        final cropDetections =
_isQuantizedTensor(outputType)
        ? _parseOutputInt(
            output,
            prep,
            inputSize,
            outputScale,
            outputZeroPoint,
            confidenceThreshold:
confidenceThreshold,
            maxDetections:
maxDetections,
            strictGeometryFilters:
strictGeometryFilters,
        )
        : _parseOutputFloat(
            output,
            prep,
            inputSize,
            confidenceThreshold:
confidenceThreshold,
            maxDetections:
maxDetections,
            strictGeometryFilters:
strictGeometryFilters,
        );

        final detections =
_mapCropDetectionsToFullImage(
            cropDetections,
            cropInfo,
            oriented.width,
            oriented.height,
        );

        postSw.stop();
        perf['postprocessMs'] =
postSw.elapsedMilliseconds;

        totalSw.stop();
        perf['totalMs'] =
totalSw.elapsedMilliseconds;

        mainSendPort.send({
            'type': 'result',
            'requestId': requestId,
            'ok': true,
            'detections': detections,
            'perf': perf,
        });
    } catch (e, st) {
        mainSendPort.send({
            'type': 'result',
            'requestId': requestId,
            'ok': false,
            'error': '$e\n$st',
        });
    }
}

class _WorkerCameraImage {
    final int width;
    final int height;
    final int formatGroupIndex;
    final List<_WorkerPlane>
planes;

    const _WorkerCameraImage({
        required this.width,
        required this.height,
        required
this.formatGroupIndex,
        required this.planes,
    });
}

class _WorkerPlane {
    final Uint8List bytes;

    final int bytesPerRow;
    final int? bytesPerPixel;

    const _WorkerPlane({
        required this.bytes,
        required this.bytesPerRow,
        required this.bytesPerPixel,
    });
}

img.Image?
_cameraImageToImage(_WorkerCameraIm
age cameraImage) {
    if
(cameraImage.formatGroupIndex ==
ImageFormatGroup.bgra8888.index) {
        return img.Image.fromBytes(
            width: cameraImage.width,
            height: cameraImage.height,
            bytes:
cameraImage.planes.first.bytes.buffer,
            order:
img.ChannelOrder.bgra,
        );
    }

    if
(cameraImage.formatGroupIndex !=
ImageFormatGroup.yuv420.index) {
        return null;
    }

    final width =
cameraImage.width;
    final height =
cameraImage.height;
    final yPlane =
cameraImage.planes[0];
    final uPlane =
cameraImage.planes[1];
    final vPlane =
cameraImage.planes[2];

    final image = img.Image(width:
width, height: height);

    for (var y = 0; y < height; y++)
{

```



```

        final uvRow = (y >> 1) *
uPlane.bytesPerRow;
        final yRow = y *
yPlane.bytesPerRow;

        for (var x = 0; x < width; x++)
        {
            final uvIndex = uvRow + (x
>> 1) * (uPlane.bytesPerPixel ?? 1);
            final index = yRow + x;

            final yp =
yPlane.bytes[index];
            final up =
uPlane.bytes[uvIndex];
            final vp =
vPlane.bytes[uvIndex];

            final r = (yp + 1.402 * (vp -
128)).round();
            final g = (yp - 0.344136 *
(up - 128) - 0.714136 * (vp - 128)).round();
            final b = (yp + 1.772 * (up -
128)).round();

            image.setPixelRgb(x, y,
_clampColor(r), _clampColor(g),
_clampColor(b));
        }
    }

    return image;
}

```

```

img.Image
_applyRotation(img.Image image, int
rotationDegrees) {
    final normalized =
((rotationDegrees % 360) + 360) % 360;
    switch (normalized) {
        case 90:
            return
img.copyRotate(image, angle: 90);
        case 180:
            return
img.copyRotate(image, angle: 180);
        case 270:

```

```

        return
img.copyRotate(image, angle: 270);
    }
    default:
        return image;
    }
}

_LetterboxResult
_letterbox(img.Image image, int inputSize)
{
    final srcWidth = image.width;
    final srcHeight = image.height;
    final scale =
math.min(inputSize / srcWidth, inputSize /
srcHeight);

    final resizedWidth =
math.max(1, (srcWidth * scale).round());
    final resizedHeight =
math.max(1, (srcHeight * scale).round());
    final padX = ((inputSize -
resizedWidth) / 2).floor();
    final padY = ((inputSize -
resizedHeight) / 2).floor();

    final resized = img.copyResize(
image,
width: resizedWidth,
height: resizedHeight,
interpolation:
img.Interpolation.linear,
);

```

```

    final canvas =
img.Image(width: inputSize, height:
inputSize);
    img.fill(canvas, color:
img.ColorRgb8(114, 114, 114));
    img.compositeImage(canvas,
resized, dstX: padX, dstY: padY);

    return _LetterboxResult(
image: canvas,
originalWidth: srcWidth,
originalHeight: srcHeight,
scale: scale,
padX: padX.toDouble(),
padY: padY.toDouble(),

```

```

    );
}

List<List<List<List<double>>>>
> _buildFloatInput(img.Image image, int
inputSize) {
    final rgbBytes =
image.getBytes(order:
img.ChannelOrder.rgb);
    var offset = 0;

    return [
        List.generate(
            inputSize,
            (_) => List.generate(
                inputSize,
                (_) {
                    final pixel = <double>[
                        rgbBytes[offset] / 255.0,
                        rgbBytes[offset + 1] /
255.0,
                        rgbBytes[offset + 2] /
255.0,
                    ];
                    offset += 3;
                    return pixel;
                },
                growable: false,
            ),
            growable: false,
        ),
    ];
}

```

```

List<List<List<List<int>>>>
_buildQuantizedInput(
img.Image image,
int inputSize,
double inputScale,
int inputZeroPoint,
TensorType inputType,
) {
    final rgbBytes =
image.getBytes(order:
img.ChannelOrder.rgb);
    var offset = 0;

    return [

```

```

List.generate(
  inputSize,
  () => List.generate(
    inputSize,
    () {
      final r =
        _quantizeInput(rgbBytes[offset],
          inputScale, inputZeroPoint, inputType);
      final g =
        _quantizeInput(rgbBytes[offset + 1],
          inputScale, inputZeroPoint, inputType);
      final b =
        _quantizeInput(rgbBytes[offset + 2],
          inputScale, inputZeroPoint, inputType);
      offset += 3;
      return <int>[r, g, b];
    },
    growable: false,
  ),
  growable: false,
),
];
}

int _quantizeInput(
  int channelValue,
  double inputScale,
  int inputZeroPoint,
  TensorType inputType,
) {
  final normalized =
    channelValue / 255.0;
  final quantized = (normalized /
    inputScale + inputZeroPoint).round();
  if (inputType ==
    TensorType.uint8) {
    return quantized.clamp(0,
      255);
  }
  return quantized.clamp(-128,
    127);
}

bool
_isQuantizedTensor(TensorType type) {
  return type == TensorType.int8
    || type == TensorType.uint8;
}

```

```

dynamic
_createOutputContainerFloat(List<int>
  shape) {
  if (shape.length == 3) {
    return List.generate(
      shape[0],
      () => List.generate(
        shape[1],
        () =>
          List<double>.filled(shape[2], 0.0,
            growable: false),
        growable: false,
      ),
      growable: false,
    );
  }
  if (shape.length == 2) {
    return List.generate(
      shape[0],
      () =>
        List<double>.filled(shape[1], 0.0,
          growable: false),
      growable: false,
    );
  }
  throw
    UnsupportedError('Unsupported float
      output shape: $shape');
}

dynamic
_createOutputContainerInt(List<int> shape,
  int zeroPoint) {
  // Важно: для quantized output
  значение float 0.0 кодируется не всегда
  как int 0,
  // а как zeroPoint. Если
  заполнить буфер нулями,
  неинициализированные/пустые
  // строки NMS могут после
  dequantization стать score≈1.0 и
  рисоваться как 100%.
  if (shape.length == 3) {
    return List.generate(
      shape[0],

```

```

    () => List.generate(
      shape[1],
      () =>
        List<int>.filled(shape[2], zeroPoint,
          growable: false),
      growable: false,
    ),
    growable: false,
  );
}

if (shape.length == 2) {
  return List.generate(
    shape[0],
    () =>
      List<int>.filled(shape[1], zeroPoint,
        growable: false),
    growable: false,
  );
}

throw
  UnsupportedError('Unsupported int output
    shape: $shape');
}

List<Map<String, Object>>
_parseOutputFloat(
  dynamic output,
  _LetterboxResult prep,
  int inputSize, {
    required double
      confidenceThreshold,
    required int maxDetections,
    required bool
      strictGeometryFilters,
  }) {
  final rows =
    _extractFloatRows(output);
  final results = <Map<String,
    Object>>[];

  for (final row in rows) {
    if (row.length < 6) continue;

    final confidence = row[4];

```

```

        if (confidence.isNaN ||
confidence < confidenceThreshold ||
confidence > 1.0001) {
            continue;
        }

        final classIndex =
row[5].round();
        if (classIndex != 0) {
            continue;
        }

        // Ultralytics TFLite c
nms=True обычно отдаёт [x1, y1, x2, y2,
score, class].

        // На всякий случай ниже
есть fallback для [cx, cy, w, h, score,
class].

        var left = _mapX(row[0],
prep, inputSize);
        var top = _mapY(row[1],
prep, inputSize);
        var right = _mapX(row[2],
prep, inputSize);
        var bottom = _mapY(row[3],
prep, inputSize);

        if (right <= left || bottom <=
top) {

            final cx = row[0];
            final cy = row[1];
            final w = row[2];
            final h = row[3];
            left = _mapX(cx - w / 2.0,
prep, inputSize);
            top = _mapY(cy - h / 2.0,
prep, inputSize);
            right = _mapX(cx + w / 2.0,
prep, inputSize);
            bottom = _mapY(cy + h /
2.0, prep, inputSize);
        }

        if (right <= left || bottom <=
top) {
            continue;
        }

```

```

        final boxWidth = right - left;
        final boxHeight = bottom -
top;

        final area = boxWidth *
boxHeight;
        final aspect = boxWidth /
boxHeight;

        // Для live-сканирования
фильтры нужны, чтобы не рисовать
мусор.

        // Для режима лупы они
могут быть отключены: там важнее
найти даже слабый bbox около клика.
        if (strictGeometryFilters) {
            if (area < 0.00002 || area >
0.25) continue;
            if (aspect < 1.2) continue;
        } else {
            if (area < 0.000005 || area >
0.75) continue;
        }

        results.add({
            'confidence':
confidence.clamp(0.0, 1.0),
            'classIndex': classIndex,
            'left': left,
            'top': top,
            'right': right,
            'bottom': bottom,
        });

        results.sort(
            (a, b) => ((b['confidence'] as
num).toDouble()).compareTo((a['confidenc
e'] as num).toDouble()),
        );
        return results.take(math.max(1,
maxDetections)).toList(growable: false);
    }

    List<Map<String, Object>>
_parseOutputInt(
        dynamic output,
        _LetterboxResult prep,
        int inputSize,

```

```

        double outputScale,
        int outputZeroPoint, {
            required double
confidenceThreshold,
            required int maxDetections,
            required bool
strictGeometryFilters,
        }) {
            final rows =
_extractIntRows(output);
            final dequantized = rows
.map((row) => row.map((v)
=> (v - outputZeroPoint) *
outputScale).toList(growable: false))
.toList(growable: false);
            return _parseOutputFloat(
                [dequantized],
                prep,
                inputSize,
                confidenceThreshold:
confidenceThreshold,
                maxDetections:
maxDetections,
                strictGeometryFilters:
strictGeometryFilters,
            );
        }

        List<List<double>>>
_extractFloatRows(dynamic output) {
            if (output is List &&
output.isNotEmpty) {
                final first = output.first;
                if (first is List &&
first.isNotEmpty && first.first is List) {
                    if (first.first is List<double>)
{
                        return
List<List<double>>>.from(first);
                    }
                    return first
.map<List<double>>>((row) => (row as
List).map((v) => (v as
num).toDouble()).toList(growable: false))
.toList(growable: false);
                }
            }

```

```

        if (first is List && first.length
== 6) {
            return output

        .map<List<double>>>((row) => (row as
List).map((v) => (v as
num).toDouble()).toList(growable: false))
            .toList(growable: false);
        }
    }

    throw
UnsupportedError('Unsupported float
output container');
}

    List<List<int>>
_extractIntRows(dynamic output) {
        if (output is List &&
output.isNotEmpty) {
            final first = output.first;
            if (first is List &&
first.isNotEmpty && first.first is List) {
                return first
                    .map<List<int>>>((row) =>
(row as List).map((v) => (v as
num).toInt()).toList(growable: false))
                        .toList(growable: false);
            }

            if (first is List && first.length
== 6) {
                return output
                    .map<List<int>>>((row) =>
(row as List).map((v) => (v as
num).toInt()).toList(growable: false))
                        .toList(growable: false);
            }
        }

    throw
UnsupportedError('Unsupported int output
container');
}

    double _mapX(double
rawValue, _LetterboxResult prep, int
inputSize) {

```

```

        final modelX = rawValue <=
1.2 ? rawValue * inputSize : rawValue;
        final originalX = ((modelX -
prep.padX) / prep.scale).clamp(0.0,
prep.originalWidth.toDouble());
        return (originalX /
prep.originalWidth).clamp(0.0, 1.0);
    }

    double _mapY(double
rawValue, _LetterboxResult prep, int
inputSize) {
        final modelY = rawValue <=
1.2 ? rawValue * inputSize : rawValue;
        final originalY = ((modelY -
prep.padY) / prep.scale).clamp(0.0,
prep.originalHeight.toDouble());
        return (originalY /
prep.originalHeight).clamp(0.0, 1.0);
    }

    int _clampColor(int value) =>
math.max(0, math.min(255, value));

    class _FocusCropResult {
        final img.Image image;
        final int x;
        final int y;
        final int width;
        final int height;

        const _FocusCropResult({
            required this.image,
            required this.x,
            required this.y,
            required this.width,
            required this.height,
        });
    }

```

```

    _FocusCropResult
_cropImageByFocus(img.Image source,
Map<String, Object?>? focus, double
paddingFactor) {
        if (focus == null) {

```

```

            return
_focusCropResult(image: source, x: 0, y: 0,
width: source.width, height: source.height);
        }

        final left = ((focus['left'] as
num?)?.toDouble() ?? 0.0).clamp(0.0,
1.0).toDouble();
        final top = ((focus['top'] as
num?)?.toDouble() ?? 0.0).clamp(0.0,
1.0).toDouble();
        final right = ((focus['right'] as
num?)?.toDouble() ?? 1.0).clamp(0.0,
1.0).toDouble();
        final bottom = ((focus['bottom']
as num?)?.toDouble() ?? 1.0).clamp(0.0,
1.0).toDouble();

        if (right <= left || bottom <=
top) {
            return
_focusCropResult(image: source, x: 0, y: 0,
width: source.width, height: source.height);
        }

        final centerX = (left + right) /
2.0;
        final centerY = (top + bottom) /
2.0;
        final boxW = math.max(0.03,
right - left);
        final boxH = math.max(0.03,
bottom - top);
        final factor = math.max(1.0,
paddingFactor);

        var cropWNorm =
math.min(1.0, boxW * factor);
        var cropHNorm =
math.min(1.0, boxH * factor);

        // Дата часто очень низкая по
высоте; добавляем минимальный
контекст вокруг строки,
        // чтобы повторный инференс
видел не только пиксели символов, но и
часть упаковки.

```

```

        cropWNorm =
math.max(cropWNorm, 0.18);
        cropHNorm =
math.max(cropHNorm, 0.08);
        cropWNorm =
math.min(cropWNorm, 1.0);
        cropHNorm =
math.min(cropHNorm, 1.0);

        final cropLeft = (centerX -
cropWNorm / 2.0).clamp(0.0, 1.0 -
cropWNorm).toDouble();
        final cropTop = (centerY -
cropHNorm / 2.0).clamp(0.0, 1.0 -
cropHNorm).toDouble();

        final x = (cropLeft *
source.width).floor().clamp(0, source.width
- 1).toInt();
        final y = (cropTop *
source.height).floor().clamp(0,
source.height - 1).toInt();
        final w = math.max(1,
(cropWNorm *
source.width).round()).clamp(1,
source.width - x).toInt();
        final h = math.max(1,
(cropHNorm *
source.height).round()).clamp(1,
source.height - y).toInt();

        final cropped =
img.copyCrop(source, x: x, y: y, width: w,
height: h);
        return
_FocusCropResult(image: cropped, x: x, y:
y, width: w, height: h);
    }

    List<Map<String, Object>>
_mapCropDetectionsToFullImage(
    List<Map<String, Object>>
cropDetections,
    _FocusCropResult crop,
    int fullWidth,
    int fullHeight,
  ) {
    return cropDetections

```

```

    .map((detection) {
        final left = (crop.x +
(detection['left'] as num).toDouble() *
crop.width) / fullWidth;
        final top = (crop.y +
(detection['top'] as num).toDouble() *
crop.height) / fullHeight;
        final right = (crop.x +
(detection['right'] as num).toDouble() *
crop.width) / fullWidth;
        final bottom = (crop.y +
(detection['bottom'] as num).toDouble() *
crop.height) / fullHeight;
        return <String, Object>{
            'confidence':
detection['confidence'] as Object,
            'classIndex':
detection['classIndex'] as Object,
            'left': left.clamp(0.0,
1.0).toDouble(),
            'top': top.clamp(0.0,
1.0).toDouble(),
            'right': right.clamp(0.0,
1.0).toDouble(),
            'bottom':
bottom.clamp(0.0, 1.0).toDouble(),
        };
    })
    .where((detection) =>
(detection['right'] as
double) > (detection['left'] as double) &&
(detection['bottom'] as
double) > (detection['top'] as double))
    .toList(growable: false);
  }

  class _LetterboxResult {
    final img.Image image;
    final int originalWidth;
    final int originalHeight;
    final double scale;
    final double padX;
    final double padY;

    const _LetterboxResult({
      required this.image,
      required this.originalWidth,
      required this.originalHeight,

```

```

      required this.scale,
      required this.padX,
      required this.padY,
    });
  }

  =====
  =====
  =====
  =====
  Файл: lib/api/api_client.dart
  =====
  =====
  =====
  =====

  import 'dart:convert';
  import 'package:http/http.dart' as
http;

  import 'api_config.dart';

  class ApiClient {
    final String? token;
    ApiClient({required
this.token});

    static String get baseUrl =>
ApiConfig.baseUrl;

    Map<String, String>
_headers({bool auth = false}) {
      final h = {'Content-Type':
'application/json'};
      if (auth && token != null) {
        h['Authorization'] = 'Bearer
$token';
      }
      return h;
    }

    Future<String> register(String
email, String password) async {
      final r = await http.post(
Uri.parse('$baseUrl/api/Auth/register'),
        headers: _headers(),

```

```

        body: jsonEncode({'email':
email, 'password': password})),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
    return
jsonDecode(r.body)['accessToken'];
}

```

```

Future<String> login(String
email, String password) async {
    final r = await http.post(

Uri.parse('$baseUrl/api/Auth/login'),
    headers: _headers(),
    body: jsonEncode({'email':
email, 'password': password})),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
    return
jsonDecode(r.body)['accessToken'];
}

```

```

Future<Map<String,
dynamic>> me() async {
    final r = await http.get(

Uri.parse('$baseUrl/api/Users/me'),
    headers: _headers(auth:
true),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
    return jsonDecode(r.body);
}

```

```

Future<List<dynamic>>
products() async {
    final r = await http.get(

Uri.parse('$baseUrl/api/Products'),
    headers: _headers(),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
    return jsonDecode(r.body);
}

```

```

Future<List<dynamic>>
storage() async {
    final r = await http.get(

Uri.parse('$baseUrl/api/Storage'),
    headers: _headers(auth:
true),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
    return jsonDecode(r.body);
}

```

```

Future<void>
addStoredProductByName(String
productName, DateTime expiryAt) async {
    final r = await http.post(

Uri.parse('$baseUrl/api/Storage'),
    headers: _headers(auth:
true),
    body: jsonEncode({
        'productName':
productName,
        'manufactureAt': null,
        'expiryAt':
expiryAt.toIso8601String(),
    })),
    );
    if (r.statusCode != 200) throw
Exception(r.body);
}

```

```

Future<void>
uploadDatasetZip(String zipPath, {String
comment = ""}) async {
    final uri =
Uri.parse('$baseUrl/api/error-
reports/upload-dataset');
    final req =
http.MultipartRequest('POST', uri);

    if (token != null) {
        req.headers['Authorization']
= 'Bearer $token';
    }
}

```

```

req.fields['comment'] =
comment;

req.files.add(await
http.MultipartFile.fromPath('datasetZip',
zipPath));

    final res = await req.send();
    final body = await
res.stream.bytesToString();
    if (res.statusCode != 200 &&
res.statusCode != 201) {
        throw Exception(body);
    }
}

=====
=====
=====
Файл: lib/api/api_config.dart
=====
=====
=====
import
'../storage/file_key_value_store.dart';

class ApiConfig {
    static const defaultBaseUrl =
'http://111.88.146.2:5099';
    static const _baseUrlKey =
'api_base_url';

    static String _baseUrl =
defaultBaseUrl;

    static String get baseUrl =>
_baseUrl;

    static Future<void> load()
async {
        try {
            final stored = await
FileKeyValueStore.getString(_baseUrlKey)
;

```

```

        _baseUrl =
_normalize(stored ?? defaultBaseUrl);
    } catch (_) {
        _baseUrl = defaultBaseUrl;
    }
}

static Future<void>
setBaseUrl(String value) async {
    _baseUrl = _normalize(value);
    await
FileKeyValueStore.setString(_baseUrlKey,
_baseUrl);
}

static Future<void> reset() =>
setBaseUrl(defaultBaseUrl);

static String
normalizeForDisplay(String value) =>
_normalize(value);

static String _normalize(String
value) {
    var normalized = value.trim();
    if (normalized.isEmpty) {
        normalized =
defaultBaseUrl;
    }

    if
(!normalized.startsWith('http://') &&
!normalized.startsWith('https://')) {
        normalized =
'http://$normalized';
    }

    while
(normalized.endsWith('/')) {
        normalized =
normalized.substring(0, normalized.length -
1);
    }

    return normalized;
}

```

```

=====
=====
=====
=====
Файл:
lib/api/model_sync_service.dart
=====
=====
=====

import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as
http;

import
'./storage/file_key_value_store.dart';
import 'api_client.dart';

class ModelSyncResult {
    final bool success;
    final bool downloaded;
    final String message;
    final Map<String, dynamic>?
remoteInfo;
    final Map<String, dynamic>?
localInfo;

    const ModelSyncResult({
        required this.success,
        required this.downloaded,
        required this.message,
        this.remoteInfo,
        this.localInfo,
    });
}

class ModelSyncService {
    static const _modelVersionKey =
'mobile_model_version';
    static const _modelPathKey =
'mobile_model_path';
    static const _trainedAtKey =
'mobile_model_trained_at';
    static const _formatKey =
'mobile_model_format';

```

```

static const _metricsJsonKey =
'mobile_model_metrics_json';
static const _sourceUrlKey =
'mobile_model_source_url';
static const _syncedAtKey =
'mobile_model_synced_at';

Future<ModelSyncResult>
trySyncLatestModel({bool force = false})
async {
    Map<String, dynamic>?
remoteInfo;

    try {
        remoteInfo = await
fetchLatestModelInfo();
        if (remoteInfo == null) {
            return ModelSyncResult(
                success: false,
                downloaded: false,
                message: 'На сервере
пока нет опубликованной мобильной
модели.',
                localInfo: await
readLocalModelInfo(),
            );
        }

        final latestVersion =
_asInt(remoteInfo['modelVersionId']);
        if (latestVersion == null) {
            return ModelSyncResult(
                success: false,
                downloaded: false,
                message: 'Сервер вернул
неполные данные о модели.',
                remoteInfo: remoteInfo,
                localInfo: await
readLocalModelInfo(),
            );
        }

        final localInfo = await
readLocalModelInfo();
        final currentVersion =
_asInt(localInfo?['modelVersionId']);
        final currentPath =
localInfo?['modelPath']?.toString();

```

```

        final hasLocalFile =
currentPath != null && await
File(currentPath).exists();

        if (!force && currentVersion
== latestVersion && hasLocalFile) {
            return ModelSyncResult(
                success: true,
                downloaded: false,
                message: 'Уже
используется актуальная версия ИИ.',
                remoteInfo: remoteInfo,
                localInfo: localInfo,
            );
        }

        final modelResponse = await
http.get(

Uri.parse('${ApiClient.baseUrl}/api/mobile
-models/latest/download'),
        );

        if
(modelResponse.statusCode != 200) {
            return ModelSyncResult(
                success: false,
                downloaded: false,
                message: 'Не удалось
скачать модель: HTTP
${modelResponse.statusCode}.',
                remoteInfo: remoteInfo,
                localInfo: await
readLocalModelInfo(),
            );
        }

        final modelsDir = await
_modelDirectory();

        final format =
(remoteInfo['mobileFormat']?.toString() ??
'tflite').toLowerCase();

        final remoteFileName =
remoteInfo['fileName']?.toString();

        final safeFileName =
_safeModelFileName(remoteFileName,
format, latestVersion);

```

```

        final file =
File('${modelsDir.path}/${safeFileName}');

        await
file.writeAsBytes(modelResponse.bodyByt
es, flush: true);

        await
FileKeyValueStore.setInt(_modelVersionK
ey, latestVersion);

        await
FileKeyValueStore.setString(_modelPathK
ey, file.path);

        await
FileKeyValueStore.setString(_trainedAtKe
y, remoteInfo['trainedAt']?.toString() ?? "");

        await
FileKeyValueStore.setString(_formatKey,
format);

        await
FileKeyValueStore.setString(_metricsJson
Key, remoteInfo['metricsJson']?.toString()
?? "");

        await
FileKeyValueStore.setString(_sourceUrlKe
y, ApiClient.baseUrl);

        await
FileKeyValueStore.setString(_syncedAtKe
y, DateTime.now().toIso8601String());

        final updatedLocalInfo =
await readLocalModelInfo();

        return ModelSyncResult(
            success: true,
            downloaded: true,
            message: 'Модель ИИ
обновлена до версии #${latestVersion}.',
            remoteInfo: remoteInfo,
            localInfo:
updatedLocalInfo,
        );
    } catch (e) {
        return ModelSyncResult(
            success: false,
            downloaded: false,
            message: 'Не удалось
обновить ИИ: $e',
            remoteInfo: remoteInfo,

```

```

        localInfo: await
readLocalModelInfo(),
    );
}

Future<Map<String,
dynamic>?> fetchLatestModelInfo() async
{
    try {
        final metaResponse = await
http.get(

Uri.parse('${ApiClient.baseUrl}/api/mobile
-models/latest'),
        );

        if (metaResponse.statusCode
!= 200) {
            return null;
        }

        final decoded =
jsonDecode(metaResponse.body);

        return decoded is
Map<String, dynamic> ? decoded : null;
    } catch (_) {
        return null;
    }
}

Future<Map<String,
dynamic>?> readLocalModelInfo() async {
    try {
        final version = await
FileKeyValueStore.getInt(_modelVersionK
ey);

        final path = await
FileKeyValueStore.getString(_modelPathK
ey);

        if (version == null || path ==
null || path.isEmpty) {
            return null;
        }

        final file = File(path);

```



```

        final exists = await
file.exists();

        return {
            'modelVersionId': version,
            'modelPath': path,
            'trainedAt': await
FileKeyValueStore.getString(_trainedAtKey),
            'mobileFormat': await
FileKeyValueStore.getString(_formatKey),
            'metricsJson': await
FileKeyValueStore.getString(_metricsJsonKey),
            'sourceUrl': await
FileKeyValueStore.getString(_sourceUrlKey),
            'syncedAt': await
FileKeyValueStore.getString(_syncedAtKey),
            'fileExists': exists,
            'fileSizeBytes': exists ?
await file.length() : null,
        };
    } catch (_) {
        return null;
    }
}

Future<String?>
localModelPath() async {
    try {
        final path = await
FileKeyValueStore.getString(_modelPathKey);

        if (path != null &&
path.isNotEmpty) {
            final file = File(path);
            if (await file.exists()) {
                return path;
            }
        }

        // Резервный вариант: если
метаданные не сохранились, но файл
модели есть,
        // берём самый свежий
.tflite/.lite из папки models.

```

```

        final modelsDir = await
_modelsDirectory();

        final files = await modelsDir
.list()
.where((entity) => entity is
File)

        .cast<File>()
        .where((file) {
            final name =
file.path.toLowerCase();
            return
name.endsWith('.tflite') ||
name.endsWith('.lite');
        })
        .toList();

        if (files.isEmpty) {
            return null;
        }

        files.sort((a, b) =>
b.lastModifiedSync().compareTo(a.lastMod
ifiedSync()));

        return files.first.path;
    } catch (_) {
        return null;
    }
}

Future<Directory>
_modelsDirectory() {
    return
FileKeyValueStore.namedDirectory('model
s');
}

int? _asInt(Object? value) {
    if (value is int) return value;
    if (value is num) return
value.toInt();
    return
int.tryParse(value?.toString() ?? "");
}

String
_safeModelFileName(String?
remoteFileName, String format, int version)
{

```

```

        final normalizedFormat =
format.trim().isEmpty ? 'tflite' :
format.trim().toLowerCase();

        final fallback =
'mobile_model_v$version.$normalizedFor
mat';

        if (remoteFileName == null ||
remoteFileName.trim().isEmpty) {
            return fallback;
        }

        final cleaned =
remoteFileName.split('/').last.split('\').last.tr
im();

        if (cleaned.isEmpty ||
cleaned.contains('.') ) {
            return fallback;
        }

        final dotIndex =
cleaned.lastIndexOf('.');

        final extension = dotIndex >=
0 ?
cleaned.substring(dotIndex).toLowerCase()
: '.$normalizedFormat';

        final base = dotIndex >= 0 ?
cleaned.substring(0, dotIndex) : cleaned;

        final safeBase =
base.replaceAll(RegExp(r'^A-Za-z0-9_-
]+'), '_');

        return
'model_v${version}_${safeBase}$extension';
    }
}

String prettyMetrics(String?
raw) {
    if (raw == null ||
raw.trim().isEmpty) {
        return 'Метрики пока не
сохранены.';
    }

    try {
        final decoded =
jsonDecode(raw);

        final encoder = const
JsonEncoder.withIndent(' ');

```

```

        return
encoder.convert(decoded);
    } catch (_) {
        return raw;
    }
}
}

=====

=====

=====

Файл: lib/auth/auth_state.dart
=====

=====

=====

import
'./storage/file_key_value_store.dart';

class AuthState {
    static const _tokenKey =
'access_token';

    String? _token;

    String? get token => _token;
    bool get isAuthenticated => _token !=
null && _token!.isNotEmpty;

    Future<void> load() async {
        try {
            _token = await
FileKeyValueStore.getString(_tokenKey);
        } catch (_) {
            _token = null;
        }
    }

    Future<void> setToken(String
token) async {
        _token = token;
        await
FileKeyValueStore.setString(_tokenKey,
token);
    }

    Future<void> logout() async {

```

```

        _token = null;
        await
FileKeyValueStore.remove(_tokenKey);
    }
}

=====

=====

Файл: lib/main.dart
=====

=====

import 'dart:async';
import 'dart:developer' as
developer;

import
'package:camera/camera.dart';
import
'package:flutter/material.dart';

import 'api/api_config.dart';
import
'api/model_sync_service.dart';
import 'auth/auth_state.dart';
import
'screens/scan_screen.dart';

List<CameraDescription>
cameras = const [];

Future<void> main() async {

WidgetsFlutterBinding.ensureInitialized();

String? startupError;

    try {
        await ApiConfig.load();
    } catch (e, st) {
        startupError = 'Не удалось
загрузить настройки API: $e';
        developer.log(startupError,
name: 'Startup', error: e, stackTrace: st);
    }

```

```

    }

    try {
        cameras = await
availableCameras();
    } catch (e, st) {
        startupError = 'Не удалось
получить доступ к камере: $e';
        developer.log(startupError,
name: 'Startup', error: e, stackTrace: st);
    }

    final auth = AuthState();
    try {
        await auth.load();
    } catch (e, st) {
        developer.log('Не удалось
загрузить авторизацию: $e', name:
'Startup', error: e, stackTrace: st);
    }

    // Не блокируем запуск
приложения синхронизацией модели.
    // Если сервер недоступен
или модель ещё не опубликована,
пользователь всё равно увидит
интерфейс.

    unawaited(ModelSyncService().trySyncLat
estModel());

    runApp(MyApp(auth: auth,
startupError: startupError));
}

class MyApp extends
StatelessWidget {
    final AuthState auth;
    final String? startupError;

    const MyApp( {super.key,
required this.auth, this.startupError});

    @override
    Widget build(BuildContext
context) {
        return MaterialApp(
            title: 'ProductsDate',

```

```

        theme:
ThemeData(useMaterial3: true),
        home: ScanScreen(auth:
auth, cameras: cameras, startupError:
startupError),
    );
    }
    }

=====
=====
=====
=====
Файл: lib/models/product.dart
=====
=====
=====
=====

class Product {
    final int id;
    final String name;
    final String? manufacturer;
    final String? barcode;

    Product({required this.id,
required this.name, this.manufacturer,
this.barcode});

    factory
Product.fromJson(Map<String, dynamic>
json) => Product(
        id: json['id'] as int,
        name: (json['name'] ??
'').toString(),
        manufacturer:
json['manufacturer']?.toString(),
        barcode:
json['barcode']?.toString(),
    );
    }

=====
=====
=====
=====
Файл:
lib/models/stored_product.dart

```

```

=====
=====
=====
=====
import 'product.dart';

class StoredProduct {
    final int id;
    final int productId;
    final DateTime?
manufactureAt;
    final DateTime? expiryAt;
    final DateTime createdAt;
    final Product? product;

    StoredProduct({
        required this.id,
        required this.productId,
        required this.createdAt,
        this.manufactureAt,
        this.expiryAt,
        this.product,
    });

    factory
StoredProduct.fromJson(Map<String,
dynamic> json) => StoredProduct(
        id: json['id'] as int,
        productId: json['productId']
as int,
        manufactureAt:
json['manufactureAt'] == null ? null :
DateTime.parse(json['manufactureAt']),
        expiryAt: json['expiryAt']
== null ? null :
DateTime.parse(json['expiryAt']),
        createdAt:
DateTime.parse(json['createdAt']),
        product: json['product'] ==
null ? null :
Product.fromJson(json['product']),
    );
    }

=====
=====

```

```

=====
=====
Файл:
lib/screens/cabinet_screen.dart
=====
=====
=====
=====
import
'package:flutter/material.dart';

import '../api/api_client.dart';
import '../auth/auth_state.dart';
import
'../widgets/manual_add_sheet.dart';
import 'settings_screen.dart';

class CabinetScreen extends
StatefulWidget {
    final AuthState auth;
    const
CabinetScreen({super.key, required
this.auth});

    @override
    State<CabinetScreen>
createState() => _CabinetScreenState();
    }

class _CabinetScreenState
extends State<CabinetScreen> {
    Map<String, dynamic>? _me;
    List<dynamic> _storage = [];
    String? _error;
    bool _loading = true;

    Future<void> _load() async {
        setState(() {
            _loading = true;
            _error = null;
        });

        try {
            final api = ApiClient(token:
widget.auth.token);
            _me = await api.me();

```

```

        _storage = await
api.storage();
    } catch (e) {
        _error = e.toString();
    } finally {
        if (mounted) setState(() =>
_loading = false);
    }
}

@override
void initState() {
    super.initState();
    _load();
}

Future<void>
_openManualAdd() async {
    final added = await
showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    builder: (_) =>
ManualAddSheet(auth: widget.auth),
);
    if (added == true) {
        await _load();
        if (mounted) {

ScaffoldMessenger.of(context).showSnack
Bar(
    const SnackBar(content:
Text('Продукт добавлен в личный
кабинет')),
);
    }
}

Future<void> _openSettings()
async {
    await Navigator.push(
        context,
        MaterialPageRoute(builder:
(_) => const SettingsScreen()),
    );
    await _load();
}

```

```

@override
Widget build(BuildContext
context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Личный
кабинет'),
            actions: [
                IconButton(
                    icon: const
Icon(Icons.settings),
                    onPressed:
_openSettings,
                ),
                IconButton(
                    icon: const
Icon(Icons.refresh),
                    onPressed: _load,
                ),
                IconButton(
                    icon: const
Icon(Icons.logout),
                    onPressed: () async {
                        await
widget.auth.logout();
                        if (mounted)
Navigator.pop(context);
                    },
                ),
            ],
            floatingActionButton:
FloatingActionButton(
                onPressed:
_openManualAdd,
                child: const
Icon(Icons.add),
            ),
            body: _loading
? const Center(child:
CircularProgressIndicator())
: (_error != null)
? Center(child:
Text(_error!, style: const TextStyle(color:
Colors.red)))
: ListView(

```

```

padding: const
EdgeInsets.all(16),
children: [
    ExpansionTile(
        initiallyExpanded:
true,
        title: const
Text('Данные профиля'),
        children: [
            ListTile(
                title: const
Text('Email'),
                subtitle:
Text(_me?['email']?.toString() ?? ''),
            ),
            ListTile(
                title: const
Text('Id'),
                subtitle:
Text(_me?['id']?.toString() ?? ''),
            ),
            ListTile(
                title: const
Text('Настройки'),
                subtitle: const
Text('BaseUrl, версия ИИ и обновление
модели'),
                trailing: const
Icon(Icons.chevron_right),
            ),
            onTap:
_openSettings,
        ),
    ],
    ),
    const
SizedBox(height: 12),
    const Text(
        'История
сканирований / На хранении',
        style:
TextStyle(fontSize: 16, fontWeight:
FontWeight.bold),
    ),
    const
SizedBox(height: 8),
    if
(_storage.isEmpty)

```

```

const Text('Пока
пусто. Добавь продукт вручную (+).'),
..._storage.map((x) {
  final product =
x['product'];

  final name =
product?['name']?.toString() ?? 'Без
названия';

  final rawExpiry =
x['expiryAt']?.toString();

  final expiry =
(rawExpiry == null || rawExpiry.isEmpty)
? '-'
:
rawExpiry.split('T').first;

  return Card(
    child: ListTile(
      title:
Text(name),
      subtitle:
Text('Срок годности: $expiry'),
    ),
  ),
),
}
}

```

```

Файл:
lib/screens/error_dataset_flow_screen.dart

```

```

import 'dart:async';
import 'dart:io';
import 'dart:math' as math;
import 'dart:ui' as ui;

import
'package:archive/archive_io.dart';

```

```

import
'package:camera/camera.dart';

import
'package:flutter/material.dart';

import
'package:image/image.dart' as img;

import '../api/api_client.dart';
import '../auth/auth_state.dart';
import
'../storage/file_key_value_store.dart';

class ErrorDatasetFlowScreen
extends StatefulWidget {
  final AuthState auth;
  final List<CameraDescription>
cameras;

  const
ErrorDatasetFlowScreen({super.key,
required this.auth, required this.cameras});

  @override

State<ErrorDatasetFlowScreen>
createState() =>
_ErrorDatasetFlowScreenState();

  static const int maxSeconds =
30;

  static const int fps = 3; // safer
frame capture rate for error reports
}

```

```

class
_ErrorDatasetFlowScreenState extends
State<ErrorDatasetFlowScreen> {
  CameraController? _controller;
  Future<void>? _initFuture;

  Timer? _limitTimer;
  bool _isRecording = false;
  bool _extracting = false;
  bool _uploading = false;
  bool _stopping = false;
  bool _recordingCompleted =
false;

```

```

bool _leaving = false;

Future<void>?
_captureLoopFuture;
int _captureIndex = 0;

XFile? _videoFile;

Directory? _workDir;
Directory? _imagesDir;
Directory? _labelsDir;

List<File> _frames = [];
int _index = 0;

// bboxes per frame in real
image pixel coords. One frame can contain
several date areas.

final Map<int, List<Rect>>
_bboxes = {};

final Set<int> _skipped = {};

final Map<int, Size>
_frameSizes = {};

final _comment =
TextEditingController();
String? _status;

@override
void initState() {
  super.initState();

  final back =
widget.cameras.where((c) =>
c.lensDirection ==
CameraLensDirection.back);

  final cam = back.isNotEmpty
? back.first : widget.cameras.first;

  // Audio is disabled
intentionally: error reports need frames
only.

  // This avoids
RECORD_AUDIO permission issues and
native crashes on some devices.

  _controller =
CameraController(cam,

```

```

ResolutionPreset.medium, enableAudio:
false);

    _initFuture =
_controller!.initialize();
    }

    Future<void> _releaseCamera()
async {
    _isRecording = false;
    final controller = _controller;
    _controller = null;
    _initFuture = null;

    if (controller == null) return;

    try {
        if
(controller.value.isInitialized &&
controller.value.isStreamingImages) {
            await
controller.stopImageStream();
        }
        } catch (_) {
            // Ignore camera stream races
during route disposal.
        }

        try {
            await controller.dispose();
        } catch (_) {
            // Ignore double-
dispose/native disposal races.
        }
    }

    Future<bool>
_prepareToLeave() async {
    if (_uploading || _stopping) {
        if (mounted) {

ScaffoldMessenger.of(context).showSnack
Bar(
            const SnackBar(content:
Text('Дождись завершения текущей
операции')),
        );
    }
    return false;
}

    }

    if (_leaving) return false;
    _leaving = true;
    _limitTimer?.cancel();
    _isRecording = false;

    if (mounted) {
        setState(() => _status =
'Освобождаю камеру...');
    }

    final loop =
_captureLoopFuture;
    if (loop != null) {
        try {
            await loop.timeout(const
Duration(seconds: 5), onTimeout: () {});
        } catch (_) {
            // The loop can fail if native
camera is already being disposed.
        }
    }

    await _releaseCamera();
    return true;
}

@override
void dispose() {
    _limitTimer?.cancel();
    _isRecording = false;
    unawaited(_releaseCamera());
    _comment.dispose();
    super.dispose();
}

Future<void> _startRecording()
async {
    try {
        await _initFuture;
        if (_controller == null ||
!_controller!.value.isInitialized) {
            setState(() => _status =
'Камера ещё не готова.');
```

```

        final reportsDir = await
FileKeyValueStore.namedDirectory('error_r
eports');

        final root =
Directory('${reportsDir.path}/error_report_
${DateTime.now().millisecondsSinceEpoch
}');

        await root.create(recursive:
true);

        final images =
Directory('${root.path}/images');
        final labels =
Directory('${root.path}/labels');
        await images.create(recursive:
true);
        await labels.create(recursive:
true);

        _workDir = root;
        _imagesDir = images;
        _labelsDir = labels;
    }

    Future<void>
_captureFramesLoop() async {
        final interval =
Duration(milliseconds: (1000 /
ErrorDatasetFlowScreen.fps).round());

        while (mounted &&
_isRecording) {
            try {
                final controller =
_controller;

                final images = _imagesDir;
                if (controller == null ||
images == null ||
!controller.value.isInitialized) {
                    break;
                }

                final shot = await
controller.takePicture();
                final index =
_captureIndex++;

```

```

        final frameFile =
File('${images.path}/${_frameName(index
)}');

        await
_normalizeCameraJpeg(File(shot.path),
frameFile);

        if (!mounted) return;
        setState() {
            _frames.add(frameFile);
            _status = 'Запись кадров:
${_frames.length}... Нажми «Стоп», когда
ошибка попала в кадр.';
        };
    } catch (e) {
        if (!mounted) return;
        setState(() => _status = 'Не
удалось сохранить один кадр,
продолжаю: $e');
        await Future.delayed(const
Duration(milliseconds: 700));
    }

    await
Future.delayed(interval);
}

Future<void>
_normalizeCameraJpeg(File source, File
destination) async {
    try {
        final bytes = await
source.readAsBytes();
        final decoded =
img.decodeImage(bytes);
        if (decoded == null) {
            await
source.copy(destination.path);
        } else {
            // Android camera JPEGs
can contain EXIF orientation. Flutter may
display the
            // rotated image while
backend/training reads raw pixels, which
makes boxes

```

```

        // appear shifted. Baking
orientation makes preview pixels and saved
labels

        // use the same coordinate
system.

        final normalized =
img.bakeOrientation(decoded);

        await
destination.writeAsBytes(img.encodeJpg(no
rmalized, quality: 92), flush: true);
    } finally {
        try {
            if (source.path !=
destination.path && await source.exists()) {
                await source.delete();
            }
        } catch (_) {
            // Temporary camera files
are cleaned by the OS if deletion is
unavailable.
        }
    }

    Future<void> _stopRecording()
async {
        if (!_isRecording || _stopping)
return;

        _limitTimer?.cancel();
        setState(() {
            _isRecording = false;
            _stopping = true;
            _extracting = true;
            _status = 'Останавливаю
запись и подготавливаю кадры...';
        });

        try {
            final loop =
_captureLoopFuture;
            if (loop != null) {
                await loop.timeout(const
Duration(seconds: 12), onTimeout: () {});
            }

            if (_frames.isEmpty) {

```

```

        throw Exception('Не
        удалось сохранить ни одного кадра.
        Попробуй записать чуть дольше.');
```

 }

 final sizes = <int, Size>{};
 for (var i = 0; i <
 _frames.length; i++) {
 sizes[i] = await
 _readImageSize(_frames[i]);
 }

 await _releaseCamera();

 if (!mounted) return;
 setState(() {
 _frameSizes
 ..clear()
 ..addAll(sizes);
 _index = 0;
 _recordingCompleted =
 true;

 _status = 'Кадры готовы:
 \${_frames.length}. Размечай по очереди.';
 });
 } catch (e) {
 if (!mounted) return;
 setState(() {
 _recordingCompleted =
 false;

 _status = 'Ошибка после
 остановки записи: \$e';
 });
 } finally {
 if (mounted) {
 setState(() {
 _stopping = false;
 _extracting = false;
 });
 }
 }
}

bool get _allAnnotated =>
 _frames.isNotEmpty &&

 List<int>.generate(_frames.length, (i) =>
 i).every((i) {

```

        if (_skipped.contains(i))

    return true;

    final boxes = _bboxes[i];
    return boxes != null &&
    boxes.isNotEmpty;
    });

    int get _usableFramesCount =>
    _frames.length - _skipped.length;

    String _frameName(int index) {
        final i = index + 1;
        return
        'frame_${i.toString().padLeft(5, '0')}.jpg';
    }

    Future<Size>
    _readImageSize(File file) async {
        final bytes = await
        file.readAsBytes();

        final completer =
        Completer<Size>();

        ui.decodeImageFromList(bytes, (image) {

            completer.complete(Size(image.width.toDouble(),
            image.height.toDouble()));

        });

        return completer.future;
    }

    Future<Size>
    _frameSizeForIndex(int index) async {
        final cached =
        _frameSizes[index];

        if (cached != null) return
        cached;

        final size = await
        _readImageSize(_frames[index]);
        _frameSizes[index] = size;
        return size;
    }

    Future<void>
    _writeYoloLabel(int index, List<Rect>
    rects) async {
        final labels = _labelsDir!;

```

```

        final name =
        _frameName(index).replaceAll('.jpg', '.txt');

        final path =
        '${labels.path}/${name}';

        final imageSize = await
        _frameSizeForIndex(index);

        final width =
        imageSize.width;

        final height =
        imageSize.height;

        double clampX(double v) =>
        v < 0 ? 0 : (v > width ? width : v);

        double clampY(double v) =>
        v < 0 ? 0 : (v > height ? height : v);

        final lines = <String>[];
        for (final r in rects) {
            final left = clampX(r.left);
            final top = clampY(r.top);
            final right = clampX(r.right);
            final bottom =
            clampY(r.bottom);

            final w = right - left;
            final h = bottom - top;
            if (w < 2 || h < 2) continue;

            final cx = left + w / 2.0;
            final cy = top + h / 2.0;

            final xc = cx / width;
            final yc = cy / height;
            final wn = w / width;
            final hn = h / height;

            lines.add('0
            ${xc.toStringAsFixed(6)}
            ${yc.toStringAsFixed(6)}
            ${wn.toStringAsFixed(6)}
            ${hn.toStringAsFixed(6)}');
        }

        if (lines.isEmpty) {
            throw Exception('На кадре
            нет валидных областей разметки.');
```



```

        final content = lines.join("\n")
+ "n";
        await
File(path).writeAsString(content, flush:
true);
    }

    Future<void>
_saveCurrentAndNext() async {
        final boxes =
_bboxes[_index];
        if (boxes == null ||
boxes.isEmpty) return;

        _skipped.remove(_index);
        await
_writeYoloLabel(_index, boxes);

        if (_index < _frames.length -
1) {
            setState(() => _index++);
        } else {
            setState(() => _status =
'Разметка завершена. Можно
отправлять.');
```

```

    }
    }

    Future<void>
_skipCurrentFrame() async {
        final labels = _labelsDir;
        final current = _index;

        _bboxes.remove(current);
        _skipped.add(current);

        if (labels != null) {
            final labelFile =
File('${labels.path}/${_frameName(current)
}.replaceAll('.jpg', '.txt')');
            try {
                if (await labelFile.exists())
await labelFile.delete();
            } catch (_) {
                // If deletion fails, the ZIP
step still ignores skipped frames.
            }
        }
    }
}

    }

    if (!mounted) return;
    if (current < _frames.length -
1) {
        setState() {
            _index++;
            _status = 'Кадр ${current +
1} пропущен.';
        });
    } else {
        setState(() => _status =
'Последний кадр пропущен. Можно
отправлять, если есть размеченные
кадры.');
```

```

    }
    }

    Future<String> _zipDataset()
async {
        final root = _workDir!;
        final zipPath =
'${root.path}/dataset.zip';

        if (_usableFramesCount <= 0)
{
            throw Exception('Все кадры
пропущены. Оставь хотя бы один
размеченный кадр.');
```

```

    }
    }

    for (var i = 0; i <
_frames.length; i++) {
        if (_skipped.contains(i))
continue;

        final boxes = _bboxes[i];
        if (boxes == null ||
boxes.isEmpty) {
            throw Exception('Не все
кадры размечены или пропущены.');
```

```

        }
        }

        await _writeYoloLabel(i,
boxes);
    }

    final imageFiles = <File>[];
    final labelFiles = <File>[];
    for (var i = 0; i <
_frames.length; i++) {
        if (_skipped.contains(i))
continue;
        imageFiles.add(_frames[i]);

        labelFiles.add(File('${_labelsDir!.path}/${_
frameName(i).replaceAll('.jpg', '.txt')}');
```

```

    }
    }

    debugPrint('ZIP INPUT
IMAGES FINAL: ${imageFiles.length}');
    debugPrint('ZIP INPUT
LABELS FINAL: ${labelFiles.length}');

    final encoder =
ZipFileEncoder();
    encoder.create(zipPath);

    for (final f in imageFiles) {
        encoder.addFile(f,
'images/${f.uri.pathSegments.last}');
```

```

    }
    }

    for (final f in labelFiles) {
        encoder.addFile(f,
'labels/${f.uri.pathSegments.last}');
```

```

    }
    }

    final meta =
File('${root.path}/meta.json');
    await meta.writeAsString(
'{"fps":${ErrorDatasetFlowScreen.fps},"fra
mes":${_frames.length},"preserveAspect":t
rue}',
        flush: true,
    );
    encoder.addFile(meta,
'meta.json');

    encoder.close();

    final zipCheck =
ZipDecoder().decodeBytes(await
File(zipPath).readAsBytes());

```

```

        debugPrint('ZIP ENTRIES
COUNT: ${zipCheck.files.length}');
        for (final entry in
zipCheck.files) {
            debugPrint('ZIP ENTRY:
${entry.name}');
        }
        return zipPath;
    }

    Future<void> _upload() async
{
    if (!_allAnnotated) {

ScaffoldMessenger.of(context).showSnack
Bar(const SnackBar(content: Text('Разметь
или пропусти все кадры')));
        return;
    }

    if (_usableFramesCount <= 0)
{

ScaffoldMessenger.of(context).showSnack
Bar(const SnackBar(content: Text('Нельзя
отправить отчёт без размеченных
кадров')));
        return;
    }

    setState(() {
        _uploading = true;
        _status = 'Архивация и
отправка...';
    });

    try {
        final zipPath = await
_zipDataset();
        final api = ApiClient(token:
widget.auth.token);
        await
api.uploadDatasetZip(zipPath, comment:
_comment.text.trim());

        await _releaseCamera();
        if (!mounted) return;

```

```

ScaffoldMessenger.of(context).showSnack
Bar(const SnackBar(content:
Text('Отправлено ☒'));
        Navigator.of(context).pop();
    } catch (e) {
        if (!mounted) return;

ScaffoldMessenger.of(context).showSnack
Bar(SnackBar(content: Text('Ошибка:
${e}')));
        setState(() => _status =
'Ошибка: ${e}');
    } finally {
        if (mounted) setState(() =>
_uploading = false);
    }
}

void
_setBBoxesForCurrent(List<Rect> rects) {
    setState(() {
        if (rects.isEmpty) {
            _bboxes.remove(_index);
        } else {
            _skipped.remove(_index);
            _bboxes[_index] =
List<Rect>.unmodifiable(rects);
        }
    });
}

@override
Widget build(BuildContext
context) {
    final readyToAnnotate =
_recordingCompleted &&
_frames.isNotEmpty;
    final currentFile =
readyToAnnotate ? _frames[_index] : null;

    return PopScope(
        canPop: false,
        onPopInvokedWithResult:
        (didPop, result) async {
            if (didPop) return;
            final ok = await
_prepareToLeave();

```

```

        if (ok && context.mounted)
{
    Navigator.of(context).pop(result);
    },
    child: Scaffold(
        appBar: AppBar(title: const
Text('Сообщить об ошибке')),
        body: FutureBuilder(
            future: _initFuture,
            builder: (context, snap) {
                final controller =
_controller;

                if (!readyToAnnotate &&
(controller == null || _initFuture == null)) {
                    return Center(child:
Text(_status ?? 'Камера
освобождается...'));
                }

                if (!readyToAnnotate &&
snap.connectionState !=
ConnectionState.done) {
                    return const
Center(child: CircularProgressIndicator());
                }

                return ListView(
                    padding: const
EdgeInsets.all(16),
                    children: [
                        if (!readyToAnnotate)
                        AspectRatio(
                            aspectRatio:
controller!.value.aspectRatio,
                            child:
CameraPreview(
                                controller,
                                key:
ValueKey('report-camera-
${controller.hashCode}'),
                            ),
                        ),
                        const
SizedBox(height: 12),
                        Row(

```

```

        children: [
          Expanded(
            child:
ElevatedButton.icon(
              onPressed:
(_extracting || _isRecording || _stopping) ?
null : _startRecording,
              icon: const
Icon(Icons.fiber_manual_record),
              label: const
Text("Начать запись"),
            ),
          ),
        ],
      ),
      const
SizedBox(width: 12),
      Expanded(
        child:
ElevatedButton.icon(
          onPressed:
(_isRecording && !_stopping) ?
_stopRecording : null,
          icon: const
Icon(Icons.stop),
          label: const
Text("Стоп"),
        ),
      ),
    ],
  ),
  const
SizedBox(height: 8),
  const Text("Лимит
записи: 30 секунд. Приложение
сохраняет кадры напрямую, без FFmpeg,
чтобы не падать на stop."),
  const
SizedBox(height: 8),
  if (_status != null)
Text(_status!),
  if (_extracting) const
Padding(padding: EdgeInsets.only(top: 12),
child: LinearProgressIndicator()),
] else ...[
  Text("Кадр ${_index +
1} / ${_frames.length}' +
(_skipped.contains(_index) ? ' —
пропущен' : ")),

```

```

const
SizedBox(height: 12),
_Annotator(
  imageFile:
currentFile!,
  initial:
_bboxes[_index] ?? const [],
  onChanged:
_setBBoxesForCurrent,
),
const
SizedBox(height: 12),
TextField(
  controller:
_comment,
  decoration: const
InputDecoration(
    labelText:
'Комментарий (опционально)',
    border:
OutlineInputBorder(),
  ),
  minLines: 2,
  maxLines: 4,
),
const
SizedBox(height: 12),
Row(
  children: [
    Expanded(
      child:
OutlinedButton(
        onPressed: _index
> 0 ? () => setState(() => _index--) : null,
        child: const
Text("Назад"),
      ),
    ),
  ],
  const
SizedBox(width: 8),
  OutlinedButton.icon(
    onPressed:
_skipCurrentFrame,
    icon: const
Icon(Icons.skip_next),

```

```

label: const
Text("Пропустить"),
),
),
],
),
const
SizedBox(height: 8),
SizedBox(
  width:
double.infinity,
  child:
ElevatedButton(
    onPressed:
(_bboxes[_index]?.isEmpty ?? false) ?
_saveCurrentAndNext : null,
    child: Text(_index
== _frames.length - 1 ? 'Завершить' :
'Сохранить и далее'),
  ),
),
const
SizedBox(height: 12),
ElevatedButton.icon(
  onPressed:
(_uploading || !_allAnnotated ||
_usableFramesCount <= 0) ? null : _upload,
  icon: const
Icon(Icons.upload),
  label:
Text(_uploading ? 'Отправка...' :
'Отправить'),
),
const
SizedBox(height: 8),
Text("Размечено
кадров: ${_bboxes.length}, областей:
${_bboxes.values.fold<int>(0, (sum, boxes)
=> sum + boxes.length)}, пропущено:
${_skipped.length}, всего:
${_frames.length}'),
  if (_status != null)
Text(_status!),
],
],
);
},
),

```

```

    ),
    );
}

class _Annotator extends
StatefulWidget {
    final File imageFile;
    final List<Rect> initial;
    final
ValueChanged<List<Rect>> onChanged;

    const _Annotator({required
this.imageFile, required this.initial, required
this.onChanged});

    @override
    State<_Annotator>
createState() => _AnnotatorState();
}

class _AnnotatorState extends
State<_Annotator> {
    Offset? _start;
    Rect? _draftRect;
    late List<Rect> _rects;
    late Future<Size>
_imageSizeFuture;

    @override
    void initState() {
        super.initState();
        _rects =
List<Rect>.from(widget.initial);
        _imageSizeFuture =
_readImageSize(widget.imageFile);
    }

    @override
    void
didUpdateWidget(covariant _Annotator
oldWidget) {

        super.didUpdateWidget(oldWidget);
        if (oldWidget.imageFile.path
!= widget.imageFile.path) {
            _rects =
List<Rect>.from(widget.initial);
            _draftRect = null;
            _start = null;
            _imageSizeFuture =
_readImageSize(widget.imageFile);
        } else if
(!_sameRects(oldWidget.initial,
widget.initial)) {
            _rects =
List<Rect>.from(widget.initial);
            _draftRect = null;
        }

        bool _sameRects(List<Rect> a,
List<Rect> b) {
            if (a.length != b.length) return
false;
            for (var i = 0; i < a.length;
i++) {
                if (a[i] != b[i]) return false;
            }
            return true;
        }

        Future<Size>
_readImageSize(File file) async {
            final bytes = await
file.readAsBytes();
            final completer =
Completer<Size>();
            ui.decodeImageFromList(bytes, (image) {
                completer.complete(Size(image.width.toDo
uble(), image.height.toDouble()));
            });
            return completer.future;
        }

        Rect _rectFrom(Offset a, Offset
b) {
            final left = math.min(a.dx,
b.dx);
            final top = math.min(a.dy,
b.dy);
            final right = math.max(a.dx,
b.dx);
            final bottom = math.max(a.dy,
b.dy);

            return Rect.fromLTRB(left,
top, right, bottom);
        }

        void _publish() =>
widget.onChanged(List<Rect>.unmodifiable(
_rects));

        void _clearAll() {
            setState(() {
                _rects.clear();
                _draftRect = null;
                _start = null;
            });
            _publish();
        }

        void _undoLast() {
            if (_rects.isEmpty) return;
            setState(() =>
_rects.removeLast());
            _publish();
        }

        @override
        Widget build(BuildContext
context) {
            return FutureBuilder<Size>(<
future: _imageSizeFuture,
builder: (context, snap) {
                if (!snap.hasData) {
                    return const AspectRatio(
aspectRatio: 1,
child: Center(child:
CircularProgressIndicator()),
);
                }

                final imageSize =
snap.data!;
                return LayoutBuilder(
builder: (context,
constraints) {
                    final maxWidth =
math.min(constraints.maxWidth, 360.0);

```



```

        FilledButton.tonal(
            onPressed:
            _rects.isEmpty && _draftRect == null ?
            null : _clearAll,
            child: const
            Text('Сброс'),
        ),
    ],
),
),
),
),
);
},
);
}
}

class _RectPainter extends
CustomPainter {
    final List<Rect> rects;
    final Rect? draft;

    _RectPainter({required
this.rects, required this.draft});

    @override
    void paint(Canvas canvas, Size
size) {
        final paint = Paint()
        ..style = PaintingStyle.stroke
        ..strokeWidth = 3
        ..color = const
Color(0xFF1565C0);

        for (final rect in rects) {
            canvas.drawRect(rect, paint);
        }

        final draftRect = draft;
        if (draftRect != null) {
            final draftPaint = Paint()
            ..style =
PaintingStyle.stroke
            ..strokeWidth = 2
            ..color = const
Color(0xFF00C853);

            canvas.drawRect(draftRect,
draftPaint);
        }
    }

    @override
    bool shouldRepaint(covariant
_RectPainter oldDelegate) =>
        oldDelegate.draft != draft ||
        !_sameRects(oldDelegate.rects, rects);

    bool _sameRects(List<Rect> a,
List<Rect> b) {
        if (a.length != b.length) return
false;

        for (var i = 0; i < a.length;
i++) {
            if (a[i] != b[i]) return false;
        }

        return true;
    }
}

=====
Файл:
lib/screens/login_screen.dart
=====

import
'package:flutter/material.dart';
import '../auth/auth_state.dart';
import '../api/api_client.dart';
import '../api/api_config.dart';
import 'register_screen.dart';
import 'settings_screen.dart';

class LoginScreen extends
StatefulWidget {
    final AuthState auth;

    final String after;
    const LoginScreen({super.key,
required this.auth, required this.after});

    @override
    State<LoginScreen>
createState() => _LoginScreenState();
}

class _LoginScreenState extends
State<LoginScreen> {
    final _email =
TextEditingController();
    final _pass =
TextEditingController();

    String? _error;
    bool _loading = false;

    Future<void> _openSettings()
async {
        await Navigator.push(
            context,
            MaterialPageRoute(builder:
(_) => const SettingsScreen()),
        );
        if (mounted) setState(() {});
    }

    Future<void> _login() async {
        setState(() {
            _loading = true;
            _error = null;
        });

        try {
            final api = ApiClient(token:
null);

            final token = await
api.login(_email.text, _pass.text);
            await
widget.auth.setToken(token);

            if (mounted)
Navigator.pop(context);
        } catch (e) {
            setState(() => _error =
e.toString());
        }
    }
}

```

```

    } finally {
      if (mounted) setState(() =>
_loading = false);
    }
  }

  @override
  void dispose() {
    _email.dispose();
    _pass.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext
context) {
    return Scaffold(
      appBar: AppBar(
        title: const
Text('Авторизация'),
        actions: [
          IconButton(
            onPressed:
_openSettings,
            icon: const
Icon(Icons.settings),
            tooltip: 'Настройки API',
          ),
        ],
        body: Padding(
          padding: const
EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _email,
                decoration: const
InputDecoration(labelText: 'Email'),
                keyboardType:
TextInputType.emailAddress,
              ),
              TextField(
                controller: _pass,
                decoration: const
InputDecoration(labelText: 'Пароль'),
                obscureText: true,
              ),
            ],
          ),
        ),
      ),
    ),
  ),

```

```

    ),
    const SizedBox(height:
12),
    Card(
      child: ListTile(
        contentPadding: const
EdgeInsets.symmetric(horizontal: 12,
vertical: 4),
        leading: const
Icon(Icons.lan),
        title: const
Text('Адрес API'),
        subtitle:
Text(ApiConfig.baseUrl),
        trailing: TextButton(
          onPressed: _loading ?
null : _openSettings,
          child: const
Text('Изменить'),
        ),
      ),
    ),
    const SizedBox(height:
12),
    if (_error != null)
Text(_error!, style: const TextStyle(color:
Colors.red)),
    const SizedBox(height:
12),
    ElevatedButton(
      onPressed: _loading ?
null : _login,
      child: Text(_loading ?
'Вход...' : 'Войти'),
    ),
    TextButton(
      onPressed: () =>
Navigator.push(
        context,
        MaterialPageRoute(builder: (_) =>
RegisterScreen(auth: widget.auth)),
      ),
    ),
    child: const Text('Нет
аккаунта? Регистрация'),
  ],
),
),

```

```

    );
  }
}

=====
=====
=====
Файл:
lib/screens/register_screen.dart
=====
=====
=====
import
'package:flutter/material.dart';
import '../auth/auth_state.dart';
import '../api/api_client.dart';

class RegisterScreen extends
StatefulWidget {
  final AuthState auth;
  const
RegisterScreen({super.key, required
this.auth});

  @override
  State<RegisterScreen>
createState() => _RegisterScreenState();
}

class _RegisterScreenState
extends State<RegisterScreen> {
  final _email =
TextEditingController();
  final _pass =
TextEditingController();

  String? _error;
  bool _loading = false;

  Future<void> _register() async
{
  setState(() {
    _loading = true;
    _error = null;
  });
}

```

```

        try {
          final api = ApiClient(token:
null);

          final token = await
api.register(_email.text, _pass.text);

          await
widget.auth.setToken(token);

          if (mounted)
Navigator.pop(context);
        } catch (e) {
          setState(() => _error =
e.toString());
        } finally {
          if (mounted) setState(() =>
_loading = false);
        }
      }

      @override
      void dispose() {
        _email.dispose();
        _pass.dispose();
        super.dispose();
      }

      @override
      Widget build(BuildContext
context) {
        return Scaffold(
          appBar: AppBar(title: const
Text("Регистрация")),
          body: Padding(
            padding: const
EdgeInsets.all(16),
            child: Column(
              children: [
                TextField(
                  controller: _email,
                  decoration: const
InputDecoration(labelText: 'Email'),
                  keyboardType:
TextInputType.emailAddress,
                ),
                TextField(
                  controller: _pass,
                    decoration: const
InputDecoration(labelText: 'Пароль'),
                    obscureText: true,
                  ),
                const SizedBox(height:
12),
                if (_error != null)
Text(_error!, style: const TextStyle(color:
Colors.red)),
                const SizedBox(height:
12),
                ElevatedButton(
                  onPressed: _loading ?
null : _register,
                  child: Text(_loading ?
'Создание...' : 'Создать аккаунт'),
                ),
              ],
            ),
          ),
        );
      }
    }

    =====
    =====
    =====
    Файл:
lib/screens/scan_screen.dart
    =====
    =====
    =====

    import 'dart:async';
    import 'dart:io';
    import 'dart:math' as math;

    import
'package:camera/camera.dart';
    import
'package:flutter/material.dart';
    import
'package:flutter/painting.dart';
    import
'package:image/image.dart' as img;

    import
'../ai/date_reader_tflite.dart';
    import
'../ai/detection_result.dart';
    import
'../ai/tflite_date_detector.dart';
    import
'../api/model_sync_service.dart';
    import '../auth/auth_state.dart';
    import
'../widgets/manual_add_sheet.dart';
    import 'cabinet_screen.dart';
    import
'error_dataset_flow_screen.dart';
    import 'login_screen.dart';
    import 'settings_screen.dart';

    class ScanScreen extends
StatefulWidget {
      final AuthState auth;
      final List<CameraDescription>
cameras;
      final String? startupError;

      const ScanScreen({
        super.key,
        required this.auth,
        required this.cameras,
        this.startupError,
      });

      @override
      State<ScanScreen>
createState() => _ScanScreenState();
    }

    class _ScanScreenState extends
State<ScanScreen> with
WidgetsBindingObserver {
      CameraController? _controller;
      Future<void>? _initFuture;
      CameraDescription?
_selectedCamera;

      final _modelSync =
ModelSyncService();

      late final TfliteDateDetector
_detector;

```


| | | |
|--|---|---------------------------------------|
| late final DateReaderTflite | @override | _resumeRequestedAfterInit |
| _dateReader; | void initState() { | = true; |
| | super.initState(); | return; |
| | | } |
| bool _aiReady = false; | WidgetsBinding.instance.addObserver(this) | |
| bool _aiBusy = false; | ; | _initializingCamera = true; |
| bool _scanActive = false; | | |
| bool _coveredByChildRoute = | _detector = | |
| false; | TfliteDateDetector(modelSyncService: | try { |
| | _modelSync); | if (forceRecreate) { |
| bool _initializingCamera = | _dateReader = | await |
| false; | DateReaderTflite.instance; | _disposeCameraController(updateState: |
| | if (widget.startupError != null | false); |
| _resumeRequestedAfterInit = false; | && widget.startupError!.isNotEmpty) { | } |
| int _cameraEpoch = 0; | _aiStatus = | |
| int _previewKeySeed = 0; | widget.startupError!; | if (widget.cameras.isEmpty) |
| String _aiStatus = 'Подготовка | } | { |
| III...'; | _initialize(); | if (mounted) { |
| | } | setState(() => _aiStatus = |
| DateTime? _lastInferenceAt; | | 'Камера не найдена или нет разрешения |
| Duration? | | на камеру.'); |
| _lastInferenceDuration; | | } |
| | Future<void> _refreshAiState() | |
| FramePerf _lastPerf = const | async { | return; |
| FramePerf(); | _localModelInfo = await | } |
| List<DetectionResult> | _modelSync.readLocalModelInfo(); | |
| _detections = const []; | final ready = await | |
| bool _magnifierBusy = false; | _detector.loadLatestModel(); | |
| int _magnifierEpoch = 0; | | if (_coveredByChildRoute) { |
| File? _magnifierImageFile; | | return; |
| Rect? _magnifierCropRect; | if (!mounted) return; | } |
| Size? _magnifierImageSize; | | |
| bool | setState() { | if (!forceRecreate && |
| _magnifierNeedsInitialTransform = false; | _aiReady = ready; | _controller != null && |
| final TransformationController | if (!ready) { | _controller!.value.isInitialized) { |
| _magnifierTransformController = | _aiStatus = 'Локальная | await _refreshAiState(); |
| TransformationController(); | модель не найдена. Зайди в настройки и | return; |
| | обнови версию III.'; | } |
| String _magnifierStatus = "; | | |
| DetectionResult? | _detections = const []; | |
| _magnifierSourceDetection; | _lastInferenceAt = null; | final back = |
| DateTime? | } | widget.cameras.where((c) => |
| _magnifierRecognizedDate; | }); | c.lensDirection == |
| String? | | CameraLensDirection.back); |
| _magnifierRecognizedDateText; | if (ready) { | final cam = back.isNotEmpty |
| bool _manualAddSheetOpen = | await _startImageStream(); | ? back.first : widget.cameras.first; |
| false; | } | _selectedCamera = cam; |
| Map<String, dynamic>? | } | |
| _localModelInfo; | | |
| DateTime _lastInferenceStarted | Future<void> _initialize({bool | final localEpoch = |
| = | forceRecreate = false}) async { | ++_cameraEpoch; |
| DateTime.fromMillisecondsSinceEpoch(0); | if (_initializingCamera) { | final controller = |
| | | CameraController(|
| | | cam, |

```

        ResolutionPreset.low,
        enableAudio: false,
        imageFormatGroup:
ImageFormatGroup.yuv420,
    );

    _controller = controller;
    _initFuture =
controller.initialize();
    _previewKeySeed++;
    if (mounted) {
        setState(() {
            _detections = const [];
            _lastInferenceAt = null;
            _aiStatus = 'Запускаю
камеру...';
        });
    }

    await _initFuture;

    if (!mounted ||
_coveredByChildRoute || localEpoch !=
_cameraEpoch) {
        try {
            await controller.dispose();
        } catch (_) {
            // Controller was
superseded or the route is covered; native
camera will be released by dispose.
        }
        return;
    }

    _localModelInfo = await
_modelSync.readLocalModelInfo();
    final ready = await
_detector.loadLatestModel();

    if (!mounted ||
_coveredByChildRoute || localEpoch !=
_cameraEpoch) return;

    setState(() {
        _aiReady = ready;
        _aiStatus = ready
        ? 'ИИ готова'

```

```

        : 'Локальная модель не
найдена. Зайди в настройки и обнови
версию ИИ.';
    });

    if (ready) {
        await
_startImageStream(expectedEpoch:
localEpoch);
    }
    } catch (e) {
        await
_disposeCameraController(updateState:
false);

        if (!mounted) return;
        setState(() {
            _controller = null;
            _initFuture = null;
            _aiReady = false;
            _scanActive = false;
            _aiStatus = 'Ошибка
запуска камеры: $e';
        });
    } finally {
        _initializingCamera = false;

        if
(_resumeRequestedAfterInit && mounted
&& !_coveredByChildRoute) {
            _resumeRequestedAfterInit
= false;

            unawaited(_resumeScanning(recreateCamer
a: true));

        } else {
            _resumeRequestedAfterInit
= false;
        }
    }

    Future<void>
_stopImageStreamIfNeeded() async {
        final controller = _controller;
        if (controller == null ||
!controller.value.isInitialized ||
!controller.value.isStreamingImages) {
            return;
        }
    }

```

```

    try {
        await
controller.stopImageStream();
    } catch (_) {
        // The camera plugin may
throw if the stream is already
stopping/disposed.
    }

    Future<void>
_disposeCameraController({bool
updateState = true}) async {
        _scanActive = false;
        _aiBusy = false;
        _cameraEpoch++;

        final controller = _controller;
        _controller = null;
        _initFuture = null;
        _previewKeySeed++;

        if (updateState && mounted)
        {
            setState(() {
                _detections = const [];
                _lastInferenceAt = null;
            });
        }

        if (controller == null) return;

        try {
            if
(controller.value.isInitialized &&
controller.value.isStreamingImages) {
                await
controller.stopImageStream();
            }
        } catch (_) {
            // Ignore stop errors during
disposal; disposing releases the native
camera.
        }

        try {
            await controller.dispose();
        }
    }

```

```

    } catch (_) {
        // Ignore double-
dispose/native disposal races.
    }
}

Future<void>
_pauseScanning({bool releaseCamera =
false}) async {
    _scanActive = false;
    _aiBusy = false;
    _cameraEpoch++;

    if (mounted) {
        setState(() {
            _detections = const [];
            _aiStatus = releaseCamera ?
'Камера освобождена для другого
экрана.' : 'Сканирование
приостановлено.';
        });
    }

    if (releaseCamera) {
        await
_disposeCameraController(updateState:
true);
    } else {
        await
_stopImageStreamIfNeeded();
    }
}

Future<void>
_resumeScanning({required bool
recreateCamera}) async {
    if (!mounted ||
_coveredByChildRoute || _magnifierActive)
return;

    // Ждем первого
кадра после закрытия route, чтобы
CameraPreview получил новый Surface,
    // а предыдущий экран успел
освободить CameraX use cases до
инициализации новой камеры.

    await
WidgetsBinding.instance.endOfFrame;

```

```

    if (!mounted ||
_coveredByChildRoute) return;

    if (recreateCamera ||
_controller == null ||
!_controller!.value.isInitialized) {
        await
_initialize(forceRecreate: recreateCamera);
    } else {
        await _refreshAiState();
    }
}

Future<T?>
_pushWithScanPaused<T>(Route<T>
route, {bool releaseCamera = false}) async
{
    _coveredByChildRoute = true;
    await
_pauseScanning(releaseCamera:
releaseCamera);

    try {
        return await
Navigator.push(context, route);
    } finally {
        _coveredByChildRoute =
false;
        await
_resumeScanning(recreateCamera:
releaseCamera);
    }
}

Future<T?>
_showModalWithScanPaused<T>(Future<
T?> Function() show) async {
    _coveredByChildRoute = true;
    await
_pauseScanning(releaseCamera: false);

    try {
        return await show();
    } finally {
        _coveredByChildRoute =
false;
        await
_resumeScanning(recreateCamera: false);
    }
}

```

```

}

@override
void
didChangeAppLifecycleState(AppLifecycle
State state) {
    if (state ==
AppLifecycleState.inactive ||
state ==
AppLifecycleState.paused ||
state ==
AppLifecycleState.detached ||
state ==
AppLifecycleState.hidden) {
        unawaited(_pauseScanning(releaseCamera:
true));
        return;
    }

    if (state ==
AppLifecycleState.resumed &&
!_coveredByChildRoute &&
!_magnifierActive) {
        unawaited(_resumeScanning(recreateCamer
a: true));
    }
}

Future<void>
_startImageStream({int? expectedEpoch})
async {
    final controller = _controller;
    if (controller == null ||
!controller.value.isInitialized ||
_coveredByChildRoute) {
        _scanActive = false;
        return;
    }

    if
(controller.value.isStreamingImages) {
        if (_scanActive) return;
        try {
            await
controller.stopImageStream();

```

```

        } catch (_) {
            await
        }
        _disposeCameraController(updateState:
true);
        if (mounted &&
!_coveredByChildRoute) {
            await
        }
        _initialize(forceRecreate: true);
    }
    return;
}
}

final streamEpoch =
expectedEpoch ?? _cameraEpoch;
_scanActive = true;
if (mounted) {
    setState(() => _aiStatus =
_aiReady ? 'ИИ готова' : 'ИИ не готова');
}

await
controller.startImageStream((image) async
{
    if (!_scanActive ||
_cameraEpoch != streamEpoch || !_aiReady
|| _aiBusy || !mounted) {
        return;
    }

    final now = DateTime.now();
    if
(now.difference(_lastInferenceStarted).inMi
lliseconds < 1000) {
        return;
    }
    _lastInferenceStarted = now;
    _aiBusy = true;
    if (mounted) setState(() =>
_aiStatus = 'ИИ работает');
    final startedAt =
DateTime.now();

    try {
        final rotationDegrees =
_selectedCamera?.sensorOrientation ?? 0;

```

```

        final mirror =
_selectedCamera?.lensDirection ==
CameraLensDirection.front;
        final frameResult = await
_detector.detectFromCameraImage(
            image,
            rotationDegrees:
rotationDegrees,
            mirrorHorizontally:
mirror,
        );
        if (!mounted || !_scanActive
|| _cameraEpoch != streamEpoch) return;

        final perf =
frameResult.perf;
        final elapsed =
DateTime.now().difference(startedAt);
        final totalDuration =
perf.totalMs > 0 ? Duration(milliseconds:
perf.totalMs) : elapsed;

        setState() {
            _detections =
frameResult.detections;
            _lastInferenceAt =
DateTime.now();
            _lastInferenceDuration =
totalDuration;
            _lastPerf = perf;
            _aiStatus = 'ИИ готова';
        });
    } on TimeoutException {
        if (!mounted || !_scanActive
|| _cameraEpoch != streamEpoch) return;
        debugPrint('[SCAN] Frame
inference timeout');
        setState() {
            _lastPerf = const
FramePerf();
            _aiStatus = 'ИИ готова';
        });
    } catch (e) {
        if (!mounted || !_scanActive
|| _cameraEpoch != streamEpoch) return;
        debugPrint('[SCAN] AI
error: $e');
        setState() {

```

```

            _lastPerf = const
FramePerf();
            _aiStatus = 'Ошибка ИИ';
        });
    } finally {
        if (_cameraEpoch ==
streamEpoch) {
            _aiBusy = false;
            if (mounted &&
_scanActive) setState(() => _aiStatus =
_aiReady ? 'ИИ готова' : 'ИИ не готова');
        }
    }
});
}

bool get _magnifierActive =>
_magnifierBusy || _magnifierImageFile !=
null;

Rect
_displayRectForDetection(DetectionResult
detection, Size viewportSize, Size
previewContentSize) {
    if
(previewContentSize.isEmpty ||
viewportSize.isEmpty) {
        return Rect.zero;
    }

    final fittedSizes =
applyBoxFit(BoxFit.cover,
previewContentSize, viewportSize);
    final destination =
fittedSizes.destination;
    final dx = (viewportSize.width
- destination.width) / 2;
    final dy =
(viewportSize.height - destination.height) /
2;
    final dstRect = Offset(dx, dy)
& destination;

    return Rect.fromLTRB(
        dstRect.left + detection.left *
dstRect.width,

```

```

        dstRect.top + detection.top *
dstRect.height,
        dstRect.left + detection.right
* dstRect.width,
        dstRect.top +
detection.bottom * dstRect.height,
    );
}

```

```

DetectionResult?
_hitTestDetection(Offset position, Size
viewportSize, Size previewContentSize) {
    if (_detections.isEmpty) return
null;

```

```

        final hits =
<({DetectionResult detection, Rect
rect})>[];
        for (final detection in
_detections) {
            final rect =
_displayRectForDetection(detection,
viewportSize,
previewContentSize).inflate(14);
            if (rect.contains(position)) {
                hits.add((detection:
detection, rect: rect));
            }
        }

```

```

        if (hits.isEmpty) return null;
        hits.sort((a, b) => (a.rect.width
* a.rect.height).compareTo(b.rect.width *
b.rect.height));
        return hits.first.detection;
    }
}

```

```

Future<void>
_onDetectionTap(DetectionResult
detection) async {
    if (_magnifierBusy ||
!mounted) return;

```

```

        final magnifierEpoch =
++_magnifierEpoch;

```

```

        setState(() {
            _magnifierBusy = true;

```

```

        _magnifierStatus = 'Делаю
снимок, пожалуйста не двигайте';
        _magnifierSourceDetection
= detection;
        _magnifierImageFile = null;
        _magnifierCropRect = null;
        _magnifierImageSize = null;

```

```

        _magnifierNeedsInitialTransform = false;

```

```

        _magnifierTransformController.value =
Matrix4.identity();
        _magnifierRecognizedDate =
null;

```

```

        _magnifierRecognizedDateText = null;
    });

```

```

        XFile? highResShot;
        File? cropFile;

```

```

        try {
            await
_pauseScanning(releaseCamera: true);
            await
Future<void>.delayed(const
Duration(milliseconds: 250));

```

```

        highResShot = await
_captureHighResolutionStill(focusDetectio
n: detection);

```

```

        if (!mounted ||
magnifierEpoch != _magnifierEpoch) {

```

```

            try {
                await
File(highResShot.path).delete();
            } catch (_) {}
            return;
        }

```

```

        setState() {
            _magnifierStatus =
'Обрабатываю изображение';
        });

```

```

        final refined = await
_detector.detectFromImageFile(

```

```

        highResShot.path,
        focusRegion: detection,
        focusPadding: 1.6,
        confidenceThreshold: 0.01,
        maxDetections: 30,
        strictGeometryFilters: false,
    );

```

```

        final refinedDetection =
_pickBestRefinedDetection(refined.detectio
ns, detection);

```

```

        final hasRefinedDetection =
refinedDetection != null;

```

```

        final cropResult = await
_createMagnifierCrop(
            imagePath:
highResShot.path,
            detection: refinedDetection
?? detection,

```

```

            fallbackDetection:
detection,

```

```

            refined:
hasRefinedDetection,
        );
        cropFile =
cropResult.cropFile;

```

```

        DateReaderResult?
readerResult;
        DateTime? recognizedDate;
        String? recognizedText;
        double readerConfidence =
0.0;

```

```

        try {
            readerResult = await
_dateReader.recognizeFile(cropFile);
            recognizedText =
readerResult?.normalizedText;
            readerConfidence =
readerResult?.confidence ?? 0.0;

```

```

            if (readerResult != null) {
                recognizedDate =
_parseDateReaderText(readerResult.normal
izedText);
            }
        } catch (e) {

```

```

debugPrint('[DATE_READER] error: $e');
    }

    if (!mounted ||
magnifierEpoch != _magnifierEpoch) {
        try {
            if (cropFile.existsSync())
await cropFile.delete();
            await
File(hiResShot.path).delete();
        } catch (e) {}
        return;
    }

    const autoFillThreshold =
0.55;

    final canAutoFill =
recognizedDate != null &&
readerConfidence >= autoFillThreshold;

    debugPrint(
'[DATE_READER]
raw=${readerResult?.rawText}
normalized=${recognizedText} '

'confidence=${readerConfidence.toStringA
sFixed(3)} parsed=${recognizedDate} '

'detectorRefined=${hasRefinedDetection
detectorCount=${refined.detections.length}'
,
    );

    try {
        if (cropFile.existsSync())
await cropFile.delete();
    } catch (e) {}
    cropFile = null;

    setState() {
        _magnifierBusy = false;
        _magnifierImageFile =
File(hiResShot!.path);
        _magnifierCropRect =
cropResult.cropRect;
        _magnifierImageSize =
cropResult.imageSize;

        _magnifierNeedsInitialTransform = true;
        _magnifierRecognizedDate
= canAutoFill ? recognizedDate : null;

        _magnifierRecognizedDateText =
recognizedText;
        _magnifierStatus =
canAutoFill
            ? 'Дата распознана:
${_formatDateRu(recognizedDate!)}'
            : 'Дата не распознана';
    });

    if (canAutoFill) {
        await
Future<void>.delayed(const
Duration(milliseconds: 150));
        if (mounted &&
magnifierEpoch == _magnifierEpoch &&
_magnifierImageFile != null) {
            unawaited(_openManualAdd(initialExpiry:
recognizedDate, keepMagnifier: true));
        }
    } else {
        _showSnackBar('Дата не
распознана');
    }
} catch (e) {
    debugPrint('[MAGNIFIER]
error: $e');
    try {
        if (hiResShot != null)
await File(hiResShot.path).delete();
    } catch (e) {}
    try {
        if (cropFile != null &&
await cropFile.existsSync()) await
cropFile.delete();
    } catch (e) {}

    if (!mounted ||
magnifierEpoch != _magnifierEpoch)
return;

    setState() {
        _magnifierBusy = false;
        _magnifierImageFile = null;

        _magnifierCropRect = null;
        _magnifierImageSize =
null;

        _magnifierNeedsInitialTransform = false;

        _magnifierTransformController.value =
Matrix4.identity();
        _magnifierSourceDetection
= null;
        _magnifierRecognizedDate
= null;

        _magnifierRecognizedDateText = null;
        _magnifierStatus = '';
        _aiStatus = 'Не удалось
открыть лупу';
    });
    await
_resumeScanning(recreateCamera: true);
}

    DetectionResult?
_pickBestRefinedDetection(
        List<DetectionResult>
candidates,
        DetectionResult source,
    ) {
        if (candidates.isEmpty) return
null;

        final sourceCenterX =
(source.left + source.right) / 2.0;
        final sourceCenterY =
(source.top + source.bottom) / 2.0;
        final sourceW =
math.max(0.001, source.right - source.left);
        final sourceH =
math.max(0.001, source.bottom -
source.top);
        final expandedSource =
Rect.fromLTRB(
            (source.left - sourceW *
1.2).clamp(0.0, 1.0).toDouble(),
            (source.top - sourceH *
2.0).clamp(0.0, 1.0).toDouble(),

```

```

        (source.right + sourceW *
1.2).clamp(0.0, 1.0).toDouble(),
        (source.bottom + sourceH *
2.0).clamp(0.0, 1.0).toDouble(),
    );

    final scored =
candidates.map((candidate) {
    final centerX =
(candidate.left + candidate.right) / 2.0;
    final centerY =
(candidate.top + candidate.bottom) / 2.0;
    final dx = (centerX -
sourceCenterX) / math.max(0.05, sourceW
* 2.0);
    final dy = (centerY -
sourceCenterY) / math.max(0.05, sourceH *
4.0);
    final distance = math.sqrt(dx
* dx + dy * dy);
    final candidateCenter =
Offset(centerX, centerY);
    final insideBonus =
expandedSource.contains(candidateCenter)
? 0.35 : 0.0;
    final score =
candidate.confidence + insideBonus -
distance * 0.22;
    return (candidate: candidate,
score: score);
}).toList(growable: false)
..sort((a, b) =>
b.score.compareTo(a.score));

    final best = scored.first;
    // Если лучший кандидат
совсем далеко от точки клика, лучше
показать исходный фрагмент,
    // чем уверенно обрезать
чужой текст на упаковке.
    if (best.score < -0.35) return
null;
    return best.candidate;
}

Future<XFile>
_captureHighResolutionStill({DetectionRes
ult? focusDetection}) async {

```

```

    if (widget.cameras.isEmpty) {
        throw StateError('Камера не
найдена.');
```

```

    }

    final cam = _selectedCamera
??
    widget.cameras.firstWhere(
(c) => c.lensDirection ==
CameraLensDirection.back,
orElse: () =>
widget.cameras.first,
);

    final highResController =
CameraController(
    cam,
    ResolutionPreset.max,
    enableAudio: false,
    imageFormatGroup:
ImageFormatGroup.jpeg,
);

    try {
        await
highResController.initialize();
        try {
            await
highResController.setFlashMode(FlashMod
e.off);
        } catch (_) {
            // Not all devices support
explicit flash configuration for this
temporary controller.
        }

        final focus = focusDetection;
        if (focus != null) {
            final focusPoint = Offset(
((focus.left + focus.right) /
2.0).clamp(0.0, 1.0).toDouble(),
((focus.top +
focus.bottom) / 2.0).clamp(0.0,
1.0).toDouble(),
);

```

```

        await
highResController.setFocusMode(FocusMo
de.auto);
    } catch (_) {}
    try {
        await
highResController.setExposureMode(Expos
ureMode.auto);
    } catch (_) {}
    try {
        await
highResController.setFocusPoint(focusPoi
nt);
    } catch (_) {}
    try {
        await
highResController.setExposurePoint(focusP
oint);
    } catch (_) {}

    // Flutter camera не отдаёт
событие "фокус навёлся". Поэтому не
ждём фиксированную секунду:
    // даём короткую паузу
только на применение focus/exposure
point и сразу снимаем.
        await
Future<void>.delayed(const
Duration(milliseconds: 180));
    } else {
        await
Future<void>.delayed(const
Duration(milliseconds: 350));
    }

    return await
highResController.takePicture();
    } finally {
        try {
            await
highResController.dispose();
        } catch (_) {}
    }

    Future<_MagnifierCropResult>
_createMagnifierCrop({
    required String imagePath,

```

```

        required DetectionResult
detection,
        required DetectionResult
fallbackDetection,
        required bool refined,
    }) async {
        final sourceBytes = await
File(imagePath).readAsBytes();
        final decoded =
img.decodeImage(sourceBytes);
        if (decoded == null) {
            throw StateError('He
удалось прочитать снимок высокого
разрешения.');
```

```

        }

        final oriented =
img.bakeOrientation(decoded);
        await
File(imagePath).writeAsBytes(img.encodeJ
pg(oriented, quality: 96), flush: true);

        final cropRect =
_normalizedDetectionToCropRect(
            refined ? detection :
fallbackDetection,
            oriented.width,
            oriented.height,
            padding: refined ? 0.20 :
0.30,
        );

        final cropped =
img.copyCrop(
            oriented,
            x: cropRect.left.round(),
            y: cropRect.top.round(),
            width: math.max(1,
cropRect.width.round()),
            height: math.max(1,
cropRect.height.round()),
        );

        final outPath =
'${imagePath}_date_reader_crop.jpg';
        final outFile = File(outPath);

```

```

        await
outFile.writeAsBytes(img.encodeJpg(cropp
ed, quality: 95), flush: true);

        return _MagnifierCropResult(
            cropFile: outFile,
            cropRect: cropRect,
            imageSize:
Size(oriented.width.toDouble(),
oriented.height.toDouble()),
        );
    }

    Rect
_normalizedDetectionToCropRect(
        DetectionResult detection,
        int imageWidth,
        int imageHeight, {
        required double padding,
    }) {
        final left =
detection.left.clamp(0.0, 1.0).toDouble();
        final top =
detection.top.clamp(0.0, 1.0).toDouble();
        final right =
detection.right.clamp(0.0, 1.0).toDouble();
        final bottom =
detection.bottom.clamp(0.0,
1.0).toDouble();

        final centerX = (left + right) /
2.0;
        final centerY = (top + bottom)
/ 2.0;
        final boxW = math.max(0.02,
right - left);
        final boxH = math.max(0.02,
bottom - top);

        var cropW = math.min(1.0,
boxW * (1.0 + padding * 2.0));
        var cropH = math.min(1.0,
boxH * (1.0 + padding * 2.0));

        // Минимальный контекст
        нужен для маленьких дат: иначе crop
        получится слишком тонким.

```

```

        cropW = math.max(cropW,
0.05);
        cropH = math.max(cropH,
0.025);
        cropW = math.min(cropW,
1.0);
        cropH = math.min(cropH,
1.0);

        final cropLeft = (centerX -
cropW / 2.0).clamp(0.0, 1.0 -
cropW).toDouble();
        final cropTop = (centerY -
cropH / 2.0).clamp(0.0, 1.0 -
cropH).toDouble();

        final x = (cropLeft *
imageWidth).floor().clamp(0, imageWidth -
1).toInt();
        final y = (cropTop *
imageHeight).floor().clamp(0, imageHeight
- 1).toInt();
        final w = math.max(1, (cropW
* imageWidth).round()).clamp(1,
imageWidth - x).toInt();
        final h = math.max(1, (cropH
* imageHeight).round()).clamp(1,
imageHeight - y).toInt();

        return
Rect.fromLTWH(x.toDouble(),
y.toDouble(), w.toDouble(), h.toDouble());
    }

    Future<void>
_closeMagnifier({bool resumeCamera =
true}) async {
        _magnifierEpoch++;
        final imageFile =
_magnifierImageFile;

        if (mounted) {
            setState(() {
                _magnifierBusy = false;
                _magnifierImageFile = null;
                _magnifierCropRect = null;
                _magnifierImageSize =
null;

```



```

        _magnifierNeedsInitialTransform = false;

        _magnifierTransformController.value =
        Matrix4.identity();

        _magnifierSourceDetection
        = null;

        _magnifierRecognizedDate
        = null;

        _magnifierRecognizedDateText = null;
        _magnifierStatus = "";
    });
    } else {
        _magnifierBusy = false;
        _magnifierImageFile = null;
        _magnifierCropRect = null;
        _magnifierImageSize = null;

        _magnifierNeedsInitialTransform = false;

        _magnifierTransformController.value =
        Matrix4.identity();

        _magnifierSourceDetection
        = null;

        _magnifierRecognizedDate =
        null;

        _magnifierRecognizedDateText = null;
        _magnifierStatus = "";
    }

    try {
        if (imageFile != null &&
        await imageFile.exists()) {
            await imageFile.delete();
        }
        } catch (_) {}

        if (resumeCamera &&
        mounted && !_coveredByChildRoute) {
            await
            _resumeScanning(recreateCamera: true);
        }
    }

Future<void>
_closeMagnifierForNavigation() async {
    if (!_magnifierActive) return;
    await
    _closeMagnifier(resumeCamera: false);
}

DateTime?
_parseDateReaderText(String text) {
    var s = text.trim();
    if (s.isEmpty) return null;
    s = s.replaceAll(' ', '');
    s = s.replaceAll('/',
    '.').replaceAll('-', '.');
    s = s.replaceAll(RegExp(r'^0-
    9.']), "");
    s =
    s.replaceAll(RegExp(r'[.]{2,}', ''), '');
    s =
    s.replaceAll(RegExp(r'^\.\$'), "");

    final parts =
    s.split('.').where((p) =>
    p.isNotEmpty).toList();
    if (parts.length == 3) {
        final a =
        int.tryParse(parts[0]);
        final b =
        int.tryParse(parts[1]);
        final c =
        int.tryParse(parts[2]);
        if (a == null || b == null || c
        == null) return null;

        if (parts[0].length == 4) {
            return _validDate(year: a,
            month: b, day: c);
        }
        if (parts[2].length == 4) {
            return _validDate(year: c,
            month: b, day: a);
        }
        final yy =
        _normalizeYear(c);
        return _validDate(year: yy,
        month: b, day: a);
    }

    if (parts.length == 2) {
        final a =
        int.tryParse(parts[0]);
        final b =
        int.tryParse(parts[1]);
        if (a == null || b == null)
        return null;

        if (parts[0].length == 4) {
            return
            _validMonthYear(year: a, month: b);
        }
        if (parts[1].length == 4) {
            return
            _validMonthYear(year: b, month: a);
        }
        return
        _validMonthYear(year: _normalizeYear(b),
        month: a);
    }

    final digits = s.replaceAll('.',
    "");
    if (digits.length == 8) {
        if (digits.startsWith('20')) {
            return _validDate(
            year:
            int.parse(digits.substring(0, 4)),
            month:
            int.parse(digits.substring(4, 6)),
            day:
            int.parse(digits.substring(6, 8)),
            );
        }
        return _validDate(
            year:
            int.parse(digits.substring(4, 8)),
            month:
            int.parse(digits.substring(2, 4)),
            day:
            int.parse(digits.substring(0, 2)),
            );
    }

    if (digits.length == 6) {
        if (digits.startsWith('20')) {
            return _validMonthYear(

```

```

        year:
    int.parse(digits.substring(0, 4)),
        month:
    int.parse(digits.substring(4, 6)),
    );
    }
    if
    (digits.substring(2).startsWith('20')) {
        return _validMonthYear(
            year:
    int.parse(digits.substring(2, 6)),
            month:
    int.parse(digits.substring(0, 2)),
        );
    }
    return _validDate(
        year:
    _normalizeYear(int.parse(digits.substring(4,
6))),
        month:
    int.parse(digits.substring(2, 4)),
        day:
    int.parse(digits.substring(0, 2)),
    );
    }

    return null;
    }

    int _normalizeYear(int y) {
        if (y >= 100) return y;
        return y >= 70 ? 1900 + y :
2000 + y;
    }

```

```

    DateTime?
    _validDate({required int year, required int
month, required int day}) {
        if (year < 2000 || year > 2100 ||
month < 1 || month > 12 || day < 1 || day >
31) {
            return null;
        }
        final d = DateTime(year,
month, day);
        if (d.year != year || d.month !=
month || d.day != day) return null;
        return d;
    }

```

```

    DateTime?
    _validMonthYear({required int year,
required int month}) {
        if (year < 2000 || year > 2100 ||
month < 1 || month > 12) return null;
        // Для срока годности
формата "месяц/год" берём последний
день месяца.
        return DateTime(year, month
+ 1, 0);
    }

```

```

    String
    _formatDateRu(DateTime date) {
        final d =
date.day.toString().padLeft(2, '0');
        final m =
date.month.toString().padLeft(2, '0');
        return '$d.$m.$ {date.year}';
    }

```

```

    void _showSnackBar(String
message) {
        if (!mounted) return;
    }

```

```

    ScaffoldMessenger.of(context).showSnack
Bar(SnackBar(content: Text(message)));
    }

```

```

    Future<void>
    _openManualAdd({DateTime?
initialExpiry, bool keepMagnifier = false})
    async {
        if (_manualAddSheetOpen)
return;
        if (!widget.auth.isAuthenticated) {
            if (keepMagnifier) {
                _showSnackBar('Для
добавления продукта нужно
авторизоваться');
            }
            return;
        }
        _goLogin(after: 'manual');
        return;
    }

```

```

        _manualAddSheetOpen =
true;
        try {
            if (!keepMagnifier) {
                await
_closeMagnifierForNavigation();
                final added = await
_showModalWithScanPaused<bool>(<
=>
showModalBottomSheet<bool>(<
context: context,
isScrollControlled: true,
builder: (_) =>
ManualAddSheet(
auth: widget.auth,
initialExpiry:
initialExpiry,
),
),
);
            if (added == true &&
mounted) {
                _showSnackBar('Продукт
добавлен в личный кабинет');
            }
            return;
        }

        final added = await
showModalBottomSheet<bool>(<
context: context,
isScrollControlled: true,
builder: (_) =>
ManualAddSheet(
auth: widget.auth,
initialExpiry:
initialExpiry,
),
);
            if (added == true &&
mounted) {
                _showSnackBar('Продукт
добавлен в личный кабинет');
            }
        } finally {
            _manualAddSheetOpen =
false;
        }
    }

```

```

    }
  }

  void
  _scheduleMagnifierInitialTransform(Size
  viewportSize) {
    if
    (!_magnifierNeedsInitialTransform) return;
    final imageSize =
    _magnifierImageSize;
    final cropRect =
    _magnifierCropRect;
    if (imageSize == null ||
    cropRect == null || viewportSize.isEmpty ||
    cropRect.isEmpty) return;

    _magnifierNeedsInitialTransform = false;

    WidgetsBinding.instance.addPostFrameCall
    back(_) {
      if (!mounted ||
      !_magnifierActive || _magnifierImageSize
      != imageSize || _magnifierCropRect !=
      cropRect) {
        return;
      }

      final fitScale = math.min(
        viewportSize.width /
        math.max(1.0, imageSize.width),
        viewportSize.height /
        math.max(1.0, imageSize.height),
      );
      final cropScale = math.min(
        viewportSize.width /
        math.max(1.0, cropRect.width),
        viewportSize.height /
        math.max(1.0, cropRect.height),
      ) *
      0.72;
      final scale =
      cropScale.clamp(fitScale, 18.0).toDouble();
      final cropCenter =
      cropRect.center;
      final dx =
      viewportSize.width / 2.0 - cropCenter.dx *
      scale;

```

```

      final dy =
      viewportSize.height / 2.0 - cropCenter.dy *
      scale;

      final matrix =
      Matrix4.identity()
        ..setEntry(0, 0, scale)
        ..setEntry(1, 1, scale)
        ..setEntry(0, 3, dx)
        ..setEntry(1, 3, dy);

      _magnifierTransformController.value =
      matrix;
    }

    Widget
    _buildMagnifierOverlay() {
      final imageFile =
      _magnifierImageFile;
      final imageSize =
      _magnifierImageSize;
      final busy = _magnifierBusy;

      return Positioned.fill(
        child: Container(
          color: Colors.black,
          child: Stack(
            fit: StackFit.expand,
            children: [
              if (busy || imageFile ==
              null || imageSize == null)
                Center(
                  child: Padding(
                    padding: const
                    EdgeInsets.all(24),
                    child: Column(
                      mainAxisAlignment:
                      MainAxisAlignment.min,
                      children: [
                        const
                        CircularProgressIndicator(),
                        const
                        SizedBox(height: 18),
                        Text(
                          _magnifierStatus.isEmpty ? 'Обрабатываю
                          изображение' : _magnifierStatus,

```

```

                          textAlign:
                          TextAlign.center,
                          style: const
                          TextStyle(color: Colors.white, fontSize:
                          18),
                        ),
                      ],
                    ),
                  )
                else
                  LayoutBuilder(
                    builder: (context,
                    constraints) {
                      final viewportSize =
                      Size(constraints.maxWidth,
                      constraints.maxHeight);
                      _scheduleMagnifierInitialTransform(viewp
                      ortSize);

                      return
                      InteractiveViewer(
                        transformationController:
                        _magnifierTransformController,
                        constrained: false,
                        boundaryMargin:
                        const EdgeInsets.all(2500),
                        minScale: 0.03,
                        maxScale: 20,
                        clipBehavior:
                        Clip.none,
                        child: Image.file(
                          imageFile,
                          width:
                          imageSize.width,
                          height:
                          imageSize.height,
                          fit: BoxFit.fill,
                          filterQuality:
                          FilterQuality.high,
                        ),
                      );
                    },
                  ),
                SafeArea(
                  child: Align(

```

```

alignment:
Alignment.bottomCenter,
child: Padding(
padding: const
EdgeInsets.fromLTRB(16, 0, 16, 156),
child: Container(
padding: const
EdgeInsets.symmetric(horizontal: 14,
vertical: 10),
decoration:
BoxDecoration(
color:
Colors.black.withOpacity(0.62),
borderRadius:
BorderRadius.circular(16),
),
child: Text(
_magnifierStatus,
textAlign:
TextAlign.center,
style: const
TextStyle(color: Colors.white, fontSize: 16,
fontWeight: FontWeight.w600),
),
),
),
),
),
SafeArea(
child: Align(
alignment:
Alignment.bottomCenter,
child: Padding(
padding: const
EdgeInsets.fromLTRB(16, 0, 16, 92),
child:
FilledButton.icon(
onPressed: busy ?
null : () => unawaited(_closeMagnifier()),
icon: const
Icon(Icons.camera_alt),
label: const
Text('Назад к сканированию'),
),
),
),
),
],

),
),
);
}

@override
void dispose() {
WidgetsBinding.instance.removeObserver(t
his);
unawaited(_disposeCameraController(updat
eState: false));
_detector.close();
_dateReader.dispose();
_magnifierTransformController.dispose();
super.dispose();
}

void _goLogin({required String
after}) async {
await
_closeMagnifierForNavigation();
await _pushWithScanPaused(
MaterialPageRoute(builder:
() => LoginScreen(auth: widget.auth,
after: after)),
);
if (mounted) setState(() {});
}

void _openSettings() async {
await
_closeMagnifierForNavigation();
await _pushWithScanPaused(
MaterialPageRoute(builder:
() => const SettingsScreen()),
);
}

void _onProfile() async {
if (!widget.auth.isAuthenticated)
return _goLogin(after: 'profile');
await
_closeMagnifierForNavigation();
await _pushWithScanPaused(

MaterialPageRoute(builder:
() => CabinetScreen(auth: widget.auth)),
);
}

void _onManualAdd() async {
await _openManualAdd(
initialExpiry:
_magnifierActive ?
_magnifierRecognizedDate : null,
keepMagnifier:
_magnifierActive,
);
}

void _onError() async {
if (!widget.auth.isAuthenticated)
return _goLogin(after: 'error');
await
_closeMagnifierForNavigation();
await _pushWithScanPaused(
MaterialPageRoute(
builder: () =>
ErrorDatasetFlowScreen(auth: widget.auth,
cameras: widget.cameras),
),
releaseCamera: true,
);
}

Widget _buildAiPanel() {
final text = !_aiReady
? 'ИИ не готова'
: _aiBusy
? 'ИИ работает'
: 'ИИ готова';

return Container(
margin: const
EdgeInsets.all(16),
padding: const
EdgeInsets.symmetric(horizontal: 12,
vertical: 8),
decoration: BoxDecoration(
color:
Colors.black.withOpacity(0.48),
borderRadius:
BorderRadius.circular(999),

```

```

    ),
    child: Text(
      text,
      style: const TextStyle(color:
Colors.white, fontSize: 14, fontWeight:
FontWeight.w600),
    ),
  );
}

```

```

@override
Widget build(BuildContext
context) {
  return PopScope(
    canPop: !_magnifierActive,
    onPopInvokedWithResult:
  (didPop, result) async {
    if (didPop) return;
    if (_magnifierActive) {
      await _closeMagnifier();
    }
  },
  child: Scaffold(
    body: Stack(
      children: [
        Positioned.fill(
          child: () {
            final controller =
_controller;

            if (controller == null) {
              return Container(
                color: Colors.black,
                alignment:
Alignment.center,
                padding: const
EdgeInsets.all(24),
                child: Text(
                  _aiStatus,
                  textAlign:
TextAlign.center,
                  style: const
TextStyle(color: Colors.white, fontSize:
16),
                ),
              );
            }

```

```

      return
FutureBuilder<void>(
        key:
        ValueKey('camera-future-
$_previewKeySeed'),
        future: _initFuture,
        builder: (context,
snap) {
          if (snap.hasError) {
            return Container(
              color:
Colors.black,
              alignment:
Alignment.center,
              padding: const
EdgeInsets.all(24),
              child: Text(
                'Ошибка
камеры: ${snap.error}',
                textAlign:
TextAlign.center,
                style: const
TextStyle(color: Colors.white, fontSize:
16),
              ),
            );
          }

          if
(snap.connectionState !=
ConnectionState.done) {
            return const
ColoredBox(
              color:
Colors.black,
              child:
Center(child: CircularProgressIndicator()),
            );
          }

          final previewSize =
controller.value.previewSize;
          if (previewSize ==
null) {
            return const
ColoredBox(
              color:
Colors.black,

```

```

            child:
Center(child: CircularProgressIndicator()),
          );
        }
      )
    final childSize =
Size(previewSize.height,
previewSize.width);

    return Stack(
      fit:
StackFit.expand,
      children: [
        FittedBox(
          fit:
BoxFit.cover,
          child:
SizedBox(
            width:
childSize.width,
            height:
childSize.height,
            child:
CameraPreview(
              controller,
              key:
ValueKey('camera-preview-
$_previewKeySeed'),
            ),
          ),
        ),
        ),
        LayoutBuilder(
          builder:
(context, constraints) {
            final
viewportSize = Size(constraints.maxWidth,
constraints.maxHeight);
            return
GestureDetector(
              behavior:
HitTestBehavior.translucent,
              onTapUp:
(details) {
                final
detection = _hitTestDetection(
                  details.localPosition,

```

| | | | |
|--|------------------------|---|--------------------------------|
| viewportSize, | | padding: | child: const |
| | childSize, | EdgeInsets.zero, | Icon(Icons.add), |
| |); | |), |
| | if | _openSettings, |), |
| (detection != null) { | | icon: const |), |
| | | Icon(Icons.settings, size: 20), |), |
| | | tooltip: | SafeArea(|
| unawaited(_onDetectionTap(detection)); | | 'Настройки API', | child: Align(|
| | |), | alignment: |
| | |), | Alignment.bottomRight, |
| | child: | const | child: Padding(|
| CustomPaint(| | SizedBox(width: 8), | padding: const |
| | painter: | GestureDetector(| EdgeInsets.all(16), |
| _DetectionOverlayPainter(| | onTap: _onProfile, | child: |
| | detections: | child: | FloatingActionButton.extended(|
| _detections, | | CircleAvatar(| heroTag: 'error', |
| | | radius: 20, | onPressed: _onError, |
| previewContentSize: childSize, | | child: | icon: const |
| | | Icon(widget.auth.isAuthenticated ? Icons.person : | Icon(Icons.report), |
| | size: | Icons.login), | label: const |
| Size.infinite, | |), | Text('Сообщить'), |
| |), |), |), |
| |); |], |), |
| | }, |), |), |
| |), |), |), |
| |], |), |], |
| |); |), |), |
| | }, | if (!_magnifierActive) |), |
| |); | SafeArea(|); |
| | })(); | child: Align(| } |
| |), | alignment: | } |
| | if (!_magnifierActive) | Alignment.topLeft, | |
| _buildMagnifierOverlay(), | | child: _buildAiPanel(), | class _MagnifierCropResult { |
| SafeArea(| |), | final File cropFile; |
| child: Align(| |), | final Rect cropRect; |
| alignment: | | SafeArea(| final Size imageSize; |
| Alignment.topRight, | | child: Align(| |
| | child: Padding(| alignment: | const _MagnifierCropResult({ |
| | padding: const | Alignment.bottomLeft, | required this.cropFile, |
| EdgeInsets.all(12), | | child: Padding(| required this.cropRect, |
| | child: Row(| padding: const | required this.imageSize, |
| | mainAxisSize: | EdgeInsets.all(16), | }); |
| MainAxisSize.min, | | child: | } |
| | children: [| FloatingActionButton(| |
| | CircleAvatar(| heroTag: | class _DetectionOverlayPainter |
| | radius: 20, | 'manualAdd', | extends CustomPainter { |
| | child: IconButton(| onPressed: | final List<DetectionResult> |
| | _onManualAdd, | | detections; |

```

        final Size previewContentSize;

        const
        _DetectionOverlayPainter({
            required this.detections,
            required
            this.previewContentSize,
        });

        @override
        void paint(Canvas canvas, Size
size) {
            if (detections.isEmpty ||
previewContentSize.isEmpty) {
                return;
            }

            final fittedSizes =
applyBoxFit(BoxFit.cover,
previewContentSize, size);

            final destination =
fittedSizes.destination;

            final dx = (size.width -
destination.width) / 2;

            final dy = (size.height -
destination.height) / 2;

            final dstRect = Offset(dx, dy)
& destination;

            final paint = Paint()
                ..style = PaintingStyle.stroke
                ..strokeWidth = 3
                ..color = const
Color(0xFF6BEE8A);

            for (final detection in
detections) {
                final left = dstRect.left +
detection.left * dstRect.width;

                final top = dstRect.top +
detection.top * dstRect.height;

                final right = dstRect.left +
detection.right * dstRect.width;

                final bottom = dstRect.top +
detection.bottom * dstRect.height;

                final rect = Rect.fromLTRB(

```

```

                left.clamp(dstRect.left,
dstRect.right),
                top.clamp(dstRect.top,
dstRect.bottom),
                right.clamp(dstRect.left,
dstRect.right),
                bottom.clamp(dstRect.top,
dstRect.bottom),
            );

            if (rect.width < 4 ||
rect.height < 4) {
                continue;
            }

            canvas.drawRect(rect, paint);
        }
    }

    @override
    bool shouldRepaint(covariant
_DetectionOverlayPainter oldDelegate) {
        return oldDelegate.detections
!= detections ||
oldDelegate.previewContentSize !=
previewContentSize;
    }
}

=====
Файл:
lib/screens/settings_screen.dart
=====

import
'package:flutter/material.dart';

import './api/api_config.dart';
import
'./api/model_sync_service.dart';

```

```

class SettingsScreen extends
StatefulWidget {
    const
SettingsScreen({super.key});

    @override
    State<SettingsScreen>
createState() => _SettingsScreenState();
}

class _SettingsScreenState
extends State<SettingsScreen> {
    final _modelSync =
ModelSyncService();

    final _baseUrlController =
TextEditingController();

    Map<String, dynamic>?
_remoteInfo;

    Map<String, dynamic>?
_localInfo;

    String? _status;
    bool _loading = true;
    bool _busy = false;

    @override
    void initState() {
        super.initState();
        _baseUrlController.text =
ApiConfig.baseUrl;
        _loadInfo();
    }

    @override
    void dispose() {
        _baseUrlController.dispose();
        super.dispose();
    }

    Future<void> _loadInfo()
async {
        setState(() {
            _loading = true;
            _status = null;
        });

        final remote = await
_modelSync.fetchLatestModelInfo();
    }
}

```

```

        final local = await
_modelSync.readLocalModelInfo();

        if (!mounted) return;
        setState(() {
            _remoteInfo = remote;
            _localInfo = local;
            _loading = false;
        });
    }

    Future<void> _saveBaseUrl()
    async {

        setState(() => _busy = true);

        try {
            final normalized =
ApiConfig.normalizeForDisplay(_baseUrlC
ontroller.text);

            await
ApiConfig.setBaseUrl(normalized);
            _baseUrlController.text =
ApiConfig.baseUrl;

            await _loadInfo();
            if (!mounted) return;
            setState(() => _status =
'Адрес сервера сохранён.');
```

```

        _status = 'Возвращён адрес
по умолчанию.';

        _busy = false;
    });
}

    Future<void>
_updateAiVersion() async {
        setState(() {
            _busy = true;
            _status = 'Проверяю новую
версию ИИ...';
        });

        final result = await
_modelSync.trySyncLatestModel(force:
true);

        final local = await
_modelSync.readLocalModelInfo();

        if (!mounted) return;
        setState(() {
            _remoteInfo =
result.remoteInfo ?? _remoteInfo;
            _localInfo = local;
            _status = result.message;
            _busy = false;
        });

        Widget _infoTile(String title,
String value) {
            return ListTile(
                dense: true,
                contentPadding:
EdgeInsets.zero,
                title: Text(title),
                subtitle: Text(value.isEmpty
? '—' : value),
            );
        }

        Widget _buildAiInfoCard({
            required String title,
            required Map<String,
dynamic>? info,
            required bool local,
        }) {
```

```

        if (info == null) {
            return Card(
                child: Padding(
                    padding: const
EdgeInsets.all(16),
                    child: Text(local
? 'Локальная модель
ещё не загружена на устройство.'
: 'На сервере пока нет
опубликованной мобильной модели или
сервер недоступен.'),
                ),
            );
        }

        final metrics =
_modelSync.prettyMetrics(info['metricsJso
n']?.toString());

        return Card(
            child: Padding(
                padding: const
EdgeInsets.all(16),
                child: Column(
                    crossAxisAlignment:
CrossAxisAlignment.start,
                    children: [
                        Text(
                            title,
                            style: const
TextStyle(fontSize: 16, fontWeight:
FontWeight.bold),
                        ),
                        const SizedBox(height:
8),
                        _infoTile('Версия',
'#{info['modelVersionId']?.toString() ??
'—}'),
                        _infoTile('Формат',
info['mobileFormat']?.toString() ?? ''),
                        _infoTile('Обучена',
info['trainedAt']?.toString() ?? ''),
                        if (local) ...[
                            _infoTile('Путь к
файлу', info['modelPath']?.toString() ?? ''),
                            _infoTile('Источник',
info['sourceUrl']?.toString() ?? ''),
```



```

        _infoTile('Синхронизирована',
info['syncedAt']?.toString() ?? ''),
        _infoTile('Файл
существует', (info['fileExists'] ?? false) ?
'Да' : 'Нет'),
        _infoTile('Размер
файла', '${info['fileSizeBytes']?.toString()
?? '—'} байт'),
    ],
    const SizedBox(height:
8),
    const Text('Метрики /
справка', style: TextStyle(fontWeight:
FontWeight.w600)),
    const SizedBox(height:
6),
    Container(
width: double.infinity,
padding: const
EdgeInsets.all(12),
decoration:
BoxDecoration(
color:
Theme.of(context).colorScheme.surfaceCon
tainerHighest,
borderRadius:
BorderRadius.circular(12),
),
child:
SelectableText(metrics),
),
),
);
}

@override
Widget build(BuildContext
context) {
return Scaffold(
appBar: AppBar(
title: const
Text('Настройки'),
actions: [
IconButton(
icon: const
Icon(Icons.refresh),
onPressed: _busy ? null :
_loadInfo,
),
],
body: _loading
? const Center(child:
CircularProgressIndicator())
: ListView(
padding: const
EdgeInsets.all(16),
children: [
TextField(
controller:
_baseUrlController,
decoration: const
InputDecoration(
labelText: 'BaseUrl
сервера',
hintText:
'http://111.88.146.2:5099',
border:
OutlineInputBorder(),
),
const
SizedBox(height: 12),
Wrap(
spacing: 12,
runSpacing: 12,
children: [
ElevatedButton.icon(
onPressed: _busy ?
null : _saveBaseUrl,
icon: const
Icon(Icons.save),
label: const
Text('Сохранить'),
),
OutlinedButton.icon(
onPressed: _busy ?
null : _resetBaseUrl,
icon: const
Icon(Icons.restore),
label: const
Text('По умолчанию'),
),
FilledButton.icon(
onPressed: _busy ?
null : _updateAiVersion,
icon: const
Icon(Icons.system_update_alt),
label: const
Text('Обновить версию ИИ'),
),
const
SizedBox(height: 12),
if (_status != null)
Card(
child: Padding(
padding: const
EdgeInsets.all(12),
child:
Text(_status!),
),
const
SizedBox(height: 8),
_buildAiInfoCard(
title: 'Текущая
версия ИИ на устройстве',
info: _localInfo,
local: true,
),
const
SizedBox(height: 12),
_buildAiInfoCard(
title: 'Последняя
версия ИИ на сервере',
info: _remoteInfo,
local: false,
),
),
),
);
}
=====
=====

```

```

=====
=====
Файл:
lib/storage/file_key_value_store.dart
=====
=====
=====
=====

import 'dart:convert';
import 'dart:io';

/// Локальное хранилище без
platform plugins.
///
/// Важно: не использует
shared_preferences/path_provider, потому
что на части
/// Android-сборок они падали
с Pigeon channel-error.

class FileKeyValueStore {
  static const String
_androidPackageId =
'com.example.mobile_app';

  static Map<String, dynamic>?
_cache;

  static Future<Directory>
appDataDirectory() async {
    if (Platform.isAndroid) {
      final candidates = <String>[

'/data/user/0/$_androidPackageId/files',

'/data/data/$_androidPackageId/files',
      ];

      for (final path in candidates)
      {
        try {
          final dir = Directory(path);
          if (!await dir.exists()) {
            await
dir.create(recursive: true);
          }
          if (await dir.exists()) {
            return dir;
          }
        }
      }
    }
  }
}

```

```

    } catch (_) {
      // Пробуем следующий
вариант.
    }
  }

  // Резерв для debug/desktop.
Directory.systemTemp берётся из dart:io и
не
  // требует platform channel.
  final fallback =
Directory('${Directory.systemTemp.path}/
mobile_app_state');
  if (!await fallback.exists()) {
    await
fallback.create(recursive: true);
  }
  return fallback;
}

  static Future<Directory>
namedDirectory(String name) async {
    final safeName =
name.replaceAll(RegExp(r'^a-zA-Z0-9_\-
|'), '_');

    final root = await
appDataDirectory();
    final dir =
Directory('${root.path}/${safeName}');
    if (!await dir.exists()) {
      await dir.create(recursive:
true);
    }
    return dir;
  }

  static Future<File> _stateFile()
async {
    final dir = await
appDataDirectory();
    return
File('${dir.path}/app_state.json');
  }

  static Future<Map<String,
dynamic>> _readAll() async {
    final cached = _cache;

```

```

    if (cached != null) {
      return Map<String,
dynamic>.from(cached);
    }

    try {
      final file = await _stateFile();
      if (!await file.exists()) {
        _cache = <String,
dynamic>{};
        return <String,
dynamic>{};
      }

      final raw = await
file.readAsString();
      if (raw.trim().isEmpty) {
        _cache = <String,
dynamic>{};
        return <String,
dynamic>{};
      }

      final decoded =
jsonDecode(raw);
      if (decoded is Map<String,
dynamic>) {
        _cache = decoded;
        return Map<String,
dynamic>.from(decoded);
      }
    } catch (_) {
      // Если файл повреждён
или временно недоступен, не ломаем
приложение.
    }

    _cache = <String,
dynamic>{};
    return <String, dynamic>{};
  }

  static Future<void>
_writeAll(Map<String, dynamic> data)
async {
    _cache = Map<String,
dynamic>.from(data);

```

```

    try {
      final file = await _stateFile();
      await file.writeAsString(
        const
JsonEncoder.withIndent(' ').convert(data),
        flush: true,
      );
    } catch (_) {
      // Запись не должна ронять
приложение. В худшем случае значение
будет

      // жить только в памяти до
перезапуска.
    }
  }
}

```

```

static Future<String?>
getString(String key) async {
  final data = await _readAll();
  final value = data[key];
  return value == null ? null :
value.toString();
}

```

```

static Future<int?>
getInt(String key) async {
  final data = await _readAll();
  final value = data[key];
  if (value is int) return value;
  if (value is num) return
value.toInt();
  return
int.tryParse(value?.toString() ?? "");
}

```

```

static Future<void>
setString(String key, String value) async {
  final data = await _readAll();
  data[key] = value;
  await _writeAll(data);
}

```

```

static Future<void>
setInt(String key, int value) async {
  final data = await _readAll();
  data[key] = value;
  await _writeAll(data);
}

```

```

static Future<void>
remove(String key) async {
  final data = await _readAll();
  data.remove(key);
  await _writeAll(data);
}
}

```

```

=====
=====
=====
Файл:
lib/widgets/manual_add_sheet.dart
=====
=====
=====

```

```

import
'package:flutter/material.dart';

import '../api/api_client.dart';
import '../auth/auth_state.dart';

```

```

class ManualAddSheet extends
StatefulWidget {
  final AuthState auth;
  final DateTime? initialExpiry;

```

```

const ManualAddSheet( {
  super.key,
  required this.auth,
  this.initialExpiry,
});

```

```

@override
State<ManualAddSheet>
createState() => _ManualAddSheetState();
}

```

```

class _ManualAddSheetState
extends State<ManualAddSheet> {
  final _nameController =
TextEditingController();
  DateTime? _expiry;

```

```

String? _error;
bool _saving = false;

@override
void initState() {
  super.initState();
  _expiry =
widget.initialExpiry;
}

```

```

@override
void dispose() {
  _nameController.dispose();
  super.dispose();
}

```

```

DateTime _dateOnly(DateTime
value) => DateTime(value.year,
value.month, value.day);

```

```

Future<void> _pickDate()
async {
  final today =
_dateOnly(DateTime.now());
  final initial =
_dateOnly(_expiry ?? today.add(const
Duration(days: 7)));

```

```

// showDatePicker падает,
если initialDate раньше firstDate.
// OCR может распознать
уже прошедшую дату, поэтому
календарь должен уметь
// открываться и для таких
значений, чтобы пользователь мог
исправить дату.

```

```

final firstDate =
DateTime(2000, 1, 1);
final defaultLastDate =
today.add(const Duration(days: 3650));
final lastDate =
initial.isAfter(defaultLastDate) ? initial :
defaultLastDate;

```

```

final picked = await
showDatePicker(
  context: context,
  firstDate: firstDate,

```

```

        lastDate: lastDate,
        initialDate: initial,
    );
    if (picked != null) setState(()
=> _expiry = picked);
    }

    Future<void> _save() async {
        final productName =
        _nameController.text.trim();

        if (productName.isEmpty) {
            setState(() => _error =
'Введи название продукта');
            return;
        }

        if (_expiry == null) {
            setState(() => _error =
'Выбери срок годности');
            return;
        }

        setState(() {
            _error = null;
            _saving = true;
        });

        try {
            final api = ApiClient(token:
widget.auth.token);
            await
            api.addStoredProductByName(productNam
e, _expiry!);

            if (!mounted) return;
            Navigator.pop(context, true);
        } catch (e) {
            if (mounted) setState(() =>
            _error = e.toString());
        } finally {
            if (mounted) setState(() =>
            _saving = false);
        }
    }

    @override
    Widget build(BuildContext
context) {

```

```

        final pad =
        MediaQuery.of(context).viewInsets.bottom;

        return Padding(
            padding:
            EdgeInsets.fromLTRB(16, 16, 16, 16 +
            pad),
            child:
            SingleChildScrollView(
                child: Column(
                    mainAxisAlignment:
                    MainAxisAlignment.min,
                    crossAxisAlignment:
                    CrossAxisAlignment.stretch,
                    children: [
                        const Text(
                            'Добавить продукт',
                            style:
                            TextStyle(fontSize: 18, fontWeight:
                            FontWeight.bold),
                        ),
                        const SizedBox(height:
                        12),
                        if (_error != null) ...[
                            Text(_error!, style:
                            const TextStyle(color: Colors.red)),
                            const SizedBox(height:
                            8),
                        ],
                        TextField(
                            controller:
                            _nameController,
                            textInputAction:
                            TextInputAction.done,
                            decoration: const
                            InputDecoration(
                                labelText: 'Название
                                продукта',
                                hintText: 'Например:
                                молоко, йогурт, сыр',
                                border:
                                OutlineInputBorder(),
                            ),
                        ),
                        const SizedBox(height:
                        12),
                        OutlinedButton.icon(

```

```

onPressed: _saving ?
null : _pickDate,
            icon: const
            Icon(Icons.calendar_month),
            label: Text(
                _expiry == null
                ? 'Выбрать срок
                годности'
                : 'Срок годности:
                ${_expiry!.toLocal().toString().split('
                ').first}',
            ),
        ),
        const SizedBox(height:
        12),
        ElevatedButton.icon(
            onPressed: _saving ?
            null : _save,
            icon: _saving
            ? const SizedBox(
                width: 18,
                height: 18,
                child:
                CircularProgressIndicator(strokeWidth: 2),
            )
            : const
            Icon(Icons.save),
            label: Text(_saving ?
            'Сохраняем...' : 'Добавить в личный
            кабинет'),
        ),
    ],
),
),
);
}
}

=====
=====
=====
=====
Файл: pubspec.yaml
=====
=====
=====
=====

```

```

name: mobile_app
description: "A new Flutter
project."

# The following line prevents the
package from being accidentally published
to

# pub.dev using `flutter pub
publish`. This is preferred for private
packages.

publish_to: 'none' # Remove this
line if you wish to publish to pub.dev

# The following defines the
version and build number for your
application.

# A version number is three
numbers separated by dots, like 1.2.43
# followed by an optional build
number separated by a +.

# Both the version and the
builder number may be overridden in flutter
# build by specifying --build-
name and --build-number, respectively.

# In Android, build-name is used
as versionName while build-number used as
versionCode.

# Read more about Android
versioning at
https://developer.android.com/studio/publish/versioning

# In iOS, build-name is used as
CFBundleShortVersionString while build-
number is used as CFBundleVersion.

# Read more about iOS
versioning at
#
https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFoundationKeys.html

# In Windows, build-name is
used as the major, minor, and patch parts
# of the product and file versions
while build-number is used as the build
suffix.

version: 1.0.0+1

environment:

```

```

sdk: ^3.11.0

# Dependencies specify other
packages that your package needs in order
to work.

# To automatically upgrade your
package dependencies to the latest versions
# consider running `flutter pub
upgrade --major-versions`. Alternatively,
# dependencies can be manually
updated by changing the version numbers
below to

# the latest version available on
pub.dev. To see which dependencies have
newer

# versions available, run `flutter
pub outdated`.

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  camera: ^0.11.0+2
  archive: ^4.0.9
  tfite_flutter: ^0.12.1
  image: ^4.8.0
  google_mlkit_text_recognition:
^0.15.0

# The following adds the
Cupertino Icons font to your application.

# Use with the CupertinoIcons
class for iOS style icons.

cupertino_icons: ^1.0.8

dev_dependencies:
  flutter_test:
    sdk: flutter

# The "flutter_lints" package
below contains a set of recommended lints
to

# encourage good coding
practices. The lint set provided by the
package is

# activated in the
`analysis_options.yaml` file located at the
root of your

```

```

# package. See that file for
information about deactivating specific lint
# rules and activating additional
ones.

flutter_lints: ^6.0.0

# For information on the generic
Dart part of this file, see the
# following page:
https://dart.dev/tools/pub/pubspec

# The following section is
specific to Flutter packages.

flutter:
  assets:
    - assets/ai/date_reader.tffile
    -
assets/ai/date_reader_meta.json

# The following line ensures
that the Material Icons font is
# included with your
application, so that you can use the icons in
# the material Icons class.
uses-material-design: true

# To add assets to your
application, add an assets section, like this:

# assets:
# - images/a_dot_burr.jpeg
# - images/a_dot_ham.jpeg

# An image asset can refer to
one or more resolution-specific "variants",
see
#
https://flutter.dev/to/resolution-aware-images

# For details regarding adding
assets from package dependencies, see
# https://flutter.dev/to/asset-from-package

# To add custom fonts to your
application, add a fonts section here,
# in this "flutter" section. Each
entry in this list should have a

```

```

# "family" key with the font
family name, and a "fonts" key with a
# list giving the asset and other
descriptors for the font. For
# example:
# fonts:
#   - family: Schyler
#   fonts:
#     - asset: fonts/Schyler-
Regular.ttf
#     - asset: fonts/Schyler-
Italic.ttf
#       style: italic
#   - family: Trajan Pro
#   fonts:
#     - asset: fonts/TrajanPro.ttf
#     - asset:
fonts/TrajanPro_Bold.ttf
#       weight: 700
#
# For details regarding fonts
from package dependencies,
# see https://flutter.dev/to/font-
from-package

```

```

=====
=====
=====
=====
Файл: README.md
=====
=====
=====
=====

```

```

# mobile_app

A new Flutter project.

## Getting Started

This project is a starting point
for a Flutter application.

```

Листинг Д.2 – Листинг кода сервиса обучения

```

from pathlib import Path
from datetime import datetime
import argparse

```

A few resources to get you
started if this is your first Flutter project:

```

- [Learn
Flutter](https://docs.flutter.dev/get-
started/learn-flutter)
- [Write your first Flutter
app](https://docs.flutter.dev/get-
started/codelab)
- [Flutter learning
resources](https://docs.flutter.dev/reference/
learning-resources)

```

For help getting started with
Flutter development, view the
[online
documentation](https://docs.flutter.dev/),
which offers tutorials,
samples, guidance on mobile
development, and a full API reference.

```

=====
=====
=====
=====
Файл: test/widget_test.dart
=====
=====
=====
=====

```

```

// This is a basic Flutter widget
test.
//
// To perform an interaction with
a widget in your test, use the WidgetTester
// utility in the flutter_test
package. For example, you can send tap and
scroll
// gestures. You can also use
WidgetTester to find child widgets in the
widget

```

```

import fnmatch

EXCLUDE_DIR_NAMES = {

```

// tree, read text, and verify that
the values of widget properties are correct.

```

import
'package:flutter/material.dart';

import
'package:flutter_test/flutter_test.dart';

import
'package:mobile_app/main.dart';

void main() {
  testWidgets('Counter
increments smoke test', (WidgetTester
tester) async {
    // Build our app and trigger a
frame.
    await
tester.pumpWidget(const MyApp());

    // Verify that our counter
starts at 0.
    expect(find.text('0'),
findsOneWidget);
    expect(find.text('1'),
findsNothing);

    // Tap the '+' icon and trigger a
frame.
    await
tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // Verify that our counter has
incremented.
    expect(find.text('0'),
findsNothing);
    expect(find.text('1'),
findsOneWidget);
  });
}

```

```

".git",
".idea",
".vscode",

```

```

".venv",
"venv",
"env",
"__pycache__",
".pytest_cache",
".mypy_cache",
".ruff_cache",
".cache",

"data",
"datasets",
"dataset",
"uploads",
"extracted",
"labels",
"images",
"videos",
"frames",

"runs",
"weights",
"models",
"checkpoints",
"artifacts",
"outputs",
"results",
"logs",

"tmp",
"temp",
"build",
"dist",
}

CODE_EXTENSIONS = {
    ".py",
    ".ipynb",
}

CONFIG_EXTENSIONS = {
    ".json",
    ".yaml",
    ".yml",
    ".toml",
    ".ini",
    ".cfg",
}

PROJECT_EXTENSIONS = {
    ".txt", # requirements.txt
    ".md", # README, если
    ".dockerfile",
}

ALLOWED_ROOT_FILES = {
    "requirements.txt",
    "pyproject.toml",
    "setup.py",
    "setup.cfg",
    "Dockerfile",
    "docker-compose.yml",
    "docker-compose.yaml",
    ".gitignore",
    ".dockerignore",
    "README.md",
}

EXCLUDE_FILE_NAMES = {
    ".env",
    ".env.local",
    ".env.development",
    ".env.production",

    "debug.log",
    "train.log",
    "last.pt",
    "best.pt",
}

EXCLUDE_FILE_PATTERNS
= {
    "*.pt",
    "*.pth",
    "*.onnx",
    "*.tflite",
    "*.engine",
    "*.weights",

    "*.jpg",
    "*.jpeg",
    "*.png",
    "*.webp",
    "*.bmp",
    "*.gif",
    "*.mp4",
    "*.mov",
    "*.avi",
    "*.mkv",

    "*.zip",
    "*.rar",
    "*.7z",
    "*.tar",
    "*.gz",

    "*.db",
    "*.sqlite",
    "*.sqlite3",

    "*.log",
    "*.cache",
    "*.tmp",

    "*.csv",
    "*.tsv",

    # YOLO-разметка кадров,
    # обычно это не код
    "frame_*.txt",
}

GENERATED_FILE_PATTERN
NS = {
    "_pb2.py",
    "_pb2_grpc.py",
}

def to_posix(path: Path) -> str:
    return str(path).replace("\\",
"/")

def has_excluded_parent(path:
Path, root: Path) -> bool:
    parts =
path.relative_to(root).parts
    return any(part in
EXCLUDE_DIR_NAMES for part in parts)

```

```

def matches_any(name: str,
patterns: set[str]) -> bool:
    return
any(fnmatch.fnmatch(name, pattern) for
pattern in patterns)

def is_allowed_file(path: Path,
root: Path, include_configs: bool,
include_docs: bool) -> bool:
    relative =
path.relative_to(root)
    name = path.name
    suffix = path.suffix.lower()

    if len(relative.parts) == 1 and
name in ALLOWED_ROOT_FILES:
        if name ==
"README.md":
            return include_docs
        return True

    if suffix in
CODE_EXTENSIONS:
        return True

    if include_configs and suffix
in CONFIG_EXTENSIONS:
        return True

    if include_docs and suffix ==
".md":
        return True

    # requirements-*.txt
оставляем, но обычные .txt не берём
    if
name.startswith("requirements") and suffix
== ".txt":
        return True

    return False

def should_skip_file(path: Path,
root: Path, include_generated: bool) ->
bool:
    name = path.name

```

```

    if has_excluded_parent(path,
root):
        return True

    if name in
EXCLUDE_FILE_NAMES:
        return True

    if matches_any(name,
EXCLUDE_FILE_PATTERNS):
        return True

    if not include_generated and
matches_any(name,
GENERATED_FILE_PATTERNS):
        return True

    return False

def read_text_file(path: Path) ->
str:
    for encoding in ("utf-8", "utf-
8-sig", "cp1251", "latin-1"):
        try:
            return
path.read_text(encoding=encoding)
        except
UnicodeDecodeError:
            continue

    return
path.read_bytes().decode("utf-8",
errors="replace")

def collect_files(root: Path,
include_configs: bool, include_docs: bool,
include_generated: bool) -> list[Path]:
    files = []

    for path in root.rglob("*"):
        if not path.is_file():
            continue

    if should_skip_file(path,
root, include_generated):

```

```

        continue

    if is_allowed_file(path,
root, include_configs, include_docs):
        files.append(path)

    return sorted(files,
key=lambda p:
to_posix(p.relative_to(root)).lower())

def main():
    parser =
argparse.ArgumentParser(
        description="Собирает код
сервиса обучения из training_service без
датасетов, весов моделей и артефактов
обучения."
    )

    parser.add_argument(
        "folder",
        nargs="?",
        default="training_service",
        help="Папка сервиса
обучения. По умолчанию:
training_service",
    )

    parser.add_argument(
        "-o",
        "--output",
        default="training_service_code_dump.txt",
        help="Файл результата.
По умолчанию:
training_service_code_dump.txt",
    )

    parser.add_argument(
        "--no-configs",
        action="store_true",
        help="Не включать
json/yaml/toml/ini/cfg конфига.",
    )

    parser.add_argument(
        "--include-docs",

```



```

        action="store_true",
        help="Включить
README и markdown-документацию.",
    )

    parser.add_argument(
        "--include-generated",
        action="store_true",
        help="Включить
сгенерированные *_pb2.py и
*_pb2_grpc.py.",
    )

    args = parser.parse_args()

    root =
Path(args.folder).resolve()
    output =
Path(args.output).resolve()

    if not root.exists():
        raise
FileNotFoundError(f"Папка не найдена:
{root}")

    if not root.is_dir():
        raise
NotADirectoryError(f"Это не папка:
{root}")

    files = collect_files(
        root=root,
        include_configs=not
args.no_configs,

        include_docs=args.include_docs,

        include_generated=args.include_generated,
    )

    with output.open("w",
encoding="utf-8", newline="\n") as out:
        out.write("# Сборка кода
сервиса обучения\n\n")

        out.write(f"Папка:
{root}\n")

```

```

        out.write(f"Дата сборки:
{datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n")

        out.write(f"Количество
файлов: {len(files)}\n\n")

        out.write("## Включённые
файлы\n\n")

        out.write("```text\n")
        for file in files:

            out.write(to_posix(file.relative_to(root)) +
"\n")

            out.write("```\n\n")

        out.write("## Содержимое
файлов\n\n")

        for file in files:
            relative =
to_posix(file.relative_to(root))

            out.write("\n")
            out.write("=" * 100)
            out.write("\n")
            out.write(f"Файл:
{relative}\n")

            out.write("=" * 100)
            out.write("\n\n")

            try:
                content =
read_text_file(file)

            except Exception as exc:
                out.write(f"[He
удалось прочитать файл: {exc}]\n")

                continue

            out.write(content)

            if not
content.endswith("\n"):
                out.write("\n")

        print(f"Готово. Собрано
файлов: {len(files)}")

        print(f"Результат сохранён в:
{output}")

```

```

if __name__ == "__main__":
    main()

    from __future__ import
annotations

    import argparse
    import csv
    import json
    import re
    from pathlib import Path

    import numpy as np
    from PIL import Image

    try:
        import tensorflow as tf
    except Exception as e:
        raise SystemExit("TensorFlow
is required: pip install tensorflow pillow
numpy") from e

    def norm_text(text: str, alphabet:
str) -> str:
        text = (text or
"".strip().upper()

        text = text.replace("\\",
"/").replace("|", "/").replace(" ", "")

        allowed = set(alphabet)

        return "".join(ch for ch in text
if ch in allowed)

    def preprocess_image(path:
Path, width: int, height: int) -> np.ndarray:
        img =
Image.open(path).convert("L")

        w, h = img.size

        scale = min(width / max(1,
w), height / max(1, h))

        nw, nh = max(1, int(w *
scale)), max(1, int(h * scale))

        img = img.resize((nw, nh),
Image.BILINEAR)

```

```

        canvas = Image.new("L",
(width, height), color=255)
        canvas.paste(img, (0, (height -
nh) // 2))
        arr =
np.asarray(canvas).astype("float32") / 255.0
        return arr[None, ..., None]

```

```

def greedy_decode(logits:
np.ndarray, alphabet: str) -> str:
    ids = logits.argmax(axis=-1)
    if ids.ndim == 2:
        ids = ids[0]
    blank = len(alphabet)
    out = []
    prev = None
    for idx in ids.tolist():
        if idx != prev and idx !=
blank and 0 <= idx < len(alphabet):

out.append(alphabet[idx])
        prev = idx
    return "".join(out)

```

```

def edit_distance(a: str, b: str) ->
int:
    dp = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        ndp = [i]
        for j, cb in enumerate(b, 1):
            ndp.append(min(
                dp[j] + 1,
                ndp[j - 1] + 1,
                dp[j - 1] + (ca != cb),
            ))
        dp = ndp
    return dp[-1]

```

```

def load_rows(labels: Path,
base_dir: Path) -> list[tuple[Path, str, str]]:
    rows = []
    with labels.open("r",
encoding="utf-8-sig", newline="") as f:
        reader = csv.DictReader(f)
        for r in reader:

```

```

        rel = r.get("image") or
r.get("path") or r.get("file") or
r.get("filename")
        text = r.get("text") or
r.get("label") or r.get("transcription") or ""
        if not rel:
            continue
        p = Path(rel)
        if not p.is_absolute():
            p = base_dir / p
        rows.append((p, text,
rel))
    return rows

```

```

def main() -> None:
    ap =
argparse.ArgumentParser()
    ap.add_argument("--model",
required=True, type=Path)
    ap.add_argument("--meta",
required=True, type=Path)
    ap.add_argument("--labels",
type=Path, help="CSV with image/text or
path/text")
    ap.add_argument("--base-dir",
type=Path, default=None, help="Base dir
for relative image paths in CSV")
    ap.add_argument("--images-
dir", type=Path, help="Predict all images in
folder without GT")
    ap.add_argument("--limit",
type=int, default=0)
    ap.add_argument("--show-
all", action="store_true")
    args = ap.parse_args()

    meta =
json.loads(args.meta.read_text(encoding="u
tf-8"))
    alphabet =
meta.get("alphabet", "0123456789./-")
    width =
int(meta.get("input_width", 256))
    height =
int(meta.get("input_height", 48))

```

```

    interpreter =
tf.lite.Interpreter(model_path=str(args.mode
l))
    interpreter.allocate_tensors()
    input_details =
interpreter.get_input_details()
    output_details =
interpreter.get_output_details()
    input_index =
input_details[0]["index"]
    output_index =
output_details[0]["index"]

    if args.labels:
        base_dir = args.base_dir or
args.labels.parent
        rows =
load_rows(args.labels, base_dir)
        elif args.images_dir:
            exts = {".jpg", ".jpeg",
".png", ".bmp", ".webp"}
            rows = [(p, "", p.name) for
p in sorted(args.images_dir.rglob("*")) if
p.suffix.lower() in exts]
            else:
                raise SystemExit("Use --
labels or --images-dir")

    if args.limit and args.limit > 0:
        rows = rows[:args.limit]

    total = 0
    exact = 0
    cer_sum = 0.0
    errors = []

    for p, raw_gt, rel in rows:
        if not p.exists():
            errors.append((rel,
raw_gt, "<missing file>", 1.0))
            continue
        x = preprocess_image(p,
width, height)

    interpreter.set_tensor(input_index, x)
    interpreter.invoke()
    logits =
interpreter.get_tensor(output_index)

```

```

        pred =
greedy_decode(logits, alphabet)

        if args.labels:
            gt = norm_text(raw_gt,
alphabet)
            ed = edit_distance(gt,
pred)
            cer = ed / max(1, len(gt))
            total += 1
            exact += int(pred == gt)
            cer_sum += cer

            if args.show_all or pred
!= gt:
                errors.append((rel, gt,
pred, cer))
            else:
                print(f'{rel}: {pred}')
            if args.labels:
                print(f'TOTAL: {total}')
                print(f'EXACT: {exact /
max(1, total):.3f}')

        print(f'CER: {cer_sum /
max(1, total):.3f}')

print("\nОшибки/примеры:")
for rel, gt, pred, cer in
errors[:50]:
    print(f'{rel}: GT={gt} |
PRED={pred} | CER={cer:.3f}')

if __name__ == "__main__":
    main()

```

Листинг Д.3 – Листинг кода админского приложения

```

'''
=====
=====
=====
=====
Файл:
AdminApp/AdminApiClient.cs
=====
=====
=====
=====

using System.Net.Http.Headers;
using System.Net.Http;
using System.Net.Http.Json;
using System.Text;
using System.Text.Json;
using Contracts.Dtos;

namespace AdminApp;

public class AdminApiClient
{
    private readonly HttpClient
_http;

    private readonly
JsonSerializerOptions _jsonOptions = new()
{
    PropertyNameCaseInsensitive = true
};

    public AdminApiClient(string
baseUrl, string jwtToken)
    {
        _http = new HttpClient
        {
            BaseAddress = new
Uri(baseUrl),
            Timeout =
TimeSpan.FromMinutes(30)
        };

        _http.DefaultRequestHeaders.Authorization
=
new
AuthenticationHeaderValue("Bearer",
jwtToken);
    }

    public async
Task<List<AdminErrorReportListItemDto>>
GetErrorReportsAsync(CancellationTok
en ct = default)
    {
        var result = await
_http.GetFromJsonAsync<List<AdminErro
rReportListItemDto>>("api/admin/error-
reports", _jsonOptions, ct);

        return result ?? new
List<AdminErrorReportListItemDto>();
    }

    public async
Task<AdminErrorReportDetailsDto?>
GetErrorReportAsync(int reportId,
CancellationTok en ct = default)
    {
        return await
_http.GetFromJsonAsync<AdminErrorRep
ortDetailsDto>("api/admin/error-
reports/{reportId}", _jsonOptions, ct);
    }

    public async Task<Image?>
GetFrameImageAsync(int reportId, int
frameIndex, CancellationTok en ct =
default)
    {
        using var response = await
_http.GetAsync(
            $"api/admin/error-
reports/{reportId}/frames/{frameIndex}",
ct);

        if
(!response.IsSuccessStatusCode)
            return null;
    }
}
'''

```

```

        await using var stream =
await
response.Content.ReadAsStreamAsync(ct);

        return
Image.FromStream(stream);
    }

    public async
Task<List<YoloBboxDto>>
GetFrameBoxesAsync(int reportId, int
frameIndex, CancellationToken ct =
default)
    {
        using var response = await
_http.GetAsync(
            $"api/admin/error-
reports/{reportId}/frames/{frameIndex}/bb
oxes", ct);

        if
(response.IsSuccessStatusCode)
        {
            await using var stream =
await
response.Content.ReadAsStreamAsync(ct);
            var bboxes = await
JsonSerializer.DeserializeAsync<List<Yolo
BboxDto>>(stream, _jsonOptions, ct);
            return bboxes ?? new
List<YoloBboxDto>();
        }

        // Fallback для старой
версии API, где возвращалась только
одна рамка.

        using var fallbackResponse
= await _http.GetAsync(
            $"api/admin/error-
reports/{reportId}/frames/{frameIndex}/bb
ox", ct);

        if
(!fallbackResponse.IsSuccessStatusCode)
        {
            return new
List<YoloBboxDto>();
        }

        await using var
fallbackStream = await

```

```

fallbackResponse.Content.ReadAsStreamA
sync(ct);

        var bbox = await
JsonSerializer.DeserializeAsync<YoloBbox
Dto>({fallbackStream, _jsonOptions, ct);
        return bbox is null ? new
List<YoloBboxDto>() : new
List<YoloBboxDto> { bbox };
    }

    public async Task
SetReportApprovedAsync(int reportId, bool
approved, CancellationToken ct = default)
    {
        var body =
JsonSerializer.Serialize(new
ApproveErrorReportRequest(approved));
        using var content = new
StringContent(body, Encoding.UTF8,
"application/json");

        using var response = await
_http.PutAsync(
            $"api/admin/error-
reports/{reportId}/approve", content, ct);

        response.EnsureSuccessStatusCode();
    }

    public async Task
DeleteReportAsync(int reportId,
CancellationToken ct = default)
    {
        using var response = await
_http.DeleteAsync(
            $"api/admin/error-
reports/{reportId}", ct);

        response.EnsureSuccessStatusCode();
    }

    public async Task
BlockUserAsync(int userId, bool isBlocked,
CancellationToken ct = default)
    {

```

```

        var body =
JsonSerializer.Serialize(new
SetBlockedRequest(isBlocked));
        using var content = new
StringContent(body, Encoding.UTF8,
"application/json");

        using var response = await
_http.PutAsync(
            $"api/admin/users/{userId}/block", content,
ct);

        response.EnsureSuccessStatusCode();
    }

    public async
Task<TrainingJobStartResponse>
StartTrainingAsync(StartTrainingRequest
request, CancellationToken ct = default)
    {
        using var response = await
_http.PostAsJsonAsync("api/admin/training
/start", request, ct);

        response.EnsureSuccessStatusCode();

        var result = await
response.Content.ReadFromJsonAsync<Tra
iningJobStartResponse>(_jsonOptions, ct);
        return result ?? throw new
Exception("Пустой ответ от сервера.");
    }

    public async
Task<List<TrainingJobStatusResponse>>
GetTrainingJobsAsync(CancellationToken
ct = default)
    {
        var result = await
_http.GetFromJsonAsync<List<TrainingJob
StatusResponse>>(
            "api/admin/training/jobs", _jsonOptions,
ct);
    }

```

```

        return result ?? new
List<TrainingJobStatusResponse>();
    }

    public async
Task<TrainingJobStatusResponse>
GetTrainingJobAsync(string jobId,
CancellationTokens ct = default)
=> await
_http.GetFromJsonAsync<TrainingJobStatu
sResponse>($"api/admin/training/jobs/{jobI
d}", _jsonOptions, ct);

    public async
Task<TrainingJobStatusResponse>
CancelTrainingJobAsync(string jobId,
CancellationTokens ct = default)
    {
        using var response = await
_http.PostAsync($"api/admin/training/jobs/
{jobId}/cancel", content: null, ct);

        response.EnsureSuccessStatusCode();

        var result = await
response.Content.ReadFromJsonAsync<Tra
iningJobStatusResponse>(_jsonOptions,
ct);

        return result ?? throw new
Exception("Пустой ответ от сервера при
остановке задачи.");
    }

    public async
Task<List<ModelVersionAdminResponse>
>
GetModelVersionsAsync(CancellationTokes
n ct = default)
    {
        var result = await
_http.GetFromJsonAsync<List<ModelVersi
onAdminResponse>>(
"api/admin/training/model-versions",
_jsonOptions, ct);

```

```

        return result ?? new
List<ModelVersionAdminResponse>();
    }

    public async
Task<ModelVersionAdminResponse>
PublishModelVersionAsync(int id,
CancellationTokens ct = default)
    {
        using var response = await
_http.PostAsync($"api/admin/training/mode
l-versions/{id}/publish", content: null, ct);

        response.EnsureSuccessStatusCode();

        return await
ReadModelVersionResponseAsync(respons
e, ct);
    }

    public async
Task<ModelVersionAdminResponse>
UnpublishModelVersionAsync(int id,
CancellationTokens ct = default)
    {
        using var response = await
_http.PostAsync($"api/admin/training/mode
l-versions/{id}/unpublish", content: null,
ct);

        response.EnsureSuccessStatusCode();

        return await
ReadModelVersionResponseAsync(respons
e, ct);
    }

    public async
Task<ModelVersionAdminResponse>
PinModelVersionAsync(int id,
CancellationTokens ct = default)
    {
        using var response = await
_http.PostAsync($"api/admin/training/mode
l-versions/{id}/pin", content: null, ct);

        response.EnsureSuccessStatusCode();

        return await
ReadModelVersionResponseAsync(respons
e, ct);
    }

```

```

    }

    public async
Task<ModelVersionAdminResponse>
UnpinModelVersionAsync(int id,
CancellationTokens ct = default)
    {
        using var response = await
_http.PostAsync($"api/admin/training/mode
l-versions/{id}/unpin", content: null, ct);

        response.EnsureSuccessStatusCode();

        return await
ReadModelVersionResponseAsync(respons
e, ct);
    }

    public async Task
DeleteModelVersionAsync(int id, bool
force = false, CancellationTokens ct =
default)
    {
        using var response = await
_http.DeleteAsync($"api/admin/training/mo
del-
versions/{id}?force={force.ToString().ToL
owerInvariant()}", ct);

        response.EnsureSuccessStatusCode();
    }

    public async Task<string>
ExportBackupAsync(string destinationPath,
CancellationTokens ct = default)
    {
        using var response = await
_http.GetAsync("api/admin/backup/export",
HttpCompletionOption.ResponseHeadersRe
ad, ct);

        response.EnsureSuccessStatusCode();

        var fileName =
response.Content.Headers.ContentDispositi
on?.FileNameStar

```

```

        ??
        response.Content.Headers.ContentDisposition?.FileName?.Trim("\\"))
        ??
        Path.GetFileName(destinationPath);

        var directory =
        Path.GetDirectoryName(destinationPath);
        if
        (!string.IsNullOrEmptyOrWhiteSpace(directory))

        Directory.CreateDirectory(directory);

        await using var source =
        await
        response.Content.ReadAsStreamAsync(ct);
        await using var destination
        = File.Create(destinationPath);
        await
        source.CopyToAsync(destination, ct);

        return fileName;
    }

    public async
    Task<BackupImportResultDto>
    ImportBackupAsync(string backupZipPath,
    bool replaceExisting, CancellationToken ct
    = default)
    {
        await using var fileStream =
        File.OpenRead(backupZipPath);
        using var content = new
        MultipartFormDataContent();

        var fileContent = new
        StreamContent(fileStream);

        fileContent.Headers.ContentType = new
        MediaTypeHeaderValue("application/zip");

        content.Add(fileContent,
        "BackupZip",
        Path.GetFileName(backupZipPath));
        content.Add(new
        StringContent(replaceExisting.ToString().To
        oLowerInvariant()), "ReplaceExisting");

```

```

        using var response = await
        _http.PostAsync("api/admin/backup/import
        ", content, ct);

        if
        (!response.IsSuccessStatusCode)
        {
            var error = await
            response.Content.ReadAsStringAsync(ct);
            throw new
            HttpRequestException(string.IsNullOrEmptyWhit
            eSpace(error)
            ? $"Ошибка импорта
            backup: {(int)response.StatusCode}
            {response.ReasonPhrase}"
            : error);
        }

        var result = await
        response.Content.ReadFromJsonAsync<Ba
        ckupImportResultDto>(_jsonOptions, ct);
        return result ?? throw new
        Exception("Пустой ответ от сервера
        после импорта backup.");
    }

    private async
    Task<ModelVersionAdminResponse>
    ReadModelVersionResponseAsync(HttpRe
    sponseMessage response,
    CancellationToken ct)
    {
        var result = await
        response.Content.ReadFromJsonAsync<Mo
        delVersionAdminResponse>(_jsonOptions,
        ct);
        return result ?? throw new
        Exception("Пустой ответ от сервера по
        версии модели.");
    }

    public static async
    Task<string> LoginAsync(string baseUrl,
    string email, string password,
    CancellationToken ct = default)
    {

```

```

        using var http = new
        HttpClient { BaseAddress = new
        Uri(baseUrl) };

        var body =
        JsonSerializer.Serialize(new
        LoginRequest(email, password));
        using var content = new
        StringContent(body, Encoding.UTF8,
        "application/json");

        using var response = await
        http.PostAsync("api/Auth/login", content,
        ct);
        response.EnsureSuccessStatusCode();

        await using var stream =
        await
        response.Content.ReadAsStreamAsync(ct);
        var auth = await
        JsonSerializer.DeserializeAsync<AuthResp
        onse>(stream, new JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true
        }, ct);

        if (auth == null ||
        string.IsNullOrEmptyOrWhiteSpace(auth.AccessTok
        en))
            throw new
            Exception("Не удалось получить токен.");

        return auth.AccessToken;
    }
}

=====
=====
=====
=====
Файл:
AdminApp/AdminApp.csproj
=====
=====
=====
=====

```

| | | |
|---|--|---|
| <pre> <Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>WinExe</OutputType> <TargetFramework>net8.0- windows</TargetFramework> <Nullable>enable</Nullable> <UseWindowsForms>true</UseWindowsF orms> <ImplicitUsings>enable</ImplicitUsings> </PropertyGroup> <ItemGroup> <ProjectReference Include="..\Contracts\Contracts.csproj" /> </ItemGroup> <ItemGroup> <None Update="Settings.json"> <CopyToOutputDirectory>PreserveNewest </CopyToOutputDirectory> </None> </ItemGroup> </Project> ===== ===== ===== ===== Файл: AdminApp/AdminBackupDtos.cs ===== ===== ===== ===== namespace AdminApp; public sealed record BackupImportResultDto(</pre> | <pre> int Users, int Products, int StoredProducts, int VideoSamples, int ErrorReports, int ModelVersions, int TrainingJobs, bool ReplacedExisting); ===== ===== ===== ===== Файл: AdminApp/AdminSettings.cs ===== ===== ===== ===== using System.Text.Json; namespace AdminApp; public sealed class AdminSettings { private const string SettingsFileName = "Settings.json"; public string ApiBaseUrl { get; set; } = "http://localhost:5099/"; public static AdminSettings Load() { var path = Path.Combine(AppContext.BaseDirectory, SettingsFileName); if (!File.Exists(path)) { var defaultSettings = new AdminSettings(); defaultSettings.Save(path); return defaultSettings; } </pre> | <pre> var json = File.ReadAllText(path); var settings = JsonSerializer.Deserialize<AdminSettings> (json, new JsonSerializerOptions { PropertyNameCaseInsensitive = true }) ?? new AdminSettings(); settings.ApiBaseUrl = NormalizeBaseUrl(settings.ApiBaseUrl); return settings; } private void Save(string path) { var json = JsonSerializer.Serialize(this, new JsonSerializerOptions { WriteIndented = true }); File.WriteAllText(path, json); } private static string NormalizeBaseUrl(string? value) { if (string.IsNullOrEmpty(value)) throw new InvalidOperationException(\$"B {SettingsFileName} не указан ApiBaseUrl."); var trimmed = value.Trim(); if (!Uri.TryCreate(trimmed, UriKind.Absolute, out var uri) (uri.Scheme != Uri.UriSchemeHttp && uri.Scheme != Uri.UriSchemeHttps)) { </pre> |
|---|--|---|

```

        throw new
InvalidOperationException(
        $"Некорректный
ApiBaseUrl в {SettingsFileName}:
'{value}'. Пример:
http://111.88.146.2:5099/");
    }

    return
trimmed.EndsWith('/') ? trimmed : trimmed
+ "/";
    }
}

=====
=====
=====
=====

Файл:
AdminApp/FormBackup.cs

=====
=====
=====
=====

namespace AdminApp;

public sealed class FormBackup
: Form
{
    private readonly
AdminApiClient _api;
    private readonly Label
_lblStatus = new();
    private readonly Button
_btnExport = new();
    private readonly Button
_btnImport = new();
    private readonly CheckBox
_chkReplaceExisting = new();

    public
FormBackup(AdminApiClient api)
    {
        _api = api;

        Text = "Резервная копия";
        Width = 620;

```

```

        Height = 330;
        StartPosition =
FormStartPosition.CenterParent;
        FormBorderStyle =
FormBorderStyle.FixedDialog;
        MaximizeBox = false;
        MinimizeBox = false;

        BuildUi();
    }

    private void BuildUi()
    {
        Controls.Clear();

        var title = new Label
        {
            Text = "Экспорт /
импорт данных",
            AutoSize = true,
            Font = new Font("Segoe
UI", 15, FontStyle.Bold),
            Location = new Point(20,
20)
        };

        var description = new Label
        {
            Text = "Backup ZIP
содержит базу данных, модели, датасеты
обучения и сообщения об ошибках.",
            AutoSize = false,
            Width = 560,
            Height = 45,
            Location = new Point(20,
60)
        };

        _btnExport.Text =
"Экспортировать в ZIP";
        _btnExport.Width = 230;
        _btnExport.Height = 44;
        _btnExport.Location = new
Point(20, 120);
        _btnExport.Click += async
(, _) => await ExportAsync();

```

```

        _btnImport.Text =
"Импортировать ZIP";
        _btnImport.Width = 230;
        _btnImport.Height = 44;
        _btnImport.Location = new
Point(270, 120);
        _btnImport.Click += async
(, _) => await ImportAsync();

        _chkReplaceExisting.Text
= "При импорте полностью заменить
текущую базу и файлы";

        _chkReplaceExisting.AutoSize = true;

        _chkReplaceExisting.Location = new
Point(20, 185);

        var warning = new Label
        {
            Text = "Внимание:
режим полной замены удалит текущие
записи, модели и загруженные файлы на
backend перед восстановлением из ZIP.",
            AutoSize = false,
            Width = 560,
            Height = 45,
            Location = new Point(40,
212)
        };

        _lblStatus.Text =
"Готово.";
        _lblStatus.AutoSize = false;
        _lblStatus.Width = 560;
        _lblStatus.Height = 36;
        _lblStatus.Location = new
Point(20, 255);

        Controls.Add(title);
        Controls.Add(description);
        Controls.Add(_btnExport);
        Controls.Add(_btnImport);

        Controls.Add(_chkReplaceExisting);
        Controls.Add(warning);
        Controls.Add(_lblStatus);
    }

```



```

private async Task
ExportAsync()
{
    using var dialog = new
SaveFileDialog
{
    Title = "Сохранить
backup ZIP",
    Filter = "ZIP archive
(*.zip)|*.zip",
    FileName =
$"productsdate_backup_{DateTime.Now:y
yyyMMdd_HHmss}.zip",
    AddExtension = true,
    DefaultExt = ".zip",
    OverwritePrompt = true
};

    if (dialog.ShowDialog(this)
!= DialogResult.OK)
        return;

    await
RunOperationAsync(async () =>
{
    _lblStatus.Text =
"Экспортирую backup с сервера...";

    await
_api.ExportBackupAsync(dialog.FileName)
;

    _lblStatus.Text =
$"Backup сохранён: {dialog.FileName}";

    MessageBox.Show(this,
"Backup успешно экспортирован.",
"Экспорт", MessageBoxButtons.OK,
MessageBoxIcon.Information);
});
}

private async Task
ImportAsync()
{
    using var dialog = new
OpenFileDialog
{
    Title = "Выбрать backup
ZIP",
    Filter = "ZIP archive
(*.zip)|*.zip",
    CheckFileExists = true,
    Multiselect = false
};

    if (dialog.ShowDialog(this)
!= DialogResult.OK)
        return;

    var replaceExisting =
_chkReplaceExisting.Checked;

    var message =
replaceExisting
    ? "Импорт с полной
заменой удалит текущую базу и файлы
на сервере. Продолжить?"
    : "Импорт будет
выполнен только если база на сервере
пустая. Продолжить?";

    var confirm =
MessageBox.Show(this, message,
"Подтверждение импорта",
MessageBoxButtons.YesNo,
MessageBoxIcon.Warning);

    if (confirm !=
DialogResult.Yes)
        return;

    await
RunOperationAsync(async () =>
{
    _lblStatus.Text =
"Загружаю backup на сервер и
восстанавливаю данные...";

    var result = await
_api.ImportBackupAsync(dialog.FileName,
replaceExisting);

    _lblStatus.Text =
BuildImportSummary(result);

    MessageBox.Show(this,
BuildImportSummary(result), "Импорт
завершён", MessageBoxButtons.OK,
MessageBoxIcon.Information);
});
}

private async Task
RunOperationAsync(Func<Task>
operation)
{
    SetBusy(true);

    try
    {
        await operation();
    }
    catch (Exception ex)
    {
        _lblStatus.Text =
"Ошибка.";

        MessageBox.Show(this,
ex.Message, "Ошибка",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
    finally
    {
        SetBusy(false);
    }
}

private void SetBusy(bool
busy)
{
    _btnExport.Enabled =
!busy;

    _btnImport.Enabled =
!busy;

    _chkReplaceExisting.Enabled = !busy;

    Cursor = busy ?
Cursors.WaitCursor : Cursors.Default;
}

private static string
BuildImportSummary(BackupImportResult
Dto result)
{
    return $"Импорт
завершён. Пользователи: {result.Users},
продукты: {result.Products}, " +
        $"запасы:
{result.StoredProducts}, видео:
{result.VideoSamples}, ошибки:
{result.ErrorReports}, " +

```

```

        $"версии моделей:
{result.ModelVersions}, задачи обучения:
{result.TrainingJobs}.";
    }
}

=====
=====
=====

Файл:
AdminApp/FormErrorReportDetails.cs
=====
=====
=====

using Contracts.Dtos;

namespace AdminApp;

public partial class
FormErrorReportDetails : Form
{
    private readonly
AdminApiClient _api;
    private readonly int _reportId;

    private
AdminErrorReportDetailsDto? _report;
    private int
_currentFrameIndex = 1;

    private Label _lblInfo = null!;
    private Label
_lblFrameNumber = null!;
    private TextBox _txtComment
= null!;
    private PictureBox
_pictureBox = null!;
    private Button _btnPrev =
null!;
    private Button _btnNext =
null!;
    private Button _btnApprove =
null!;
    private Button _btnDelete =
null!;

```

```

private Button _btnBlockUser
= null!;
private Button _btnRefresh =
null!;

private Image? _currentImage;
private List<YoloBboxDto>
_currentBboxes = new();

public
FormErrorReportDetails(AdminApiClient
api, int reportId)
{
    InitializeComponent();

    _api = api;
    _reportId = reportId;

    Text = $"Сообщение
#{reportId}";
    Width = 1100;
    Height = 850;
    StartPosition =
FormStartPosition.CenterParent;

    BuildUi();

    Shown += async (_, _) =>
await LoadAllAsync();
}

private void BuildUi()
{
    Controls.Clear();

    _lblInfo = new Label
{
        AutoSize = false,
        Width = 1040,
        Height = 80,
        Location = new Point(20,
20),
        Font = new Font("Segoe
UI", 10, FontStyle.Regular)
    };

    _txtComment = new
TextBox

```

```

{
    Location = new Point(20,
110),
    Width = 1040,
    Height = 70,
    Multiline = true,
    ReadOnly = true
};

_pictureBox = new
PictureBox
{
    Location = new Point(20,
200),
    Width = 800,
    Height = 600,
    BorderStyle =
BorderStyle.FixedSingle,
    SizeMode =
PictureBoxSizeMode.Zoom
};
_pictureBox.Paint +=
PictureBox_Paint;

_lblFrameNumber = new
Label
{
    Location = new
Point(840, 220),
    Width = 220,
    Height = 30,
    Font = new Font("Segoe
UI", 10, FontStyle.Bold)
};

_btnPrev = new Button
{
    Text = "←",
    Width = 80,
    Height = 40,
    Location = new
Point(840, 270)
};
_btnPrev.Click += async (_,
_) =>
{
    if (_report is null ||
_currentFrameIndex <= 1) return;

```

```

        _currentFrameIndex--;
        await
LoadFrameAsync();
    };

    _btnNext = new Button
    {
        Text = "→",
        Width = 80,
        Height = 40,
        Location = new
Point(940, 270)
    };
    _btnNext.Click += async
(, _) =>
    {
        if (_report is null ||
_currentFrameIndex >=
_report.FramesCount) return;
        _currentFrameIndex++;
        await
LoadFrameAsync();
    };

    _btnApprove = new Button
    {
        Text = "Approve /
Unapprove",
        Width = 220,
        Height = 40,
        Location = new
Point(840, 350)
    };
    _btnApprove.Click +=
async (, _) => await
ToggleApproveAsync();

    _btnDelete = new Button
    {
        Text = "Удалить
репорт",
        Width = 220,
        Height = 40,
        Location = new
Point(840, 410)
    };
    _btnDelete.Click += async
(, _) => await DeleteReportAsync();

```

```

        _btnBlockUser = new
Button
    {
        Text = "Заблокировать
пользователя",
        Width = 220,
        Height = 40,
        Location = new
Point(840, 470)
    };
    _btnBlockUser.Click +=
async (, _) => await BlockUserAsync();

    _btnRefresh = new Button
    {
        Text = "Обновить",
        Width = 220,
        Height = 40,
        Location = new
Point(840, 530)
    };
    _btnRefresh.Click += async
(, _) => await LoadAllAsync();

    Controls.Add(_lblInfo);
    Controls.Add(_txtComment);
    Controls.Add(_pictureBox);
    Controls.Add(_lblFrameNumber);
    Controls.Add(_btnPrev);
    Controls.Add(_btnNext);
    Controls.Add(_btnApprove);
    Controls.Add(_btnDelete);
    Controls.Add(_btnBlockUser);
    Controls.Add(_btnRefresh);
    }

    private async Task
LoadAllAsync()
    {
        try
        {
            UseWaitCursor = true;

```

```

        _report = await
_api.GetErrorReportAsync(_reportId);
        if (_report == null)
        {
            MessageBox.Show(this, "Репорт не
найден", "Ошибка",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
            Close();
            return;
        }

        if (_currentFrameIndex <
1) _currentFrameIndex = 1;
        if (_currentFrameIndex >
_report.FramesCount) _currentFrameIndex
= _report.FramesCount;

        _lblInfo.Text =
 $"ReportId:
{_report.Id}\r\n" +
 $"UserId:
{_report.UserId}\r\n" +
 $"CreatedAt:
{_report.CreatedAt}\r\n" +
 $"FramesCount:
{_report.FramesCount}\r\n" +
 $"Approved:
{_report.Approved}";

        _txtComment.Text =
_report.Comment ?? "";

        await
LoadFrameAsync();
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
ex.Message, "Ошибка",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
    finally
    {
        UseWaitCursor = false;
    }

```

```

    }

    private async Task
LoadFrameAsync()
    {
        if (_report == null) return;

        try
        {
            UseWaitCursor = true;

            _currentImage?.Dispose();
            _currentImage = await
            _api.GetFrameImageAsync(_report.Id,
            _currentFrameIndex);

            _currentBboxes = await
            _api.GetFrameBboxesAsync(_report.Id,
            _currentFrameIndex);

            _pictureBox.Image =
            _currentImage;
            _lblFrameNumber.Text =
            $"Кадр { _currentFrameIndex } /
            { _report.FramesCount }";
            _pictureBox.Invalidate();
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
            ex.Message, "Ошибка загрузки кадра",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
        finally
        {
            UseWaitCursor = false;
        }
    }

    private async Task
ToggleApproveAsync()
    {
        if (_report == null) return;

        try
        {

```

```

            await
            _api.SetReportApprovedAsync(_report.Id,
            !_report.Approved);

            await LoadAllAsync();
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
            ex.Message, "Ошибка",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }

    private async Task
DeleteReportAsync()
    {
        if (_report == null) return;

        var confirm =
        MessageBox.Show(
            this,
            $"Удалить репорт
            #{_report.Id}?",
            "Подтверждение",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (confirm !=
        DialogResult.Yes)
            return;

        try
        {
            await
            _api.DeleteReportAsync(_report.Id);

            MessageBox.Show(this,
            "Репорт удалён", "OK",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information);

            Close();
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
            ex.Message, "Ошибка удаления",

```

```

            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }

    private async Task
BlockUserAsync()
    {
        if (_report == null) return;

        var confirm =
        MessageBox.Show(
            this,
            $"Заблокировать
            пользователя #{_report.UserId}?",
            "Подтверждение",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (confirm !=
        DialogResult.Yes)
            return;

        try
        {
            await
            _api.BlockUserAsync(_report.UserId, true);

            MessageBox.Show(this,
            "Пользователь заблокирован", "OK",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information);
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
            ex.Message, "Ошибка блокировки",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }

    private void
PictureBox_Paint(object? sender,
PaintEventArgs e)
    {

```

```

        if (_pictureBox.Image ==
null || _currentBboxes.Count == 0)
            return;

        var img =
_pictureBox.Image;
        var imgW = img.Width;
        var imgH = img.Height;
        var rect =
GetImageDisplayRectangle(_pictureBox);

        float scaleX = rect.Width /
(float)imgW;
        float scaleY = rect.Height /
(float)imgH;

        using var pen = new
Pen(Color.Red, 3);

        foreach (var bbox in
_currentBboxes)
        {
            var bboxW =
(float)(bbox.W * imgW);
            var bboxH =
(float)(bbox.H * imgH);
            var centerX =
(float)(bbox.Xc * imgW);
            var centerY =
(float)(bbox.Yc * imgH);

            var left = centerX -
bboxW / 2f;
            var top = centerY -
bboxH / 2f;

            var drawLeft = rect.X +
left * scaleX;
            var drawTop = rect.Y +
top * scaleY;
            var drawWidth = bboxW
* scaleX;
            var drawHeight = bboxH
* scaleY;

e.Graphics.DrawRectangle(pen, drawLeft,
drawTop, drawWidth, drawHeight);

```

```

        }
    }

    private static Rectangle
GetImageDisplayRectangle(PictureBox pb)
    {
        if (pb.Image == null)
            return
pb.ClientRectangle;

        var img = pb.Image;
        float imageRatio =
(float)img.Width / img.Height;
        float boxRatio =
(float)pb.ClientSize.Width /
pb.ClientSize.Height;

        if (imageRatio > boxRatio)
        {
            int width =
pb.ClientSize.Width;
            int height = (int)(width /
imageRatio);
            int top =
(pb.ClientSize.Height - height) / 2;
            return new Rectangle(0,
top, width, height);
        }
        else
        {
            int height =
pb.ClientSize.Height;
            int width = (int)(height *
imageRatio);
            int left =
(pb.ClientSize.Width - width) / 2;
            return new
Rectangle(left, 0, width, height);
        }
    }
}

=====
=====
=====

```

Файл:
AdminApp/FormErrorReportDetails.Designer.cs

```

=====
=====
=====
=====

namespace AdminApp
{
    partial class
FormErrorReportDetails
    {
        /// <summary>
        /// Required designer
variable.
        /// </summary>
        private
System.ComponentModel.IContainer
components = null;

        /// <summary>
        /// Clean up any resources
being used.
        /// </summary>
        /// <param
name="disposing">true if managed
resources should be disposed; otherwise,
false.</param>

        protected override void
Dispose(bool disposing)
        {
            if (disposing &&
(components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form
Designer generated code

        /// <summary>
        /// Required method for
Designer support - do not modify
        /// the contents of this
method with the code editor.

```

```

        /// </summary>
        private void
InitializeComponent()
        {
            this.components = new
System.ComponentModel.Container();
            this.AutoScaleMode =
System.Windows.Forms.AutoScaleMode.F
ont;
            this.ClientSize = new
System.Drawing.Size(800, 450);
            this.Text =
"FormErrorReportDetails";
        }

        #endregion
    }
}

```

```

=====
=====
=====
=====
Файл:
AdminApp/FormErrorReports.cs
=====
=====
=====
=====

```

```

using Contracts.Dtos;

namespace AdminApp;

public partial class
FormErrorReports : Form
{
    private readonly
AdminApiClient _api;
    private readonly
BindingSource _binding = new();

    private DataGridView _grid =
null!;
    private Button _btnRefresh =
null!;
    private Button _btnOpen =
null!;

```

```

        public
FormErrorReports(AdminApiClient api)
        {
            InitializeComponent();

            _api = api;

            Text = "Сообщения об
ошибках";
            Width = 900;
            Height = 600;
            StartPosition =
FormStartPosition.CenterParent;

            BuildUi();

            Shown += async (_, _) =>
await LoadReportsAsync();
        }

```

```

        private void BuildUi()
        {
            Controls.Clear();

            _grid = new DataGridView
            {
                Dock = DockStyle.Top,
                Height = 480,
                ReadOnly = true,
                SelectionMode =
DataGridViewSelectionMode.FullRowSele
ct,

                MultiSelect = false,
                AutoGenerateColumns =
false,

                AllowUserToAddRows =
false,

                AllowUserToDeleteRows = false
            };

```

```

            _grid.Columns.Add(new
DataGridViewTextBoxColumn
            {
                HeaderText = "Id",
                DataPropertyName =
"Id",

```

```

                Width = 80
            });

            _grid.Columns.Add(new
DataGridViewTextBoxColumn
            {
                HeaderText = "UserId",
                DataPropertyName =
"UserId",
                Width = 80
            });

            _grid.Columns.Add(new
DataGridViewTextBoxColumn
            {
                HeaderText = "Дата",
                DataPropertyName =
"CreatedAt",
                Width = 200
            });

            _grid.Columns.Add(new
DataGridViewTextBoxColumn
            {
                HeaderText = "Кол-во
кадров",
                DataPropertyName =
"FramesCount",
                Width = 120
            });

            _grid.Columns.Add(new
DataGridViewCheckBoxColumn
            {
                HeaderText =
"Approved",
                DataPropertyName =
"Approved",
                Width = 90
            });

            _grid.DataSource =
_binding;

            _grid.DoubleClick += (_, _)
=> OpenSelected();

            _btnRefresh = new Button
            {

```

```

        Text = "Обновить",
        Width = 120,
        Height = 35,
        Location = new Point(20,
500)
    };
    _btnRefresh.Click += async
    (_, _) => await LoadReportsAsync();

    _btnOpen = new Button
    {
        Text = "Открыть
подробнее",
        Width = 180,
        Height = 35,
        Location = new
Point(160, 500)
    };
    _btnOpen.Click += (_, _)
=> OpenSelected();

    Controls.Add(_grid);
    Controls.Add(_btnRefresh);
    Controls.Add(_btnOpen);
    }

    private async Task
LoadReportsAsync()
    {
        try
        {
            UseWaitCursor = true;
            var items = await
_api.GetErrorReportsAsync();
            _binding.DataSource =
items;
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
ex.Message, "Ошибка",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
        }
        finally
        {
            UseWaitCursor = false;
        }
    }

```

```

    }

    private void OpenSelected()
    {
        if (_binding.Current is not
AdminErrorReportListItemDto item)
            return;

        var form = new
FormErrorReportDetails(_api, item.Id);
        form.ShowDialog(this);
    }

    =====
    =====
    =====
    =====

    Файл:
AdminApp/FormErrorReports.Designer.cs
    =====
    =====
    =====
    =====

    namespace AdminApp
    {
        partial class
FormErrorReports
        {
            /// <summary>
            /// Required designer
variable.
            /// </summary>
            private
System.ComponentModel.IContainer
components = null;

            /// <summary>
            /// Clean up any resources
being used.
            /// </summary>
            /// <param
name="disposing">true if managed
resources should be disposed; otherwise,
false.</param>
            protected override void
Dispose(bool disposing)

```

```

        {
            if (disposing &&
(components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form
Designer generated code

        /// <summary>
        /// Required method for
Designer support - do not modify
        /// the contents of this
method with the code editor.
        /// </summary>
        private void
InitializeComponent()
        {
            this.components = new
System.ComponentModel.Container();
            this.AutoScaleMode =
System.Windows.Forms.AutoScaleMode.F
ont;
            this.ClientSize = new
System.Drawing.Size(800, 450);
            this.Text =
"FormErrorReports";
        }

        #endregion
    }

    =====
    =====
    =====
    =====

    Файл: AdminApp/FormLogin.cs
    =====
    =====
    =====
    =====

    namespace AdminApp;

```

```

        public partial class FormLogin :
        Form
        {
            private readonly
            AdminSettings _settings;

            public string? Token { get;
            private set; }

            private TextBox _txtEmail =
            null!;

            private TextBox _txtPassword
            = null!;

            private Button _btnLogin =
            null!;

            public FormLogin()
            {
                _settings =
                AdminSettings.Load();

                InitializeComponent();

                Text = "Вход в админку";
                Width = 400;
                Height = 220;
                StartPosition =
                FormStartPosition.CenterScreen;

                BuildUi();
            }

            private void BuildUi()
            {
                Controls.Clear();

                var lblEmail = new Label
                {
                    Text = "Email",
                    Left = 20,
                    Top = 20,
                    Width = 100
                };

                _txtEmail = new TextBox
                {
                    Left = 20,
                    Top = 45,
                    Width = 320
                };

                var lblPassword = new
                Label
                {
                    Text = "Пароль",
                    Left = 20,
                    Top = 80,
                    Width = 100
                };

                _txtPassword = new
                TextBox
                {
                    Left = 20,
                    Top = 105,
                    Width = 320,
                    UseSystemPasswordChar
                    = true
                };

                _btnLogin = new Button
                {
                    Text = "Войти",
                    Left = 20,
                    Top = 145,
                    Width = 120,
                    Height = 35
                };

                _btnLogin.Click += async
                (_, _) => await LoginAsync();

                Controls.Add(lblEmail);
                Controls.Add(_txtEmail);
                Controls.Add(lblPassword);
                Controls.Add(_txtPassword);
                Controls.Add(_btnLogin);
            }

            private async Task
            LoginAsync()
            {
                try
                {
                    UseWaitCursor = true;

                    _btnLogin.Enabled =
                    false;

                    var token = await
                    AdminApiClient.LoginAsync(
                        _settings.ApiBaseUrl,
                        _txtEmail.Text.Trim(),
                        _txtPassword.Text
                    );

                    Token = token;
                    DialogResult =
                    DialogResult.OK;
                    Close();
                }
                catch (Exception ex)
                {
                    MessageBox.Show(this,
                    ex.Message, "Ошибка входа",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Error);
                }
                finally
                {
                    UseWaitCursor = false;
                    _btnLogin.Enabled =
                    true;
                }
            }
        }
    }

    namespace AdminApp
    {
        partial class FormLogin
        {
            /// <summary>

```


| | | |
|--|--|--|
| <pre> /// Required designer variable. /// </summary> private System.ComponentModel.IContainer components = null; /// <summary> /// Clean up any resources being used. /// </summary> /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param> protected override void Dispose(bool disposing) { if (disposing && (components != null)) { components.Dispose(); } base.Dispose(disposing); } #region Windows Form Designer generated code /// <summary> /// Required method for Designer support - do not modify /// the contents of this method with the code editor. /// </summary> private void InitializeComponent() { this.components = new System.ComponentModel.Container(); this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.F ont; this.ClientSize = new System.Drawing.Size(800, 450); this.Text = "FormLogin"; } </pre> | <pre> #endregion } } ===== ===== ===== Файл: AdminApp/FormMain.cs ===== ===== ===== namespace AdminApp; public partial class FormMain : Form { private readonly string _token; private readonly AdminApiClient _api; public FormMain(string token) { InitializeComponent(); _token = token; var settings = AdminSettings.Load(); _api = new AdminApiClient(settings.ApiBaseUrl, _token); Text = "Админка"; Width = 560; Height = 390; StartPosition = FormStartPosition.CenterScreen; BuildUi(); } private void BuildUi() { Controls.Clear(); </pre> | <pre> var lblTitle = new Label { Text = "Панель администратора", AutoSize = true, Font = new Font("Segoe UI", 16, FontStyle.Bold), Location = new Point(20, 20) }; var btnErrorReports = new Button { Text = "Сообщения об ошибках", Width = 220, Height = 50, Location = new Point(20, 80) }; btnErrorReports.Click += (, _) => { var form = new FormErrorReports(_api); form.ShowDialog(this); }; var btnTraining = new Button { Text = "Обучение модели", Width = 220, Height = 50, Location = new Point(260, 80) }; btnTraining.Click += (, _) => { using var form = new FormTraining(_api); form.ShowDialog(this); } </pre> |
|--|--|--|

```

        };

        var btnModelVersions =
new Button
    {
        Text = "Версии
моделей",
        Width = 220,
        Height = 50,
        Location = new Point(20,
150)
    };

    btnModelVersions.Click +=
(, _) =>
    {
        using var form = new
FormModelVersions(_api);
        form.ShowDialog(this);
    };

    var btnBackup = new
Button
    {
        Text = "Backup /
восстановление",
        Width = 220,
        Height = 50,
        Location = new
Point(260, 150)
    };

    btnBackup.Click += (, _)
=>
    {
        using var form = new
FormBackup(_api);
        form.ShowDialog(this);
    };

    Controls.Add(lblTitle);

    Controls.Add(btnErrorReports);
    Controls.Add(btnTraining);

    Controls.Add(btnModelVersions);
    Controls.Add(btnBackup);
}

```

```

    }

    =====
    =====
    =====
    =====
    Файл:
AdminApp/FormMain.Designer.cs
    =====
    =====
    =====

namespace AdminApp
{
    partial class FormMain
    {
        /// <summary>
        /// Required designer
variable.
        /// </summary>
        private
System.ComponentModel.IContainer
components = null;

        /// <summary>
        /// Clean up any resources
being used.
        /// </summary>
        /// <param
name="disposing">true if managed
resources should be disposed; otherwise,
false.</param>
        protected override void
Dispose(bool disposing)
        {
            if (disposing &&
(components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form
Designer generated code

        /// <summary>

```

```

        /// Required method for
Designer support - do not modify
        /// the contents of this
method with the code editor.
        /// </summary>
        private void
InitializeComponent()
        {
            this.components = new
System.ComponentModel.Container();
            this.AutoScaleMode =
System.Windows.Forms.AutoScaleMode.F
ont;

            this.ClientSize = new
System.Drawing.Size(800, 450);
            this.Text = "FormMain";
        }

        #endregion
    }
}

    =====
    =====
    =====
    Файл:
AdminApp/FormModelVersionDetails.cs
    =====
    =====
    =====

using Contracts.Dtos;
using System.Globalization;
using System.Text.Json;

namespace AdminApp;

public sealed class
FormModelVersionDetails : Form
{
    private readonly
AdminApiClient _api;
    private readonly int
_modelVersionId;
    private
ModelVersionAdminResponse? _model;

```

```

        private readonly Label
        _lblIdValue = CreateValueLabel();
        private readonly Label
        _lblStateValue = CreateValueLabel();
        private readonly Label
        _lblTrainedAtValue = CreateValueLabel();
        private readonly Label
        _lblDeletedAtValue = CreateValueLabel();
        private readonly Label
        _lblBaseModelValue =
        CreateValueLabel();
        private readonly Label
        _lblMobileFormatValue =
        CreateValueLabel();
        private readonly Label
        _lblJobIdValue = CreateValueLabel();
        private readonly Label
        _lblContentTypeValue =
        CreateValueLabel();

        private readonly
        DataGridView _gridMetrics = new()
        {
            Dock = DockStyle.Fill,
            ReadOnly = true,
            AllowUserToAddRows =
            false,
            AllowUserToDeleteRows =
            false,
            AutoGenerateColumns =
            false,
            SelectionMode =
            DataGridViewSelectionMode.FullRowSele
            ct,
            MultiSelect = false,
            RowHeadersVisible = false,
            AutoSizeRowsMode =
            DataGridViewAutoSizeRowsMode.AllCell
            s,
            DefaultCellStyle = new
            DataGridViewCellStyle { WrapMode =
            DataGridViewTriState.True },
        };

        private readonly TextBox
        _txtFiles = new()
        {
            Multiline = true,
            ReadOnly = true,
            ScrollBars =
            ScrollBars.Vertical,
            Dock = DockStyle.Fill,
        };

        private readonly TextBox
        _txtRelatedJob = new()
        {
            Multiline = true,
            ReadOnly = true,
            ScrollBars =
            ScrollBars.Vertical,
            Dock = DockStyle.Fill,
        };

        private readonly Button
        _btnRefresh = new() { Text = "Обновить",
            Width = 120, Height = 36 };
        private readonly Button
        _btnOpenJob = new() { Text = "Открыть
        задачу обучения", Width = 210, Height =
        36 };
        private readonly Button
        _btnClose = new() { Text = "Закрыть",
            Width = 120, Height = 36 };

        public
        FormModelVersionDetails(AdminApiClient
        t api, ModelVersionAdminResponse model)
        : this(api, model.Id)
        {
            _model = model;
        }

        public
        FormModelVersionDetails(AdminApiClient
        t api, int modelVersionId)
        {
            _api = api;
            _modelVersionId =
            modelVersionId;

            Text = $"Версия модели
            #{modelVersionId}";
            Width = 1120;
            Height = 780;

            StartPosition =
            FormStartPosition.CenterParent;

            BuildUi();

            _btnRefresh.Click += async
            (_, _) => await LoadModelAsync();
            _btnOpenJob.Click += (_,
            _) => OpenRelatedJob();
            _btnClose.Click += (_, _)
            => Close();

            Shown += async (_, _) =>
            await LoadModelAsync();
        }

        private void BuildUi()
        {
            ConfigureMetricsGrid();

            var header = new
            TableLayoutPanel
            {
                Dock = DockStyle.Top,
                AutoSize = true,
                Padding = new
                Padding(12),
                ColumnCount = 4,
            };

            header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Absolute, 170));

            header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Percent, 50));

            header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Absolute, 170));

            header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Percent, 50));

            AddInfoRow(header, 0, "ID
            версии", _lblIdValue, "Состояние",
            _lblStateValue);

            AddInfoRow(header, 1,
            "Обучена", _lblTrainedAtValue,
            "Удалена", _lblDeletedAtValue);

```

```

        AddInfoRow(header, 2,
"Базовая модель", _lblBaseModelValue,
"Mobile формат",
_lblMobileFormatValue);
        AddInfoRow(header, 3,
"Job ID", _lblJobIdValue, "Content-Type",
_lblContentTypeValue);

```

```

        var actions = new
FlowLayoutPanel
        {
            Dock = DockStyle.Top,
            AutoSize = true,
            Padding = new
Padding(12, 0, 12, 12),
            FlowDirection =
FlowDirection.LeftToRight,
        };

```

```

actions.Controls.AddRange([_btnRefresh,
_btnOpenJob, _btnClose]);

```

```

        var content = new
TableLayoutPanel
        {
            Dock = DockStyle.Fill,
            Padding = new
Padding(12, 0, 12, 12),
            ColumnCount = 1,
            RowCount = 6,
        };

```

```

content.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

```

```

content.RowStyles.Add(new
RowStyle(SizeType.Percent, 52));

```

```

content.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

```

```

content.RowStyles.Add(new
RowStyle(SizeType.Percent, 24));

```

```

content.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

```

```

content.RowStyles.Add(new
RowStyle(SizeType.Percent, 24));

```

```

content.Controls.Add(CreateSectionLabel("
Метрики качества"), 0, 0);

```

```

content.Controls.Add(_gridMetrics, 0, 1);

```

```

content.Controls.Add(CreateSectionLabel("
Файлы модели на backend"), 0, 2);

```

```

content.Controls.Add(_txtFiles, 0, 3);

```

```

content.Controls.Add(CreateSectionLabel("
Связанная задача обучения"), 0, 4);

```

```

content.Controls.Add(_txtRelatedJob, 0, 5);

```

```

        Controls.Add(content);
        Controls.Add(actions);
        Controls.Add(header);
    }

```

```

        private void
ConfigureMetricsGrid()
        {

```

```

            _gridMetrics.Columns.Add(new
DataGridViewTextBoxColumn
            {

```

```

                HeaderText =
"Метрика",
                DataPropertyName =
nameof(MetricRow.Name),
                Width = 190
            });

```

```

            _gridMetrics.Columns.Add(new
DataGridViewTextBoxColumn
            {

```

```

                HeaderText =
"Значение",
                DataPropertyName =
nameof(MetricRow.Value),
                Width = 140
            });

```

```

            _gridMetrics.Columns.Add(new
DataGridViewTextBoxColumn
            {
                HeaderText =
"Пояснение",
                DataPropertyName =
nameof(MetricRow.Description),
                AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill
            });
        }

```

```

        private async Task
LoadModelAsync()
        {
            try
            {
                ToggleBusy(true);

```

```

                var versions = await
_api.GetModelVersionsAsync();
                var latest =
versions.FirstOrDefault(x => x.Id ==
_modelVersionId);
                if (latest is not null)
                    _model = latest;
                if (_model is null)
                {

```

```

                    MessageBox.Show(this, "Версия модели
не найдена на сервере.", "Версия
модели", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
                    Close();
                    return;
                }

```

```

                Render(_model);
                await
LoadRelatedJobAsync(_model.ExternalJobI
d);
            }
            catch (Exception ex)
            {
                MessageBox.Show(this,
ex.Message, "Ошибка загрузки версии

```

```

        модели", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private void
Render(ModelVersionAdminResponse
model)
{
    Text = $"Версия модели
#{model.Id}";
    _lblIdValue.Text =
model.Id.ToString(CultureInfo.InvariantCulture);
    _lblStateValue.Text =
BuildState(model);
    _lblTrainedAtValue.Text =
ToLocalString(model.TrainedAt);
    _lblDeletedAtValue.Text =
ToLocalString(model.DeletedAt);
    _lblBaseModelValue.Text
= model.BaseModel ?? "-";

    _lblMobileFormatValue.Text =
model.MobileFormat ?? "-";
    _lblJobIdValue.Text =
model.ExternalJobId ?? "-";
    _lblContentTypeValue.Text
= model.MobileModelContentType ?? "-";

    _txtFiles.Text =
string.Join(Environment.NewLine, new[]
{
        $"Mobile file name:
{model.MobileModelFileName ?? "-"}",
        $"Mobile model path:
{model.MobileModelPath ?? "-"}",
        $"Best weights path:
{model.BestWeightsPath ?? "-"}",
        $"Content-Type:
{model.MobileModelContentType ?? "-"}"
    });
}

```

```

        _gridMetrics.DataSource =
BuildMetricRows(model.MetricsJson);
        _btnOpenJob.Enabled =
!string.IsNullOrEmpty(model.External
JobId);
    }

    private async Task
LoadRelatedJobAsync(string? jobId)
    {
        if
(string.IsNullOrEmpty(jobId))
        {
            _txtRelatedJob.Text = "У
этой версии нет связанной задачи
обучения.";
            return;
        }

        try
        {
            var job = await
_api.GetTrainingJobAsync(jobId);
            if (job is null)
            {
                _txtRelatedJob.Text =
$"Связанная задача {jobId} не найдена.";
                return;
            }

            _txtRelatedJob.Text =
BuildJobSummary(job);
        }
        catch (Exception ex)
        {
            _txtRelatedJob.Text =
$"Не удалось загрузить связанную
задачу {jobId}." + Environment.NewLine
+ ex.Message;
        }
    }

    private void OpenRelatedJob()
    {
        var jobId =
_model?.ExternalJobId;
        if
(string.IsNullOrEmpty(jobId))

```

```

        return;

        using var form = new
FormTrainingJobDetails(_api, jobId);
        form.ShowDialog(this);
    }

    private void ToggleBusy(bool
busy)
    {
        UseWaitCursor = busy;
        _btnRefresh.Enabled =
!busy;
        _btnClose.Enabled = !busy;
        _btnOpenJob.Enabled =
!busy &&
!string.IsNullOrEmpty(_model?.Exter
nalJobId);
    }

    private static string
BuildState(ModelVersionAdminResponse
model)
    {
        var parts = new
List<string>();
        if (model.IsPublished)
            parts.Add("текущая для пользователей");
        if (model.IsPinned)
            parts.Add("зафиксирована");
        if (model.IsDeleted)
            parts.Add("удалена");
        return parts.Count == 0 ?
"обычная версия" : string.Join(", ", parts);
    }

    private static string
BuildJobSummary(TrainingJobStatusRespo
nse job)
    {
        =>
string.Join(Environment.NewLine, new[]
{
            $"Job ID: {job.JobId}",
            $"Статус:
{TranslateStatus(job.Status)}",
            $"Клиент обучения:
{job.ClientId ?? "-"}",

```

```

        $"Создано:
{ToLocalString(job.CreatedAt)}",
        $"Назначено клиенту:
{ToLocalString(job.AssignedAt)}",
        $"Запущено:
{ToLocalString(job.StartedAt)}",
        $"Последний heartbeat:
{ToLocalString(job.HeartbeatAt)}",
        $"Завершено:
{ToLocalString(job.FinishedAt)}",
        $"Датасет кадров:
{job.ImagesCount}",
        $"Параметры:
epochs={job.Epochs?.ToString(CultureInfo.
InvariantCulture) ?? "-"},
imgsz={job.ImgSize?.ToString(CultureInfo.
InvariantCulture) ?? "-"},
batch={job.Batch?.ToString(CultureInfo.In
variantCulture) ?? "-"}, device={job.Device
?? "-"}",
        $"Mobile export:
format={job.MobileFormat ?? "-"},
int8={BoolText(job.ExportInt8)},
nms={BoolText(job.ExportNms)},
fraction={job.QuantizationFraction?.ToStri
ng("0.###", CultureInfo.InvariantCulture)
?? "-"}",
        $"Сообщение:
{job.Message ?? "-}"
    });

    private static
List<MetricRow> BuildMetricRows(string?
metricsJson)
    {
        if
(string.IsNullOrEmpty(metricsJson))
        {
            return
            [
                new MetricRow
                {
                    Name = "Метрики
не сохранены",
                    Value = "-",
                    Description = "Для
этой версии модели backend не получил
JSON метрик от клиента обучения."
                }
            ];
        }
    }

```

```

    }
};

}

try
{
    using var document =
JsonDocument.Parse(metricsJson);
    var root =
document.RootElement;
    var rows = new
List<MetricRow>();

    AddMetric(rows, root,
"mAP50_95", "mAP@0.50:0.95",
"Главная итоговая метрика детектора.
Среднее качество обнаружения на
разных уровнях IoU от 0.50 до 0.95. Чем
выше, тем лучше.");

    AddMetric(rows, root,
"mAP50", "mAP@0.50", "Качество
обнаружения при мягком критерии
совпадения рамок. Чем выше, тем
лучше.");

    AddMetric(rows, root,
"mAP75", "mAP@0.75", "Качество
обнаружения при более строгом
сравнении рамок. Чем выше, тем
лучше.");

    AddMetric(rows, root,
"precision", "Precision", "Доля
правильных срабатываний среди всех
найденных моделью областей. Высокое
значение означает меньше ложных
срабатываний.");

    AddMetric(rows, root,
"recall", "Recall", "Доля реально
существующих областей даты, которые
модель смогла найти. Высокое значение
означает меньше пропусков.");

    AddMetric(rows, root,
"fitness", "Fitness", "Сводная служебная
метрика Ultralytics для сравнения
запусков обучения.");

    foreach (var property in
root.EnumerateObject())
    {

```

```

        if (rows.Any(x =>
x.RawKey == property.Name))
            continue;

        rows.Add(new
MetricRow
        {
            RawKey =
property.Name,
            Name =
property.Name,
            Value =
FormatMetricValue(property.Value),
            Description =
"Дополнительная метрика из JSON,
который вернул клиент обучения."
        });

        return rows.Count > 0
? rows
: [new MetricRow {
Name = "Метрики не распознаны", Value
= "-", Description = "JSON метрик есть,
но ожидаемые значения в нём не
найжены." }];
    }
    catch
    {
        return
        [
            new MetricRow
            {
                Name = "Metrics
JSON",
                Value =
metricsJson,
                Description = "Не
удалось разобрать JSON автоматически,
поэтому показано исходное значение."
            }
        ];
    }
}

private static void
AddMetric(List<MetricRow> rows,

```

```

JsonElement root, string key, string
displayName, string description)
{
    if
(!root.TryGetProperty(key, out var value))
        return;

    rows.Add(new MetricRow
    {
        RawKey = key,
        Name = displayName,
        Value =
FormatMetricValue(value),
        Description = description
    });
}

```

```

private static string
FormatMetricValue(JsonElement value)
{
    if (value.ValueKind ==
JsonValueKind.Number &&
value.TryGetDouble(out var number))
        return number >= 0 &&
number <= 1
        ? $"{{number:P2}}
({{number:F4}})"
        :
number.ToString("0.####",
CultureInfo.InvariantCulture);

    return value.ToString();
}

```

```

private static string
TranslateStatus(string? status)
=>
status?.ToLowerInvariant() switch
{
    "queued" => "В
очереди",
    "running" =>
"Выполняется",
    "completed" =>
"Завершена успешно",
    "failed" => "Завершена
с ошибкой",
}

```

```

"canceled" =>
"Остановлена",
_ => status ?? "-"
};

private static string
BoolText(bool? value)
=> value.HasValue ?
(value.Value ? "да" : "нет") : "-";

private static string
ToLocalString(DateTime value)
=>
value.ToLocalTime().ToString("dd.MM.yy
yy HH:mm:ss",
CultureInfo.InvariantCulture);

```

```

private static string
ToLocalString(DateTime? value)
=>
value?.ToLocalTime().ToString("dd.MM.y
yyy HH:mm:ss",
CultureInfo.InvariantCulture) ?? "-";

```

```

private static Label
CreateSectionLabel(string text)
=> new()
{
    Text = text,
    Dock = DockStyle.Fill,
    AutoSize = true,
    Font = new
Font(SystemFonts.DefaultFont,
FontStyle.Bold),
    Padding = new
Padding(0, 8, 0, 8)
};

```

```

private static Label
CreateValueLabel()
=> new()
{
    AutoSize = true,
    Anchor =
AnchorStyles.Left,
    MaximumSize = new
Size(430, 0)
};

```

```

private static void
AddInfoRow(TableLayoutPanel layout, int
row, string leftTitle, Control leftValue,
string rightTitle, Control rightValue)
{
    layout.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

    layout.Controls.Add(CreateTitleLabel(leftT
itle), 0, row);

    leftValue.Margin = new
Padding(0, 8, 12, 8);

    layout.Controls.Add(leftValue, 1, row);
}

```

```

layout.Controls.Add(CreateTitleLabel(right
Title), 2, row);

    rightValue.Margin = new
Padding(0, 8, 0, 8);

    layout.Controls.Add(rightValue, 3, row);
}

```

```

private static Label
CreateTitleLabel(string text)
=> new()
{
    Text = text,
    AutoSize = true,
    Anchor =
AnchorStyles.Left,
    Font = new
Font(SystemFonts.DefaultFont,
FontStyle.Bold),
    Margin = new Padding(0,
8, 8, 8)
};

```

```

private sealed class
MetricRow
{
    public string RawKey { get;
set; } = string.Empty;

    public string Name { get;
set; } = string.Empty;
}

```

```

        public string Value { get;
set; } = string.Empty;
        public string Description {
get; set; } = string.Empty;
    }
}

=====
=====
=====
=====

Файл:
AdminApp/FormModelVersions.cs

=====
=====
=====
=====

using System.Globalization;
using System.Text.Json;
using Contracts.Dtos;

namespace AdminApp;

public class FormModelVersions
: Form
{
    private readonly
AdminApiClient _api;
    private readonly
DataGridView _grid = new()
    {
        Dock = DockStyle.Fill,
        ReadOnly = true,
        AllowUserToAddRows =
false,
        AllowUserToDeleteRows =
false,
        AutoGenerateColumns =
false,
        SelectionMode =
DataGridViewSelectionMode.FullRowSele
ct,
        MultiSelect = false,
        RowHeadersVisible = false,
    };
    private readonly TextBox
_txtDetails = new()

```

```

    {
        Dock = DockStyle.Fill,
        Multiline = true,
        ReadOnly = true,
        ScrollBars =
ScrollBars.Vertical,
    };
    private readonly Button
_btnRefresh = new() { Text = "Обновить",
Width = 120, Height = 36 };
    private readonly Button
_btnDetails = new() { Text = "Подробнее",
Width = 120, Height = 36 };
    private readonly Button
_btnPublish = new() { Text = "Сделать
текущей", Width = 160, Height = 36 };
    private readonly Button
_btnUnpublish = new() { Text = "Снять
публикацию", Width = 160, Height = 36 };
    private readonly Button
_btnPin = new() { Text =
"Зафиксировать", Width = 140, Height =
36 };
    private readonly Button
_btnUnpin = new() { Text = "Снять
фиксацию", Width = 150, Height = 36 };
    private readonly Button
_btnDelete = new() { Text = "Удалить",
Width = 120, Height = 36 };
    private readonly Button
_btnClose = new() { Text = "Закрыть",
Width = 120, Height = 36 };

    public
FormModelVersions(AdminApiClient api)
    {
        _api = api;
        Text = "Управление
версиями модели";
        Width = 1250;
        Height = 760;
        StartPosition =
FormStartPosition.CenterParent;

        BuildUi();
        _btnRefresh.Click += async
(, _) => await RefreshAsync();

```

```

        _btnDetails.Click += (, _)
=> OpenDetails();
        _btnPublish.Click += async
(, _) => await PublishSelectedAsync();
        _btnUnpublish.Click +=
async (, _) => await
UnpublishSelectedAsync();
        _btnPin.Click += async (,
_) => await PinSelectedAsync();
        _btnUnpin.Click += async
(, _) => await UnpinSelectedAsync();
        _btnDelete.Click += async
(, _) => await DeleteSelectedAsync();
        _btnClose.Click += (, _)
=> Close();
        _grid.SelectionChanged +=
(, _) => RenderSelected();
        _grid.CellDoubleClick +=
(, _) => OpenDetails();
        Shown += async (, _) =>
await RefreshAsync();
    }

    private void BuildUi()
    {
        ConfigureGrid();

        var actions = new
FlowLayoutPanel
    {
        Dock = DockStyle.Top,
        AutoSize = true,
        Padding = new
Padding(12),
        FlowDirection =
FlowDirection.LeftToRight,
    };
        actions.Controls.AddRange([_btnRefresh,
_btnDetails, _btnPublish, _btnUnpublish,
_btnPin, _btnUnpin, _btnDelete,
_btnClose]);

        var split = new
SplitContainer
    {
        Dock = DockStyle.Fill,

```



```

        Orientation =
Orientation.Horizontal,
        SplitterDistance = 420,
    };

split.Panel1.Controls.Add(_grid);

split.Panel2.Controls.Add(_txtDetails);

Controls.Add(split);
Controls.Add(actions);
}

private void ConfigureGrid()
{
    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "ID", DataPropertyName =
nameof(ModelVersionGridRow.Id), Width
= 70 });

    _grid.Columns.Add(new
DataGridViewCheckBoxColumn {
HeaderText = "Текущая",
DataPropertyName =
nameof(ModelVersionGridRow.IsPublished
), Width = 90 });

    _grid.Columns.Add(new
DataGridViewCheckBoxColumn {
HeaderText = "Фикс.", DataPropertyName
=
nameof(ModelVersionGridRow.IsPinned),
Width = 70 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "Обучена",
DataPropertyName =
nameof(ModelVersionGridRow.TrainedAt),
Width = 160 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "Job ID", DataPropertyName
=
nameof(ModelVersionGridRow.ExternalJo
bId), Width = 240 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "База", DataPropertyName =

```

```

nameof(ModelVersionGridRow.BaseModel
), Width = 120 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "Формат",
DataPropertyName =
nameof(ModelVersionGridRow.MobileFor
mat), Width = 80 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "Файл", DataPropertyName
=
nameof(ModelVersionGridRow.MobileMo
delFileName), Width = 220 });

    _grid.Columns.Add(new
DataGridViewTextBoxColumn {
HeaderText = "Метрики",
DataPropertyName =
nameof(ModelVersionGridRow.MetricsSu
mmmary), AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill
});

    }

    private async Task
RefreshAsync(int? selectId = null)
    {
        try
        {
            ToggleBusy(true);
            var selectedId = selectId
?? GetSelected()?.Id;

            var models = await
_api.GetModelVersionsAsync();

            _grid.DataSource =
models.Select(x => new
ModelVersionGridRow
            {
                Id = x.Id,
                ExternalJobId =
x.ExternalJobId ?? "-",
                TrainedAt =
ToLocalString(x.TrainedAt),
                BaseModel =
x.BaseModel ?? "-",
                MobileFormat =
x.MobileFormat ?? "-",

```

```

MobileModelFileName =
x.MobileModelFileName ?? "-",
            IsPublished =
x.IsPublished,
            IsPinned = x.IsPinned,
            MetricsSummary =
BuildMetricsShort(x.MetricsJson),
            Source = x,
        }).ToList();

RestoreSelection(selectedId);
        RenderSelected();
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
ex.Message, "Ошибка загрузки версий",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private void OpenDetails()
{
    var selected =
GetSelected();
    if (selected is null) return;

    using var form = new
FormModelVersionDetails(_api,
selected.Source);
    form.ShowDialog(this);
}

private async Task
PublishSelectedAsync()
{
    var selected =
GetSelected();
    if (selected is null) return;

    var confirm =
MessageBox.Show(

```

```

        this,
        $"Сделать версию
#{selected.Id} текущей для скачивания
мобильными пользователями?",
        "Публикация модели",

        MessageBoxButtons.YesNo,

        MessageBoxIcon.Question);
        if (confirm !=
        DialogResult.Yes) return;

        await
        RunActionAsync(async () => await
        _api.PublishModelVersionAsync(selected.I
        d), selected.Id);
    }

    private async Task
    UnpublishSelectedAsync()
    {
        var selected =
        GetSelected();

        if (selected is null) return;

        await
        RunActionAsync(async () => await
        _api.UnpublishModelVersionAsync(selecte
        d.Id), selected.Id);
    }

    private async Task
    PinSelectedAsync()
    {
        var selected =
        GetSelected();

        if (selected is null) return;

        await
        RunActionAsync(async () => await
        _api.PinModelVersionAsync(selected.Id),
        selected.Id);
    }

    private async Task
    UnpinSelectedAsync()
    {

```

```

        var selected =
        GetSelected();

        if (selected is null) return;

        await
        RunActionAsync(async () => await
        _api.UnpinModelVersionAsync(selected.Id)
        , selected.Id);
    }

    private async Task
    DeleteSelectedAsync()
    {
        var selected =
        GetSelected();

        if (selected is null) return;

        var text = selected.IsPinned
        ? $"Версия
#{selected.Id} зафиксирована. Удалить
принудительно? Мобильные
пользователи больше не смогут скачать
её, если она текущая."
        : $"Удалить версию
#{selected.Id}? Файлы модели будут
удалены с backend-хранилища.";

        var confirm =
        MessageBox.Show(this, text, "Удаление
версии", MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning);

        if (confirm !=
        DialogResult.Yes) return;

        try
        {
            ToggleBusy(true);

            await
            _api.DeleteModelVersionAsync(selected.Id
            , force: selected.IsPinned);

            await RefreshAsync();
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
            ex.Message, "Ошибка удаления",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);

```

```

    }
    finally
    {
        ToggleBusy(false);
    }
}

private async Task
RunActionAsync(Func<Task<ModelVersio
nAdminResponse>> action, int selectId)
{
    try
    {
        ToggleBusy(true);
        await action();
        await
        RefreshAsync(selectId);
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
        ex.Message, "Ошибка управления
версией", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private void RenderSelected()
{
    var selected =
    GetSelected();

    var hasSelected = selected
    is not null;

    _btnDetails.Enabled =
    hasSelected;

    _btnPublish.Enabled =
    hasSelected && !selected!.IsPublished;

    _btnUnpublish.Enabled =
    hasSelected && selected!.IsPublished;

    _btnPin.Enabled =
    hasSelected && !selected!.IsPinned;

    _btnUnpin.Enabled =
    hasSelected && selected!.IsPinned;

```

```

        _btnDelete.Enabled =
hasSelected;

        if (selected is null)
        {
            _txtDetails.Text =
"Выберите версию модели.";
            return;
        }

        var src = selected.Source;
        _txtDetails.Text =
string.Join(Environment.NewLine, new[]
        {
            $"ID: {src.Id}",
            $"Job ID:
{src.ExternalJobId ?? "-"}",
            $"Текущая для
пользователей: {{src.IsPublished ? "Да" :
"Нет"}}",
            $"Зафиксирована:
{{src.IsPinned ? "Да" : "Нет"}}",
            $"Обучена:
{ToLocalString(src.TrainedAt)}",
            $"Базовая модель:
{src.BaseModel ?? "-"}",
            $"Mobile формат:
{src.MobileFormat ?? "-"}",
            $"Mobile файл:
{src.MobileModelFileName ?? "-"}",
            $"Mobile path на
backend: {src.MobileModelPath ?? "-"}",
            $"Best weights path на
backend: {src.BestWeightsPath ?? "-"}",
            "",
            "Метрики:",

            PrettyJson(src.Metrics.Json),
        });
    }

    private
ModelVersionGridRow? GetSelected()
=>
_grid.CurrentRow?.DataBoundItem as
ModelVersionGridRow;

```

```

    private void
RestoreSelection(int? id)
    {
        if (id is null) return;

        foreach
(DataGridViewRow row in _grid.Rows)
        {
            if (row.DataBoundItem is
ModelVersionGridRow item && item.Id
== id.Value)
            {
                row.Selected = true;
                _grid.CurrentCell =
row.Cells[0];
                break;
            }
        }

        private void ToggleBusy(bool
busy)
        {
            UseWaitCursor = busy;
            _btnRefresh.Enabled =
!busy;
            _btnClose.Enabled = !busy;
            if (busy)
            {
                _btnDetails.Enabled =
false;
                _btnPublish.Enabled =
false;
                _btnUnpublish.Enabled =
false;
                _btnPin.Enabled = false;
                _btnUnpin.Enabled =
false;
                _btnDelete.Enabled =
false;
            }
            else
            {
                RenderSelected();
            }
        }

        private static string
ToLocalString(DateTime value)

```

```

=>
value.ToLocalTime().ToString("dd.MM.yy
yy HH:mm:ss",
CultureInfo.InvariantCulture);

    private static string
BuildMetricsShort(string? metricsJson)
    {
        if
(string.IsNullOrEmpty(metricsJson))
return "-";

        try
        {
            using var document =
JsonDocument.Parse(metricsJson);
            var root =
document.RootElement;
            var parts = new
List<string>();

            AddMetric(parts, root,
"mAP50_95", "mAP");
            AddMetric(parts, root,
"mAP50", "mAP50");
            AddMetric(parts, root,
"precision", "P");
            AddMetric(parts, root,
"recall", "R");

            return parts.Count == 0 ?
"json" : string.Join(" · ", parts);
        }
        catch
        {
            return
metricsJson.Length > 120 ?
metricsJson[..120] + "... " : metricsJson;
        }
    }

    private static void
AddMetric(List<string> parts, JsonElement
root, string key, string title)
    {
        {
            if
(root.TryGetProperty(key, out var value)
&& value.TryGetDouble(out var number))
                parts.Add($"{{title}:
{number:F4}}");
        }
    }

```

```

        private static string
PrettyJson(string? json)
    {
        if
(string.IsNullOrEmpty(json)) return
"Метрики не сохранены.";
        try
        {
            using var document =
JsonDocument.Parse(json);
            return
JsonSerializer.Serialize(document.RootEle
ment, new JsonSerializerOptions {
WriteIndented = true });
        }
        catch
        {
            return json;
        }
    }

    private sealed class
ModelVersionGridRow
    {
        public int Id { get; set; }
        public string ExternalJobId
{ get; set; } = string.Empty;
        public string TrainedAt {
get; set; } = string.Empty;
        public string BaseModel {
get; set; } = string.Empty;
        public string MobileFormat
{ get; set; } = string.Empty;
        public string
MobileModelFileName { get; set; } =
string.Empty;
        public bool IsPublished {
get; set; }
        public bool IsPinned { get;
set; }
        public string
MetricsSummary { get; set; } =
string.Empty;
        public required
ModelVersionAdminResponse Source {
get; set; }
    }
}

}

=====
=====
=====
Файл:
AdminApp/FormTraining.cs
=====
=====
=====
using Contracts.Dtos;
using System.Text.Json;

namespace AdminApp;

public class FormTraining :
Form
    {
        private readonly
AdminApiClient _api;
        private readonly TextBox
_txtBaseModel = new() { Text =
"yolov8n.pt", Width = 180 };
        private readonly
NumericUpDown _numEpochs = new() {
Minimum = 1, Maximum = 1000, Value =
50 };
        private readonly
NumericUpDown _numImgsz = new() {
Minimum = 320, Maximum = 1280,
Increment = 32, Value = 640 };
        private readonly
NumericUpDown _numBatch = new() {
Minimum = 1, Maximum = 128, Value = 16
};
        private readonly TextBox
_txtDevice = new() { Text = "auto", Width
= 80 };
        private readonly CheckBox
_chkInt8 = new() { Text = "INT8 для
mobile", Checked = true, AutoSize = true };
        private readonly CheckBox
_chkNms = new() { Text = "Добавить
NMS", Checked = true, AutoSize = true };

        private readonly TextBox
_txtJobId = new() { ReadOnly = true,
Width = 360 };
        private readonly TextBox
_txtStatus = new() { Multiline = true,
ScrollBars = ScrollBars.Vertical, Dock =
DockStyle.Fill, ReadOnly = true };
        private readonly Button
_btnStart = new() { Text = "Запустить
обучение", Width = 180, Height = 36 };
        private readonly Button
_btnRefresh = new() { Text = "Обновить
список", Width = 140, Height = 36 };
        private readonly Button
_btnDetails = new() { Text = "Подробнее
по задаче", Width = 180, Height = 36 };
        private readonly Button
_btnStop = new() { Text = "Остановить
задачу", Width = 170, Height = 36 };
        private readonly
DataGridView _gridJobs = new()
        {
            Dock = DockStyle.Fill,
            ReadOnly = true,
            AllowUserToAddRows =
false,
            AllowUserToDeleteRows =
false,
            AutoGenerateColumns =
false,
            SelectionMode =
DataGridViewSelectionMode.FullRowSele
ct,
            MultiSelect = false,
            RowHeadersVisible = false,
        };
        private readonly
System.Windows.Forms.Timer _timer =
new() { Interval = 4000 };

        public
FormTraining(AdminApiClient api)
        {
            _api = api;

            Text = "Обучение
модели";

            Width = 1200;

```

```

        Height = 700;
        StartPosition =
FormStartPosition.CenterParent;

        BuildUi();

        _btnStart.Click += async (_,
_) => await StartAsync();
        _btnRefresh.Click += async
(_, _) => await RefreshJobsAsync();
        _btnDetails.Click += async
(_, _) => await OpenDetailsAsync();
        _btnStop.Click += async (_,
_) => await CancelSelectedJobAsync();

        _gridJobs.SelectionChanged += async (_, _)
=> await ShowSelectedJobAsync();
        _gridJobs.CellDoubleClick
+= async (_, _) => await
OpenDetailsAsync();
        _timer.Tick += async (_, _)
=> await RefreshJobsAsync(silent: true);
        Shown += async (_, _) =>
await RefreshJobsAsync();
        FormClosed += (_, _) =>
        _timer.Stop();
    }

    private void BuildUi()
    {
        var topLayout = new
TableLayoutPanel
        {
            Dock = DockStyle.Top,
            ColumnCount = 4,
            AutoSize = true,
            Padding = new
Padding(12),
        };
    }

```

```

topLayout.ColumnStyles.Add(new
ColumnStyle(SizeType.Absolute, 160));

```

```

topLayout.ColumnStyles.Add(new
ColumnStyle(SizeType.Absolute, 220));

```

```

topLayout.ColumnStyles.Add(new
ColumnStyle(SizeType.Absolute, 160));

```

```

topLayout.ColumnStyles.Add(new
ColumnStyle(SizeType.Percent, 100));

```

```

        AddRow(topLayout, 0,
"Базовая модель", _txtBaseModel,
"Epochs", _numEpochs);
        AddRow(topLayout, 1,
"Размер изображения", _numImgsz,
"Batch", _numBatch);
        AddRow(topLayout, 2,
"Устройство", _txtDevice, "Job ID",
_txtJobId);

```

```

        var checksPanel = new
FlowLayoutPanel { AutoSize = true,
FlowDirection =
FlowDirection.LeftToRight };

```

```

checksPanel.Controls.Add(_chkInt8);

```

```

checksPanel.Controls.Add(_chkNms);

```

```

        var buttons = new
FlowLayoutPanel { AutoSize = true,
FlowDirection =
FlowDirection.LeftToRight };

```

```

buttons.Controls.Add(_btnStart);

```

```

buttons.Controls.Add(_btnRefresh);

```

```

buttons.Controls.Add(_btnDetails);

```

```

buttons.Controls.Add(_btnStop);

```

```

        AddRow(topLayout, 3,
"Mobile export", checksPanel, "Действия",
buttons);

```

```

ConfigureGrid();

```

```

        var split = new
SplitContainer
    {

```

```

        Dock = DockStyle.Fill,
        Orientation =
Orientation.Horizontal,
        SplitterDistance = 320,
    };

```

```

split.Panel1.Controls.Add(_gridJobs);

```

```

split.Panel2.Controls.Add(_txtStatus);

```

```

        Controls.Add(split);
        Controls.Add(topLayout);
    }

```

```

    private void ConfigureGrid()
    {

```

```

        _gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"JobId", HeaderText = "Job ID",
DataPropertyName = "JobId", Width = 240
});

```

```

        _gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"Status", HeaderText = "Статус",
DataPropertyName = "Status", Width = 140
});

```

```

        _gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"CreatedAt", HeaderText = "Создано",
DataPropertyName = "CreatedAt", Width =
150 });

```

```

        _gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"StartedAt", HeaderText = "Старт",
DataPropertyName = "StartedAt", Width =
150 });

```

```

        _gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"FinishedAt", HeaderText = "Финиш",
DataPropertyName = "FinishedAt", Width
= 150 });

```

```

_gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"ImagesCount", HeaderText = "Кадров",
DataPropertyName = "ImagesCount",
Width = 80 });

_gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"BaseModel", HeaderText = "Модель",
DataPropertyName = "BaseModel", Width
= 120 });

_gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"MobileFormat", HeaderText = "Mobile",
DataPropertyName = "MobileFormat",
Width = 80 });

_gridJobs.Columns.Add(new
DataGridViewTextBoxColumn { Name =
"Message", HeaderText = "Сообщение",
DataPropertyName = "Message",
AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill
});

private static void
AddRow(TableLayoutPanel layout, int row,
string labelLeft, Control controlLeft, string
labelRight, Control controlRight)
{
    layout.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

    layout.Controls.Add(new
Label
    {
        Text = labelLeft,
        AutoSize = true,
        Anchor =
AnchorStyles.Left,
        Margin = new Padding(0,
8, 8, 8)
    }, 0, row);

    controlLeft.Anchor =
AnchorStyles.Left | AnchorStyles.Right;

controlLeft.Margin = new
Padding(0, 4, 16, 4);

layout.Controls.Add(controlLeft, 1, row);

    layout.Controls.Add(new
Label
    {
        Text = labelRight,
        AutoSize = true,
        Anchor =
AnchorStyles.Left | AnchorStyles.Right;
        controlRight.Anchor =
AnchorStyles.Left | AnchorStyles.Right;
        controlRight.Margin = new
Padding(0, 4, 0, 4);
    }, 2, row);

    controlRight.Anchor =
AnchorStyles.Left | AnchorStyles.Right;
    controlRight.Margin = new
Padding(0, 4, 0, 4);

    layout.Controls.Add(controlRight, 3, row);
}

private async Task
StartAsync()
{
    try
    {
        ToggleBusy(true);
        AppendStatus("Создаю
задачу обучения на backend. Python-
клиент заберёт её исходящим
запросом...");

        var response = await
_api.StartTrainingAsync(new
StartTrainingRequest(
_txtBaseModel.Text.Trim(),
(int)_numEpochs.Value,
(int)_numImgsz.Value,
(int)_numBatch.Value,
_txtDevice.Text.Trim(),
_chkInt8.Checked,
_chkNms.Checked,
"tf-lite",
0.3
));

        _txtJobId.Text =
response.JobId;
        AppendStatus($"Job
{response.JobId}:
{TranslateStatus(response.Status)}.
{response.Message}");
        await
RefreshJobsAsync(selectJobId:
response.JobId);
        _timer.Start();
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
ex.Message, "Ошибка обучения",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
        AppendStatus("Ошибка:
" + ex.Message);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private async Task
RefreshJobsAsync(bool silent = false,
string? selectJobId = null)
{
    try
    {
        var jobs = await
_api.GetTrainingJobsAsync();
        var selectedJobId =
selectJobId ?? GetSelectedJobId() ??
_txtJobId.Text.Trim();

        _gridJobs.DataSource =
jobs
.Select(x => new
TrainingJobGridRow
{
    JobId = x.JobId,

```

```

        Status =
TranslateStatus(x.Status),
        CreatedAt =
ToLocalString(x.CreatedAt),
        StartedAt =
ToLocalString(x.StartedAt),
        FinishedAt =
ToLocalString(x.FinishedAt),
        ImagesCount =
x.ImagesCount,
        BaseModel =
x.BaseModel,
        MobileFormat =
x.MobileFormat,
        Message =
x.Message,
        Source = x,
    })
    .ToList();

RestoreSelection(selectedJobId);
UpdateButtonsState();

if (!silent)

AppendStatus($"Список задач обучения
обновлён. Всего: {jobs.Count}.");
}
catch (Exception ex)
{
    if (!silent)

AppendStatus("Ошибка обновления
списка: " + ex.Message);
}

private async Task
ShowSelectedJobAsync()
{
    var selected =
GetSelectedJob();
    UpdateButtonsState();
    if (selected is null)
        return;

```

```

        _txtJobId.Text =
selected.JobId;

try
{
    var job = await
_api.GetTrainingJobAsync(selected.JobId);
    if (job is null)
    {
        _txtStatus.Text =
"Задача не найдена на backend.";
        return;
    }

    var lines = new
List<string>
{
    $"Job ID:
{job.JobId}",
    $"Статус:
{TranslateStatus(job.Status)}",
    $"Создано:
{ToLocalString(job.CreatedAt)}",
    $"Старт:
{ToLocalString(job.StartedAt)}",
    $"Финиш:
{ToLocalString(job.FinishedAt)}",
    $"Кадров в датасете:
{job.ImagesCount}",
    $"Базовая модель:
{job.BaseModel}",
    $"Mobile формат:
{job.MobileFormat}",
    $"Файл лучших
весов: {job.BestWeightsPath ?? "-"}",
    $"Файл мобильной
модели: {job.MobileModelPath ?? "-"}",
    "",
    "Кратко по
метрикам:",
    BuildMetricsSummary(job.MetricsJson),
    "",
    "Сообщение
сервиса:",
    job.Message ?? "-",
    ""

```

```

        "Подсказка: дважды
щёлкните по задаче в списке выше,
чтобы открыть подробное окно с
пояснениями по каждой метрике.
Готовые модели появятся в окне
«Версии модели»."
    };

    _txtStatus.Text =
string.Join(Environment.NewLine, lines);

    if
(IsTerminal(job.Status))
    {
        var rows =
(_gridJobs.DataSource as
List<TrainingJobGridRow>) ?? [];
        if (rows.All(x =>
IsTerminal(x.Source.Status)))
            _timer.Stop();
    }
    catch (Exception ex)
    {
        _txtStatus.Text =
"Ошибка получения задачи: " +
ex.Message;
    }

    private async Task
OpenDetailsAsync()
    {
        var selected =
GetSelectedJob();
        if (selected is null)
        {
            MessageBox.Show(this,
"Сначала выберите задачу обучения в
списке.", "Обучение",
MessageBoxButtons.OK,
MessageBoxIcon.Information);
            return;
        }

        using var dialog = new
FormTrainingJobDetails(_api,
selected.JobId);

```

```

        dialog.ShowDialog(this);
        await
RefreshJobsAsync(selectJobId:
selected.JobId);
    }

    private async Task
CancelSelectedJobAsync()
    {
        var selected =
GetSelectedJob();
        if (selected is null)
        {
            MessageBox.Show(this,
"Сначала выберите задачу обучения в
списке.", "Обучение",
MessageBoxButtons.OK,
MessageBoxIcon.Information);
            return;
        }

        if
(!CanBeCanceled(selected.Source.Status))
        {
            MessageBox.Show(this,
"Эту задачу уже нельзя остановить: она
завершена, остановлена или завершилась
с ошибкой.", "Обучение",
MessageBoxButtons.OK,
MessageBoxIcon.Information);
            return;
        }

        var confirm =
MessageBox.Show(
this,
$"Остановить задачу
{selected.JobId}?",
"Подтверждение
остановки",

MessageBoxButtons.YesNo,

MessageBoxIcon.Question);

        if (confirm !=
DialogResult.Yes)
            return;

```

```

        try
        {
            ToggleBusy(true);
            var response = await
_api.CancelTrainingJobAsync(selected.JobI
d);
            AppendStatus($"Задача
{selected.JobId}:
{TranslateStatus(response.Status)}.
{response.Message}");
            await
RefreshJobsAsync(selectJobId:
selected.JobId);
        }
        catch (Exception ex)
        {
            MessageBox.Show(this,
ex.Message, "Ошибка остановки",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
            AppendStatus("Ошибка
остановки задачи: " + ex.Message);
        }
        finally
        {
            ToggleBusy(false);
        }

        private TrainingJobGridRow?
GetSelectedJob()
        =>
_gridJobs.CurrentRow?.DataBoundItem as
TrainingJobGridRow;

        private string?
GetSelectedJobId()
        =>
GetSelectedJob()?.JobId;

        private void
RestoreSelection(string? jobId)
        {
            if
(string.IsNullOrEmpty(jobId))
                return;

```

```

        foreach
(DataGridViewRow row in
_gridJobs.Rows)
        {
            if (row.DataBoundItem is
TrainingJobGridRow item && item.JobId
== jobId)
            {
                row.Selected = true;
                _gridJobs.CurrentCell
= row.Cells[0];
                break;
            }
        }

        private void
UpdateButtonsState()
        {
            var selected =
GetSelectedJob();
            _btnDetails.Enabled =
selected is not null;
            _btnStop.Enabled =
selected is not null &&
CanBeCanceled(selected.Source.Status);
        }

        private static bool
IsTerminal(string? status)
            => string.Equals(status,
"completed",
StringComparison.OrdinalIgnoreCase)
            || string.Equals(status,
"failed",
StringComparison.OrdinalIgnoreCase)
            || string.Equals(status,
"canceled",
StringComparison.OrdinalIgnoreCase);

        private static bool
CanBeCanceled(string? status)
            => string.Equals(status,
"queued",
StringComparison.OrdinalIgnoreCase)
            || string.Equals(status,
"running",
StringComparison.OrdinalIgnoreCase);

```



```

        private static string
TranslateStatus(string? status)
    =>
status?.ToLowerInvariant() switch
    {
        "queued" => "В
очереди",
        "running" =>
"Выполняется",
        "completed" =>
"Завершена успешно",
        "failed" => "Завершена
с ошибкой",
        "canceled" =>
"Остановлена",
        _ => status ?? "-"
    };

        private static string
ToLocalString(DateTime? value)
    =>
value?.ToLocalTime().ToString("dd.MM.y
yy HH:mm:ss") ?? "-";

        private static string
BuildMetricsSummary(string? metricsJson)
    {
        if
(string.IsNullOrEmpty(metricsJson))
            return "Метрики пока не
рассчитаны.";

        try
        {
            using var document =
JsonDocument.Parse(metricsJson);
            var root =
document.RootElement;
            var lines = new
List<string>();

            AppendMetric(lines,
root, "mAP50_95", "mAP@0.50:0.95 —
основная итоговая метрика качества
детектора");

            AppendMetric(lines,
root, "mAP50", "mAP@0.50 — качество

```

```

при мягком критерии совпадения
рамок");

            AppendMetric(lines,
root, "mAP75", "mAP@0.75 — качество
при более строгом совпадении рамок");

            AppendMetric(lines,
root, "precision", "Precision — доля
правильных срабатываний среди всех
найденных");

            AppendMetric(lines,
root, "recall", "Recall — доля реальных
дат, которые модель нашла");

            AppendMetric(lines,
root, "fitness", "Fitness — сводная
служебная метрика Ultralytics");

            return lines.Count > 0 ?
string.Join(Environment.NewLine, lines) :
PrettyJson(metricsJson);
        }
        catch
        {
            return
PrettyJson(metricsJson);
        }
    }

        private static void
AppendMetric(List<string> lines,
JsonElement root, string key, string title)
    {
        if
(!root.TryGetProperty(key, out var value) ||
!value.TryGetDouble(out var number))
            return;

        lines.Add($"{title}:
{number:P2} ({number:F4})");
    }

        private static string
PrettyJson(string json)
    {
        try
        {
            using var document =
JsonDocument.Parse(json);

```

```

            return
JsonSerializer.Serialize(document.RootEle
ment, new JsonSerializerOptions {
WriteIndented = true });
        }
        catch
        {
            return json;
        }
    }

        private void
AppendStatus(string text)
    {
        _txtStatus.AppendText(text
+ Environment.NewLine);
    }

        private void ToggleBusy(bool
busy)
    {
        UseWaitCursor = busy;
        _btnStart.Enabled = !busy;
        _btnRefresh.Enabled =
!busy;

        if (busy)
        {
            _btnDetails.Enabled =
false;
            _btnStop.Enabled =
false;
        }
        else
        {
            UpdateButtonsState();
        }
    }

        private sealed class
TrainingJobGridRow
    {
        public string JobId { get;
set; } = string.Empty;
        public string Status { get;
set; } = string.Empty;
        public string CreatedAt {
get; set; } = string.Empty;
    }

```

```

        public string StartedAt {
get; set; } = string.Empty;
        public string FinishedAt {
get; set; } = string.Empty;
        public int ImagesCount {
get; set; }
        public string? BaseModel {
get; set; }
        public string?
MobileFormat { get; set; }
        public string? Message {
get; set; }
        public required
TrainingJobStatusResponse Source { get;
set; }
    }
}

```

```

=====
=====
=====
=====
=====
Файл:
AdminApp/FormTrainingJobDetails.cs
=====
=====
=====
=====

```

```

using Contracts.Dtos;
using System.Globalization;
using System.Text.Json;

namespace AdminApp;

public sealed class
FormTrainingJobDetails : Form
{
    private readonly
AdminApiClient _api;
    private readonly string _jobId;
    private
TrainingJobStatusResponse? _job;

    private readonly Label
_lblJobIdValue = CreateValueLabel();
    private readonly Label
_lblStatusValue = CreateValueLabel();

```

```

    private readonly Label
_lblCreatedAtValue = CreateValueLabel();
    private readonly Label
_lblStartedAtValue = CreateValueLabel();
    private readonly Label
_lblFinishedAtValue = CreateValueLabel();
    private readonly Label
_lblAssignedAtValue =
CreateValueLabel();
    private readonly Label
_lblHeartbeatAtValue =
CreateValueLabel();
    private readonly Label
_lblImagesCountValue =
CreateValueLabel();
    private readonly Label
_lblBaseModelValue =
CreateValueLabel();
    private readonly Label
_lblMobileFormatValue =
CreateValueLabel();
    private readonly Label
_lblClientIdValue = CreateValueLabel();
    private readonly Label
_lblParamsValue = CreateValueLabel();
    private readonly Label
_lblMobileExportValue =
CreateValueLabel();
    private readonly Label
_lblCancellationValue =
CreateValueLabel();
    private readonly TextBox
_txtArtifacts = new() { Multiline = true,
ReadOnly = true, ScrollBars =
ScrollBars.Vertical, Dock = DockStyle.Fill,
Height = 80 };
    private readonly TextBox
_txtMessage = new() { Multiline = true,
ReadOnly = true, ScrollBars =
ScrollBars.Vertical, Dock = DockStyle.Fill,
Height = 130 };
    private readonly
DataGridView _gridMetrics = new()
{
    Dock = DockStyle.Fill,
    ReadOnly = true,
    AllowUserToAddRows =
false,

```

```

AllowUserToDeleteRows =
false,
    AutoGenerateColumns =
false,
    SelectionMode =
DataGridViewSelectionMode.FullRowSele
ct,
    MultiSelect = false,
    RowHeadersVisible = false,
    AutoSizeRowsMode =
DataGridViewAutoSizeRowsMode.AllCell
s,
    DefaultCellStyle = new
DataGridViewCellStyle { WrapMode =
DataGridViewTriState.True },
    };
    private readonly Button
_btnRefresh = new() { Text = "Обновить",
Width = 120, Height = 36 };
    private readonly Button
_btnStop = new() { Text = "Остановить
задачу", Width = 180, Height = 36 };
    private readonly Button
_btnOpenModel = new() { Text =
"Открыть версию модели", Width = 210,
Height = 36 };
    private readonly Button
_btnClose = new() { Text = "Закрыть",
Width = 120, Height = 36 };

    public
FormTrainingJobDetails(AdminApiClient
api, string jobId)
    {
        _api = api;
        _jobId = jobId;

        Text = $"Задача обучения:
{jobId}";
        Width = 1100;
        Height = 820;
        StartPosition =
FormStartPosition.CenterParent;

        BuildUi();

        _btnRefresh.Click += async
(, _) => await LoadJobAsync();

```

```

        _btnStop.Click += async (_, _) => await CancelJobAsync();

        _btnOpenModel.Click += async (_, _) => await
            OpenModelVersionAsync();

        _btnClose.Click += (_, _) => Close();

        Shown += async (_, _) => await LoadJobAsync();
    }

    private void BuildUi()
    {
        ConfigureMetricsGrid();

        var header = new
            TableLayoutPanel
            {
                Dock = DockStyle.Top,
                AutoSize = true,
                Padding = new
                    Padding(12),
                ColumnCount = 4,
            };

        header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Absolute, 160));

        header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Percent, 50));

        header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Absolute, 160));

        header.ColumnStyles.Add(new
            ColumnStyle(SizeType.Percent, 50));

        AddInfoRow(header, 0,
            "Job ID", _lblJobIdValue, "Cratyc",
            _lblStatusValue);

        AddInfoRow(header, 1,
            "Создано", _lblCreatedAtValue,
            "Назначено клиенту",
            _lblAssignedAtValue);

        AddInfoRow(header, 2,
            "Запущено", _lblStartedAtValue,

```

```

            "Последний heartbeat",
            _lblHeartbeatAtValue);

        AddInfoRow(header, 3,
            "Завершено", _lblFinishedAtValue,
            "Кадров в датасете",
            _lblImagesCountValue);

        AddInfoRow(header, 4,
            "Клиент обучения", _lblClientIdValue,
            "Остановка", _lblCancellationValue);

        AddInfoRow(header, 5,
            "Базовая модель", _lblBaseModelValue,
            "Mobile формат",
            _lblMobileFormatValue);

        AddInfoRow(header, 6,
            "Параметры", _lblParamsValue, "Mobile
            export", _lblMobileExportValue);

        var actionsPanel = new
            FlowLayoutPanel
            {
                Dock = DockStyle.Top,
                AutoSize = true,
                Padding = new
                    Padding(12, 0, 12, 12),
                FlowDirection =
                    FlowDirection.LeftToRight,
            };

        actionsPanel.Controls.AddRange([_btnRefr
            esh, _btnStop, _btnOpenModel,
            _btnClose]);

        var content = new
            TableLayoutPanel
            {
                Dock = DockStyle.Fill,
                Padding = new
                    Padding(12, 0, 12, 12),
                ColumnCount = 1,
                RowCount = 6,
            };

        content.RowStyles.Add(new
            RowStyle(SizeType.AutoSize));

        content.RowStyles.Add(new
            RowStyle(SizeType.Percent, 55));

```

```

        content.RowStyles.Add(new
            RowStyle(SizeType.AutoSize));

        content.RowStyles.Add(new
            RowStyle(SizeType.Absolute, 100));

        content.RowStyles.Add(new
            RowStyle(SizeType.AutoSize));

        content.RowStyles.Add(new
            RowStyle(SizeType.Percent, 45));

        content.Controls.Add(CreateSectionLabel("
            Метрики качества"), 0, 0);

        content.Controls.Add(_gridMetrics, 0, 1);

        content.Controls.Add(CreateSectionLabel("
            Файлы и артефакты"), 0, 2);

        content.Controls.Add(_txtArtifacts, 0, 3);

        content.Controls.Add(CreateSectionLabel("
            Сообщение сервиса"), 0, 4);

        content.Controls.Add(_txtMessage, 0, 5);

        Controls.Add(content);

        Controls.Add(actionsPanel);

        Controls.Add(header);
    }

    private void
        ConfigureMetricsGrid()
    {
        _gridMetrics.Columns.Add(new
            DataGridViewTextBoxColumn
            {
                HeaderText =
                    "Метрика",

                DataPropertyName =
                    nameof(MetricRow.Name),

                Width = 180
            });

```

```

_gridMetrics.Columns.Add(new
DataGridViewTextBoxColumn
    {
        HeaderText =
"Значение",
        DataPropertyName =
nameof(MetricRow.Value),
        Width = 120
    });

_gridMetrics.Columns.Add(new
DataGridViewTextBoxColumn
    {
        HeaderText =
"Пояснение",
        DataPropertyName =
nameof(MetricRow.Description),
        AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill
    });

private async Task
LoadJobAsync()
{
    try
    {
        ToggleBusy(true);
        var job = await
_api.GetTrainingJobAsync(_jobId);
        if (job is null)
        {

            MessageBox.Show(this, "Задача не
            найдена на сервере.", "Обучение",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
            Close();
            return;
        }

        Render(job);
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
        ex.Message, "Ошибка загрузки задачи",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private async Task
CancelJobAsync()
{
    var confirm =
    MessageBox.Show(
        this,
        "Остановить
        выбранную задачу обучения?
        Запущенный процесс будет прерван.",
        "Подтверждение
        остановки",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    if (confirm !=
    DialogResult.Yes)
        return;

    try
    {
        ToggleBusy(true);
        var job = await
_api.CancelTrainingJobAsync(_jobId);
        Render(job);
        MessageBox.Show(this,
        "Команда остановки отправлена.",
        "Обучение", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
        ex.Message, "Ошибка остановки",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private void
Render(TrainingJobStatusResponse job)
{
    _job = job;
    _lblJobIdValue.Text =
job.JobId;

    _lblStatusValue.Text =
TranslateStatus(job.Status);
    _lblCreatedAtValue.Text =
ToLocalString(job.CreatedAt);
    _lblStartedAtValue.Text =
ToLocalString(job.StartedAt);
    _lblFinishedAtValue.Text =
ToLocalString(job.FinishedAt);
    _lblAssignedAtValue.Text
= ToLocalString(job.AssignedAt);
    _lblHeartbeatAtValue.Text
= ToLocalString(job.HeartbeatAt);

    _lblImagesCountValue.Text =
job.ImagesCount.ToString(CultureInfo.Inva
riantCulture);
    _lblBaseModelValue.Text
= job.BaseModel ?? "-";

    _lblMobileFormatValue.Text =
job.MobileFormat ?? "-";
    _lblClientIdValue.Text =
job.ClientId ?? "-";
    _lblCancellationValue.Text
= job.CancellationRequested ?
"запрошена" : "нет";
    _lblParamsValue.Text =
BuildTrainingParams(job);

    _lblMobileExportValue.Text =
BuildMobileExport(job);

    _txtArtifacts.Text =
string.Join(Environment.NewLine, new[]
{
        $"Dataset ZIP:
        {job.DatasetZipPath ?? "-"}",

```

```

        $"Файл весов лучшей
модели: {job.BestWeightsPath ?? "-"}",
        $"Файл мобильной
модели: {job.MobileModelPath ?? "-"}",
        $"Имя mobile-файла:
{job.MobileModelFileName ?? "-"}",
        $"Mobile Content-Type:
{job.MobileModelContentType ?? "-"}"
    });

    _txtMessage.Text =
job.Message ?? "-";

    _gridMetrics.DataSource =
BuildMetricRows(job.MetricsJson);
    _btnStop.Enabled =
CanBeCanceled(job.Status);
    _btnOpenModel.Enabled =
HasPossibleModel(job);
}

private async Task
OpenModelVersionAsync()
{
    if (_job is null)
        return;

    try
    {
        ToggleBusy(true);
        var versions = await
_api.GetModelVersionsAsync();
        var model =
versions.FirstOrDefault(x =>
string.Equals(x.ExternalJobId, _job.JobId,
StringComparison.OrdinalIgnoreCase));
        if (model is null)
        {

            MessageBox.Show(this, "Для этой задачи
пока нет сохранённой версии модели.",
"Версия модели",
MessageBoxButtons.OK,
MessageBoxIcon.Information);

            return;
        }

        using var form = new
FormModelVersionDetails(_api, model);

```

```

        form.ShowDialog(this);
    }
    catch (Exception ex)
    {
        MessageBox.Show(this,
ex.Message, "Ошибка открытия версии
модели", MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
    finally
    {
        ToggleBusy(false);
    }
}

private void ToggleBusy(bool
busy)
{
    UseWaitCursor = busy;
    _btnRefresh.Enabled =
!busy;

    _btnClose.Enabled = !busy;
    if (busy)
    {
        _btnStop.Enabled =
false;

        _btnOpenModel.Enabled
= false;
    }
    else
    {
        _btnOpenModel.Enabled
= _job is not null &&
HasPossibleModel(_job);
    }
}

private static
List<MetricRow> BuildMetricRows(string?
metricsJson)
{
    if
(string.IsNullOrEmpty(metricsJson))
    {
        return
[
            new MetricRow
{

```

```

            Name = "Метрики
пока недоступны",
            Value = "-",
            Description =
"Обычно метрики появляются после
этапа валидации. Пока обучение ещё не
дошло до оценки качества или задача
завершилась до расчёта метрик."
        }
    ];
}

try
{
    using var document =
JsonDocument.Parse(metricsJson);
    var root =
document.RootElement;
    var rows = new
List<MetricRow>();

    AddMetric(rows, root,
"mAP50_95", "mAP@0.50:0.95",
"Главная итоговая метрика детектора.
Показывает среднее качество
обнаружения на нескольких уровнях
строгости IoU от 0.50 до 0.95. Чем выше,
тем лучше.");

    AddMetric(rows, root,
"mAP50", "mAP@0.50", "Качество
обнаружения при более мягком критерии
совпадения рамок. Обычно заметно
выше, чем mAP@0.50:0.95. Чем выше,
тем лучше.");

    AddMetric(rows, root,
"mAP75", "mAP@0.75", "Качество
обнаружения при более строгом
сравнении рамок. Показывает, насколько
точно модель позиционирует область
даты. Чем выше, тем лучше.");

    AddMetric(rows, root,
"precision", "Precision", "Доля
правильных срабатываний среди всех
найденных моделью областей. Высокое
значение означает меньше ложных
срабатываний.");

    AddMetric(rows, root,
"recall", "Recall", "Доля реально

```

существующих областей даты, которые модель смогла найти. Высокое значение означает меньше пропусков.");

AddMetric(rows, root, "fitness", "Fitness", "Сводная служебная метрика Ultralytics для сравнения запусков обучения. Используется как интегральная оценка качества модели.");

```
foreach (var property in
root.EnumerateObject())
{
    if (rows.Any(x =>
x.RawKey == property.Name))
        continue;

    rows.Add(new
MetricRow
    {
        RawKey =
property.Name,
        Name =
property.Name,
        Value =
FormatMetricValue(property.Value),
        Description =
"Дополнительная метрика, сохранённая
сервисом обучения."
    });
}
```

```
return rows.Count > 0
? rows
: [new MetricRow {
Name = "Метрики не распознаны", Value
= "-", Description = "Сервис вернул JSON
метрик, но он не содержал ожидаемых
числовых значений." }];
```

```
}
catch
{
    return
[
    new MetricRow
    {
        Name =
"Метрики",
```

```
Value =
metricsJson,
        Description = "Не
удалось разобрать JSON автоматически,
поэтому показано исходное значение,
которое вернул сервис обучения."
    }
];
}
```

```
private static void
AddMetric(List<MetricRow> rows,
JsonElement root, string key, string
displayName, string description)
{
    if
(!root.TryGetProperty(key, out var value))
        return;

    rows.Add(new MetricRow
    {
        RawKey = key,
        Name = displayName,
        Value =
FormatMetricValue(value),
        Description = description
    });
}
```

```
private static string
FormatMetricValue(JsonElement value)
{
    if (value.ValueKind ==
JsonValueKind.Number &&
value.TryGetDouble(out var number))
        return $"{{number:P2}}
({number:F4})";
```

```
return value.ToString();
}
```

```
private static string
BuildTrainingParams(TrainingJobStatusRes
ponse job)
=>
$"epochs={job.Epochs?.ToString(CultureIn
fo.InvariantCulture) ?? "-"}, "
```

```
+
$"imgsz={job.ImageSize?.ToString(CultureIn
fo.InvariantCulture) ?? "-"}, "
+
$"batch={job.Batch?.ToString(CultureInfo.
InvariantCulture) ?? "-"}, "
+ $"device={job.Device
?? "-"}";
```

```
private static string
BuildMobileExport(TrainingJobStatusResp
onse job)
=>
$"int8={BoolText(job.ExportInt8)}, "
+
$"nms={BoolText(job.ExportNms)}, "
+
$"quant={job.QuantizationFraction?.ToStri
ng("0.###", CultureInfo.InvariantCulture)
?? "-"}";
```

```
private static string
BoolText(bool? value)
=> value.HasValue ?
(value.Value ? "да" : "нет") : "-";
```

```
private static bool
HasPossibleModel(TrainingJobStatusRespo
nse job)
=> string.Equals(job.Status,
"completed",
StringComparison.OrdinalIgnoreCase)
||
!string.IsNullOrEmpty(job.MobileMo
delPath)
||
!string.IsNullOrEmpty(job.BestWeigh
tsPath);
```

```
private static bool
CanBeCanceled(string? status)
=> string.Equals(status,
"queued",
StringComparison.OrdinalIgnoreCase)
|| string.Equals(status,
"running",
StringComparison.OrdinalIgnoreCase);
```

```

        private static string
TranslateStatus(string? status)
    =>
status?.ToLowerInvariant() switch
    {
        "queued" => "В
очереди",
        "running" =>
"Выполняется",
        "completed" =>
"Завершена успешно",
        "failed" => "Завершена
с ошибкой",
        "canceled" =>
"Остановлена",
        _ => status ?? "-"
    };

```

```

        private static string
ToLocalString(DateTime? value)
    =>
value?.ToLocalTime().ToString("dd.MM.y
yyy HH:mm:ss") ?? "-";

```

```

        private static Label
CreateSectionLabel(string text)
    => new()
    {
        Text = text,
        Dock = DockStyle.Fill,
        AutoSize = true,
        Font = new
Font(SystemFonts.DefaultFont,
FontStyle.Bold),
        Padding = new
Padding(0, 8, 0, 8)
    };

```

```

        private static Label
CreateValueLabel()
    => new()
    {
        AutoSize = true,
        Anchor =
AnchorStyles.Left,
        MaximumSize = new
Size(420, 0)
    };

```

```

        private static void
AddInfoRow(TableLayoutPanel layout, int
row, string leftTitle, Control leftValue,
string rightTitle, Control rightValue)
    {
        layout.RowStyles.Add(new
RowStyle(SizeType.AutoSize));

        layout.Controls.Add(CreateTitleLabel(leftT
itle), 0, row);

        leftValue.Margin = new
Padding(0, 8, 12, 8);

        layout.Controls.Add(leftValue, 1, row);

```

```

        layout.Controls.Add(CreateTitleLabel(right
Title), 2, row);

        rightValue.Margin = new
Padding(0, 8, 0, 8);

        layout.Controls.Add(rightValue, 3, row);
    }

```

```

        private static Label
CreateTitleLabel(string text)
    => new()
    {
        Text = text,
        AutoSize = true,
        Anchor =
AnchorStyles.Left,
        Font = new
Font(SystemFonts.DefaultFont,
FontStyle.Bold),
        Margin = new Padding(0,
8, 8, 8)
    };

```

```

        private sealed class
MetricRow
    {
        public string RawKey { get;
set; } = string.Empty;

        public string Name { get;
set; } = string.Empty;

```

```

        public string Value { get;
set; } = string.Empty;

        public string Description {
get; set; } = string.Empty;
    }
}

```

```

=====
=====
=====
=====
Файл: AdminApp/Program.cs
=====
=====
=====
=====

```

```

namespace AdminApp;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();

        using var loginForm = new
FormLogin();
        if (loginForm.ShowDialog()
== DialogResult.OK &&
!string.IsNullOrEmpty(loginForm.Tok
en))
        {
            Application.Run(new
FormMain(loginForm.Token));
        }
    }
}

```

```

=====
=====
=====
=====
Файл: AdminApp/Settings.json
=====
=====
=====

```

```

=====
=====

{
    "ApiBaseUrl":
"http://111.88.146.2:5099/"
}

=====
=====
=====
=====

Файл:
Contracts/Contracts.csproj
=====
=====
=====
=====

<Project
Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>

<TargetFramework>net8.0</TargetFramework>

<ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
</PropertyGroup>

</Project>

=====
=====
=====
=====

Файл:
Contracts/Dtos/AdminErrorReportDtos.cs
=====
=====
=====
=====

namespace Contracts.Dtos;

public record
AdminErrorReportListItemDto(

```

```

    int Id,
    int UserId,
    DateTime CreatedAt,
    int FramesCount,
    bool Approved
);

public record
AdminErrorReportDetailsDto(
    int Id,
    int UserId,
    DateTime CreatedAt,
    int FramesCount,
    bool Approved,
    string? Comment,
    string DatasetFolder
);

public record
ApproveErrorReportRequest(bool
Approved);

public record YoloBboxDto(
    int ClassId,
    double Xc,
    double Yc,
    double W,
    double H
);

=====
=====
=====
=====

Файл:
Contracts/Dtos/AdminUserDtos.cs
=====
=====
=====
=====

namespace Contracts.Dtos;

public record
AdminUserListItem(int Id, string Email,
bool IsBlocked, DateTime? CreatedAt);

public record
SetBlockedRequest(bool IsBlocked);

```

```

=====
=====
=====
=====

Файл:
Contracts/Dtos/AuthDtos.cs
=====
=====
=====
=====

namespace Contracts.Dtos;

public record
RegisterRequest(string Email, string
Password);

public record
LoginRequest(string Email, string
Password);

public record
AuthResponse(string AccessToken);

=====
=====
=====
=====

Файл:
Contracts/Dtos/ErrorReportUploadDto.cs
=====
=====
=====
=====

namespace Contracts.Dtos;

public record
UploadDatasetResponse(int ReportId, int
VideoSampleId, string Folder);

=====
=====
=====
=====

Файл:
Contracts/Dtos/ProductDtos.cs
=====
=====
=====
=====

```



```

=====
=====

namespace Contracts.Dtos;

public record
ProductCreateRequest(string Name, string?
Manufacturer, string? Barcode);

public record
ProductUpdateRequest(string Name, string?
Manufacturer, string? Barcode);

=====
=====
=====
=====

Файл:
Contracts/Dtos/StoredProductDtos.cs

=====
=====
=====
=====

namespace Contracts.Dtos;

public sealed class
StoredProductCreateRequest
{
    public int? ProductId { get;
init; }

    public string? ProductName {
get; init; }

    public DateTime?
ManufactureAt { get; init; }
    public DateTime? ExpiryAt {
get; init; }
}

public sealed class
ProductSummaryDto
{
    public int Id { get; init; }
    public string Name { get; init;
} = string.Empty;
    public string? Manufacturer {
get; init; }

    public string? Barcode { get;
init; }

```

```

}

public sealed class
StoredProductDto
{
    public int Id { get; init; }
    public int UserId { get; init; }
    public int ProductId { get; init;
}

    public DateTime?
ManufactureAt { get; init; }
    public DateTime? ExpiryAt {
get; init; }

    public DateTime CreatedAt {
get; init; }

    public ProductSummaryDto?
Product { get; init; }
}

=====
=====
=====
=====

Файл:
Contracts/Dtos/TrainingDtos.cs

=====
=====
=====
=====

namespace Contracts.Dtos;

public record
StartTrainingRequest(
    string? BaseModel,
    int? Epochs,
    int? ImgSize,
    int? Batch,
    string? Device,
    bool? ExportInt8,
    bool? ExportNms,
    string? MobileFormat,
    double? QuantizationFraction
);

public record
TrainingJobStartResponse(
    string JobId,

```

```

    string Status,
    int ImagesCount,
    string Message
);

public record
TrainingJobStatusResponse(
    string JobId,
    string Status,
    string? Message,
    DateTime CreatedAt,
    DateTime? StartedAt,
    DateTime? FinishedAt,
    int ImagesCount,
    string? BaseModel,
    string? BestWeightsPath,
    string? MobileModelPath,
    string? MobileFormat,
    string? MetricsJson,
    bool CancellationRequested =
false,

    string? ClientId = null,
    DateTime? AssignedAt = null,
    DateTime? HeartbeatAt =
null,

    string? DatasetZipPath = null,
    int? Epochs = null,
    int? ImgSize = null,
    int? Batch = null,
    string? Device = null,
    bool? ExportInt8 = null,
    bool? ExportNms = null,
    double? QuantizationFraction
= null,

    string?
MobileModelFileName = null,
    string?
MobileModelContentType = null
);

public record
TrainingClientJobResponse(
    string JobId,
    int ImagesCount,
    string? BaseModel,
    int Epochs,
    int ImgSize,
    int Batch,

```

| | | |
|--|--|---|
| <pre> string? Device, bool ExportInt8, bool ExportNms, string? MobileFormat, double QuantizationFraction, bool CancellationRequested); public record TrainingClientStatusRequest(string Status, string? Message, string? MetricsJson = null); public record TrainingClientStatusResponse(string JobId, string Status, bool CancellationRequested, string? Message); public record ModelVersionAdminResponse(int Id, string? ExternalJobId, DateTime TrainedAt, string? MetricsJson, string? BaseModel, string? BestWeightsPath, string? MobileModelPath, string? MobileModelFileName, string? MobileModelContentType, string? MobileFormat, bool IsPublished, bool IsPinned, bool IsDeleted, DateTime? DeletedAt = null); public record LatestMobileModelResponse(int ModelVersionId, DateTime TrainedAt, string? MobileFormat, </pre> | <pre> string? MetricsJson, string? FileName = null, bool IsPinned = false); ===== Файл: docker-compose.yml ===== Файл: Contracts/Dtos/UserDtos.cs ===== namespace Contracts.Dtos; public record UserProfileResponse(int Id, string Email, bool IsBlocked, string? SettingsJson); ===== Файл: Contracts/IRepository.cs ===== namespace Contracts; public interface IRepository<T> { Task<T?> GetByIdAsync(int id, CancellationToken ct = default); Task<List<T>> GetAllAsync(CancellationToken ct = default); Task<T> AddAsync(T entity, CancellationToken ct = default); Task UpdateAsync(T entity, CancellationToken ct = default); Task DeleteAsync(int id, CancellationToken ct = default); } </pre> | <pre> ===== services: postgres: image: postgres:16-alpine container_name: productsdate-postgres restart: unless-stopped environment: POSTGRES_DB: \${POSTGRES_DB:-productsdate} POSTGRES_USER: \${POSTGRES_USER:-postgres} POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env} volumes: - productsdate_data:/var/lib/postgresql/data healthcheck: test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER:- postgres} -d \${POSTGRES_DB:- productsdate}"] interval: 10s timeout: 5s retries: 10 api: build: context: . dockerfile: Dockerfile image: products-date- api:latest container_name: productsdate-api restart: unless-stopped depends_on: postgres: </pre> |
|--|--|---|

```

        condition: service_healthy
    ports:
        - "${API_PORT:-5099}:8080"
    environment:

    ASPNETCORE_ENVIRONMENT: Docker
    ASPNETCORE_URLS:
    http://+:8080
    ConnectionStrings__Default:
    Host=postgres;Port=5432;Database=${POSTGRES_DB:-productsdate};Username=${POSTGRES_USER:-postgres};Password=${POSTGRES_PASSWORD}

    Jwt__Issuer:
    ${JWT_ISSUER:-ProductsDateAPI}
    Jwt__Audience:
    ${JWT_AUDIENCE:-ProductsDateAPI}
    Jwt__Secret:
    ${JWT_SECRET:?set JWT_SECRET in .env}
    Jwt__AccessTokenMinutes:
    ${JWT_ACCESS_TOKEN_MINUTES:-240}
    TrainingService__ApiKey:
    ${TRAINING_CLIENT_API_KEY:?set TRAINING_CLIENT_API_KEY in .env}

    TrainingService__TempRoot:
    /app/temp/training

    TrainingService__SeedDatasetPath:
    /app/dataset

    TrainingService__StorageRoot:
    /app/storage/training

    Database__ApplyMigrations:
    ${APPLY_MIGRATIONS:-true}
    Swagger__Enabled:
    ${SWAGGER_ENABLED:-true}

    volumes:
        - api_storage:/app/storage
        - api_uploads:/app/uploads
        - api_temp:/app/temp
        - ./dataset:/app/dataset:ro

```

```

volumes:
    postgres_data:
    api_storage:
    api_uploads:
    api_temp:

=====
Файл:
Models/Entities/BaseEntity.cs
=====
namespace Models.Entities;

public abstract class BaseEntity
{
    public int Id { get; set; }
}

=====
Файл:
Models/Entities/ErrorResponse.cs
=====
using
System.ComponentModel.DataAnnotations;

namespace Models.Entities;

public class ErrorResponse :
BaseEntity
{
    [Required]
    public int UserId { get; set; }
    public User? User { get; set; }
}

```

```

        public int? VideoId { get; set; }
    }

    public VideoSample? Video {
    get; set; }

    public int? ModelVersionId {
    get; set; }

    public ModelVersion?
    ModelVersion { get; set; }

    [MaxLength(2000)]
    public string? Comment { get;
    set; }

    [Required]
    public DateTime CreatedAt {
    get; set; } = DateTime.UtcNow;

    public bool Resolved { get;
    set; } = false;

    public bool Approved { get;
    set; } = false;

    public int FramesCount { get;
    set; } = 0;
    }

=====
Файл:
Models/Entities/ModelVersion.cs
=====
using
System.ComponentModel.DataAnnotations;

namespace Models.Entities;

public class ModelVersion :
BaseEntity
{
    [Required]

```

```

        public DateTime TrainedAt {
get; set; } = DateTime.UtcNow;

        /// <summary>Метрики в
JSON (например:
mAP/precision/recall)</summary>
        public string? MetricsJson {
get; set; }

        [MaxLength(128)]
        public string? ExternalJobId {
get; set; }

        [MaxLength(256)]
        public string? BaseModel {
get; set; }

        [MaxLength(2048)]
        public string?
BestWeightsPath { get; set; }

        [MaxLength(2048)]
        public string?
MobileModelPath { get; set; }

        [MaxLength(32)]
        public string? MobileFormat {
get; set; }

        [MaxLength(512)]
        public string?
MobileModelFileName { get; set; }

        [MaxLength(128)]
        public string?
MobileModelContentType { get; set; }

        public bool IsPublished { get;
set; }

        public bool IsPinned { get;
set; }

        public bool IsDeleted { get;
set; }

        public DateTime? DeletedAt {
get; set; }

```

```

        public
ICollection<ErrorReport> ErrorReports {
get; set; } = new List<ErrorReport>();
    }

        Файл:
Models/Entities/Product.cs

        using
System.ComponentModel.DataAnnotations
;

        namespace Models.Entities;

        public class Product : BaseEntity
        {
            [Required, MaxLength(256)]
            public string Name { get; set;
} = string.Empty;

            [MaxLength(256)]
            public string? Manufacturer {
get; set; }

            [MaxLength(64)]
            public string? Barcode { get;
set; }

        public
ICollection<StoredProduct> StoredProducts
{ get; set; } = new List<StoredProduct>();
    }

        Файл:
Models/Entities/StoredProduct.cs

```

```

        using
System.ComponentModel.DataAnnotations
;

        namespace Models.Entities;

        public class StoredProduct :
BaseEntity
        {
            [Required]
            public int UserId { get; set; }
            public User? User { get; set; }

            [Required]
            public int ProductId { get; set;

            public Product? Product { get;
set; }

            /// <summary>Дата
изготовления (если известна)</summary>
            public DateTime?
ManufactureAt { get; set; }

            /// <summary>Срок годности
(дата истечения)</summary>
            public DateTime? ExpiryAt {
get; set; }

            [Required]
            public DateTime CreatedAt {
get; set; } = DateTime.UtcNow;
        }

        Файл:
Models/Entities/TrainingJob.cs

```

```

=====
=====

using
System.ComponentModel.DataAnnotations
;

namespace Models.Entities;

public class TrainingJob :
BaseEntity
{
    [Required]
    [MaxLength(128)]
    public string JobId { get; set;
} = Guid.NewGuid().ToString("N");

    [Required]
    [MaxLength(32)]
    public string Status { get; set;
} = "queued";

    public string? Message { get;
set; }

    public DateTime CreatedAt {
get; set; } = DateTime.UtcNow;
    public DateTime? StartedAt {
get; set; }
    public DateTime? FinishedAt
{ get; set; }
    public DateTime? AssignedAt
{ get; set; }
    public DateTime? HeartbeatAt
{ get; set; }

    public int ImagesCount { get;
set; }

    [MaxLength(256)]
    public string? ClientId { get;
set; }

    [MaxLength(2048)]
    public string DatasetZipPath {
get; set; } = string.Empty;

    [MaxLength(256)]

```

```

public string? BaseModel {
get; set; }

    public int Epochs { get; set; }
    = 50;
    public int ImgSize { get; set; }
    = 640;
    public int Batch { get; set; } =
16;

    [MaxLength(64)]
    public string? Device { get;
set; } = "auto";

    public bool ExportInt8 { get;
set; } = true;
    public bool ExportNms { get;
set; } = true;

    [MaxLength(32)]
    public string? MobileFormat {
get; set; } = "tf-lite";

    public double
QuantizationFraction { get; set; } = 0.3;

    [MaxLength(2048)]
    public string?
BestWeightsPath { get; set; }

    [MaxLength(2048)]
    public string?
MobileModelPath { get; set; }

    [MaxLength(512)]
    public string?
MobileModelFileName { get; set; }

    [MaxLength(128)]
    public string?
MobileModelContentType { get; set; }

    public string? MetricsJson {
get; set; }

    public bool
CancellationRequested { get; set; }
}

```

```

=====
=====
=====
Файл: Models/Entities/User.cs
=====
=====
=====

using
System.ComponentModel.DataAnnotations
;

namespace Models.Entities;

public class User : BaseEntity
{
    [Required, EmailAddress,
MaxLength(256)]
    public string Email { get; set;
} = string.Empty;

    [Required, MaxLength(256)]
    public string PasswordHash {
get; set; } = string.Empty;

    public bool IsBlocked { get;
set; } = false;

    /// <summary>Расширяемые
настройки в JSON (может быть
null/пусто)</summary>
    public string? SettingsJson {
get; set; }

    public DateTime CreatedAt {
get; set; } = DateTime.UtcNow;

    public
ICollection<StoredProduct> StoredProducts
{ get; set; } = new List<StoredProduct>();
    public
ICollection<ErrorReport> ErrorReports {
get; set; } = new List<ErrorReport>();
}

```

```

=====
=====
=====
=====
Файл:
Models/Entities/VideoSample.cs
=====
=====
=====
=====

using
System.ComponentModel.DataAnnotations
;

namespace Models.Entities;

public class VideoSample :
BaseEntity
{
    [Required, MaxLength(1024)]
    public string VideoPath { get;
set; } = string.Empty;

    /// <summary>camera/file/test
и т.п.</summary>
    [MaxLength(64)]
    public string? Source { get;
set; }

    public
ICollection<ErrorReport> ErrorReports {
get; set; } = new List<ErrorReport>();
}

=====
=====
=====
=====
Файл: Models/Models.csproj
=====
=====
=====
=====

<Project
Sdk="Microsoft.NET.Sdk">

```

```

<PropertyGroup>

<TargetFramework>net8.0</TargetFramework>

<ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
</PropertyGroup>

</Project>

=====
=====
=====
=====
Файл: netSolution.sln
=====
=====
=====
=====

Microsoft Visual Studio Solution
File, Format Version 12.00
# Visual Studio Version 17
VisualStudioVersion =
17.14.36717.8
MinimumVisualStudioVersion =
10.0.40219.1
Project("{FAE04EC0-301F-
11D3-BF4B-00C04F79EFBC}") =
"AdminApp",
"AdminApp\AdminApp.csproj",
"{221FD622-2A96-4622-8CF9-
8F7B1E612DB6}"
EndProject
Project("{FAE04EC0-301F-
11D3-BF4B-00C04F79EFBC}") =
"Models", "Models\Models.csproj",
"{B47F731F-92CA-45F0-A70E-
4D86D209D5A3}"
EndProject
Project("{FAE04EC0-301F-
11D3-BF4B-00C04F79EFBC}") =
"ProductsDateAPI",
"ProductsDateAPI\ProductsDateAPI.csproj",
"{E1ECA3E2-5982-43A1-9FE0-
B52291AF7510}"

```

```

EndProject
Project("{FAE04EC0-301F-
11D3-BF4B-00C04F79EFBC}") =
"PostgreSQLRepository",
"PostgreSQLRepository\PostgreSQLReposi
tory.csproj", "{8731B5F6-950E-438B-
9747-E97B11D138A7}"
EndProject
Project("{FAE04EC0-301F-
11D3-BF4B-00C04F79EFBC}") =
"Contracts", "Contracts\Contracts.csproj",
"{A73A4B9F-623F-43E3-8218-
B6A80EB904DD}"
EndProject
Global
GlobalSection(SolutionConfigur
ationPlatforms) = preSolution
    Debug|Any CPU =
Debug|Any CPU
    Release|Any CPU =
Release|Any CPU
EndGlobalSection
GlobalSection(ProjectConfigurat
ionPlatforms) = postSolution
    {221FD622-2A96-
4622-8CF9-8F7B1E612DB6}.Debug|Any
CPU.ActiveCfg = Debug|Any CPU
    {221FD622-2A96-
4622-8CF9-8F7B1E612DB6}.Debug|Any
CPU.Build.0 = Debug|Any CPU
    {221FD622-2A96-
4622-8CF9-8F7B1E612DB6}.Release|Any
CPU.ActiveCfg = Release|Any CPU
    {221FD622-2A96-
4622-8CF9-8F7B1E612DB6}.Release|Any
CPU.Build.0 = Release|Any CPU
    {B47F731F-92CA-
45F0-A70E-4D86D209D5A3}.Debug|Any
CPU.ActiveCfg = Debug|Any CPU
    {B47F731F-92CA-
45F0-A70E-4D86D209D5A3}.Debug|Any
CPU.Build.0 = Debug|Any CPU
    {B47F731F-92CA-
45F0-A70E-4D86D209D5A3}.Release|Any
CPU.ActiveCfg = Release|Any CPU
    {B47F731F-92CA-
45F0-A70E-4D86D209D5A3}.Release|Any
CPU.Build.0 = Release|Any CPU

```

```

        {E1ECA3E2-5982-
43A1-9FE0-B52291AF7510}.Debug|Any
CPU.ActiveCfg = Debug|Any CPU
        {E1ECA3E2-5982-
43A1-9FE0-B52291AF7510}.Debug|Any
CPU.Build.0 = Debug|Any CPU
        {E1ECA3E2-5982-
43A1-9FE0-B52291AF7510}.Release|Any
CPU.ActiveCfg = Release|Any CPU
        {E1ECA3E2-5982-
43A1-9FE0-B52291AF7510}.Release|Any
CPU.Build.0 = Release|Any CPU
        {8731B5F6-950E-
438B-9747-E97B11D138A7}.Debug|Any
CPU.ActiveCfg = Debug|Any CPU
        {8731B5F6-950E-
438B-9747-E97B11D138A7}.Debug|Any
CPU.Build.0 = Debug|Any CPU
        {8731B5F6-950E-
438B-9747-E97B11D138A7}.Release|Any
CPU.ActiveCfg = Release|Any CPU
        {8731B5F6-950E-
438B-9747-E97B11D138A7}.Release|Any
CPU.Build.0 = Release|Any CPU
        {A73A4B9F-623F-
43E3-8218-B6A80EB904DD}.Debug|Any
CPU.ActiveCfg = Debug|Any CPU
        {A73A4B9F-623F-
43E3-8218-B6A80EB904DD}.Debug|Any
CPU.Build.0 = Debug|Any CPU
        {A73A4B9F-623F-
43E3-8218-B6A80EB904DD}.Release|Any
CPU.ActiveCfg = Release|Any CPU
        {A73A4B9F-623F-
43E3-8218-B6A80EB904DD}.Release|Any
CPU.Build.0 = Release|Any CPU
    EndGlobalSection
    GlobalSection(SolutionPropertie
s) = preSolution
        HideSolutionNode =
FALSE
    EndGlobalSection
    GlobalSection(ExtensibilityGlob
als) = postSolution
        SolutionGuid =
{1579380D-819E-458B-831B-
955CAF98C365}
    EndGlobalSection

```

```

EndGlobal

=====

=====

=====

Файл:
PostgreSQLRepository/AppDbContext.cs
=====

=====

=====

using
Microsoft.EntityFrameworkCore;

using Models.Entities;

namespace
PostgreSQLRepository;

public class AppDbContext :
DbContext
{
    public
AppDbContext(DbContextOptions<AppDb
Context> options) : base(options) { }

    public DbSet<User> Users =>
Set<User>();

    public DbSet<Product>
Products => Set<Product>();

    public DbSet<StoredProduct>
StoredProducts => Set<StoredProduct>();

    public DbSet<VideoSample>
VideoSamples => Set<VideoSample>();

    public DbSet<ModelVersion>
ModelVersions => Set<ModelVersion>();

    public DbSet<TrainingJob>
TrainingJobs => Set<TrainingJob>();

    public DbSet<ErrorReport>
ErrorReports => Set<ErrorReport>();

    protected override void
OnModelCreating(ModelBuilder
modelBuilder)
{

    base.OnModelCreating(modelBuilder);

```

```

modelBuilder.Entity<User>()
    .HasIndex(x => x.Email)
    .IsUnique();

modelBuilder.Entity<Product>()
    .HasIndex(x =>
x.Barcode);

modelBuilder.Entity<ModelVersion>()
    .HasIndex(x =>
x.ExternalJobId)
    .IsUnique();

modelBuilder.Entity<ModelVersion>()
    .HasIndex(x =>
x.IsPublished);

modelBuilder.Entity<ModelVersion>()
    .HasIndex(x =>
x.IsDeleted);

modelBuilder.Entity<TrainingJob>()
    .HasIndex(x => x.JobId)
    .IsUnique();

modelBuilder.Entity<TrainingJob>()
    .HasIndex(x =>
x.Status);

modelBuilder.Entity<StoredProduct>()
    .HasOne(x => x.User)
    .WithMany(x =>
x.StoredProducts)
    .HasForeignKey(x =>
x.UserId)

.OnDelete(DeleteBehavior.Cascade);

```

| | | |
|--|---|--|
| <pre> modelBuilder.Entity<StoredProduct>() .HasOne(x => x.Product) .WithMany(x => x.StoredProducts) .HasForeignKey(x => x.ProductId) .OnDelete(DeleteBehavior.Restrict); modelBuilder.Entity<ErrorReport>() .HasOne(x => x.User) .WithMany(x => x.ErrorReports) .HasForeignKey(x => x.UserId) .OnDelete(DeleteBehavior.Cascade); modelBuilder.Entity<ErrorReport>() .HasOne(x => x.Video) .WithMany(x => x.ErrorReports) .HasForeignKey(x => x.VideoId) .OnDelete(DeleteBehavior.SetNull); modelBuilder.Entity<ErrorReport>() .HasOne(x => x.ModelVersion) .WithMany(x => x.ErrorReports) .HasForeignKey(x => x.ModelVersionId) .OnDelete(DeleteBehavior.SetNull); </pre> | <pre> Файл: PostgreSQLRepository/Migrations/2026021 1191844_InitialCreate.cs ===== ===== ===== using System; using Microsoft.EntityFrameworkCore.Migration s; using Npgsql.EntityFrameworkCore.PostgreSQL. Metadata; #nullable disable namespace PostgreSQLRepository.Migrations { /// <inheritdoc /> public partial class InitialCreate : Migration { /// <inheritdoc /> protected override void Up(MigrationBuilder migrationBuilder) { migrationBuilder.CreateTable(name: "ModelVersions", columns: table => new { Id = table.Column<int>(type: "integer", nullable: false) .Annotation("Npgsql:ValueGenerationStrat egy", Npgsql.ValueGenerationStrategy.IdentityBy DefaultColumn), TrainedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false), } } </pre> | <pre> MetricsJson = table.Column<string>(type: "text", nullable: true) }, constraints: table => { table.PrimaryKey("PK_ModelVersions", x => x.Id); }); migrationBuilder.CreateTable(name: "Products", columns: table => new { Id = table.Column<int>(type: "integer", nullable: false) .Annotation("Npgsql:ValueGenerationStrat egy", Npgsql.ValueGenerationStrategy.IdentityBy DefaultColumn), Name = table.Column<string>(type: "character varying(256)", maxLength: 256, nullable: false), Manufacturer = table.Column<string>(type: "character varying(256)", maxLength: 256, nullable: true), Barcode = table.Column<string>(type: "character varying(64)", maxLength: 64, nullable: true) }, constraints: table => { table.PrimaryKey("PK_Products", x => x.Id); }); migrationBuilder.CreateTable(name: "Users", columns: table => new </pre> |
|--|---|--|


```

        {
            Id =
table.Column<int>(type: "integer", nullable:
false)

.Annotation("Npgsql:ValueGenerationStrategyStrat
egy",
NpgsqlValueGenerationStrategy.IdentityBy
DefaultColumn),
            Email =
table.Column<string>(type: "character
varying(256)", maxLength: 256, nullable:
false),
            PasswordHash =
table.Column<string>(type: "character
varying(256)", maxLength: 256, nullable:
false),
            IsBlocked =
table.Column<bool>(type: "boolean",
nullable: false),
            SettingsJson =
table.Column<string>(type: "text", nullable:
true)
        },
        constraints: table =>
        {

table.PrimaryKey("PK_Users", x => x.Id);
        });

migrationBuilder.CreateTable(
            name:
"VideoSamples",
            columns: table => new
            {
                Id =
table.Column<int>(type: "integer", nullable:
false)

.Annotation("Npgsql:ValueGenerationStrategyStrat
egy",
NpgsqlValueGenerationStrategy.IdentityBy
DefaultColumn),
                VideoPath =
table.Column<string>(type: "character
varying(1024)", maxLength: 1024, nullable:
false),

```

```

Source =
table.Column<string>(type: "character
varying(64)", maxLength: 64, nullable:
true)
            },
            constraints: table =>
            {

table.PrimaryKey("PK_VideoSamples", x
=> x.Id);
            });

migrationBuilder.CreateTable(
            name:
"StoredProducts",
            columns: table => new
            {
                Id =
table.Column<int>(type: "integer", nullable:
false)

.Annotation("Npgsql:ValueGenerationStrategyStrat
egy",
NpgsqlValueGenerationStrategy.IdentityBy
DefaultColumn),
                UserId =
table.Column<int>(type: "integer", nullable:
false),
                ProductId =
table.Column<int>(type: "integer", nullable:
false),
                ManufactureAt =
table.Column<DateTime>(type: "timestamp
with time zone", nullable: true),
                ExpiryAt =
table.Column<DateTime>(type: "timestamp
with time zone", nullable: true),
                CreatedAt =
table.Column<DateTime>(type: "timestamp
with time zone", nullable: false)
            },
            constraints: table =>
            {

table.PrimaryKey("PK_StoredProducts", x
=> x.Id);

table.ForeignKey(

```

```

            name:
"FK_StoredProducts_Products_ProductId",
            column: x =>
x.ProductId,
            principalTable:
"Products",
            principalColumn:
"Id",
            onDelete:
ReferentialAction.Restrict);
            table.ForeignKey(
            name:
"FK_StoredProducts_Users_UserId",
            column: x =>
x.UserId,
            principalTable:
"Users",
            principalColumn:
"Id",
            onDelete:
ReferentialAction.Cascade);
        });

migrationBuilder.CreateTable(
            name: "ErrorReports",
            columns: table => new
            {
                Id =
table.Column<int>(type: "integer", nullable:
false)

.Annotation("Npgsql:ValueGenerationStrategyStrat
egy",
NpgsqlValueGenerationStrategy.IdentityBy
DefaultColumn),
                UserId =
table.Column<int>(type: "integer", nullable:
false),
                VideoId =
table.Column<int>(type: "integer", nullable:
true),
                ModelVersionId =
table.Column<int>(type: "integer", nullable:
true),
                Comment =
table.Column<string>(type: "character

```

```

        varying(2000)", maxLength: 2000, nullable:
        true),

        CreatedAt =
        table.Column<DateTime>(type: "timestamp
        with time zone", nullable: false),

        Resolved =
        table.Column<bool>(type: "boolean",
        nullable: false)

    },
    constraints: table =>
    {

        table.PrimaryKey("PK_ErrorReports", x =>
        x.Id);

        table.ForeignKey(
            name:
            "FK_ErrorReports_ModelVersions_Model
            VersionId",

            column: x =>
            x.ModelVersionId,

            principalTable:
            "ModelVersions",

            principalColumn:
            "Id",

            onDelete:
            ReferentialAction.SetNull);

        table.ForeignKey(
            name:
            "FK_ErrorReports_Users_UserId",

            column: x =>
            x.UserId,

            principalTable:
            "Users",

            principalColumn:
            "Id",

            onDelete:
            ReferentialAction.Cascade);

        table.ForeignKey(
            name:
            "FK_ErrorReports_VideoSamples_VideoId
            ",

            column: x =>
            x.VideoId,

            principalTable:
            "VideoSamples",

            principalColumn:
            "Id",

```

```

        onDelete:
        ReferentialAction.SetNull);
    });

    migrationBuilder.CreateIndex(
        name:
        "IX_ErrorReports_ModelVersionId",
        table: "ErrorReports",
        column:
        "ModelVersionId");

    migrationBuilder.CreateIndex(
        name:
        "IX_ErrorReports_UserId",
        table: "ErrorReports",
        column: "UserId");

    migrationBuilder.CreateIndex(
        name:
        "IX_ErrorReports_VideoId",
        table: "ErrorReports",
        column: "VideoId");

    migrationBuilder.CreateIndex(
        name:
        "IX_Products_Barcode",
        table: "Products",
        column: "Barcode");

    migrationBuilder.CreateIndex(
        name:
        "IX_StoredProducts_ProductId",
        table:
        "StoredProducts",
        column: "ProductId");

    migrationBuilder.CreateIndex(
        name:
        "IX_StoredProducts_UserId",
        table:
        "StoredProducts",
        column: "UserId");

```

```

migrationBuilder.CreateIndex(
    name:
    "IX_Users_Email",
    table: "Users",
    column: "Email",
    unique: true);

    }

    /// <inheritdoc />
    protected override void
    Down(MigrationBuilder migrationBuilder)
    {

        migrationBuilder.DropTable(
            name: "ErrorReports");

        migrationBuilder.DropTable(
            name:
            "StoredProducts");

        migrationBuilder.DropTable(
            name:
            "ModelVersions");

        migrationBuilder.DropTable(
            name:
            "VideoSamples");

        migrationBuilder.DropTable(
            name: "Products");

        migrationBuilder.DropTable(
            name: "Users");
    }
}

=====
=====
=====
=====

```

```

Файл:
PostgreSQLRepository/Migrations/2026021
1191844_InitialCreate.Designer.cs
=====

// <auto-generated />
using System;
using
Microsoft.EntityFrameworkCore;
using
Microsoft.EntityFrameworkCore.Infrastruct
ure;
using
Microsoft.EntityFrameworkCore.Migration
s;
using
Microsoft.EntityFrameworkCore.Storage.V
alueConversion;
using
Npgsql.EntityFrameworkCore.PostgreSQL.
Metadata;
using PostgreSQLRepository;

#nullable disable

namespace
PostgreSQLRepository.Migrations
{

[DbContext(typeof(AppDbContext))]

[Migration("20260211191844_InitialCreate
")]

    partial class InitialCreate
    {
        /// <inheritdoc />
        protected override void
BuildTargetModel(ModelBuilder
modelBuilder)
        {
            #pragma warning disable 612,
618

            modelBuilder

```

```

.HasAnnotation("ProductVersion",
"9.0.13")

.HasAnnotation("Relational:MaxIdentifierL
ength", 63);

NpgsqlModelBuilderExtensions.UseIdentit
yByDefaultColumns(modelBuilder);

modelBuilder.Entity("Models.Entities.Error
Report", b =>
{
    b.Property<int>("Id")

    .ValueGeneratedOnAdd()

    .HasColumnType("integer");

    NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

    b.Property<string>("Comment")

    .HasMaxLength(2000)

    .HasColumnType("character
varying(2000)");

    b.Property<DateTime>("CreatedAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<int?>("ModelVersionId")

    .HasColumnType("integer");

    b.Property<bool>("Resolved")

```

```

.HasColumnType("boolean");

b.Property<int>("UserId")

.HasColumnType("integer");

b.Property<int?>("VideoId")

.HasColumnType("integer");

    b.HasKey("Id");

    b.HasIndex("ModelVersionId");

    b.HasIndex("UserId");

    b.HasIndex("VideoId");

    b.ToTable("ErrorReports");
});

modelBuilder.Entity("Models.Entities.Mod
elVersion", b =>
{
    b.Property<int>("Id")

    .ValueGeneratedOnAdd()

    .HasColumnType("integer");

    NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

    b.Property<string>("MetricsJson")

    .HasColumnType("text");

```

```

        b.Property<DateTime>("TrainedAt")

        .HasColumnType("timestamp with time
zone");

        b.HasKey("Id");

        b.ToTable("ModelVersions");
    });

modelBuilder.Entity("Models.Entities.Prod
uct", b =>
    {
        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<string>("Barcode")

        .HasMaxLength(64)

        .HasColumnType("character varying(64)");

        b.Property<string>("Manufacturer")

        .HasMaxLength(256)

        .HasColumnType("character
varying(256)");

        b.Property<string>("Name")

        .IsRequired()

```

```

        .HasMaxLength(256)

        .HasColumnType("character
varying(256)");

        b.HasKey("Id");

        b.HasIndex("Barcode");

        b.ToTable("Products");
    });

modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
    {
        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<DateTime>("CreatedAt")

        .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ExpiryAt")

        .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ManufactureAt")

        .HasColumnType("timestamp with time
zone");

```

```

        b.Property<int>("ProductId")

        .HasColumnType("integer");

        b.Property<int>("UserId")

        .HasColumnType("integer");

        b.HasKey("Id");

        b.HasIndex("ProductId");

        b.HasIndex("UserId");

        b.ToTable("StoredProducts");
    });

modelBuilder.Entity("Models.Entities.User
", b =>
    {
        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<string>("Email")

        .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character
varying(256)");

```

| | | |
|--|--|--|
| b.Property<bool>("IsBlocked") | .HasColumnType("character varying(64)"); | .HasForeignKey("VideoId") |
| .HasColumnType("boolean"); | b.Property<string>("VideoPath") | .OnDelete(DeleteBehavior.SetNull); |
| b.Property<string>("PasswordHash") | .IsRequired() | b.Navigation("ModelVersion"); |
| .IsRequired() | .HasMaxLength(1024) | b.Navigation("User"); |
| .HasMaxLength(256) | .HasColumnType("character varying(1024)"); | b.Navigation("Video"); |
| .HasColumnType("character varying(256)"); | b.HasKey("Id"); | }); |
| b.Property<string>("SettingsJson") | b.ToTable("VideoSamples"); | modelBuilder.Entity("Models.Entities.Store dProduct", b => |
| .HasColumnType("text"); | }); | { |
| b.HasKey("Id"); | modelBuilder.Entity("Models.Entities.Error Report", b => | b.HasOne("Models.Entities.Product", "Product") |
| b.HasIndex("Email") | { | .WithMany("StoredProducts") |
| .IsUnique(); | b.HasOne("Models.Entities.ModelVersion", "ModelVersion") | .HasForeignKey("ProductId") |
| b.ToTable("Users"); | .WithMany("ErrorReports") | .OnDelete(DeleteBehavior.Restrict) |
| }); | .HasForeignKey("ModelVersionId") | .IsRequired(); |
| modelBuilder.Entity("Models.Entities.VideoSample", b => | .OnDelete(DeleteBehavior.SetNull); | b.HasOne("Models.Entities.User", "User") |
| { | b.HasOne("Models.Entities.User", "User") | .WithMany("StoredProducts") |
| b.Property<int>("Id") | .WithMany("ErrorReports") | .HasForeignKey("UserId") |
| .ValueGeneratedOnAdd() | .HasForeignKey("UserId") | .OnDelete(DeleteBehavior.Cascade) |
| .HasColumnType("integer"); | .OnDelete(DeleteBehavior.Cascade) | .IsRequired(); |
| NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id")); | .IsRequired(); | b.Navigation("Product"); |
| b.Property<string>("Source") | b.HasOne("Models.Entities.VideoSample", "Video") | b.Navigation("User"); |
| .HasMaxLength(64) | .WithMany("ErrorReports") | }); |

```
modelBuilder.Entity("Models.Entities.ModelVersion", b =>
```

```
{
```

```
    b.Navigation("ErrorReports");
}
```

```
modelBuilder.Entity("Models.Entities.Product", b =>
```

```
{
```

```
    b.Navigation("StoredProducts");
}
```

```
modelBuilder.Entity("Models.Entities.User", b =>
```

```
{
```

```
    b.Navigation("ErrorReports");
```

```
    b.Navigation("StoredProducts");
}
```

```
modelBuilder.Entity("Models.Entities.VideoSample", b =>
```

```
{
```

```
    b.Navigation("ErrorReports");
}
```

```
    #pragma warning restore 612, 618
```

```
}
```

```
}
```

```
}
```

```
=====
=====
=====
```

```
Файл:
```

```
PostgreSQLRepository/Migrations/20260211195602_mig2.cs
```

```
=====
=====
=====
```

```
using System;
using
Microsoft.EntityFrameworkCore.Migrations;
```

```
#nullable disable
```

```
namespace
PostgreSQLRepository.Migrations
{
    /// <inheritdoc />
    public partial class mig2 :
Migration
    {
        /// <inheritdoc />
        protected override void
Up(MigrationBuilder migrationBuilder)
        {
```

```
            migrationBuilder.AddColumn<DateTime>(
                name: "CreatedAt",
                table: "Users",
                type: "timestamp with
time zone",
                nullable: false,
                defaultValue: new
DateTime(1, 1, 1, 0, 0, 0, 0,
DateTimeKind.Unspecified));
        }
```

```
        /// <inheritdoc />
        protected override void
Down(MigrationBuilder migrationBuilder)
        {
```

```
            migrationBuilder.DropColumn(
                name: "CreatedAt",
                table: "Users");
        }
    }
}
```

```
=====
=====
=====
```

```
Файл:
```

```
PostgreSQLRepository/Migrations/20260211195602_mig2.Designer.cs
```

```
// <auto-generated />
using System;
using
Microsoft.EntityFrameworkCore;
using
Microsoft.EntityFrameworkCore.Infrastructure;
using
Microsoft.EntityFrameworkCore.Migrations;
using
Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using
Npgsql.EntityFrameworkCore.PostgreSQL.Metadata;
using PostgreSQLRepository;

#nullable disable
```

```
namespace
PostgreSQLRepository.Migrations
{
    [DbContext(typeof(AppDbContext))]
    [Migration("20260211195602_mig2")]
    partial class mig2
    {
        /// <inheritdoc />
        protected override void
BuildTargetModel(ModelBuilder
modelBuilder)
        {
            #pragma warning disable 612, 618
```

| | | |
|---|---|--|
| <pre> modelBuilder .HasAnnotation("ProductVersion", "9.0.13") .HasAnnotation("Relational:MaxIdentifierL ength", 63); NpgsqlModelBuilderExtensions.UseIdentit yByDefaultColumns(modelBuilder); modelBuilder.Entity("Models.Entities.Error Report", b => { b.Property<int>("Id") .ValueGeneratedOnAdd() .HasColumnType("integer"); NpgsqlPropertyBuilderExtensions.UseIdent ityByDefaultColumn(b.Property<int>("Id")); b.Property<string>("Comment") .HasMaxLength(2000) .HasColumnType("character varying(2000)"); b.Property<DateTime>("CreatedAt") .HasColumnType("timestamp with time zone"); b.Property<int?>("ModelVersionId") .HasColumnType("integer"); </pre> | <pre> b.Property<bool>("Resolved") .HasColumnType("boolean"); b.Property<int>("UserId") .HasColumnType("integer"); b.Property<int?>("VideoId") .HasColumnType("integer"); b.HasKey("Id"); b.HasIndex("ModelVersionId"); b.HasIndex("UserId"); b.HasIndex("VideoId"); b.ToTable("ErrorReports"); modelBuilder.Entity("Models.Entities.Mod elVersion", b => { b.Property<int>("Id") .ValueGeneratedOnAdd() .HasColumnType("integer"); NpgsqlPropertyBuilderExtensions.UseIdent ityByDefaultColumn(b.Property<int>("Id")); b.Property<string>("MetricsJson") </pre> | <pre> .HasColumnType("text"); b.Property<DateTime>("TrainedAt") .HasColumnType("timestamp with time zone"); b.HasKey("Id"); b.ToTable("ModelVersions"); modelBuilder.Entity("Models.Entities.Prod uct", b => { b.Property<int>("Id") .ValueGeneratedOnAdd() .HasColumnType("integer"); NpgsqlPropertyBuilderExtensions.UseIdent ityByDefaultColumn(b.Property<int>("Id")); b.Property<string>("Barcode") .HasMaxLength(64) .HasColumnType("character varying(64)"); b.Property<string>("Manufacturer") .HasMaxLength(256) .HasColumnType("character varying(256)"); b.Property<string>("Name") </pre> |
|---|---|--|

```

        .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character
        varying(256)");

        b.HasKey("Id");

        b.HasIndex("Barcode");

        b.ToTable("Products");
    });

    modelBuilder.Entity("Models.Entities.Store
    dProduct", b =>
    {

        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
        ityByDefaultColumn(b.Property<int>("Id")
        );

        b.Property<DateTime>("CreatedAt")

        .HasColumnType("timestamp with time
        zone");

        b.Property<DateTime?>("ExpiryAt")

        .HasColumnType("timestamp with time
        zone");

        b.Property<DateTime?>("ManufactureAt")

        .HasColumnType("timestamp with time
        zone");

        .HasColumnType("timestamp with time
        zone");

        b.Property<int>("ProductId")

        .HasColumnType("integer");

        b.Property<int>("UserId")

        .HasColumnType("integer");

        b.HasIndex("ProductId");

        b.HasIndex("UserId");

        b.ToTable("StoredProducts");
    });

    modelBuilder.Entity("Models.Entities.User
    ", b =>
    {

        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
        ityByDefaultColumn(b.Property<int>("Id")
        );

        b.Property<DateTime>("CreatedAt")

        .HasColumnType("timestamp with time
        zone");

        b.Property<string>("Email")

        .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character
        varying(256)");

        b.Property<string>("PasswordHash")

        .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character
        varying(256)");

        b.Property<bool>("IsBlocked")

        .HasColumnType("boolean");

        b.Property<string>("PasswordHash")

        .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character
        varying(256)");

        b.Property<string>("SettingsJson")

        .HasColumnType("text");

        b.HasKey("Id");

        b.HasIndex("Email")

        .IsUnique();

        b.ToTable("Users");
    });

    modelBuilder.Entity("Models.Entities.Vide
    oSample", b =>
    {

        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

```



```
NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"))
);
```

```
b.Property<string>("Source")
```

```
.HasMaxLength(64)
```

```
.HasColumnType("character varying(64)");
```

```
b.Property<string>("VideoPath")
    .IsRequired()
```

```
.HasMaxLength(1024)
```

```
.HasColumnType("character
varying(1024)");
```

```
b.HasKey("Id");
```

```
b.ToTable("VideoSamples");
    });
```

```
modelBuilder.Entity("Models.Entities.Error
Report", b =>
    {
```

```
b.HasOne("Models.Entities.ModelVersion",
"ModelVersion")
```

```
.WithMany("ErrorReports")
```

```
.HasForeignKey("ModelVersionId")
```

```
.OnDelete(DeleteBehavior.SetNull);
```

```
b.HasOne("Models.Entities.User", "User")
```

```
.WithMany("ErrorReports")
```

```
.HasForeignKey("UserId")
```

```
.OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

```
b.HasOne("Models.Entities.VideoSample",
"Video")
```

```
.WithMany("ErrorReports")
```

```
.HasForeignKey("VideoId")
```

```
.OnDelete(DeleteBehavior.SetNull);
```

```
b.Navigation("ModelVersion");
```

```
b.Navigation("User");
```

```
b.Navigation("Video");
    });
```

```
modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
    {
```

```
b.HasOne("Models.Entities.Product",
"Product")
```

```
.WithMany("StoredProducts")
```

```
.HasForeignKey("ProductId")
```

```
.OnDelete(DeleteBehavior.Restrict)
    .IsRequired();
```

```
b.HasOne("Models.Entities.User", "User")
```

```
.WithMany("StoredProducts")
```

```
.HasForeignKey("UserId")
```

```
.OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

```
b.Navigation("Product");
```

```
b.Navigation("User");
    });
```

```
modelBuilder.Entity("Models.Entities.Mod
elVersion", b =>
    {
```

```
b.Navigation("ErrorReports");
    });
```

```
modelBuilder.Entity("Models.Entities.Prod
uct", b =>
    {
```

```
b.Navigation("StoredProducts");
    });
```

```
modelBuilder.Entity("Models.Entities.User
", b =>
    {
```

```
b.Navigation("ErrorReports");
```

```
b.Navigation("StoredProducts");
    });
```

```
modelBuilder.Entity("Models.Entities.Vide
oSample", b =>
    {
```

```
b.Navigation("ErrorReports");
    });
#pragma warning restore 612,
618
    }
    }
}
```

```

=====
=====
=====
=====
Файл:
PostgreSQLRepository/Migrations/2026031
5184736_AddApprovedAndFramesCountT
oErrorReport.cs
=====
=====
=====
=====

using
Microsoft.EntityFrameworkCore.Migration
s;

#nullable disable

namespace
PostgreSQLRepository.Migrations
{
    /// <inheritdoc />
    public partial class
AddApprovedAndFramesCountToErrorRep
ort : Migration
    {
        /// <inheritdoc />
        protected override void
Up(MigrationBuilder migrationBuilder)
        {

migrationBuilder.AddColumn<bool>(<
        name: "Approved",
        table: "ErrorReports",
        type: "boolean",
        nullable: false,
        defaultValue: false);

migrationBuilder.AddColumn<int>(<
        name: "FramesCount",
        table: "ErrorReports",
        type: "integer",
        nullable: false,
        defaultValue: 0);
    }
}

```

```

    /// <inheritdoc />
    protected override void
Down(MigrationBuilder migrationBuilder)
    {

migrationBuilder.DropColumn(<
        name: "Approved",
        table: "ErrorReports");

migrationBuilder.DropColumn(<
        name: "FramesCount",
        table: "ErrorReports");
    }
}

=====
=====
=====
=====
Файл:
PostgreSQLRepository/Migrations/2026031
5184736_AddApprovedAndFramesCountT
oErrorReport.Designer.cs
=====
=====
=====
=====

// <auto-generated />
using System;

using
Microsoft.EntityFrameworkCore;

using
Microsoft.EntityFrameworkCore.Infrastruct
ure;

using
Microsoft.EntityFrameworkCore.Migration
s;

using
Microsoft.EntityFrameworkCore.Storage.V
alueConversion;

using
Npgsql.EntityFrameworkCore.PostgreSQL.
Metadata;

using PostgreSQLRepository;

```

```

#nullable disable

namespace
PostgreSQLRepository.Migrations
{
    [DbContext(typeof(AppDbContext))]

[Migration("20260315184736_AddApprov
edAndFramesCountToErrorReport")]
    partial class
AddApprovedAndFramesCountToErrorRep
ort
    {
        /// <inheritdoc />
        protected override void
BuildTargetModel(ModelBuilder
modelBuilder)
        {
            #pragma warning disable 612,
618

            modelBuilder

.HasAnnotation("ProductVersion",
"9.0.13")

.HasAnnotation("Relational:MaxIdentifierL
ength", 63);

NpgsqlModelBuilderExtensions.UseIdentit
yByDefaultColumns(modelBuilder);

modelBuilder.Entity("Models.Entities.Error
Report", b =>
    {

b.Property<int>("Id")

.ValueGeneratedOnAdd()

.HasColumnType("integer");

NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);
}
}

```

| | | |
|---|--|--|
| b.Property<bool>("Approved") | b.HasIndex("ModelVersionId"); | modelBuilder.Entity("Models.Entities.Product", b => |
| | | { |
| .HasColumnType("boolean"); | b.HasIndex("UserId"); | b.Property<int>("Id") |
| | | .ValueGeneratedOnAdd() |
| b.Property<string>("Comment") | b.HasIndex("VideoId"); | .HasColumnType("integer"); |
| .HasMaxLength(2000) | | |
| .HasColumnType("character varying(2000)"); | b.ToTable("ErrorReports"); | NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id") |
| | }); |); |
| b.Property<DateTime>("CreatedAt") | modelBuilder.Entity("Models.Entities.ModelVersion", b => | |
| | { | b.Property<string>("Barcode") |
| .HasColumnType("timestamp with time zone"); | b.Property<int>("Id") | .HasMaxLength(64) |
| | .ValueGeneratedOnAdd() | .HasColumnType("character varying(64)"); |
| b.Property<int>("FramesCount") | .HasColumnType("integer"); | |
| .HasColumnType("integer"); | | b.Property<string>("Manufacturer") |
| | NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id") | .HasMaxLength(256) |
| b.Property<int?>("ModelVersionId") |); | .HasColumnType("character varying(256)"); |
| .HasColumnType("integer"); | | |
| | b.Property<string>("MetricsJson") | |
| b.Property<bool>("Resolved") | .HasColumnType("text"); | b.Property<string>("Name") |
| .HasColumnType("boolean"); | | .IsRequired() |
| | | .HasMaxLength(256) |
| b.Property<int>("UserId") | b.Property<DateTime>("TrainedAt") | |
| .HasColumnType("integer"); | .HasColumnType("timestamp with time zone"); | .HasColumnType("character varying(256)"); |
| | | |
| | b.HasKey("Id"); | b.HasKey("Id"); |
| b.Property<int?>("VideoId") | | |
| .HasColumnType("integer"); | b.ToTable("ModelVersions"); | b.HasIndex("Barcode"); |
| | }); | |
| b.HasKey("Id"); | | b.ToTable("Products"); |
| | | }); |

```

modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<DateTime>("CreatedAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ExpiryAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ManufactureAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<int>("ProductId")
            .HasColumnType("integer");

        b.Property<int>("UserId")
            .HasColumnType("integer");

        b.HasKey("Id");

        b.HasIndex("ProductId");

        b.HasIndex("UserId");

        b.ToTable("StoredProducts");
    });

modelBuilder.Entity("Models.Entities.User
", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<DateTime>("CreatedAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<string>("Email")
            .IsRequired()
            .HasMaxLength(256)
            .HasColumnType("character
varying(256)");

        b.Property<bool>("IsBlocked")
            .HasColumnType("boolean");

        b.Property<string>("PasswordHash")
            .IsRequired()

        b.Property<string>("SettingsJson")
            .HasColumnType("text");

        b.HasKey("Id");

        b.HasIndex("Email")
            .IsUnique();

        b.ToTable("Users");

modelBuilder.Entity("Models.Entities.Vide
oSample", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<string>("Source")
            .HasMaxLength(64)
            .HasColumnType("character varying(64)");

        b.Property<string>("VideoPath")
            .IsRequired()

```

| | | |
|---|---|---|
| .HasMaxLength(1024) | | modelBuilder.Entity("Models.Entities.Product", b => |
| | b.Navigation("User"); | { |
| .HasColumnType("character
varying(1024)"); | | |
| | b.Navigation("Video"); | b.Navigation("StoredProducts"); |
| b.HasKey("Id"); | }); | }); |
| | | |
| b.ToTable("VideoSamples"); | modelBuilder.Entity("Models.Entities.Store
dProduct", b => | modelBuilder.Entity("Models.Entities.User", b => |
| }); | { | { |
| | | |
| modelBuilder.Entity("Models.Entities.Error
Report", b => | b.HasOne("Models.Entities.Product",
"Product") | b.Navigation("ErrorReports"); |
| { | | |
| | .WithMany("StoredProducts") | b.Navigation("StoredProducts"); |
| b.HasOne("Models.Entities.ModelVersion",
"ModelVersion") | .HasForeignKey("ProductId") | }); |
| | | |
| .WithMany("ErrorReports") | .OnDelete(DeleteBehavior.Restrict) | modelBuilder.Entity("Models.Entities.Vide
oSample", b => |
| | .IsRequired(); | { |
| .HasForeignKey("ModelVersionId") | | |
| | | |
| .OnDelete(DeleteBehavior.SetNull); | b.HasOne("Models.Entities.User", "User") | b.Navigation("ErrorReports"); |
| | | }); |
| | .WithMany("StoredProducts") | #pragma warning restore 612, |
| b.HasOne("Models.Entities.User", "User") | | 618 |
| | .HasForeignKey("UserId") | } |
| .WithMany("ErrorReports") | | } |
| | .OnDelete(DeleteBehavior.Cascade) | } |
| .HasForeignKey("UserId") | .IsRequired(); | |
| | | ===== |
| .OnDelete(DeleteBehavior.Cascade) | | ===== |
| .IsRequired(); | b.Navigation("Product"); | ===== |
| | | ===== |
| | | |
| b.HasOne("Models.Entities.VideoSample",
"Video") | b.Navigation("User"); | Файл: |
| | }); | PostgreSQLRepository/Migrations/2026031
7000100__AddTrainingArtifactsToModelVe
rsion.cs |
| | | |
| .WithMany("ErrorReports") | | ===== |
| | | ===== |
| .HasForeignKey("VideoId") | modelBuilder.Entity("Models.Entities.Mod
elVersion", b => | ===== |
| | { | |
| | | |
| .OnDelete(DeleteBehavior.SetNull); | b.Navigation("ErrorReports"); | using |
| | }); | Microsoft.EntityFrameworkCore.Migration
s; |
| b.Navigation("ModelVersion"); | | |

| | | |
|--|--|---|
| #nullable disable | name: | migrationBuilder.DropColumn(|
| namespace | table: | name: |
| PostgreSQLRepository.Migrations | type: "character | "BestWeightsPath", |
| { | varying(32)", | table: |
| public partial class | maxLength: 32, | "ModelVersions"); |
| AddTrainingArtifactsToModelVersion : | nullable: true); | |
| Migration | migrationBuilder.AddColumn<string>(<string>("MobileModelPath", | migrationBuilder.DropColumn(|
| { | name: | name: |
| protected override void | table: | "ExternalJobId", |
| Up(MigrationBuilder migrationBuilder) | "MobileModelPath", | table: |
| { | "ModelVersions", | "ModelVersions"); |
| migrationBuilder.AddColumn<string>(<string>("BaseModel", | type: "character | migrationBuilder.DropColumn(|
| name: "BaseModel", | varying(2048)", | name: |
| table: | maxLength: 2048, | "MobileFormat", |
| "ModelVersions", | nullable: true); | table: |
| type: "character | migrationBuilder.CreateIndex(| "ModelVersions"); |
| varying(256)", | name: | migrationBuilder.DropColumn(|
| maxLength: 256, | "IX_ModelVersions_ExternalJobId", | name: |
| nullable: true); | table: | "MobileModelPath", |
| migrationBuilder.AddColumn<string>(<string>("BestWeightsPath", | "ModelVersions", | table: |
| name: | column: | "ModelVersions"); |
| "BestWeightsPath", | "ExternalJobId", | } |
| table: | unique: true); | } |
| "ModelVersions", | } | } |
| type: "character | protected override void | |
| varying(2048)", | Down(MigrationBuilder migrationBuilder) | |
| maxLength: 2048, | { | |
| nullable: true); | | |
| migrationBuilder.AddColumn<string>(<string>("ExternalJobId", | migrationBuilder.DropIndex(| |
| name: | name: | Файл: |
| "ExternalJobId", | table: | PostgreSQLRepository/Migrations/2026031 |
| table: | "IX_ModelVersions_ExternalJobId", | 7214812_train.cs |
| "ModelVersions", | table: | |
| type: "character | "ModelVersions"); | |
| varying(128)", | migrationBuilder.DropColumn(| |
| maxLength: 128, | name: "BaseModel", | |
| nullable: true); | table: | using |
| migrationBuilder.AddColumn<string>(<string>("MobileFormat", | "ModelVersions"); | Microsoft.EntityFrameworkCore.Migration |
| | | s; |
| | | #nullable disable |

| | | |
|--|---|---|
| <pre> namespace PostgreSQLRepository.Migrations { /// <inheritdoc /> public partial class train : Migration { /// <inheritdoc /> protected override void Up(MigrationBuilder migrationBuilder) { } /// <inheritdoc /> protected override void Down(MigrationBuilder migrationBuilder) { } } } ===== ===== ===== ===== Файл: PostgreSQLRepository/Migrations/2026031 7214812_train.Designer.cs ===== ===== ===== ===== // <auto-generated /> using System; using Microsoft.EntityFrameworkCore; using Microsoft.EntityFrameworkCore.Infrastruct ure; using Microsoft.EntityFrameworkCore.Migration s; </pre> | <pre> using Microsoft.EntityFrameworkCore.Storage.V alueConversion; using Npgsql.EntityFrameworkCore.PostgreSQL. Metadata; using PostgreSQLRepository; #nullable disable namespace PostgreSQLRepository.Migrations { [DbContext(typeof(AppDbContext))] [Migration("20260317214812_train")] partial class train { /// <inheritdoc /> protected override void BuildTargetModel(ModelBuilder modelBuilder) { #pragma warning disable 612, 618 modelBuilder .HasAnnotation("ProductVersion", "9.0.13") .HasAnnotation("Relational:MaxIdentifierL ength", 63); NpgsqlModelBuilderExtensions.UseIdentit yByDefaultColumns(modelBuilder); modelBuilder.Entity("Models.Entities.Error Report", b => { b.Property<int>("Id") .ValueGeneratedOnAdd() .HasColumnType("integer"); </pre> | <pre> NpgsqlPropertyBuilderExtensions.UseIdent ityByDefaultColumn(b.Property<int>("Id")); b.Property<bool>("Approved") .HasColumnType("boolean"); b.Property<string>("Comment") .HasMaxLength(2000) .HasColumnType("character varying(2000)"); b.Property<DateTime>("CreatedAt") .HasColumnType("timestamp with time zone"); b.Property<int>("FramesCount") .HasColumnType("integer"); b.Property<int?>("ModelVersionId") .HasColumnType("integer"); b.Property<bool>("Resolved") .HasColumnType("boolean"); b.Property<int>("UserId") .HasColumnType("integer"); b.Property<int?>("VideoId") </pre> |
|--|---|---|

| | | |
|--|--|--|
| .HasColumnType("integer"); | .HasColumnType("character
varying(2048)"); | modelBuilder.Entity("Models.Entities.Product", b => |
| b.HasKey("Id"); | | { |
| | b.Property<string>("ExternalJobId") | b.Property<int>("Id") |
| b.HasIndex("ModelVersionId"); | .HasMaxLength(128) | .ValueGeneratedOnAdd() |
| | .HasColumnType("character
varying(128)"); | .HasColumnType("integer"); |
| b.HasIndex("UserId"); | | NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id") |
| b.HasIndex("VideoId"); | b.Property<string>("MetricsJson") |); |
| b.ToTable("ErrorReports"); | .HasColumnType("text"); | |
| }); | | b.Property<string>("Barcode") |
| modelBuilder.Entity("Models.Entities.ModelVersion", b => | b.Property<string>("MobileFormat") | .HasMaxLength(64) |
| { | .HasMaxLength(32) | .HasColumnType("character varying(64)"); |
| b.Property<int>("Id") | .HasColumnType("character varying(32)"); | |
| .ValueGeneratedOnAdd() | b.Property<string>("MobileModelPath") | b.Property<string>("Manufacturer") |
| .HasColumnType("integer"); | .HasMaxLength(2048) | .HasMaxLength(256) |
| NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id") | .HasColumnType("character
varying(2048)"); | .HasColumnType("character
varying(256)"); |
|); | b.Property<DateTime>("TrainedAt") | b.Property<string>("Name") |
| | | .IsRequired() |
| b.Property<string>("BaseModel") | .HasColumnType("timestamp with time
zone"); | .HasMaxLength(256) |
| .HasMaxLength(256) | | .HasColumnType("character
varying(256)"); |
| .HasColumnType("character
varying(256)"); | b.HasKey("Id"); | b.HasKey("Id"); |
| | b.HasIndex("ExternalJobId") | |
| | .IsUnique(); | b.HasIndex("Barcode"); |
| b.Property<string>("BestWeightsPath") | | |
| .HasMaxLength(2048) | b.ToTable("ModelVersions"); | b.ToTable("Products"); |
| | }); | }); |


```

modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<DateTime>("CreatedAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ExpiryAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<DateTime?>("ManufactureAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<int>("ProductId")
            .HasColumnType("integer");

        b.Property<int>("UserId")
            .HasColumnType("integer");

        b.HasKey("Id");

        b.HasIndex("ProductId");

        b.HasIndex("UserId");

        b.ToTable("StoredProducts");
    });

modelBuilder.Entity("Models.Entities.User
", b =>
    {
        b.Property<int>("Id")
            .ValueGeneratedOnAdd()
            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<DateTime>("CreatedAt")
            .HasColumnType("timestamp with time
zone");

        b.Property<string>("Email")
            .IsRequired()
            .HasMaxLength(256)
            .HasColumnType("character
varying(256)");

        b.Property<bool>("IsBlocked")
            .HasColumnType("boolean");

        b.Property<string>("PasswordHash")
            .IsRequired()

        b.Property<string>("SettingsJson")
            .HasColumnType("text");

        b.HasKey("Id");

        b.HasIndex("Email")
            .IsUnique();

        b.ToTable("Users");

        modelBuilder.Entity("Models.Entities.Vide
oSample", b =>
            {
                b.Property<int>("Id")
                    .ValueGeneratedOnAdd()
                    .HasColumnType("integer");

                NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

                b.Property<string>("Source")
                    .HasMaxLength(64)
                    .HasColumnType("character varying(64)");

                b.Property<string>("VideoPath")
                    .IsRequired()

```

| | | |
|---|---|---|
| .HasMaxLength(1024) | | modelBuilder.Entity("Models.Entities.Product", b => |
| | b.Navigation("User"); | { |
| .HasColumnType("character
varying(1024)"); | | |
| | b.Navigation("Video"); | b.Navigation("StoredProducts"); |
| b.HasKey("Id"); | }); | }); |
| | | |
| b.ToTable("VideoSamples"); | modelBuilder.Entity("Models.Entities.Store
dProduct", b => | modelBuilder.Entity("Models.Entities.User", b => |
| }); | { | { |
| | | |
| modelBuilder.Entity("Models.Entities.Error
Report", b => | b.HasOne("Models.Entities.Product",
"Product") | b.Navigation("ErrorReports"); |
| { | | |
| | .WithMany("StoredProducts") | b.Navigation("StoredProducts"); |
| b.HasOne("Models.Entities.ModelVersion",
"ModelVersion") | .HasForeignKey("ProductId") | }); |
| | | |
| .WithMany("ErrorReports") | .OnDelete(DeleteBehavior.Restrict) | modelBuilder.Entity("Models.Entities.Vide
oSample", b => |
| | .IsRequired(); | { |
| .HasForeignKey("ModelVersionId") | | |
| | | |
| .OnDelete(DeleteBehavior.SetNull); | b.HasOne("Models.Entities.User", "User") | b.Navigation("ErrorReports"); |
| | | }); |
| | .WithMany("StoredProducts") | #pragma warning restore 612, |
| b.HasOne("Models.Entities.User", "User") | | 618 |
| | .HasForeignKey("UserId") | } |
| .WithMany("ErrorReports") | | } |
| | .OnDelete(DeleteBehavior.Cascade) | } |
| .HasForeignKey("UserId") | .IsRequired(); | |
| | | ===== |
| .OnDelete(DeleteBehavior.Cascade) | | ===== |
| .IsRequired(); | b.Navigation("Product"); | ===== |
| | | ===== |
| | | |
| b.HasOne("Models.Entities.VideoSample",
"Video") | b.Navigation("User"); | Файл: |
| | }); | PostgreSQLRepository/Migrations/2026031
9160444_a.cs |
| | | ===== |
| .WithMany("ErrorReports") | | ===== |
| | | ===== |
| .HasForeignKey("VideoId") | modelBuilder.Entity("Models.Entities.Mod
elVersion", b => | ===== |
| | { | |
| | | |
| .OnDelete(DeleteBehavior.SetNull); | | using |
| | b.Navigation("ErrorReports"); | Microsoft.EntityFrameworkCore.Migration
s; |
| | }); | |
| b.Navigation("ModelVersion"); | | |

| | | |
|--|--|---|
| #nullable disable | table: | name: |
| namespace | "ModelVersions", | "BestWeightsPath", |
| PostgreSQLRepository.Migrations | type: "character | table: |
| { | varying(32)", | "ModelVersions"); |
| public partial class a : | maxLength: 32, | |
| Migration | nullable: true); | migrationBuilder.DropColumn(|
| { | migrationBuilder.AddColumn<string>(<string>("MobileModelPath", | name: |
| protected override void | name: | "ExternalJobId", |
| Up(MigrationBuilder migrationBuilder) | table: | table: |
| { | "ModelVersions", | "ModelVersions"); |
| migrationBuilder.AddColumn<string>(<string>("BaseModel", | type: "character | migrationBuilder.DropColumn(|
| table: | varying(2048)", | name: |
| "ModelVersions", | maxLength: 2048, | "MobileFormat", |
| type: "character | nullable: true); | table: |
| varying(256)", | migrationBuilder.CreateIndex(| "ModelVersions"); |
| maxLength: 256, | name: | migrationBuilder.DropColumn(|
| nullable: true); | "IX_ModelVersions_ExternalJobId", | name: |
| migrationBuilder.AddColumn<string>(<string>("BestWeightsPath", | table: | "MobileModelPath", |
| name: | "ModelVersions", | table: |
| "BestWeightsPath", | column: | "ModelVersions"); |
| table: | "ExternalJobId", | } |
| "ModelVersions", | unique: true); | } |
| type: "character | } | } |
| varying(2048)", | protected override void | ===== |
| maxLength: 2048, | Down(MigrationBuilder migrationBuilder) | ===== |
| nullable: true); | { | ===== |
| migrationBuilder.AddColumn<string>(<string>("ExternalJobId", | migrationBuilder.DropIndex(| Файл: |
| name: | name: | PostgreSQLRepository/Migrations/2026031 |
| "ExternalJobId", | "IX_ModelVersions_ExternalJobId", | 9160444_a.Designer.cs |
| table: | table: | ===== |
| "ModelVersions", | "ModelVersions"); | ===== |
| type: "character | migrationBuilder.DropColumn(| ===== |
| varying(128)", | name: "BaseModel", | // <auto-generated /> |
| maxLength: 128, | table: | using System; |
| nullable: true); | "ModelVersions"); | using |
| migrationBuilder.AddColumn<string>(<string>("MobileFormat", | migrationBuilder.DropColumn(| Microsoft.EntityFrameworkCore; |
| name: | | using |
| "MobileFormat", | | Microsoft.EntityFrameworkCore.Infrastructure; |
| | | ure; |

```

        using
Microsoft.EntityFrameworkCore.Migrations;

        using
Microsoft.EntityFrameworkCore.Storage.ValueConversion;

        using
Npgsql.EntityFrameworkCore.PostgreSQL.Metadata;

        using PostgreSQLRepository;

        #nullable disable

        namespace
PostgreSQLRepository.Migrations
        {

[DbContext(typeof(AppDbContext))]

[Migration("20260319160444_a")]
        partial class a
        {
            /// <inheritdoc />
            protected override void
BuildTargetModel(ModelBuilder
modelBuilder)
            {
                #pragma warning disable 612,
618

                modelBuilder

                .HasAnnotation("ProductVersion",
"9.0.13")

                .HasAnnotation("Relational:MaxIdentifierLength", 63);

                NpgsqlModelBuilderExtensions.UseIdentityByDefaultColumns(modelBuilder);

                modelBuilder.Entity("Models.Entities.Error
Report", b =>
                    {

                        b.Property<int>("Id")

```

```

                    .ValueGeneratedOnAdd()

                    .HasColumnType("integer");

                NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id")
);

                b.Property<bool>("Approved")

                .HasColumnType("boolean");

                b.Property<string>("Comment")

                .HasMaxLength(2000)

                .HasColumnType("character
varying(2000)");

                b.Property<DateTime>("CreatedAt")

                .HasColumnType("timestamp with time
zone");

                b.Property<int>("FramesCount")

                .HasColumnType("integer");

                b.Property<int?>("ModelVersionId")

                .HasColumnType("integer");

                b.Property<bool>("Resolved")

                .HasColumnType("boolean");

                b.Property<int>("UserId")

                .HasColumnType("integer");

```

```

                    b.Property<int?>("VideoId")

                    .HasColumnType("integer");

                    b.HasKey("Id");

                    b.HasIndex("ModelVersionId");

                    b.HasIndex("UserId");

                    b.HasIndex("VideoId");

                    b.ToTable("ErrorReports");
                });

                modelBuilder.Entity("Models.Entities.ModelVersion", b =>
                    {

                        b.Property<int>("Id")

                        .ValueGeneratedOnAdd()

                        .HasColumnType("integer");

                        NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id")
);

                        b.Property<string>("BaseModel")

                        .HasMaxLength(256)

                        .HasColumnType("character
varying(256)");

                        b.Property<string>("BestWeightsPath")

```

```

        });

        .HasMaxLength(2048)

        .HasColumnType("character
        varying(2048)");

        b.Property<string>("ExternalJobId")

        .HasMaxLength(128)

        .HasColumnType("character
        varying(128)");

        b.Property<string>("MetricsJson")

        .HasColumnType("text");

        b.Property<string>("MobileFormat")

        .HasMaxLength(32)

        .HasColumnType("character varying(32)");

        b.Property<string>("MobileModelPath")

        .HasMaxLength(2048)

        .HasColumnType("character
        varying(2048)");

        b.Property<DateTime>("TrainedAt")

        .HasColumnType("timestamp with time
        zone");

        b.HasKey("Id");

        b.HasIndex("ExternalJobId")

        .IsUnique();

        b.ToTable("ModelVersions");

        });

        modelBuilder.Entity("Models.Entities.Prod
        uct", b =>

        {

            b.Property<int>("Id")

            .ValueGeneratedOnAdd()

            .HasColumnType("integer");

            NpgsqlPropertyBuilderExtensions.UseIdent
            ityByDefaultColumn(b.Property<int>("Id")
            );

            b.Property<string>("Barcode")

            .HasMaxLength(64)

            .HasColumnType("character varying(64)");

            b.Property<string>("Manufacturer")

            .HasMaxLength(256)

            .HasColumnType("character
            varying(256)");

            b.Property<string>("Name")

            .IsRequired()

            .HasMaxLength(256)

            .HasColumnType("character
            varying(256)");

            b.HasKey("Id");

            b.HasIndex("Barcode");

        });

        modelBuilder.Entity("Models.Entities.Store
        dProduct", b =>

        {

            b.Property<int>("Id")

            .ValueGeneratedOnAdd()

            .HasColumnType("integer");

            NpgsqlPropertyBuilderExtensions.UseIdent
            ityByDefaultColumn(b.Property<int>("Id")
            );

            b.Property<DateTime>("CreatedAt")

            .HasColumnType("timestamp with time
            zone");

            b.Property<DateTime?>("ExpiryAt")

            .HasColumnType("timestamp with time
            zone");

            b.Property<DateTime?>("ManufactureAt")

            .HasColumnType("timestamp with time
            zone");

            b.Property<int>("ProductId")

            .HasColumnType("integer");

            b.Property<int>("UserId")

            .HasColumnType("integer");

```

| | | |
|---|---|--|
| b.HasKey("Id"); | | .IsRequired() |
| | b.Property<string>("PasswordHash") | |
| | .IsRequired() | .HasMaxLength(1024) |
| b.HasIndex("ProductId"); | | |
| | .HasMaxLength(256) | .HasColumnType("character |
| | | varying(1024)); |
| b.HasIndex("UserId"); | .HasColumnType("character | |
| | varying(256)); | b.HasKey("Id"); |
| b.ToTable("StoredProducts"); | | |
| }); | b.Property<string>("SettingsJson") | b.ToTable("VideoSamples"); |
| | | }); |
| modelBuilder.Entity("Models.Entities.User | | |
| ", b => | b.HasKey("Id"); | modelBuilder.Entity("Models.Entities.Error |
| { | | Report", b => |
| | | { |
| b.Property<int>("Id") | b.HasIndex("Email") | |
| | .IsUnique(); | b.HasOne("Models.Entities.ModelVersion", |
| .ValueGeneratedOnAdd() | b.ToTable("Users"); | "ModelVersion") |
| | }); | .WithMany("ErrorReports") |
| .HasColumnType("integer"); | | |
| | | .HasForeignKey("ModelVersionId") |
| NpgsqlPropertyBuilderExtensions.UseIdent | modelBuilder.Entity("Models.Entities.Vide | |
| ityByDefaultColumn(b.Property<int>("Id") | oSample", b => | .OnDelete(DeleteBehavior.SetNull); |
|); | { | |
| | | |
| b.Property<DateTime>("CreatedAt") | b.Property<int>("Id") | b.HasOne("Models.Entities.User", "User") |
| | .ValueGeneratedOnAdd() | .WithMany("ErrorReports") |
| .HasColumnType("timestamp with time | | |
| zone"); | .HasColumnType("integer"); | .HasForeignKey("UserId") |
| | | .OnDelete(DeleteBehavior.Cascade) |
| b.Property<string>("Email") | NpgsqlPropertyBuilderExtensions.UseIdent | .IsRequired(); |
| .IsRequired() | ityByDefaultColumn(b.Property<int>("Id") | |
| |); | b.HasOne("Models.Entities.VideoSample", |
| .HasMaxLength(256) | | "Video") |
| .HasColumnType("character | b.Property<string>("Source") | .WithMany("ErrorReports") |
| varying(256)); | .HasMaxLength(64) | .HasForeignKey("VideoId") |
| | | .OnDelete(DeleteBehavior.SetNull); |
| b.Property<bool>("IsBlocked") | .HasColumnType("character varying(64)"); | |
| .HasColumnType("boolean"); | | |
| | b.Property<string>("VideoPath") | |

```

        });

b.Navigation("ModelVersion");

b.Navigation("User");

b.Navigation("Video");
    });

modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
    {

b.HasOne("Models.Entities.Product",
"Product")

.WithMany("StoredProducts")

.HasForeignKey("ProductId")

.OnDelete(DeleteBehavior.Restrict)
        .IsRequired();

b.HasOne("Models.Entities.User", "User")

.WithMany("StoredProducts")

.HasForeignKey("UserId")

.OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

b.Navigation("Product");

b.Navigation("User");
    });

modelBuilder.Entity("Models.Entities.Mod
elVersion", b =>
    {

b.Navigation("ErrorReports");

using
Microsoft.EntityFrameworkCore.Migrations;

using
Npgsql.EntityFrameworkCore.PostgreSQL.
Metadata;

#nullable disable

namespace
PostgreSQLRepository.Migrations
    {
        /// <inheritdoc />
        public partial class
AddPullTrainingAndModelPublishing :
Migration
        {
            /// <inheritdoc />
            protected override void
Up(MigrationBuilder migrationBuilder)
            {

migrationBuilder.AddColumn<DateTime>(
                name: "DeletedAt",
                table:
"ModelVersions",
                type: "timestamp with
time zone",
                nullable: true);

migrationBuilder.AddColumn<bool>(
                name: "IsDeleted",
                table:
"ModelVersions",
                type: "boolean",
                nullable: false,
                defaultValue: false);

migrationBuilder.AddColumn<bool>(
                name: "IsPinned",
                table:
"ModelVersions",
                type: "boolean",
                nullable: false,
                defaultValue: false);
    }
}

modelBuilder.Entity("Models.Entities.Prod
uct", b =>
    {

b.Navigation("ErrorReports");

b.Navigation("StoredProducts");
    });

modelBuilder.Entity("Models.Entities.User
", b =>
    {

b.Navigation("ErrorReports");

b.Navigation("StoredProducts");
    });

modelBuilder.Entity("Models.Entities.Vide
oSample", b =>
    {

b.Navigation("ErrorReports");
    });

#pragma warning restore 612,
618
    }
}
}

=====
=====
=====

Файл:
PostgreSQLRepository/Migrations/2026051
0142827_AddPullTrainingAndModelPublis
hing.cs

=====
=====
=====

using System;

```

```

migrationBuilder.AddColumn<bool>(
    name: "IsPublished",
    table:
        "ModelVersions",
    type: "boolean",
    nullable: false,
    defaultValue: false);

migrationBuilder.AddColumn<string>(
    name:
        "MobileModelContentType",
    table:
        "ModelVersions",
    type: "character
        varying(128)",
    maxLength: 128,
    nullable: true);

migrationBuilder.AddColumn<string>(
    name:
        "MobileModelFileName",
    table:
        "ModelVersions",
    type: "character
        varying(512)",
    maxLength: 512,
    nullable: true);

migrationBuilder.CreateTable(
    name: "TrainingJobs",
    columns: table => new
    {
        Id =
            table.Column<int>(type: "integer", nullable:
                false)

        .Annotation("Npgsql:ValueGenerationStrategy.IdentityBy
            DefaultColumn"),

        JobId =
            table.Column<string>(type: "character
                varying(128)", maxLength: 128, nullable:
                false),

                Status =
                    table.Column<string>(type: "character
                        varying(32)", maxLength: 32, nullable:
                        false),

                Message =
                    table.Column<string>(type: "text", nullable:
                        true),

                CreatedAt =
                    table.Column<DateTime>(type: "timestamp
                        with time zone", nullable: false),

                StartedAt =
                    table.Column<DateTime>(type: "timestamp
                        with time zone", nullable: true),

                FinishedAt =
                    table.Column<DateTime>(type: "timestamp
                        with time zone", nullable: true),

                AssignedAt =
                    table.Column<DateTime>(type: "timestamp
                        with time zone", nullable: true),

                HeartbeatAt =
                    table.Column<DateTime>(type: "timestamp
                        with time zone", nullable: true),

                ImagesCount =
                    table.Column<int>(type: "integer", nullable:
                        false),

                ClientId =
                    table.Column<string>(type: "character
                        varying(256)", maxLength: 256, nullable:
                        true),

                DatasetZipPath =
                    table.Column<string>(type: "character
                        varying(2048)", maxLength: 2048, nullable:
                        false),

                BaseModel =
                    table.Column<string>(type: "character
                        varying(256)", maxLength: 256, nullable:
                        true),

                Epochs =
                    table.Column<int>(type: "integer", nullable:
                        false),

                ImgSize =
                    table.Column<int>(type: "integer", nullable:
                        false),

                Batch =
                    table.Column<int>(type: "integer", nullable:
                        false),

                Device =
                    table.Column<string>(type: "character
                        varying(64)", maxLength: 64, nullable:
                        true),

                ExportInt8 =
                    table.Column<bool>(type: "boolean",
                        nullable: false),

                ExportNms =
                    table.Column<bool>(type: "boolean",
                        nullable: false),

                MobileFormat =
                    table.Column<string>(type: "character
                        varying(32)", maxLength: 32, nullable:
                        true),

                QuantizationFraction =
                    table.Column<double>(type: "double
                        precision", nullable: false),

                BestWeightsPath =
                    table.Column<string>(type: "character
                        varying(2048)", maxLength: 2048, nullable:
                        true),

                MobileModelPath =
                    table.Column<string>(type: "character
                        varying(2048)", maxLength: 2048, nullable:
                        true),

                MobileModelFileName =
                    table.Column<string>(type: "character
                        varying(512)", maxLength: 512, nullable:
                        true),

                MobileModelContentType =
                    table.Column<string>(type: "character
                        varying(128)", maxLength: 128, nullable:
                        true),

                MetricsJson =
                    table.Column<string>(type: "text", nullable:
                        true),

                CancellationRequested =
                    table.Column<bool>(type: "boolean",
                        nullable: false)
            },
    constraints: table =>
    {
        table.PrimaryKey("PK_TrainingJobs", x =>
            x.Id);
    });

```



```
migrationBuilder.CreateIndex(
    name:
    "IX_ModelVersions_IsDeleted",
    table:
    "ModelVersions",
    column: "IsDeleted");
```

```
migrationBuilder.CreateIndex(
    name:
    "IX_ModelVersions_IsPublished",
    table:
    "ModelVersions",
    column:
    "IsPublished");
```

```
migrationBuilder.CreateIndex(
    name:
    "IX_TrainingJobs_JobId",
    table: "TrainingJobs",
    column: "JobId",
    unique: true);
```

```
migrationBuilder.CreateIndex(
    name:
    "IX_TrainingJobs_Status",
    table: "TrainingJobs",
    column: "Status");
}
```

```
/// <inheritdoc />
protected override void
Down(MigrationBuilder migrationBuilder)
{
```

```
migrationBuilder.DropTable(
    name: "TrainingJobs");
```

```
migrationBuilder.DropIndex(
    name:
    "IX_ModelVersions_IsDeleted",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropIndex(
    name:
    "IX_ModelVersions_IsPublished",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name: "DeletedAt",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name: "IsDeleted",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name: "IsPinned",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name: "IsPublished",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name:
    "MobileModelContentType",
    table:
    "ModelVersions");
```

```
migrationBuilder.DropColumn(
    name:
    "MobileModelFileName",
    table:
    "ModelVersions");
}
```

```
=====
=====
=====
=====
```

```
Файл:
PostgreSQLRepository/Migrations/2026051
0142827_AddPullTrainingAndModelPublis
hing.Designer.cs
```

```
=====
=====
=====
=====
```

```
// <auto-generated />
using System;
using
Microsoft.EntityFrameworkCore;
using
Microsoft.EntityFrameworkCore.Infrastruct
ure;
using
Microsoft.EntityFrameworkCore.Migration
s;
using
Microsoft.EntityFrameworkCore.Storage.V
alueConversion;
using
Npgsql.EntityFrameworkCore.PostgreSQL.
Metadata;
using PostgreSQLRepository;

#nullable disable

namespace
PostgreSQLRepository.Migrations
{
```

```
[DbContext(typeof(AppDbContext))]
```

```
[Migration("20260510142827_AddPullTrai
ningAndModelPublishing")]
partial class
AddPullTrainingAndModelPublishing
{
    /// <inheritdoc />
```

```

        protected override void
BuildTargetModel(ModelBuilder
modelBuilder)
    {
        #pragma warning disable 612,
618
        modelBuilder

.HasAnnotation("ProductVersion",
"9.0.13")

.HasAnnotation("Relational:MaxIdentifierL
ength", 63);

NpgsqlModelBuilderExtensions.UseIdentit
yByDefaultColumns(modelBuilder);

modelBuilder.Entity("Models.Entities.Error
Report", b =>
    {
        b.Property<int>("Id")

.ValueGeneratedOnAdd()

.HasColumnType("integer");

NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<bool>("Approved")

.HasColumnType("boolean");

        b.Property<string>("Comment")

.HasMaxLength(2000)

.HasColumnType("character
varying(2000)");

```

```

        b.Property<DateTime>("CreatedAt")

.HasColumnType("timestamp with time
zone");

        b.Property<int>("FramesCount")

.HasColumnType("integer");

        b.Property<int?>("ModelVersionId")

.HasColumnType("integer");

        b.Property<bool>("Resolved")

.HasColumnType("boolean");

        b.Property<int>("UserId")

.HasColumnType("integer");

        b.Property<int?>("VideoId")

.HasColumnType("integer");

        b.HasKey("Id");

        b.HasIndex("ModelVersionId");

        b.HasIndex("UserId");

        b.HasIndex("VideoId");

        b.ToTable("ErrorReports");
    });

```

```

modelBuilder.Entity("Models.Entities.Mod
elVersion", b =>
    {
        b.Property<int>("Id")

.ValueGeneratedOnAdd()

.HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
);

        b.Property<string>("BaseModel")

.HasMaxLength(256)

.HasColumnType("character
varying(256)");

        b.Property<string>("BestWeightsPath")

.HasMaxLength(2048)

.HasColumnType("character
varying(2048)");

        b.Property<DateTime?>("DeletedAt")

.HasColumnType("timestamp with time
zone");

        b.Property<string>("ExternalJobId")

.HasMaxLength(128)

.HasColumnType("character
varying(128)");

        b.Property<bool>("IsDeleted")

```

| | | |
|--|--|--|
| .HasColumnType("boolean"); | .HasColumnType("character
varying(2048)"); | b.Property<string>("Manufacturer") |
| b.Property<bool>("IsPinned") | b.Property<DateTime>("TrainedAt") | .HasMaxLength(256) |
| .HasColumnType("boolean"); | .HasColumnType("timestamp with time
zone"); | .HasColumnType("character
varying(256)"); |
| b.Property<bool>("IsPublished") | b.HasKey("Id"); | b.Property<string>("Name")
.IsRequired() |
| .HasColumnType("boolean"); | b.HasIndex("ExternalJobId")
.IsUnique(); | .HasMaxLength(256) |
| b.Property<string>("MetricsJson") | b.HasIndex("IsDeleted"); | .HasColumnType("character
varying(256)");

b.HasKey("Id"); |
| .HasColumnType("text"); | b.HasIndex("IsPublished"); | b.HasIndex("Barcode"); |
| b.Property<string>("MobileFormat") | b.ToTable("ModelVersions");
}); | b.ToTable("Products");
}); |
| .HasMaxLength(32) | modelBuilder.Entity("Models.Entities.Prod
uct", b =>
{ | modelBuilder.Entity("Models.Entities.Store
dProduct", b =>
{ |
| .HasColumnType("character varying(32)"); | b.Property<int>("Id")

.ValueGeneratedOnAdd() | b.Property<int>("Id")

.ValueGeneratedOnAdd() |
| b.Property<string>("MobileModelContentT
ype") | .HasColumnType("integer"); | .HasColumnType("integer"); |
| .HasMaxLength(128) | NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
); | NpgsqlPropertyBuilderExtensions.UseIdent
ityByDefaultColumn(b.Property<int>("Id")
); |
| .HasColumnType("character
varying(128)"); | b.Property<string>("Barcode")

.HasMaxLength(64) | b.Property<DateTime>("CreatedAt") |
| b.Property<string>("MobileModelFileNam
e") | .HasColumnType("character varying(64)"); | .HasColumnType("timestamp with time
zone"); |
| .HasMaxLength(512) | | |
| .HasColumnType("character
varying(512)"); | | |
| b.Property<string>("MobileModelPath") | | |
| .HasMaxLength(2048) | | |

```

        ityByDefaultColumn(b.Property<int>("Id")
    );

    b.Property<DateTime?>("ExpiryAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<DateTime?>("AssignedAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<DateTime?>("ManufactureAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<string>("BaseModel")

    .HasMaxLength(256)

    .HasColumnType("character
varying(256)");

    b.Property<int>("Batch")

    .HasColumnType("integer");

    b.Property<int>("UserId")

    .HasColumnType("integer");

    b.HasKey("Id");

    b.HasIndex("ProductId");

    b.HasIndex("UserId");

    b.ToTable("StoredProducts");
    });

    modelBuilder.Entity("Models.Entities.Train
ingJob", b =>
    {
        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdent
    });

    b.Property<int>("Id")

    .HasColumnType("integer");

    b.Property<DateTime?>("AssignedAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<DateTime?>("ManufactureAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<string>("DatasetZipPath")

    .IsRequired()

    .HasMaxLength(2048)

    .HasColumnType("character
varying(2048)");

    b.Property<string>("Device")

    .HasMaxLength(64)

    .HasColumnType("character varying(64)");

    b.Property<int>("Epochs")

    .HasColumnType("integer");

    b.Property<bool>("ExportInt8")

    .HasColumnType("boolean");

    b.Property<bool>("ExportNms")

    .HasColumnType("boolean");

    b.Property<DateTime?>("FinishedAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<DateTime?>("HeartbeatAt")

    .HasColumnType("timestamp with time
zone");

    b.Property<DateTime>("CreatedAt")

```

| | | |
|--|---|--|
| b.Property<int>("ImagesCount") | .HasMaxLength(512) | modelBuilder.Entity("Models.Entities.User", b => |
| .HasColumnType("integer"); | .HasColumnType("character varying(512)"); | { |
| b.Property<int>("ImgSize") | | b.Property<int>("Id") |
| .HasColumnType("integer"); | b.Property<string>("MobileModelPath") | .ValueGeneratedOnAdd() |
| | .HasMaxLength(2048) | .HasColumnType("integer"); |
| b.Property<string>("JobId") | .HasColumnType("character varying(2048)"); | NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id") |
| .IsRequired() | |); |
| .HasMaxLength(128) | b.Property<double>("QuantizationFraction") | |
| |) | b.Property<DateTime>("CreatedAt") |
| .HasColumnType("character varying(128)"); | .HasColumnType("double precision"); | .HasColumnType("timestamp with time zone"); |
| b.Property<string>("Message") | | |
| | b.Property<DateTime?>("StartedAt") | |
| .HasColumnType("text"); | .HasColumnType("timestamp with time zone"); | b.Property<string>("Email") |
| b.Property<string>("MetricsJson") | | .IsRequired() |
| | | .HasMaxLength(256) |
| .HasColumnType("text"); | b.Property<string>("Status") | .HasColumnType("character varying(256)"); |
| | .IsRequired() | |
| b.Property<string>("MobileFormat") | .HasMaxLength(32) | |
| .HasMaxLength(32) | .HasColumnType("character varying(32)"); | b.Property<bool>("IsBlocked") |
| .HasColumnType("character varying(32)"); | b.HasKey("Id"); | .HasColumnType("boolean"); |
| | b.HasIndex("JobId") | |
| b.Property<string>("MobileModelContentType") | .IsUnique(); | b.Property<string>("PasswordHash") |
| | | .IsRequired() |
| .HasMaxLength(128) | b.HasIndex("Status"); | .HasMaxLength(256) |
| .HasColumnType("character varying(128)"); | b.ToTable("TrainingJobs"); | .HasColumnType("character varying(256)"); |
| | }); | |
| b.Property<string>("MobileModelFilename") | | b.Property<string>("SettingsJson") |

```

        });

        .HasColumnType("text");

        b.HasKey("Id");

        b.HasIndex("Email")
            .IsUnique();

        b.ToTable("Users");
    });

    modelBuilder.Entity("Models.Entities.VideoSample", b =>
    {
        b.Property<int>("Id")

            .ValueGeneratedOnAdd()

            .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id")
        );

        b.Property<string>("Source")

            .HasMaxLength(64)

            .HasColumnType("character varying(64)");

        b.Property<string>("VideoPath")
            .IsRequired()

            .HasMaxLength(1024)

            .HasColumnType("character varying(1024)");

        b.HasKey("Id");

        b.ToTable("VideoSamples");
    });

    modelBuilder.Entity("Models.Entities.ErrorReport", b =>
    {
        b.HasOne("Models.Entities.ModelVersion", "ModelVersion")

            .WithMany("ErrorReports")

            .HasForeignKey("ModelVersionId")

            .OnDelete(DeleteBehavior.SetNull);

        b.HasOne("Models.Entities.User", "User")

            .WithMany("ErrorReports")

            .HasForeignKey("UserId")

            .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();

        b.HasOne("Models.Entities.VideoSample", "Video")

            .WithMany("ErrorReports")

            .HasForeignKey("VideoId")

            .OnDelete(DeleteBehavior.SetNull);

        b.Navigation("ModelVersion");

        b.Navigation("User");

        b.Navigation("Video");
    });

    modelBuilder.Entity("Models.Entities.StoredProduct", b =>
    {
        b.HasOne("Models.Entities.Product", "Product")

            .WithMany("StoredProducts")

            .HasForeignKey("ProductId")

            .OnDelete(DeleteBehavior.Restrict)
                .IsRequired();

        b.HasOne("Models.Entities.User", "User")

            .WithMany("StoredProducts")

            .HasForeignKey("UserId")

            .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();

        b.Navigation("Product");

        b.Navigation("User");
    });

    modelBuilder.Entity("Models.Entities.ModelVersion", b =>
    {
        b.Navigation("ErrorReports");
    });

    modelBuilder.Entity("Models.Entities.Product", b =>
    {
        b.Navigation("StoredProducts");
    });

```

| | | |
|--|--|--|
| <pre> modelBuilder.Entity("Models.Entities.User ", b => { b.Navigation("ErrorReports"); b.Navigation("StoredProducts"); }); modelBuilder.Entity("Models.Entities.Vide oSample", b => { b.Navigation("ErrorReports"); }); #pragma warning restore 612, 618 } } } ===== ===== ===== ===== Файл: PostgreSQLRepository/Migrations/AppDb ContextModelSnapshot.cs ===== ===== ===== ===== // <auto-generated /> using System; using Microsoft.EntityFrameworkCore; using Microsoft.EntityFrameworkCore.Infrastruct ure; using Microsoft.EntityFrameworkCore.Storage.V alueConversion; </pre> | <pre> using Npgsql.EntityFrameworkCore.PostgreSQL. Metadata; using PostgreSQLRepository; #nullable disable namespace PostgreSQLRepository.Migrations { [DbContext(typeof(AppDbContext))] partial class AppDbContextModelSnapshot : ModelSnapshot { protected override void BuildModel(ModelBuilder modelBuilder) { #pragma warning disable 612, 618 modelBuilder .HasAnnotation("ProductVersion", "9.0.13") .HasAnnotation("Relational:MaxIdentifierL ength", 63); NpgsqlModelBuilderExtensions.UseIdentit yByDefaultColumns(modelBuilder); modelBuilder.Entity("Models.Entities.Error Report", b => { b.Property<int>("Id") .ValueGeneratedOnAdd() .HasColumnType("integer"); NpgsqlPropertyBuilderExtensions.UseIdent ityByDefaultColumn(b.Property<int>("Id")); </pre> | <pre> b.Property<bool>("Approved") .HasColumnType("boolean"); b.Property<string>("Comment") .HasMaxLength(2000) .HasColumnType("character varying(2000)"); b.Property<DateTime>("CreatedAt") .HasColumnType("timestamp with time zone"); b.Property<int>("FramesCount") .HasColumnType("integer"); b.Property<int?>("ModelVersionId") .HasColumnType("integer"); b.Property<bool>("Resolved") .HasColumnType("boolean"); b.Property<int>("UserId") .HasColumnType("integer"); b.Property<int?>("VideoId") .HasColumnType("integer"); b.HasKey("Id"); </pre> |
|--|--|--|

| | | |
|--|--|--|
| b.HasIndex("ModelVersionId"); | .HasColumnType("timestamp with time zone"); | b.Property<string>("MobileModelFileName") |
| b.HasIndex("UserId"); | b.Property<string>("ExternalJobId") | .HasMaxLength(512) |
| b.HasIndex("VideoId"); | .HasMaxLength(128) | .HasColumnType("character varying(512)"); |
| b.ToTable("ErrorReports");
}); | .HasColumnType("character varying(128)"); | b.Property<string>("MobileModelPath") |
| modelBuilder.Entity("Models.Entities.ModelVersion", b =>
{ | b.Property<bool>("IsDeleted") | .HasMaxLength(2048) |
| | .HasColumnType("boolean"); | .HasColumnType("character varying(2048)"); |
| b.Property<int>("Id") | b.Property<bool>("IsPinned") | b.Property<DateTime>("TrainedAt") |
| .ValueGeneratedOnAdd() | .HasColumnType("boolean"); | .HasColumnType("timestamp with time zone"); |
| .HasColumnType("integer"); | b.Property<bool>("IsPublished") | b.HasKey("Id"); |
| NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id")); | .HasColumnType("boolean"); | b.HasIndex("ExternalJobId").IsUnique(); |
| | b.Property<string>("MetricsJson") | |
| b.Property<string>("BaseModel") | .HasColumnType("text"); | b.HasIndex("IsDeleted"); |
| .HasMaxLength(256) | b.Property<string>("MobileFormat") | b.HasIndex("IsPublished"); |
| .HasColumnType("character varying(256)"); | .HasMaxLength(32) | b.ToTable("ModelVersions"); |
| b.Property<string>("BestWeightsPath") | .HasColumnType("character varying(32)"); | }); |
| .HasMaxLength(2048) | b.Property<string>("MobileModelContentType") | modelBuilder.Entity("Models.Entities.Product", b =>
{ |
| .HasColumnType("character varying(2048)"); | .HasMaxLength(128) | b.Property<int>("Id") |
| b.Property<DateTime?>("DeletedAt") | .HasColumnType("character varying(128)"); | .ValueGeneratedOnAdd() |


```

        });

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"));

        b.Property<string>("Barcode")

        .HasMaxLength(64)

        .HasColumnType("character varying(64)");

        b.Property<string>("Manufacturer")

        .HasMaxLength(256)

        .HasColumnType("character varying(256)");

        b.Property<string>("Name")
            .IsRequired()

        .HasMaxLength(256)

        .HasColumnType("character varying(256)");

        b.HasKey("Id");

        b.HasIndex("Barcode");

        b.ToTable("Products");
    });

    modelBuilder.Entity("Models.Entities.StoredProduct", b =>
    {

        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"));

        b.Property<DateTime>("CreatedAt")

        .HasColumnType("timestamp with time zone");

        b.Property<DateTime?>("ExpiryAt")

        .HasColumnType("timestamp with time zone");

        b.Property<DateTime?>("ManufactureAt")

        .HasColumnType("timestamp with time zone");

        b.Property<DateTime?>("AssignedAt")

        .HasColumnType("timestamp with time zone");

        b.Property<string>("BaseModel")

        .HasMaxLength(256)

        .HasColumnType("character varying(256)");

        b.Property<int>("Batch")

        .HasColumnType("integer");

        b.Property<string>("BestWeightsPath")

        .HasMaxLength(2048)

        .HasColumnType("character varying(2048)");

        b.Property<bool>("CancellationRequested")
    )
    });

    modelBuilder.Entity("Models.Entities.TrainingJob", b =>
    {

        b.Property<int>("Id")

        .ValueGeneratedOnAdd()

        .HasColumnType("integer");

        NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"));

        b.Property<DateTime?>("AssignedAt")

        .HasColumnType("timestamp with time zone");

        b.Property<string>("BaseModel")

        .HasMaxLength(256)

        .HasColumnType("character varying(256)");

        b.Property<int>("Batch")

        .HasColumnType("integer");

        b.Property<string>("BestWeightsPath")

        .HasMaxLength(2048)

        .HasColumnType("character varying(2048)");

        b.Property<bool>("CancellationRequested")
    )
    });

```

| | | |
|---|--|--|
| .HasColumnType("boolean"); | b.Property<DateTime?>("FinishedAt") | b.Property<string>("MobileModelContentT
ype") |
| b.Property<string>("ClientId") | .HasColumnType("timestamp with time
zone"); | .HasMaxLength(128) |
| .HasMaxLength(256) | b.Property<DateTime?>("HeartbeatAt") | .HasColumnType("character
varying(128)"); |
| .HasColumnType("character
varying(256)"); | .HasColumnType("timestamp with time
zone"); | b.Property<string>("MobileModelFileNam
e") |
| b.Property<DateTime>("CreatedAt") | b.Property<int>("ImagesCount") | .HasMaxLength(512) |
| .HasColumnType("timestamp with time
zone"); | .HasColumnType("integer"); | .HasColumnType("character
varying(512)"); |
| b.Property<string>("DatasetZipPath")
.IsRequired() | b.Property<int>("ImgSize") | b.Property<string>("MobileModelPath") |
| .HasMaxLength(2048) | .HasColumnType("integer"); | .HasMaxLength(2048) |
| .HasColumnType("character
varying(2048)"); | b.Property<string>("JobId")
.IsRequired() | .HasColumnType("character
varying(2048)"); |
| b.Property<string>("Device") | .HasMaxLength(128) | b.Property<double>("QuantizationFraction"
) |
| .HasMaxLength(64) | .HasColumnType("character
varying(128)"); | .HasColumnType("double precision"); |
| .HasColumnType("character varying(64)"); | b.Property<string>("Message") | b.Property<DateTime?>("StartedAt") |
| b.Property<int>("Epochs") | .HasColumnType("text"); | .HasColumnType("timestamp with time
zone"); |
| .HasColumnType("integer"); | b.Property<string>("MetricsJson") | b.Property<string>("Status")
.IsRequired() |
| b.Property<bool>("ExportInt8") | .HasColumnType("text"); | .HasMaxLength(32) |
| .HasColumnType("boolean"); | b.Property<string>("MobileFormat") | .HasColumnType("character varying(32)"); |
| b.Property<bool>("ExportNms") | .HasMaxLength(32) | b.HasKey("Id"); |
| .HasColumnType("boolean"); | .HasColumnType("character varying(32)"); | |

```

        b.HasIndex("JobId")
            .IsUnique();

        b.HasIndex("Status");

        b.ToTable("TrainingJobs");
    });

modelBuilder.Entity("Models.Entities.User", b =>
{
    b.Property<int>("Id")
        .ValueGeneratedOnAdd()
        .HasColumnType("integer");

    NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"));

    b.Property<DateTime>("CreatedAt")
        .HasColumnType("timestamp with time zone");

    b.Property<string>("Email")
        .IsRequired()
        .HasMaxLength(256)
        .HasColumnType("character varying(256)");

    b.Property<bool>("IsBlocked")
        .HasColumnType("boolean");

    b.Property<string>("PasswordHash")
        .IsRequired()
        .IsRequired()
        .HasMaxLength(256)
        .HasColumnType("character varying(256)");

    b.Property<string>("SettingsJson")
        .HasColumnType("text");

    b.HasKey("Id");

    b.HasIndex("Email")
        .IsUnique();

    b.ToTable("Users");
});

modelBuilder.Entity("Models.Entities.VideoSample", b =>
{
    b.Property<int>("Id")
        .ValueGeneratedOnAdd()
        .HasColumnType("integer");

    NpgsqlPropertyBuilderExtensions.UseIdentityByDefaultColumn(b.Property<int>("Id"));

    b.Property<string>("Source")
        .HasMaxLength(64)
        .HasColumnType("character varying(64)");

    b.Property<string>("VideoPath")
        .IsRequired()
        .IsRequired()
        .HasMaxLength(1024)
        .HasColumnType("character varying(1024)");

    b.HasKey("Id");

    b.ToTable("VideoSamples");
});

modelBuilder.Entity("Models.Entities.ErrorReport", b =>
{
    b.HasOne("Models.Entities.ModelVersion", "ModelVersion")
        .WithMany("ErrorReports")
        .HasForeignKey("ModelVersionId")
        .OnDelete(DeleteBehavior.SetNull);

    b.HasOne("Models.Entities.User", "User")
        .WithMany("ErrorReports")
        .HasForeignKey("UserId")
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

    b.HasOne("Models.Entities.VideoSample", "Video")
        .WithMany("ErrorReports")
        .HasForeignKey("VideoId")
        .OnDelete(DeleteBehavior.SetNull);

    b.Navigation("ModelVersion");
}

```

| | | |
|---|---|--|
| b.Navigation("User"); | modelBuilder.Entity("Models.Entities.Product", b => | <TargetFramework>net8.0</TargetFramework> |
| | { | |
| b.Navigation("Video"); | b.Navigation("StoredProducts"); | <ImplicitUsings>enable</ImplicitUsings> |
| }); | }); | <Nullable>enable</Nullable> |
| | | </PropertyGroup> |
| | | <ItemGroup> |
| modelBuilder.Entity("Models.Entities.StoredProduct", b => | modelBuilder.Entity("Models.Entities.User", b => | <PackageReference |
| { | { | Include="Microsoft.EntityFrameworkCore" |
| | | Version="9.0.13" /> |
| b.HasOne("Models.Entities.Product", "Product") | b.Navigation("ErrorReports"); | <PackageReference |
| | | Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.13"> |
| .WithMany("StoredProducts") | b.Navigation("StoredProducts"); | <PrivateAssets>all</PrivateAssets> |
| | }); | <IncludeAssets>runtime; |
| .HasForeignKey("ProductId") | | build; native; contentfiles; analyzers; |
| | | buildtransitive</IncludeAssets> |
| .OnDelete(DeleteBehavior.Restrict) | modelBuilder.Entity("Models.Entities.VideoSample", b => | </PackageReference> |
| .IsRequired(); | { | <PackageReference |
| | | Include="Microsoft.EntityFrameworkCore.Relational" Version="9.0.13" /> |
| b.HasOne("Models.Entities.User", "User") | b.Navigation("ErrorReports"); | <PackageReference |
| | }); | Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.13"> |
| .WithMany("StoredProducts") | #pragma warning restore 612, | |
| | 618 | |
| .HasForeignKey("UserId") | } | <PrivateAssets>all</PrivateAssets> |
| | } | <IncludeAssets>runtime; |
| .OnDelete(DeleteBehavior.Cascade) | } | build; native; contentfiles; analyzers; |
| .IsRequired(); | | buildtransitive</IncludeAssets> |
| | ===== | </PackageReference> |
| | ===== | <PackageReference |
| b.Navigation("Product"); | ===== | Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="9.0.4" /> |
| | ===== | </ItemGroup> |
| b.Navigation("User"); | Файл: | |
| }); | PostgreSQLRepository/PostgreSQLRepository.csproj | <ItemGroup> |
| | ===== | <ProjectReference |
| | ===== | Include="..\Contracts\Contracts.csproj" /> |
| | ===== | <ProjectReference |
| modelBuilder.Entity("Models.Entities.ModelVersion", b => | ===== | Include="..\Models\Models.csproj" /> |
| { | | </ItemGroup> |
| | <Project | |
| b.Navigation("ErrorReports"); | Sdk="Microsoft.NET.Sdk"> | </Project> |
| }); | | |
| | <PropertyGroup> | |

```

=====
=====
=====
=====
Файл:
PostgreSQLRepository/Repository.cs
=====
=====
=====
=====

using Contracts;
using
Microsoft.EntityFrameworkCore;
using Models.Entities;

namespace
PostgreSQLRepository;

public class Repository<T> :
IRepository<T> where T : BaseEntity
{
    protected readonly
AppDbContext Db;
    protected readonly DbSet<T>
Set;

    public
Repository(AppDbContext db)
    {
        Db = db;
        Set = db.Set<T>();
    }

    public async Task<T?>
GetByIdAsync(int id, CancellationToken ct
= default)
    => await
Set.FirstOrDefaultAsync(x => x.Id == id,
ct);

    public async Task<List<T>>
GetAllAsync(CancellationToken ct =
default)
    => await
Set.AsNoTracking().ToListAsync(ct);

```

```

public async Task<T>
AddAsync(T entity, CancellationToken ct =
default)
{
    Set.Add(entity);
    await
Db.SaveChangesAsync(ct);
    return entity;
}

public async Task
UpdateAsync(T entity, CancellationToken
ct = default)
{
    Set.Update(entity);
    await
Db.SaveChangesAsync(ct);
}

public async Task
DeleteAsync(int id, CancellationToken ct =
default)
{
    var entity = await
Set.FirstOrDefaultAsync(x => x.Id == id,
ct);

    if (entity is null) return;
    Set.Remove(entity);
    await
Db.SaveChangesAsync(ct);
}

=====
=====
=====
=====
Файл:
PostgreSQLRepository/StoredProductRepo
sitory.cs
=====
=====
=====
=====

using
Microsoft.EntityFrameworkCore;
using Models.Entities;

```

```

namespace
PostgreSQLRepository;

public class
StoredProductRepository :
Repository<StoredProduct>
{
    public
StoredProductRepository(AppDbContext
db) : base(db) { }

    public async
Task<List<StoredProduct>>
GetByUserAsync(int userId,
CancellationToken ct = default)
    {
        return await
Db.StoredProducts
.AsNoTracking()
.Include(x => x.Product)
.Where(x => x.UserId ==
userId)
.OrderByDescending(x
=> x.CreatedAt)
.ToListAsync(ct);
    }
}

=====
=====
=====
=====
Файл:
ProductsDateAPI/appsettings.json
=====
=====
=====
=====

{
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore":
"Warning"
        }
    },

```

```

        "ConnectionStrings": {
            "Default":
                "Host=localhost;Port=5432;Database=productsdate;Username=postgres;Password=postgres"
        },
        "Jwt": {
            "Issuer": "ProductsDateAPI",
            "Audience":
                "ProductsDateAPI",
            "Secret":
                "CHANGE_ME_SUPER_SECRET_32_CHARS_MIN_!!!!!!",
            "AccessTokenMinutes": 240
        },
        "TrainingService": {
            "ApiKey":
                "super_secret_training_key_2026",
            "TempRoot": "temp/training",
            "SeedDatasetPath":
                "..\\..\\dataset",
            "StorageRoot":
                "storage/training"
        },
        "AllowedHosts": "*"
    }
}

=====
Файл:
ProductsDateAPI/Controllers/Admin/AdminBackupController.cs
=====
using System.Text.Json;
using
Microsoft.AspNetCore.Authorization;
using
Microsoft.AspNetCore.Mvc;
using ProductsDateAPI.Models;
using ProductsDateAPI.Services;

```

```

namespace
ProductsDateAPI.Controllers.Admin;

[ApiController]
[Route("api/admin/backup")]
[Authorize]
public class
AdminBackupController : ControllerBase
{
    private readonly
BackupService _backupService;

    public
AdminBackupController(BackupService
backupService)
    {
        _backupService =
backupService;
    }

    [HttpGet("export")]
    [Produces("application/zip")]
    public async
Task<IActionResult>
Export(CancellationToken ct)
    {
        var zipPath = await
        _backupService.CreateBackupZipAsync(ct)
        ;

        var fileName =
        $"productsdate_backup_{DateTime.UtcNow:yyyyMMdd_HH:mm:ss}.zip";

        var stream = new
        FileStream(
            zipPath,
            FileMode.Open,
            FileAccess.Read,
            FileShare.Read,
            bufferSize: 1024 * 128,
            options:
                FileOptions.Asynchronous |
                FileOptions.DeleteOnClose);

        return File(stream,
            "application/zip", fileName);
    }

    [HttpPost("import")]

```

```

[Consumes("multipart/form-data")]

[RequestSizeLimit(2_000_000_000)]
    public async
Task<ActionResult<BackupImportResult>
> Import([FromForm] BackupImportForm
form, CancellationToken ct)
    {
        if (form.BackupZip is null ||
form.BackupZip.Length == 0)
            return
            BadRequest("BackupZip is required.");

        try
        {
            var result = await
            _backupService.ImportBackupZipAsync(form.BackupZip,
form.ReplaceExisting, ct);

            return Ok(result);
        }
        catch
        (InvalidOperationException ex)
        {
            return
            BadRequest(ex.Message);
        }
        catch
        (InvalidDataException ex)
        {
            return BadRequest($"Не
            удалось прочитать ZIP-файл:
            {ex.Message}");
        }
        catch (JsonException ex)
        {
            return BadRequest($"Не
            удалось прочитать JSON внутри backup-
            архива: {ex.Message}");
        }
    }
}

=====
=====
=====

```

```

Файл:
ProductsDateAPI/Controllers/Admin/AdminErrorReportsController.cs

=====

=====

=====

using Contracts.Dtos;
using
Microsoft.AspNetCore.Authorization;
using
Microsoft.AspNetCore.Mvc;
using
Microsoft.EntityFrameworkCore;
using PostgreSQLRepository;
using System.Globalization;

namespace
ProductsDateAPI.Controllers.Admin;

[ApiController]
[Route("api/admin/error-reports")]
[Authorize]
public class
AdminErrorReportsController :
ControllerBase
{
    private readonly
AppDbContext _db;
    private readonly
IWebHostEnvironment _env;

    public
AdminErrorReportsController(AppDbContext db, IWebHostEnvironment env)
    {
        _db = db;
        _env = env;
    }

    [HttpGet]
    public async
Task<ActionResult<List<AdminErrorReportListItemDto>>> GetList(
    [FromQuery] bool?
onlyApproved,

```

```

[FromQuery] int skip = 0,
[FromQuery] int take = 50,
CancellationToken ct =
default)
{
    if (skip < 0) skip = 0;
    if (take <= 0) take = 50;
    if (take > 200) take = 200;

    var query =
_db.ErrorReports.AsNoTracking();

    if
(onlyApproved.HasValue)
        query = query.Where(x
=> x.Approved == onlyApproved.Value);

    var items = await query
.OrderByDescending(x
=> x.CreatedAt)
.Skip(skip)
.Take(take)
.Select(x => new
AdminErrorReportListItemDto(
    x.Id,
    x.UserId,
    x.CreatedAt,
    x.FramesCount,
    x.Approved
))
.ToListAsync(ct);

    return Ok(items);
}

[HttpGet("{reportId:int}")]
public async
Task<ActionResult<AdminErrorReportDetailsDto>> GetById(
    int reportId,
    CancellationToken ct =
default)
{
    var report = await
_db.ErrorReports
.AsNoTracking()
.Include(x => x.Video)

```

```

.FirstOrDefaultAsync(x
=> x.Id == reportId, ct);

    if (report is null)
        return NotFound();

    if (report.Video is null)
        return
BadRequest("VideoSample not found for
this report.");

    return Ok(new
AdminErrorReportDetailsDto(
        report.Id,
        report.UserId,
        report.CreatedAt,
        report.FramesCount,
        report.Approved,
        report.Comment,

        Path.Combine(report.Video.VideoPath,
"extracted").Replace("\\", "/")
    ));
}

[HttpGet("{reportId:int}/frames/{frameIndex:int}")]
public async
Task<ActionResult> GetFrameImage(
    int reportId,
    int frameIndex,
    CancellationToken ct =
default)
{
    if (frameIndex <= 0)
        return
BadRequest("frameIndex must be >= 1.");

    var report = await
_db.ErrorReports
.AsNoTracking()
.Include(x => x.Video)
.FirstOrDefaultAsync(x
=> x.Id == reportId, ct);

    if (report is null)
        return NotFound();

```

```

        if (report.Video is null)
            return
        BadRequest("VideoSample not found for
this report.");

        var datasetRoot =
Path.Combine(
    _env.ContentRootPath,

report.Video.VideoPath.Replace("/",
Path.DirectorySeparatorChar.ToString()),
    "extracted"
);

        var imagePath =
Path.Combine(datasetRoot, "images",
$"frame_{frameIndex:D5}.jpg");

        if
(!System.IO.File.Exists(imagePath))
            return NotFound("Frame
image not found.");

        var stream =
System.IO.File.OpenRead(imagePath);
        return File(stream,
"image/jpeg");
    }

[HttpGet("{reportId:int}/frames/{frameIndex:int}/bbox")]
    public async
Task<ActionResult<YoloBboxDto>>
GetFrameBbox(
    int reportId,
    int frameIndex,
    CancellationToken ct =
default)
    {
        var bboxesResult = await
GetFrameBboxes(reportId, frameIndex, ct);

        if (bboxesResult.Result is
not null)
            return
bboxesResult.Result;

```

```

        var bboxes =
bboxesResult.Value ?? new
List<YoloBboxDto>();
        var first =
bboxes.FirstOrDefault();

        if (first is null)
            return NotFound("BBox
label not found or empty.");

        return Ok(first);
    }

[HttpGet("{reportId:int}/frames/{frameIndex:int}/bboxes")]
    public async
Task<ActionResult<List<YoloBboxDto>>
> GetFrameBboxes(
    int reportId,
    int frameIndex,
    CancellationToken ct =
default)
    {
        if (frameIndex <= 0)
            return
BadRequest("frameIndex must be >= 1.");

        var report = await
_db.ErrorReports
.AsNoTracking()
.Include(x => x.Video)
.FirstOrDefaultAsync(x
=> x.Id == reportId, ct);

        if (report is null)
            return NotFound();

        if (report.Video is null)
            return
BadRequest("VideoSample not found for
this report.");

        var datasetRoot =
Path.Combine(
    _env.ContentRootPath,

```

```

report.Video.VideoPath.Replace("/",
Path.DirectorySeparatorChar.ToString()),
    "extracted"
);

        var labelPath =
Path.Combine(datasetRoot, "labels",
$"frame_{frameIndex:D5}.txt");

        if
(!System.IO.File.Exists(labelPath))
            return NotFound("BBox
label not found.");

        var lines = await
System.IO.File.ReadAllLinesAsync(labelPa
th, ct);

        var bboxes = new
List<YoloBboxDto>();

        foreach (var line in
lines.Where(x =>
!string.IsNullOrEmpty(x)))
        {
            var parts =
line.Trim().Split(' ',
StringSplitOptions.RemoveEmptyEntries);

            if (parts.Length < 5)
                continue;

            if (!int.TryParse(parts[0],
out var classId))
                continue;

            if
(!double.TryParse(parts[1],
NumberStyles.Float,
CultureInfo.InvariantCulture, out var xc))
                continue;

            if
(!double.TryParse(parts[2],
NumberStyles.Float,
CultureInfo.InvariantCulture, out var yc))
                continue;

            if
(!double.TryParse(parts[3],

```



```

NumberStyles.Float,
CultureInfo.InvariantCulture, out var w))
        continue;
        if
(!double.TryParse(parts[4],
NumberStyles.Float,
CultureInfo.InvariantCulture, out var h))
            continue;

        bboxes.Add(new
YoloBboxDto(classId, xc, yc, w, h));
    }

    return Ok(bboxes);
}

[HttpPut("{reportId:int}/approve")]
public async
Task<IActionResult> SetApproved(
    int reportId,
    [FromBody]
ApproveErrorReportRequest req,
    CancellationToken ct =
default)
{
    var report = await
_db.ErrorReports.FirstOrDefaultAsync(x
=> x.Id == reportId, ct);

    if (report is null)
        return NotFound();

    report.Approved =
req.Approved;

    await
_db.SaveChangesAsync(ct);

    return NoContent();
}

[HttpDelete("{reportId:int}")]
public async
Task<IActionResult> DeleteReport(
    int reportId,
    CancellationToken ct =
default)
{

```

```

        var report = await
        _db.ErrorReports
            .Include(x => x.Video)
            .FirstOrDefaultAsync(x
=> x.Id == reportId, ct);

        if (report is null)
            return NotFound();

        string? datasetRoot = null;
        if (report.Video is not null)
        {
            datasetRoot =
Path.Combine(
    _env.ContentRootPath,

    report.Video.VideoPath.Replace("/",
Path.DirectorySeparatorChar.ToString())
);
        }

        _db.ErrorReports.Remove(report);

        if (report.Video is not null)
            _db.VideoSamples.Remove(report.Video);

        await
        _db.SaveChangesAsync(ct);

        try
        {
            if
(!string.IsNullOrEmpty(datasetRoot)
&& Directory.Exists(datasetRoot))
            {
                Directory.Delete(datasetRoot, recursive:
true);
            }
        }
        catch
        {
            return NoContent();
        }
    }
}

```

```

    }

    =====
    =====
    =====
    =====

    Файл:
ProductsDateAPI/Controllers/Admin/Admin
nTrainingController.cs

    =====
    =====
    =====

    using Contracts.Dtos;
    using
Microsoft.AspNetCore.Authorization;
    using
Microsoft.AspNetCore.Mvc;
    using
Microsoft.EntityFrameworkCore;
    using Models.Entities;
    using PostgreSQLRepository;
    using ProductsDateAPI.Services;

    namespace
ProductsDateAPI.Controllers.Admin;

    [ApiController]
    [Route("api/admin/training")]
    [Authorize]
    public class
AdminTrainingController : ControllerBase
    {
        private readonly
AppDbContext _db;
        private readonly
TrainingDatasetPackager _packager;
        private readonly
TrainingFileStorage _storage;

        public
AdminTrainingController(
            AppDbContext db,
            TrainingDatasetPackager
packager,
            TrainingFileStorage
storage)

```

```

    {
        _db = db;
        _packager = packager;
        _storage = storage;
    }

    [HttpPost("start")]
    public async
    Task<ActionResult<TrainingJobStartResponse>> StartTraining(
        [FromBody]
        StartTrainingRequest? request,
        CancellationToken ct)
    {
        TrainingDatasetPackage?
        package = null;
        var jobId =
        Guid.NewGuid().ToString("N");

        try
        {
            package = await
            _packager.CreateApprovedReportsZipAsync(ct);

            var datasetZipPath =
            _storage.SaveDatasetZip(jobId,
            package.ZipPath);

            var job = new
            TrainingJob
            {
                JobId = jobId,
                Status = "queued",
                Message = $"Задача
                создана на сервере. Python-клиент
                заберёт её при следующем опросе. В
                датасете {package.ImagesCount} кадров.",
                CreatedAt =
                DateTime.UtcNow,
                ImagesCount =
                package.ImagesCount,
                DatasetZipPath =
                datasetZipPath,
                BaseModel =
                string.IsNullOrEmpty(request?.BaseModel) ? "yolov8n.pt" : request!.BaseModel,
                Epochs =
                request?.Epochs ?? 50,

```

```

                ImgSize =
                request?.ImgSize ?? 640,
                Batch = request?.Batch
                ?? 16,
                Device =
                string.IsNullOrEmpty(request?.Device) ? "auto" : request!.Device,
                ExportInt8 =
                request?.ExportInt8 ?? true,
                ExportNms =
                request?.ExportNms ?? true,
                MobileFormat =
                string.IsNullOrEmpty(request?.MobileFormat) ? "tf lite" : request!.MobileFormat,
                QuantizationFraction =
                request?.QuantizationFraction ?? 0.3,
            };

            _db.TrainingJobs.Add(job);
            await
            _db.SaveChangesAsync(ct);

            return Ok(new
            TrainingJobStartResponse(
                job.JobId,
                job.Status,
                job.ImagesCount,
                job.Message ??
                "Задача обучения создана."));

            finally
            {
                if (package is not null
                &&
                Directory.Exists(package.WorkingDirectory))
                {
                    try
                    {
                        Directory.Delete(package.WorkingDirectory, recursive: true);
                    }
                    catch
                    {
                        // temp cleanup best
                        effort

```

```

                }
            }
        }

        [HttpGet("jobs")]
        public async
        Task<ActionResult<List<TrainingJobStatusResponse>>> GetJobs(CancellationTokens ct)
        {
            var jobs = await
            _db.TrainingJobs
            .AsNoTracking()
            .OrderByDescending(x
            => x.CreatedAt)
            .ToListAsync(ct);

            return
            Ok(jobs.Select(ToJobStatusResponse).ToList());
        }

        [HttpGet("jobs/{jobId}")]
        public async
        Task<ActionResult<TrainingJobStatusResponse>> GetJob(string jobId,
        CancellationToken ct)
        {
            var job = await
            _db.TrainingJobs.AsNoTracking().FirstOrDefaultAsync(x => x.JobId == jobId, ct);

            return job is null ?
            NotFound() :
            Ok(ToJobStatusResponse(job));
        }

        [HttpPost("jobs/{jobId}/cancel")]
        public async
        Task<ActionResult<TrainingJobStatusResponse>> CancelJob(string jobId,
        CancellationToken ct)
        {
            var job = await
            _db.TrainingJobs.FirstOrDefaultAsync(x
            => x.JobId == jobId, ct);

            if (job is null)

```

```

        return NotFound();

        if (IsTerminal(job.Status))
            return
            Ok(ToJobStatusResponse(job));

        job.CancellationRequested
        = true;

        job.Message =
        "Администратор запросил остановку
        задачи. Если задача ещё не началась, она
        будет отменена сразу; если уже идёт
        обучение, Python-клиент увидит запрос
        при следующей синхронизации.";

        if (string.Equals(job.Status,
        "queued",
        StringComparison.OrdinalIgnoreCase))
        {
            job.Status = "canceled";
            job.FinishedAt =
            DateTime.UtcNow;

            job.Message = "Задача
            отменена администратором до старта
            обучения.";
        }

        await
        _db.SaveChangesAsync(ct);

        return
        Ok(ToJobStatusResponse(job));
    }

    [HttpGet("model-versions")]
    public async
    Task<ActionResult<List<ModelVersionAd
    minResponse>>>
    GetModelVersions(CancellationTokens ct)
    {
        var models = await
        _db.ModelVersions
            .AsNoTracking()
            .Where(x =>
            !x.IsDeleted)
            .OrderByDescending(x
            => x.IsPublished)
            .ThenByDescending(x
            => x.TrainedAt)

```

```

        .ToListAsync(ct);

        return
        Ok(models.Select(ToModelVersionRespons
        e).ToList());
    }

    [HttpPost("model-
    versions/{id:int}/publish")]
    public async
    Task<ActionResult<ModelVersionAdminR
    esponse>> PublishModelVersion(int id,
    CancellationToken ct)
    {
        var selected = await
        _db.ModelVersions.FirstOrDefaultAsync(x
        => x.Id == id && !x.IsDeleted, ct);

        if (selected is null)
            return NotFound();

        if
        (string.IsNullOrEmpty(selected.Mobil
        eModelPath))
            return BadRequest("У
            этой версии нет мобильного файла
            модели.");

        var all = await
        _db.ModelVersions.Where(x =>
        !x.IsDeleted).ToListAsync(ct);

        foreach (var model in all)
            model.IsPublished =
            model.Id == selected.Id;

        selected.IsPinned = true;

        await
        _db.SaveChangesAsync(ct);

        return
        Ok(ToModelVersionResponse(selected));
    }

    [HttpPost("model-
    versions/{id:int}/unpublish")]
    public async
    Task<ActionResult<ModelVersionAdminR
    esponse>> UnpublishModelVersion(int id,
    CancellationToken ct)

```

```

    {
        var selected = await
        _db.ModelVersions.FirstOrDefaultAsync(x
        => x.Id == id && !x.IsDeleted, ct);

        if (selected is null)
            return NotFound();

        selected.IsPublished =
        false;

        await
        _db.SaveChangesAsync(ct);

        return
        Ok(ToModelVersionResponse(selected));
    }

    [HttpPost("model-
    versions/{id:int}/pin")]
    public async
    Task<ActionResult<ModelVersionAdminR
    esponse>> PinModelVersion(int id,
    CancellationToken ct)
    {
        var selected = await
        _db.ModelVersions.FirstOrDefaultAsync(x
        => x.Id == id && !x.IsDeleted, ct);

        if (selected is null)
            return NotFound();

        selected.IsPinned = true;

        await
        _db.SaveChangesAsync(ct);

        return
        Ok(ToModelVersionResponse(selected));
    }

    [HttpPost("model-
    versions/{id:int}/unpin")]
    public async
    Task<ActionResult<ModelVersionAdminR
    esponse>> UnpinModelVersion(int id,
    CancellationToken ct)
    {
        var selected = await
        _db.ModelVersions.FirstOrDefaultAsync(x
        => x.Id == id && !x.IsDeleted, ct);

        if (selected is null)
            return NotFound();
    }

```

```

        selected.IsPinned = false;
        await
        _db.SaveChangesAsync(ct);
        return
        Ok(ToModelVersionResponse(selected));
    }

    [HttpDelete("model-
versions/{id:int}")]
    public async
    Task<ActionResult>
    DeleteModelVersion(int id, [FromQuery]
    bool force = false, CancellationToken ct =
    default)
    {
        var selected = await
        _db.ModelVersions.FirstOrDefaultAsync(x
        => x.Id == id && !x.IsDeleted, ct);
        if (selected is null)
            return NotFound();

        if (selected.IsPinned &&
        !force)
            return
            BadRequest("Версия зафиксирована.
            Сначала снимите фиксацию или
            передайте force=true.");

        selected.IsDeleted = true;
        selected.IsPublished =
        false;
        selected.DeletedAt =
        DateTime.UtcNow;
        await
        _db.SaveChangesAsync(ct);

        _storage.TryDeleteFile(selected.MobileMo
        delPath);

        _storage.TryDeleteFile(selected.BestWeight
        sPath);

        return NoContent();
    }

```

```

        private static bool
        IsTerminal(string? status)
            => string.Equals(status,
            "completed",
            StringComparison.OrdinalIgnoreCase)
            || string.Equals(status,
            "failed",
            StringComparison.OrdinalIgnoreCase)
            || string.Equals(status,
            "canceled",
            StringComparison.OrdinalIgnoreCase);

        private static
        TrainingJobStatusResponse
        ToJobStatusResponse(TrainingJob job)
            => new(
            job.JobId,
            job.Status,
            job.Message,
            job.CreatedAt,
            job.StartedAt,
            job.FinishedAt,
            job.ImagesCount,
            job.BaseModel,
            job.BestWeightsPath,
            job.MobileModelPath,
            job.MobileFormat,
            job.MetricsJson,

            job.CancellationRequested,
            job.ClientId,
            job.AssignedAt,
            job.HeartbeatAt,
            job.DatasetZipPath,
            job.Epochs,
            job.ImgSize,
            job.Batch,
            job.Device,
            job.ExportInt8,
            job.ExportNms,
            job.QuantizationFraction,

            job.MobileModelFileName,

            job.MobileModelContentType);

        private static
        ModelVersionAdminResponse

```

```

        ToModelVersionResponse(ModelVersion
        model)
            => new(
            model.Id,
            model.ExternalJobId,
            model.TrainedAt,
            model.MetricsJson,
            model.BaseModel,
            model.BestWeightsPath,
            model.MobileModelPath,

            model.MobileModelFileName,

            model.MobileModelContentType,
            model.MobileFormat,
            model.IsPublished,
            model.IsPinned,
            model.IsDeleted,
            model.DeletedAt);
    }

    =====
    =====
    =====
    =====
    Файл:
    ProductsDateAPI/Controllers/Admin/Admi
    nUsersController.cs
    =====
    =====
    =====
    =====
    using Contracts.Dtos;
    using
    Microsoft.AspNetCore.Authorization;
    using
    Microsoft.AspNetCore.Mvc;
    using
    Microsoft.EntityFrameworkCore;
    using PostgreSQLRepository;

    namespace
    ProductsDateAPI.Controllers.Admin;

    [ApiController]
    [Route("api/admin/users")]

```

```

        // пока без ролей, но
        закрываем авторизацией
        [Authorize]
        public class
        AdminUsersController : ControllerBase
        {
            private readonly
            AppDbContext _db;

            public
            AdminUsersController(AppDbContext db)
            => _db = db;

            [HttpGet]
            public async
            Task<ActionResult<List<AdminUserListItem>
            m>>> GetAll(CancellationToken ct)
            {
                var items = await _db.Users
                .AsNoTracking()
                .OrderByDescending(x
            => x.Id)
                .Select(x => new
            AdminUserListItem(x.Id, x.Email,
            x.IsBlocked, x.CreatedAt))
                .ToListAsync(ct);

                return Ok(items);
            }

            [HttpGet("{id:int}")]
            public async
            Task<ActionResult<AdminUserListItem>>
            Get(int id, CancellationToken ct)
            {
                var u = await
                _db.Users.AsNoTracking().FirstOrDefaultA
            sync(x => x.Id == id, ct);
                if (u is null) return
            NotFound();

                return Ok(new
            AdminUserListItem(u.Id, u.Email,
            u.IsBlocked, u.CreatedAt));
            }

            [HttpPut("{id:int}/block")]
            public async
            Task<ActionResult> SetBlocked(int id,

```

```

            SetBlockedRequest req, CancellationToken
            ct)
            {
                var u = await
                _db.Users.FirstOrDefaultAsync(x => x.Id
            == id, ct);
                if (u is null) return
            NotFound();

                u.IsBlocked =
            req.IsBlocked;
                await
                _db.SaveChangesAsync();

                return NoContent();
            }

            [HttpDelete("{id:int}")]
            public async
            Task<ActionResult> Delete(int id,
            CancellationToken ct)
            {
                var u = await
                _db.Users.FirstOrDefaultAsync(x => x.Id
            == id, ct);
                if (u is null) return
            NotFound();

                _db.Users.Remove(u);
                await
                _db.SaveChangesAsync();

                return NoContent();
            }
        }

        =====
        =====
        =====
        =====
        Файл:
        ProductsDateAPI/Controllers/AuthControll
        er.cs
        =====
        =====
        =====

```

```

            using Contracts.Dtos;
            using
            Microsoft.AspNetCore.Mvc;
            using
            Microsoft.EntityFrameworkCore;
            using Models.Entities;
            using PostgreSQLRepository;
            using ProductsDateAPI.Security;

            namespace
            ProductsDateAPI.Controllers;

            [ApiController]
            [Route("api/[controller]")]
            public class AuthController :
            ControllerBase
            {
                private readonly
                AppDbContext _db;
                private readonly
                JwtTokenFactory _jwt;

                public
                AuthController(AppDbContext db,
                JwtTokenFactory jwt)
                {
                    _db = db;
                    _jwt = jwt;
                }

                [HttpPost("register")]
                public async
                Task<ActionResult<AuthResponse>>
                Register(RegisterRequest req,
                CancellationToken ct)
                {
                    var email =
                    req.Email.Trim().ToLowerInvariant();

                    if
                    (string.IsNullOrEmpty(email) ||
                    string.IsNullOrEmpty(req.Password))
                        return
                    BadRequest("Email/password required.");

                    var exists = await
                    _db.Users.AnyAsync(x => x.Email ==
                    email, ct);

```

```

        if (exists) return
        BadRequest("Email already used.");

        var user = new User
        {
            Email = email,
            PasswordHash =
                BCrypt.Net.BCrypt.HashPassword(req.Password),
            IsBlocked = false
        };

        _db.Users.Add(user);
        await
            _db.SaveChangesAsync(ct);

        return Ok(new
            AuthResponse(_jwt.CreateForUser(user)));
    }

    [HttpPost("login")]
    public async
        Task<ActionResult<AuthResponse>>
        Login(LoginRequest req,
            CancellationToken ct)
    {
        var email =
            req.Email.Trim().ToLowerInvariant();

        var user = await
            _db.Users.FirstOrDefaultAsync(x =>
                x.Email == email, ct);

        if (user is null) return
            Unauthorized("Invalid credentials.");
        if (user.IsBlocked) return
            Unauthorized("User blocked.");

        var ok =
            BCrypt.Net.BCrypt.Verify(req.Password,
                user.PasswordHash);
        if (!ok) return
            Unauthorized("Invalid credentials.");

        return Ok(new
            AuthResponse(_jwt.CreateForUser(user)));
    }
}

```

```

=====
=====
=====
Файл:
ProductsDateAPI/Controllers/ErrorReports
DatasetController.cs
=====
=====
=====
using Contracts.Dtos;
using
    Microsoft.AspNetCore.Authorization;
using
    Microsoft.AspNetCore.Mvc;
using Models.Entities;
using PostgreSQLRepository;
using ProductsDateAPI.Helpers;
using ProductsDateAPI.Models;
using System.IO.Compression;

namespace
    ProductsDateAPI.Controllers;

[ApiController]
[Route("api/error-reports")]
[Authorize]
public class
    ErrorReportsDatasetController :
        ControllerBase
    {
        private readonly
            AppDbContext _db;
        private readonly
            IWebHostEnvironment _env;

        public
            ErrorReportsDatasetController(AppDbContext
                ext db, IWebHostEnvironment env)
        {
            _db = db;
            _env = env;
        }

        [HttpPost("upload-dataset")]

```

```

[Consumes("multipart/form-
data")]

[RequestSizeLimit(400_000_000)]
public async
    Task<IActionResult>
    UploadDataset([FromForm]
        UploadDatasetForm form,
        CancellationToken ct)
    {
        var datasetZip =
            form.datasetZip;
        var comment =
            form.comment;

        if (datasetZip is null ||
            datasetZip.Length == 0)
            return
                BadRequest("datasetZip is required.");

        var userId =
            User.GetUserIdOrThrow();

        // 1) создаём папку
        var root =
            Path.Combine(_env.ContentRootPath,
                "uploads", "datasets");

        Directory.CreateDirectory(root);

        var folderName =
            $"{DateTime.UtcNow:yyyyMMdd_HH:mm:ss}_{Guid.NewGuid():N}";
        var datasetFolder =
            Path.Combine(root, folderName);

        Directory.CreateDirectory(datasetFolder);

        // 2) сохраняем zip
        временно
            var zipPath =
                Path.Combine(datasetFolder, "dataset.zip");
            await using (var fs =
                System.IO.File.Create(zipPath))
                await
                    datasetZip.CopyToAsync(fs, ct);

        // 3) распаковка

```

```

        var extractFolder =
Path.Combine(datasetFolder, "extracted");

Directory.CreateDirectory(extractFolder);

ZipFile.ExtractToDirectory(zipPath,
extractFolder);

// 4) валидация структуры
var imagesDir =
Path.Combine(extractFolder, "images");
var labelsDir =
Path.Combine(extractFolder, "labels");

if
(!Directory.Exists(imagesDir))
    return BadRequest("Zip
must contain /images folder.");
if
(!Directory.Exists(labelsDir))
    return BadRequest("Zip
must contain /labels folder.");

var imagesCount =
Directory.GetFiles(imagesDir, "*.*",
SearchOption.TopDirectoryOnly)
.Count(f=>
f.EndsWith(".jpg",
StringComparison.OrdinalIgnoreCase) ||
f.EndsWith(".png",
StringComparison.OrdinalIgnoreCase));

if (imagesCount == 0)
    return BadRequest("No
images found in /images.");

// 5) записываем
VideoSample, путь = папка датасета (как
ты хотел)

var vs = new VideoSample
{
    VideoPath =
Path.Combine("uploads", "datasets",
folderName).Replace("\\", "/"),
    Source =
"mobile_dataset"
};

```

```

_db.VideoSamples.Add(vs);
await
_db.SaveChangesAsync(ct);

// 6) ErrorReport
var report = new
ErrorReport
{
    UserId = userId,
    VideoId = vs.Id,
    Comment =
string.IsNullOrEmpty(comment) ?
null : comment.Trim(),
    CreatedAt =
DateTime.UtcNow,
    Resolved = false,
    Approved = false,
    FramesCount =
imagesCount
};

_db.ErrorReports.Add(report);
await
_db.SaveChangesAsync(ct);

return Ok(new
UploadDatasetResponse(report.Id, vs.Id,
vs.VideoPath));
}

=====
Файл:
ProductsDateAPI/Controllers/MobileModel
sController.cs
=====
using Contracts.Dtos;
using
Microsoft.AspNetCore.Mvc;

```

```

using
Microsoft.EntityFrameworkCore;
using PostgreSQLRepository;
using ProductsDateAPI.Services;

namespace
ProductsDateAPI.Controllers;

[ApiController]
[Route("api/mobile-models")]
public class
MobileModelsController : ControllerBase
{
    private readonly
AppDbContext _db;
    private readonly
TrainingFileStorage _storage;

    public
MobileModelsController(AppDbContext
db, TrainingFileStorage storage)
    {
        _db = db;
        _storage = storage;
    }

    [HttpGet("latest")]
    public async
Task<ActionResult<LatestMobileModelRe
sponse>> GetLatest(CancellationToken ct)
    {
        var model = await
_db.ModelVersions
.AsNoTracking()
.Where(x =>
x.IsPublished && !x.IsDeleted &&
x.MobileModelPath != null)
.OrderByDescending(x
=> x.TrainedAt)
.FirstOrDefaultAsync(ct);

if (model is null)
    return NotFound();

return Ok(new
LatestMobileModelResponse(
model.Id,

```

```

        model.TrainedAt,
        model.MobileFormat,
        model.MetricsJson,

        model.MobileModelFileName,
        model.IsPinned
    ));
}

[HttpGet("latest/download")]
public async
Task<ActionResult>
DownloadLatest(Cancellation token ct)
{
    var model = await
    _db.ModelVersions
        .AsNoTracking()
        .Where(x =>
            x.IsPublished && !x.IsDeleted &&
            x.MobileModelPath != null)
        .OrderByDescending(x
            => x.TrainedAt)

    .FirstOrDefaultAsync(ct);

    if (model is null ||
        string.IsNullOrWhiteSpace(model.MobileM
            odelPath))
        return NotFound();

    var absolute =
    _storage.ToAbsolutePath(model.MobileMo
        delPath);

    if
    (!System.IO.File.Exists(absolute))
        return
        NotFound("Published model file is missing
            on backend storage.");

    var fileName =
    string.IsNullOrWhiteSpace(model.MobileM
        odelFileName)
        ?
        $"latest_{model.MobileFormat} ??
        "bin" }"
        :
        model.MobileModelFileName;

```

```

    return File(
        System.IO.File.OpenRead(absolute),

        string.IsNullOrWhiteSpace(model.MobileM
            odelContentType) ? "application/octet-
            stream" : model.MobileModelContentType,

        fileName,
        enableRangeProcessing:
        true);
}

=====
Файл:
ProductsDateAPI/Controllers/ProductsCont
roller.cs
=====
using Contracts;
using Contracts.Dtos;
using
Microsoft.AspNetCore.Authorization;
using
Microsoft.AspNetCore.Mvc;
using Models.Entities;

namespace
ProductsDateAPI.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ProductsController :
ControllerBase
{
    private readonly
    IRepository<Product> _repo;

    public
    ProductsController(IRepository<Product>
        repo) => _repo = repo;

```

```

[HttpGet]
public async
Task<ActionResult<List<Product>>>
GetAll(Cancellation token ct)
    => Ok(await
    _repo.GetAllAsync(ct));

[HttpGet("{id:int}")]
public async
Task<ActionResult<Product>> Get(int id,
    Cancellation token ct)
{
    var p = await
    _repo.GetByIdAsync(id, ct);
    return p is null ?
    NotFound() : Ok(p);
}

// Можно оставить
открытым, но обычно
создание/редактирование — админское.
[Authorize]
[HttpPost]
public async
Task<ActionResult<Product>>
Create(ProductCreateRequest req,
    Cancellation token ct)
{
    var p = new Product
    {
        Name = req.Name,
        Manufacturer =
        req.Manufacturer,
        Barcode = req.Barcode
    };

    await _repo.AddAsync(p,
    ct);

    return
    CreatedAtAction(nameof(Get), new { id =
        p.Id }, p);
}

[Authorize]
[HttpPut("{id:int}")]
public async
Task<ActionResult> Update(int id,

```



```

ProductUpdateRequest req,
CancellationToken ct)
    {
        var p = await
_repo.GetByIdAsync(id, ct);
        if (p is null) return
NotFound();

        p.Name = req.Name;
        p.Manufacturer =
req.Manufacturer;
        p.Barcode = req.Barcode;

        await
_repo.UpdateAsync(p, ct);
        return NoContent();
    }

    [Authorize]
    [HttpDelete("{id:int}")]
    public async
Task<ActionResult> Delete(int id,
CancellationToken ct)
    {
        await
_repo.DeleteAsync(id, ct);
        return NoContent();
    }
}

=====
=====
=====
=====

Файл:
ProductsDateAPI/Controllers/StorageContr
oller.cs

=====
=====
=====
=====

using Contracts;
using Contracts.Dtos;
using
Microsoft.AspNetCore.Authorization;
using
Microsoft.AspNetCore.Mvc;

```

```

using
Microsoft.EntityFrameworkCore;
using Models.Entities;
using PostgreSQLRepository;
using ProductsDateAPI.Helpers;

namespace
ProductsDateAPI.Controllers;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class StorageController :
ControllerBase
    {
        private readonly
StoredProductRepository _storageRepo;
        private readonly
IRepository<StoredProduct> _repo;
        private readonly
AppDbContext _db;

        public StorageController(
StoredProductRepository
storageRepo,
IRepository<StoredProduct> repo,
AppDbContext db)
        {
            _storageRepo =
storageRepo;
            _repo = repo;
            _db = db;
        }

        [HttpGet]
        public async
Task<ActionResult<List<StoredProductDto
>>> GetMy(Cancellation token ct)
        {
            var userId =
User.GetUserIdOrThrow();
            var items = await
_storageRepo.GetUserAsync(userId, ct);
            return
Ok(items.Select(MapStoredProduct).ToList
());
        }
    }

```

```

[HttpPost]
public async
Task<ActionResult<StoredProductDto>>>
Add(StoredProductCreateRequest req,
CancellationToken ct)
    {
        var userId =
User.GetUserIdOrThrow();

        var product = await
ResolveProductAsync(req, ct);
        if (product is null)
        {
            return
BadRequest("Нужно передать productId
или непустое название продукта.");
        }

        var sp = new StoredProduct
        {
            UserId = userId,
            ProductId = product.Id,
            ManufactureAt =
NormalizeToUtc(req.ManufactureAt),
            ExpiryAt =
NormalizeToUtc(req.ExpiryAt),
            CreatedAt =
DateTime.UtcNow
        };

        await _repo.AddAsync(sp,
ct);

        sp.Product = product;
        return
Ok(MapStoredProduct(sp));
    }

    [HttpDelete("{id:int}")]
    public async
Task<ActionResult> Delete(int id,
CancellationToken ct)
    {
        // Защита: удалять можно
только своё

        var userId =
User.GetUserIdOrThrow();
    }

```

```

        var sp = await
_repo.GetByIdAsync(id, ct);
        if (sp is null) return
NotFound();
        if (sp.UserId != userId)
return Forbid();

        await
_repo.DeleteAsync(id, ct);
        return NoContent();
    }

    private static
StoredProductDto
MapStoredProduct(StoredProduct entity)
    {
        return new
StoredProductDto
    {
        Id = entity.Id,
        UserId = entity.UserId,
        ProductId =
entity.ProductId,
        ManufactureAt =
entity.ManufactureAt,
        ExpiryAt =
entity.ExpiryAt,
        CreatedAt =
entity.CreatedAt,
        Product = entity.Product
is null
        ? null
        : new
ProductSummaryDto
    {
        Id =
entity.Product.Id,
        Name =
entity.Product.Name,
        Manufacturer =
entity.Product.Manufacturer,
        Barcode =
entity.Product.Barcode
    }
    };
    }

```

```

    private async Task<Product?>
ResolveProductAsync(StoredProductCreate
Request req, CancellationToken ct)
    {
        if (req.ProductId is int
productId)
        {
            return await
_db.Products.FirstOrDefaultAsync(x =>
x.Id == productId, ct);
        }

        var name =
req.ProductName?.Trim();
        if
(string.IsNullOrEmpty(name))
        {
            return null;
        }

        var existing = await
_db.Products
.FirstOrDefaultAsync(x
=> x.Name.ToLower() ==
name.ToLower(), ct);

        if (existing is not null)
        {
            return existing;
        }

        var created = new Product
        {
            Name = name
        };

        _db.Products.Add(created);
        await
_db.SaveChangesAsync(ct);
        return created;
    }

    private static DateTime?
NormalizeToUtc(DateTime? value)
    {
        if (value is null)
        {
            return null;
        }
    }

```

```

        var dt = value.Value;
        return dt.Kind switch
        {
            DateTimeKind.Utc =>
dt,
            DateTimeKind.Local =>
dt.ToUniversalTime(),
            _ =>
DateTime.SpecifyKind(dt,
DateTimeKind.Utc)
        };
    }

=====
=====
=====
=====

Файл:
ProductsDateAPI/Controllers/TrainingClien
tController.cs
=====
=====
=====
=====

using Contracts.Dtos;
using
Microsoft.AspNetCore.Mvc;
using
Microsoft.EntityFrameworkCore;
using
Microsoft.Extensions.Options;
using Models.Entities;
using PostgreSQLRepository;
using ProductsDateAPI.Models;
using ProductsDateAPI.Services;

namespace
ProductsDateAPI.Controllers;

[ApiController]
[Route("api/training-client")]
public class
TrainingClientController : ControllerBase
    {

```

```

        private readonly
AppDbContext _db;

        private readonly
TrainingFileStorage _storage;

        private readonly
TrainingServiceOptions _options;

    public
TrainingClientController(
        AppDbContext db,
        TrainingFileStorage
storage,

IOptions<TrainingServiceOptions> options)
    {
        _db = db;
        _storage = storage;
        _options = options.Value;
    }

    [HttpGet("jobs/next")]
    public async
Task<IActionResult> GetNextJob(
        [FromQuery] string?
clientId,

        [FromHeader(Name = "X-
Training-Client-Key")] string? apiKey,
        CancellationToken ct)
    {
        if (!IsAuthorized(apiKey))
            return Unauthorized();

        var job = await
_db.TrainingJobs
        .Where(x => x.Status ==
"queued" && !x.CancellationRequested)
        .OrderBy(x =>
x.CreatedAt)

        .FirstOrDefaultAsync(ct);

        if (job is null)
            return NoContent();

        job.Status = "running";
        job.StartedAt ??=
DateTime.UtcNow;

```

```

        job.AssignedAt =
DateTime.UtcNow;

        job.HeartbeatAt =
DateTime.UtcNow;

        job.ClientId =
string.IsNullOrEmpty(clientId) ?
Environment.MachineName : clientId;

        job.Message = $"Задача
забрана Python-клиентом {job.ClientId}.";

        await
_db.SaveChangesAsync(ct);

        return Ok(new
TrainingClientJobResponse(
            job.JobId,
            job.ImagesCount,
            job.BaseModel,
            job.Epochs,
            job.ImgSize,
            job.Batch,
            job.Device,
            job.ExportInt8,
            job.ExportNms,
            job.MobileFormat,
            job.QuantizationFraction,

            job.CancellationRequested));
    }

    [HttpGet("jobs/{jobId}")]
    public async
Task<IActionResult> GetJob(
        string jobId,
        [FromHeader(Name = "X-
Training-Client-Key")] string? apiKey,
        CancellationToken ct)
    {
        if (!IsAuthorized(apiKey))
            return Unauthorized();

        var job = await
_db.TrainingJobs.AsNoTracking().FirstOrD
efaultAsync(x => x.JobId == jobId, ct);

        if (job is null)
            return NotFound();

        return Ok(new
TrainingClientStatusResponse(job.JobId,

```

```

        job.Status, job.CancellationRequested,
        job.Message));
    }

    [HttpGet("jobs/{jobId}/dataset")]
    public async
Task<IActionResult> DownloadDataset(
        string jobId,
        [FromHeader(Name = "X-
Training-Client-Key")] string? apiKey,
        CancellationToken ct)
    {
        if (!IsAuthorized(apiKey))
            return Unauthorized();

        var job = await
_db.TrainingJobs.AsNoTracking().FirstOrD
efaultAsync(x => x.JobId == jobId, ct);

        if (job is null)
            return NotFound();

        if
(string.IsNullOrEmpty(job.DatasetZip
Path))

            return
NotFound("Dataset file path is empty.");

        var absolute =
_storage.ToAbsolutePath(job.DatasetZipPat
h);

        if
(!System.IO.File.Exists(absolute))

            return
NotFound("Dataset file is missing on
backend storage.");

        return
File(System.IO.File.OpenRead(absolute),
"application/zip",
$"dataset_{job.JobId}.zip");
    }

    [HttpPost("jobs/{jobId}/status")]
    public async
Task<IActionResult> UpdateStatus(
        string jobId,

```

```

[FromBody]
TrainingClientStatusRequest request,
[FromHeader(Name = "X-
Training-Client-Key")] string? apiKey,
CancellationToken ct)
{
    if (!IsAuthorized(apiKey))
        return Unauthorized();

    var job = await
        _db.TrainingJobs.FirstOrDefaultAsync(x
=> x.JobId == jobId, ct);
    if (job is null)
        return NotFound();

    job.HeartbeatAt =
        DateTime.UtcNow;

    if
        (!string.IsNullOrEmpty(request.Status
        ))
    {
        var requestedStatus =
            request.Status.Trim().ToLowerInvariant();
        if (requestedStatus is
            "running" or "failed" or "canceled")
        {
            job.Status =
                requestedStatus;
            if (requestedStatus is
                "failed" or "canceled")
                job.FinishedAt =
                    DateTime.UtcNow;
        }
    }

    if
        (!string.IsNullOrEmpty(request.Messa
        ge))
        job.Message =
            request.Message;

    if
        (!string.IsNullOrEmpty(request.Metric
        sJson))
        job.MetricsJson =
            request.MetricsJson;

```

```

        await
            _db.SaveChangesAsync();

        return Ok(new
            TrainingClientStatusResponse(job.JobId,
            job.Status, job.CancellationRequested,
            job.Message));
    }

    [HttpPost("jobs/{jobId}/artifacts")]

    [RequestSizeLimit(2_147_483_648L)]

    [RequestFormLimits(MultipartBodyLength
    Limit = 2_147_483_648L)]

    public async
        Task<IActionResult> UploadArtifacts(
            string jobId,
            [FromForm]
            TrainingArtifactUploadForm form,
            [FromHeader(Name = "X-
            Training-Client-Key")] string? apiKey,
            CancellationToken ct)
    {
        if (!IsAuthorized(apiKey))
            return Unauthorized();

        if (form.MobileModel is
            null || form.MobileModel.Length == 0)
            return
                BadRequest("mobileModel is required.");

        var job = await
            _db.TrainingJobs.FirstOrDefaultAsync(x
=> x.JobId == jobId, ct);
        if (job is null)
            return NotFound();

        var bestPath =
            form.BestWeights is not null &&
            form.BestWeights.Length > 0
            ? await
                _storage.SaveArtifactAsync(jobId,
                form.BestWeights,
                form.BestWeights.FileName, ct)
            : null;

        var mobilePath = await
            _storage.SaveArtifactAsync(jobId,

```

```

            form.MobileModel,
            form.MobileModel.FileName, ct);

        var format =
            string.IsNullOrEmpty(form.MobileFor
            mat) ? job.MobileFormat :
            form.MobileFormat;

        job.Status = "completed";
        job.FinishedAt =
            DateTime.UtcNow;
        job.Message = "Python-
        клиент завершил обучение и загрузил
        артефакты на бэк.";
        job.BestWeightsPath =
            bestPath;
        job.MobileModelPath =
            mobilePath;
        job.MobileModelFileName
            = form.MobileModel.FileName;

        job.MobileModelContentType =
            string.IsNullOrEmpty(form.MobileMo
            del.ContentType) ? "application/octet-
            stream" : form.MobileModel.ContentType;
        job.MobileFormat = format;
        job.MetricsJson =
            form.MetricsJson;

        var model = await
            _db.ModelVersions.FirstOrDefaultAsync(x
=> x.ExternalJobId == job.JobId, ct);
        if (model is null)
        {
            model = new
                ModelVersion
                {
                    ExternalJobId =
                        job.JobId,

                    IsPublished = false,
                    IsPinned = false,
                };

            _db.ModelVersions.Add(model);
        }

        model.TrainedAt =
            job.FinishedAt ?? DateTime.UtcNow;

```

```

        model.MetricsJson =
form.MetricsJson;

        model.BaseModel =
job.BaseModel;

        model.BestWeightsPath =
bestPath;

        model.MobileModelPath =
mobilePath;

model.MobileModelFileName =
form.MobileModel.FileName;

model.MobileModelContentType =
job.MobileModelContentType;

        model.MobileFormat =
format;

        model.IsDeleted = false;
        model.DeletedAt = null;

        await
_db.SaveChangesAsync(ct);
        return Ok(new
TrainingClientStatusResponse(job.JobId,
job.Status, job.CancellationRequested,
job.Message));
    }

    private bool
IsAuthorized(string? apiKey)
    =>
string.IsNullOrEmpty(_options.ApiKey) || apiKey == _options.ApiKey;
    }

=====

=====

=====

Файл:
ProductsDateAPI/Controllers/UsersController.cs

=====

=====

=====

using Contracts.Dtos;

```

```

using
Microsoft.AspNetCore.Authorization;

using
Microsoft.AspNetCore.Mvc;

using
Microsoft.EntityFrameworkCore;

using PostgreSQLRepository;
using ProductsDateAPI.Helpers;

namespace
ProductsDateAPI.Controllers;

[ApiController]
[Route("api/users")]
public class UsersController :
ControllerBase
    {
        private readonly
AppDbContext _db;

        public
UsersController(AppDbContext db) => _db
= db;

        [Authorize]
        [HttpGet("me")]
        public async
Task<ActionResult<UserProfileResponse>
> Me(CancellationToken ct)
        {
            var userId =
User.GetUserIdOrThrow();

            var u = await
_db.Users.AsNoTracking().FirstOrDefaultA
sync(x => x.Id == userId, ct);

            if (u is null) return
NotFound();

            return Ok(new
UserProfileResponse(u.Id, u.Email,
u.IsBlocked, u.SettingsJson));
        }

        // опционально: обновление
своих настроек (JSON)

        [Authorize]
        [HttpPut("me/settings")]
        public async
Task<ActionResult>

```

```

UpdateSettings([FromBody] string?
settingsJson, CancellationToken ct)
    {
        var userId =
User.GetUserIdOrThrow();

        var u = await
_db.Users.FirstOrDefaultAsync(x => x.Id
== userId, ct);

        if (u is null) return
NotFound();

        u.SettingsJson =
settingsJson;

        await
_db.SaveChangesAsync(ct);

        return NoContent();
    }

    [Authorize]
    [HttpDelete("me")]
    public async
Task<ActionResult>
DeleteMe(CancellationToken ct)
    {
        var userId =
User.GetUserIdOrThrow();

        var u = await
_db.Users.FirstOrDefaultAsync(x => x.Id
== userId, ct);

        if (u is null) return
NotFound();

        _db.Users.Remove(u);

        await
_db.SaveChangesAsync(ct);

        return NoContent();
    }
}

=====

=====

=====

Файл:
ProductsDateAPI/Helpers/ClaimsExtension
s.cs

```

```

=====
=====
=====
=====

using System.Security.Claims;

namespace
ProductsDateAPI.Helpers;

public static class
ClaimsExtensions
{
    public static int
GetUserIdOrThrow(this ClaimsPrincipal
user)
    {
        var sub =
user.FindFirst("sub")?.Value ??
user.FindFirst(ClaimTypes.NameIdentifier)
?.Value;

        if
(string.IsNullOrEmpty(sub) ||
!int.TryParse(sub, out var id))

            throw new
InvalidOperationException("Invalid JWT:
missing sub");

        return id;
    }
}

=====
=====
=====
=====

Файл:
ProductsDateAPI/Models/BackupImportFor
m.cs

=====
=====
=====
=====

using
Microsoft.AspNetCore.Http;

namespace
ProductsDateAPI.Models;

```

```

public class BackupImportForm
{
    public IFormFile? BackupZip
{ get; set; }

    public bool ReplaceExisting {
get; set; } = false;
}

=====
=====
=====
=====

Файл:
ProductsDateAPI/Models/TrainingArtifact
UploadForm.cs

=====
=====
=====
=====

using
Microsoft.AspNetCore.Http;

namespace
ProductsDateAPI.Models;

public class
TrainingArtifactUploadForm
{
    public IFormFile?
BestWeights { get; set; }

    public IFormFile?
MobileModel { get; set; }

    public string? MetricsJson {
get; set; }

    public string? MobileFormat {
get; set; }
}

=====
=====
=====
=====

Файл:
ProductsDateAPI/Models/TrainingServiceO
ptions.cs

```

```

=====
=====
=====
=====

namespace
ProductsDateAPI.Models;

public class
TrainingServiceOptions
{
    public string? ApiKey { get;
set; }

    public string TempRoot { get;
set; } = "temp/training";

    public string StorageRoot {
get; set; } = "storage/training";

    public string?
SeedDatasetPath { get; set; }
}

=====
=====
=====
=====

Файл:
ProductsDateAPI/Models/UploadDatasetFo
rm.cs

=====
=====
=====
=====

namespace
ProductsDateAPI.Models;

public class UploadDatasetForm
{
    public IFormFile datasetZip {
get; set; } = default!;

    public string? comment { get;
set; }
}

=====
=====
=====
=====

```

```

Файл:
ProductsDateAPI/ProductsDateAPI.csproj
=====

<Project
Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>

<TargetFramework>net8.0</TargetFramework>

    <Nullable>enable</Nullable>

<ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference
Include="BCrypt.Net-Next"
Version="4.0.3" />
    <PackageReference
Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.23" />
    <PackageReference
Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.13">

<PrivateAssets>all</PrivateAssets>
    <IncludeAssets>runtime;
build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
  </PackageReference>
  <PackageReference
Include="Swashbuckle.AspNetCore"
Version="6.6.2" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference
Include="..\Contracts\Contracts.csproj" />
    <ProjectReference
Include="..\Models\Models.csproj" />
    <ProjectReference
Include="..\PostgreSQLRepository\PostgreSQLRepository.csproj" />

```

```

</ItemGroup>

</Project>
=====

Файл:
ProductsDateAPI/Program.cs
=====

using Contracts;
using
Microsoft.AspNetCore.Authentication.JwtBearer;
using
Microsoft.EntityFrameworkCore;
using
Microsoft.IdentityModel.Tokens;
using PostgreSQLRepository;
using ProductsDateAPI.Models;
using ProductsDateAPI.Security;
using ProductsDateAPI.Services;
using System.Text;

var builder =
WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers(
);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// DB
builder.Services.AddDbContext<AppDbContext>(opt =>
opt.UseNpgsql(builder.Configuration.GetConnectionString("Default")));

// Репозитории

```

```

builder.Services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
builder.Services.AddScoped<StoredProductRepository>();

// Training pipeline
builder.Services.Configure<TrainingServiceOptions>(builder.Configuration.GetSection("TrainingService"));
builder.Services.AddScoped<TrainingDatasetPackager>();
builder.Services.AddScoped<TrainingFileStorage>();
builder.Services.AddScoped<BackupService>();

// JWT
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));
builder.Services.AddSingleton<JwtTokenFactory>();

var jwt =
builder.Configuration.GetSection("Jwt").Get<JwtOptions>()!;
var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Secret));

builder.Services
.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(opt =>
{
opt.TokenValidationParameters = new
TokenValidationParameters
{
ValidateIssuer = true,
ValidIssuer = jwt.Issuer,
ValidateAudience = true,
ValidAudience =
jwt.Audience,
ValidateIssuerSigningKey = true,
IssuerSigningKey = key,

```

```

        ValidateLifetime = true,
        ClockSkew =
        TimeSpan.FromSeconds(30)
    };
});

builder.Services.AddAuthorization(
on());

// CORS (для Flutter/админки)
builder.Services.AddCors(opt =>
{
    opt.AddPolicy("Default", p =>

p.AllowAnyHeader().AllowAnyMethod().A
llowAnyOrigin());
});

var app = builder.Build();

if
(app.Configuration.GetValue<bool>("Datab
ase:ApplyMigrations"))
{
    using var scope =
app.Services.CreateScope();
    var db =
scope.ServiceProvider.GetRequiredService
<AppDbContext>();
    db.Database.Migrate();
}

if
(app.Environment.IsDevelopment() ||
app.Configuration.GetValue<bool>("Swagg
er:Enabled"))
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

//app.UseHttpsRedirection();
app.UseCors("Default");
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

```

```

app.Run();

=====

=====

=====

Файл:
ProductsDateAPI/Properties/launchSettings.
json
=====

=====

=====

{
    "$schema":
"http://json.schemastore.org/launchsettings.j
son",

    "iisSettings": {

        "windowsAuthentication":
false,

        "anonymousAuthentication":
true,

        "iisExpress": {

            "applicationUrl":
"http://localhost:6655",

            "sslPort": 44300
        }
    },

    "profiles": {

        "http": {

            "commandName": "Project",
            "dotnetRunMessages": true,
            "launchBrowser": true,
            "launchUrl": "swagger",
            "applicationUrl":
"http://0.0.0.0:5099",

            "environmentVariables": {

"ASPNETCORE_ENVIRONMENT":
"Development"
        }
    },

    "https": {

        "commandName": "Project",
        "dotnetRunMessages": true,
        "launchBrowser": true,
        "launchUrl": "swagger",

```

```

        "applicationUrl":
"http://localhost:7178;http://localhost:5099
",

        "environmentVariables": {

"ASPNETCORE_ENVIRONMENT":
"Development"
    }
},

    "IIS Express": {

        "commandName":
"IISExpress",

        "launchBrowser": true,
        "launchUrl": "swagger",
        "environmentVariables": {

"ASPNETCORE_ENVIRONMENT":
"Development"
    }
}

=====

=====

=====

Файл:
ProductsDateAPI/Security/JwtOptions.cs
=====

=====

namespace
ProductsDateAPI.Security;

public sealed class JwtOptions
{

    public string Issuer { get; set;
} = "ProductsDateAPI";

    public string Audience { get;
set; } = "ProductsDateAPI";

    public string Secret { get; set;
} = "CHANGE_ME";

    public int
AccessTokenMinutes { get; set; } = 60;
}

```



```

=====
=====
=====
=====

Файл:
ProductsDateAPI/Security/JwtTokenFactor
y.cs
=====
=====
=====

using
System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using
Microsoft.Extensions.Options;
using
Microsoft.IdentityModel.Tokens;
using Models.Entities;

namespace
ProductsDateAPI.Security;

public sealed class
JwtTokenFactory
{
    private readonly JwtOptions
_opt;

    public
JwtTokenFactory(IOptions<JwtOptions>
opt) => _opt = opt.Value;

    public string
CreateForUser(User user)
    {
        var claims = new
List<Claim>
    {

new(JwtRegisteredClaimNames.Sub,
user.Id.ToString()),

new(JwtRegisteredClaimNames.Email,
user.Email),

```

```

        new("blocked",
user.IsBlocked ? "1" : "0")
    };

    var key = new
SymmetricSecurityKey(Encoding.UTF8.Ge
tBytes(_opt.Secret));

    var creds = new
SigningCredentials(key,
SecurityAlgorithms.HmacSha256);

    var token = new
JwtSecurityToken(
    issuer: _opt.Issuer,
    audience: _opt.Audience,
    claims: claims,
    expires:
DateTime.UtcNow.AddMinutes(_opt.Acces
sTokenMinutes),
    signingCredentials: creds
);

    return new
JwtSecurityTokenHandler().WriteToken(to
ken);
}
}

=====
=====
=====

Файл:
ProductsDateAPI/Services/BackupService.c
s
=====
=====
=====

using System.IO.Compression;
using System.Text.Json;
using
Microsoft.AspNetCore.Http;
using
Microsoft.EntityFrameworkCore;
using Models.Entities;
using PostgreSQLRepository;

```

```

namespace
ProductsDateAPI.Services;

public sealed class
BackupService
{
    private const string
BackupFormat = "productsdate-backup";
    private const int
BackupVersion = 1;

    private static readonly
JsonSerializerOptions JsonOptions =
new(JsonSerializerDefaults.Web)
    {
        WriteIndented = true
    };

    private readonly
AppDbContext _db;
    private readonly
IWebHostEnvironment _env;
    private readonly
TrainingFileStorage _trainingStorage;

    public
BackupService(AppDbContext db,
IWebHostEnvironment env,
TrainingFileStorage trainingStorage)
    {
        _db = db;
        _env = env;
        _trainingStorage =
trainingStorage;
    }

    public async Task<string>
CreateBackupZipAsync(CancellationToken
ct)
    {
        var backupDir =
Path.Combine(Path.GetTempPath(),
"productsdate-backups");

        Directory.CreateDirectory(backupDir);
    }
}

```

```

        var zipPath =
            Path.Combine(backupDir,
                $"productsdate_backup_{DateTime.UtcNow:yyyyMMdd_HHmms}_{Guid.NewGuid():N}.zip");

        await using var zipStream =
            File.Create(zipPath);
        using var archive = new
            ZipArchive(zipStream,
                ZipArchiveMode.Create, leaveOpen: false);

        await
            AddJsonEntryAsync(archive,
                "manifest.json", new BackupManifest(
                    BackupFormat,
                    BackupVersion,
                    DateTime.UtcNow,
                    "ProductsDateAPI", ct);

        var snapshot = await
            CreateDatabaseSnapshotAsync(ct);
        await
            AddJsonEntryAsync(archive,
                "database.json", snapshot, ct);

        AddDirectoryToArchive(archive,
            _trainingStorage.RootPath,
            "storage/training");

        AddDirectoryToArchive(archive,
            UploadsRootPath, "uploads");

        return zipPath;
    }

    public async
        Task<BackupImportResult>
        ImportBackupZipAsync(IFormFile
            backupZip, bool replaceExisting,
            CancellationToken ct)
        {
            if (backupZip.Length == 0)
                throw new
                    InvalidOperationException("ZIP-файл
                        пусрой.");

```

```

        var tempRoot =
            Path.Combine(Path.GetTempPath(),
                "productsdate-backup-import",
                Guid.NewGuid().ToString("N"));
        var zipPath =
            Path.Combine(tempRoot, "backup.zip");
        var extractRoot =
            Path.Combine(tempRoot, "extracted");

        Directory.CreateDirectory(tempRoot);

        Directory.CreateDirectory(extractRoot);

        try
        {
            await using (var stream =
                File.Create(zipPath))
            await
                backupZip.CopyToAsync(stream, ct);

            ExtractZipSafely(zipPath, extractRoot);

            var manifest = await
                ReadJsonFileAsync<BackupManifest>(Pat
                    h.Combine(extractRoot, "manifest.json"),
                    ct);

            if
                (!string.Equals(manifest.Format,
                    BackupFormat,
                    StringComparison.OrdinalIgnoreCase))
                throw new
                    InvalidOperationException("Неверный
                        формат backup-архива.");

            if (manifest.Version >
                BackupVersion)
                throw new
                    InvalidOperationException($"Backup
                        создан в более новой версии формата:
                        {manifest.Version}.");

            var snapshot = await
                ReadJsonFileAsync<BackupDatabaseSnaps
                    hot>(Path.Combine(extractRoot,
                        "database.json"), ct);

```

```

            if (!replaceExisting &&
                await DatabaseHasDataAsync(ct))
            {
                throw new
                    InvalidOperationException(
                        "В базе уже есть
                        данные. Для полного восстановления
                        передайте replaceExisting=true.");
            }

            await using var tx =
                await
                    _db.Database.BeginTransactionAsync(ct);

            if (replaceExisting)
            {
                await
                    ClearDatabaseAsync(ct);

                ClearDirectoryContents(_trainingStorage.R
                    ootPath);

                ClearDirectoryContents(UploadsRootPath);
            }

            CopyDirectoryContentsIfExists(Path.Combi
                ne(extractRoot, "storage", "training"),
                _trainingStorage.RootPath);

            CopyDirectoryContentsIfExists(Path.Combi
                ne(extractRoot, "uploads",
                    UploadsRootPath);

            RestoreDatabaseSnapshot(snapshot);

            await
                _db.SaveChangesAsync(ct);
            await
                ResetSequencesAsync(ct);

            await
                tx.CommitAsync(ct);

            return new
                BackupImportResult(
                    snapshot.Users.Count,

```

```

        snapshot.Products.Count,
        snapshot.StoredProducts.Count,
        snapshot.VideoSamples.Count,
        snapshot.ErrorReports.Count,
        snapshot.ModelVersions.Count,
        snapshot.TrainingJobs.Count,
        replaceExisting);
    }
    finally
    {
        try
        {
            if
            (Directory.Exists(tempRoot))

            Directory.Delete(tempRoot, recursive:
            true);

        }
        catch
        {
            // best effort cleanup
        }
    }
}

private string
UploadsRootPath =>
Path.Combine(_env.ContentRootPath,
"uploads");

private async
Task<BackupDatabaseSnapshot>
CreateDatabaseSnapshotAsync(Cancellatio
nToken ct)
{
    return new
BackupDatabaseSnapshot
{
    Users = await
_db.Users.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
UserBackupDto(x.Id, x.Email,
x.PasswordHash, x.IsBlocked,
x.SettingsJson, x.CreatedAt))
        .ToListAsync(ct),

    Products = await
_db.Products.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
ProductBackupDto(x.Id, x.Name,
x.Manufacturer, x.Barcode))
        .ToListAsync(ct),

    StoredProducts = await
_db.StoredProducts.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
StoredProductBackupDto(x.Id, x.UserId,
x.ProductId, x.ManufactureAt, x.ExpiryAt,
x.CreatedAt))
        .ToListAsync(ct),

    VideoSamples = await
_db.VideoSamples.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
VideoSampleBackupDto(x.Id, x.VideoPath,
x.Source))
        .ToListAsync(ct),

    ErrorReports = await
_db.ErrorReports.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
ErrorReportBackupDto(x.Id, x.UserId,
x.VideoId, x.ModelVersionId, x.Comment,
x.CreatedAt, x.Resolved, x.Approved,
x.FramesCount))
        .ToListAsync(ct),

    ModelVersions = await
_db.ModelVersions.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
ModelVersionBackupDto(
x.Id,
x.TrainedAt,
x.MetricsJson,
x.ExternalJobId,
x.BaseModel,
x.BestWeightsPath,
x.MobileModelPath,
x.MobileFormat,
x.MobileModelFileName,
x.MobileModelContentType,
x.IsPublished,
x.IsPinned,
x.IsDeleted,
x.DeletedAt))
        .ToListAsync(ct),

    TrainingJobs = await
_db.TrainingJobs.AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new
TrainingJobBackupDto(
x.Id,
x.JobId,
x.Status,
x.Message,
x.CreatedAt,
x.StartedAt,
x.FinishedAt,
x.AssignedAt,
x.HeartbeatAt,
x.ImagesCount,
x.ClientId,
x.DatasetZipPath,
x.BaseModel,
x.Epochs,
x.ImgSize,
x.Batch,
x.Device,
x.ExportInt8,
x.ExportNms,
x.MobileFormat,
x.QuantizationFraction,
x.BestWeightsPath,
x.MobileModelPath,
x.MobileModelFileName,

```

```

x.MobileModelContentType,
    x.MetricsJson,

x.CancellationRequested))
    .ToListAsync(ct)
    };
}

private async Task<bool>
DatabaseHasDataAsync(CancellationTok
en ct)
{
    return await
_db.Users.AnyAsync(ct)
    || await
_db.Products.AnyAsync(ct)
    || await
_db.StoredProducts.AnyAsync(ct)
    || await
_db.VideoSamples.AnyAsync(ct)
    || await
_db.ErrorReports.AnyAsync(ct)
    || await
_db.ModelVersions.AnyAsync(ct)
    || await
_db.TrainingJobs.AnyAsync(ct);
}

private async Task
ClearDatabaseAsync(CancellationTok
en ct)
{
    await
_db.ErrorReports.ExecuteDeleteAsync(ct);
    await
_db.StoredProducts.ExecuteDeleteAsync(ct
);
    await
_db.TrainingJobs.ExecuteDeleteAsync(ct);
    await
_db.VideoSamples.ExecuteDeleteAsync(ct)
;
    await
_db.ModelVersions.ExecuteDeleteAsync(ct
);
    await
_db.Products.ExecuteDeleteAsync(ct);
    await
_db.Users.ExecuteDeleteAsync(ct);
}

private void
RestoreDatabaseSnapshot(BackupDatabase
Snapshot snapshot)
{
    _db.Users.AddRange(snapshot.Users.Select
(x => new User
{
    Id = x.Id,
    Email = x.Email,
    PasswordHash =
x.PasswordHash,
    IsBlocked = x.IsBlocked,
    SettingsJson =
x.SettingsJson,
    CreatedAt = x.CreatedAt
}));
    _db.Products.AddRange(snapshot.Products.
Select(x => new Product
{
    Id = x.Id,
    Name = x.Name,
    Manufacturer =
x.Manufacturer,
    Barcode = x.Barcode
}));
    _db.VideoSamples.AddRange(snapshot.Vid
eoSamples.Select(x => new VideoSample
{
    Id = x.Id,
    VideoPath =
x.VideoPath,
    Source = x.Source
}));
    _db.ModelVersions.AddRange(snapshot.M
odelVersions.Select(x => new
ModelVersion
{
    Id = x.Id,
    TrainedAt = x.TrainedAt,
    MetricsJson =
x.MetricsJson,
    ExternalJobId =
x.ExternalJobId,
    BaseModel =
x.BaseModel,
    BestWeightsPath =
x.BestWeightsPath,
    MobileModelPath =
x.MobileModelPath,
    MobileFormat =
x.MobileFormat,
    MobileModelFileName =
x.MobileModelFileName,
    MobileModelContentType =
x.MobileModelContentType,
    IsPublished =
x.IsPublished,
    IsPinned = x.IsPinned,
    IsDeleted = x.IsDeleted,
    DeletedAt = x.DeletedAt
}));
    _db.StoredProducts.AddRange(snapshot.St
oredProducts.Select(x => new
StoredProduct
{
    Id = x.Id,
    UserId = x.UserId,
    ProductId = x.ProductId,
    ManufactureAt =
x.ManufactureAt,
    ExpiryAt = x.ExpiryAt,
    CreatedAt = x.CreatedAt
}));
    _db.TrainingJobs.AddRange(snapshot.Train
ingJobs.Select(x => new TrainingJob
{
    Id = x.Id,
    JobId = x.JobId,
    Status = x.Status,
    Message = x.Message,

```

| | | |
|--|---|---|
| CreatedAt = x.CreatedAt, | VideoId = x.VideoId, | EXISTS(SELECT |
| StartedAt = x.StartedAt, | ModelVersionId = | 1 FROM "{quotedTable}") |
| FinishedAt = | x.ModelVersionId, |); |
| x.FinishedAt, | Comment = x.Comment, | ""; |
| AssignedAt = | CreatedAt = x.CreatedAt, | await |
| x.AssignedAt, | Resolved = x.Resolved, | _db.Database.ExecuteSqlRawAsync(sql, |
| HeartbeatAt = | Approved = x.Approved, | cancellationToken: ct); |
| x.HeartbeatAt, | FramesCount = | } |
| ImagesCount = | x.FramesCount | |
| x.ImagesCount, |)); | private static async Task |
| ClientId = x.ClientId, | } | AddJsonEntryAsync<T>(ZipArchive |
| DatasetZipPath = | | archive, string entryName, T value, |
| x.DatasetZipPath, | private async Task | CancellationToken ct) |
| BaseModel = | ResetSequencesAsync(Cancellation token | { |
| x.BaseModel, | ct) | var entry = |
| Epochs = x.Epochs, | { | archive.CreateEntry(entryName, |
| ImgSize = x.ImgSize, | await | CompressionLevel.Optimal); |
| Batch = x.Batch, | ResetSequenceAsync("Users", ct); | await using var stream = |
| Device = x.Device, | await | entry.Open(); |
| ExportInt8 = | ResetSequenceAsync("Products", ct); | await |
| x.ExportInt8, | await | JsonSerializer.SerializeAsync(stream, |
| ExportNms = | ResetSequenceAsync("StoredProducts", ct); | value, JsonOptions, ct); |
| x.ExportNms, | await | } |
| MobileFormat = | ResetSequenceAsync("VideoSamples", ct); | |
| x.MobileFormat, | await | private static async Task<T> |
| QuantizationFraction = | ResetSequenceAsync("ErrorReports", ct); | ReadJsonFileAsync<T>(string path, |
| x.QuantizationFraction, | await | CancellationToken ct) |
| BestWeightsPath = | ResetSequenceAsync("ModelVersions", ct); | { |
| x.BestWeightsPath, | await | if (!File.Exists(path)) |
| MobileModelPath = | ResetSequenceAsync("TrainingJobs", ct); | throw new |
| x.MobileModelPath, | } | InvalidOperationException(\$"B backup- |
| MobileModelFileName = | | архиве нет обязательного файла |
| x.MobileModelFileName, | private async Task | {Path.GetFileName(path)});"); |
| MobileModelContentType = | ResetSequenceAsync(string tableName, | |
| x.MobileModelContentType, | CancellationToken ct) | await using var stream = |
| MetricsJson = | { | File.OpenRead(path); |
| x.MetricsJson, | var quotedTable = | return await |
| CancellationRequested = | tableName.Replace("'", "\""); | JsonSerializer.DeserializeAsync<T>(stream |
| x.CancellationRequested | var sql = \$"" | , JsonOptions, ct) |
| }); | SELECT setval(| ?? throw new |
| | pg_get_serial_sequence("{quotedTable}", | InvalidOperationException(\$"Не удалось |
| | 'Id'), | прочитать {Path.GetFileName(path)}); |
| | | } |
| _db.ErrorReports.AddRange(snapshot.Error | GREATEST(COALESCE((SELECT | |
| Reports.Select(x => new ErrorReport | MAX("Id") FROM "{quotedTable}"), 1), | private static void |
| { | 1), | AddDirectoryToArchive(ZipArchive |
| Id = x.Id, | | archive, string sourceDirectory, string |
| UserId = x.UserId, | | entryPrefix) |

```

        {
            if
            (!Directory.Exists(sourceDirectory))
                return;

            var sourceRoot =
            Path.GetFullPath(sourceDirectory);

            foreach (var file in
            Directory.EnumerateFiles(sourceRoot, "*",
            SearchOption.AllDirectories))
            {
                var relative =
            Path.GetRelativePath(sourceRoot,
            file).Replace("\\", "/");

                var entryName =
            JoinZipPath(entryPrefix, relative);

            archive.CreateEntryFromFile(file,
            entryName, CompressionLevel.Fastest);
            }
        }

        private static void
        ExtractZipSafely(string zipPath, string
        destinationDirectory)
        {
            var destinationRoot =
            Path.GetFullPath(destinationDirectory);

            Directory.CreateDirectory(destinationRoot)
            ;

            using var archive =
            ZipFile.OpenRead(zipPath);

            foreach (var entry in
            archive.Entries)
            {
                if
                (string.IsNullOrEmpty(entry.FullNam
                e))
                    continue;

                var destinationPath =
            Path.GetFullPath(Path.Combine(destination
            Root, entry.FullName));

                if
                (!destinationPath.StartsWith(destinationRoo

```

```

            t + Path.DirectorySeparatorChar,
            StringComparison.OrdinalIgnoreCase)
                &&
                !string.Equals(destinationPath,
                destinationRoot,
                StringComparison.OrdinalIgnoreCase))
            {
                throw new
            InvalidOperationException("Backup-архив
            содержит небезопасные пути файлов.");
            }

            if
            (entry.FullName.EndsWith("/",
            StringComparison.Ordinal))
            {
                Directory.CreateDirectory(destinationPath);

                continue;
            }

            var directory =
            Path.GetDirectoryName(destinationPath);

            if
            (!string.IsNullOrEmpty(directory))
            {
                Directory.CreateDirectory(directory);

                entry.ExtractToFile(destinationPath,
                overwrite: true);
            }
        }

        private static void
        CopyDirectoryContentsIfExists(string
        sourceDirectory, string
        destinationDirectory)
        {
            if
            (!Directory.Exists(sourceDirectory))
                return;

            foreach (var sourcePath in
            Directory.EnumerateFileSystemEntries(sou
            rceDirectory, "*",
            SearchOption.AllDirectories))
            {

```

```

            var relative =
            Path.GetRelativePath(sourceDirectory,
            sourcePath);

            var destinationPath =
            Path.Combine(destinationDirectory,
            relative);

            if
            (Directory.Exists(sourcePath))
            {
                Directory.CreateDirectory(destinationPath);

                continue;
            }

            var destinationDir =
            Path.GetDirectoryName(destinationPath);

            if
            (!string.IsNullOrEmpty(destinationDir
            ))

            Directory.CreateDirectory(destinationDir);

            File.Copy(sourcePath,
            destinationPath, overwrite: true);
        }

        private static void
        ClearDirectoryContents(string directory)
        {
            Directory.CreateDirectory(directory);

            foreach (var file in
            Directory.EnumerateFiles(directory))
                File.Delete(file);

            foreach (var subDirectory
            in
            Directory.EnumerateDirectories(directory))
                Directory.Delete(subDirectory, recursive:
                true);
        }

        private static string
        JoinZipPath(string left, string right)

```

```

=> $" {left.TrimEnd('/',
"\\")} / {right.TrimStart('/', "\\")} ".Replace("\\",
'/');
}

```

```

public sealed record
BackupManifest(
    string Format,
    int Version,
    DateTime CreatedAtUtc,
    string Application);

public sealed class
BackupDatabaseSnapshot
{
    public List<UserBackupDto>
Users { get; set; } = new();
    public
List<ProductBackupDto> Products { get;
set; } = new();
    public
List<StoredProductBackupDto>
StoredProducts { get; set; } = new();
    public
List<VideoSampleBackupDto>
VideoSamples { get; set; } = new();
    public
List<ErrorReportBackupDto> ErrorReports
{ get; set; } = new();
    public
List<ModelVersionBackupDto>
ModelVersions { get; set; } = new();
    public
List<TrainingJobBackupDto> TrainingJobs
{ get; set; } = new();
}

```

```

public sealed record
UserBackupDto(
    int Id,
    string Email,
    string PasswordHash,
    bool IsBlocked,
    string? SettingsJson,
    DateTime CreatedAt);

```

```

public sealed record
ProductBackupDto(

```

```

    int Id,
    string Name,
    string? Manufacturer,
    string? Barcode);

public sealed record
StoredProductBackupDto(
    int Id,
    int UserId,
    int ProductId,
    DateTime? ManufactureAt,
    DateTime? ExpiryAt,
    DateTime CreatedAt);

```

```

public sealed record
VideoSampleBackupDto(
    int Id,
    string VideoPath,
    string? Source);

```

```

public sealed record
ErrorReportBackupDto(
    int Id,
    int UserId,
    int? VideoId,
    int? ModelVersionId,
    string? Comment,
    DateTime CreatedAt,
    bool Resolved,
    bool Approved,
    int FramesCount);

```

```

public sealed record
ModelVersionBackupDto(
    int Id,
    DateTime TrainedAt,
    string? MetricsJson,
    string? ExternalJobId,
    string? BaseModel,
    string? BestWeightsPath,
    string? MobileModelPath,
    string? MobileFormat,
    string?
MobileModelFileName,
    string?
MobileModelContentType,
    bool IsPublished,
    bool IsPinned,

```

```

    bool IsDeleted,
    DateTime? DeletedAt);

public sealed record
TrainingJobBackupDto(
    int Id,
    string JobId,
    string Status,
    string? Message,
    DateTime CreatedAt,
    DateTime? StartedAt,
    DateTime? FinishedAt,
    DateTime? AssignedAt,
    DateTime? HeartbeatAt,
    int ImagesCount,
    string? ClientId,
    string DatasetZipPath,
    string? BaseModel,
    int Epochs,
    int ImgSize,
    int Batch,
    string? Device,
    bool ExportInt8,
    bool ExportNms,
    string? MobileFormat,
    double QuantizationFraction,
    string? BestWeightsPath,
    string? MobileModelPath,
    string?
MobileModelFileName,
    string?
MobileModelContentType,
    string? MetricsJson,
    bool CancellationRequested);

```

```

public sealed record
BackupImportResult(
    int Users,
    int Products,
    int StoredProducts,
    int VideoSamples,
    int ErrorReports,
    int ModelVersions,
    int TrainingJobs,
    bool ReplacedExisting);

```

```

=====
=====

```

```

=====
=====
Файл:
ProductsDateAPI/Services/TrainingDataset
Packager.cs
=====
=====
=====
=====

using
Microsoft.EntityFrameworkCore;
using
Microsoft.Extensions.Options;
using PostgreSQLRepository;
using ProductsDateAPI.Models;
using System.IO.Compression;

namespace
ProductsDateAPI.Services;

public sealed record
TrainingDatasetPackage(string
WorkingDirectory, string ZipPath, int
ImagesCount);

public class
TrainingDatasetPackager
{
    private static readonly
HashSet<string> ImageExtensions =
new(StringComparer.OrdinalIgnoreCase)
{
    ".jpg", ".jpeg", ".png"
};

    private readonly
AppDbContext _db;
    private readonly
IWebHostEnvironment _env;
    private readonly
TrainingServiceOptions _options;

    public
TrainingDatasetPackager(
AppDbContext db,
IWebHostEnvironment env,

```

```

IOptions<TrainingServiceOptions> options)
{
    _db = db;
    _env = env;
    _options = options.Value;
}

public async
Task<TrainingDatasetPackage>
CreateApprovedReportsZipAsync(Cancellat
ionToken ct = default)
{
    var reports = await
_db.ErrorReports
.AsNoTracking()
.Include(x => x.Video)
.Where(x => x.Approved
&& x.Video != null)
.OrderBy(x => x.Id)
.ToListAsync(ct);

    var tempRoot =
ResolvePath(_options.TempRoot);

    Directory.CreateDirectory(tempRoot);

    var workDir =
Path.Combine(tempRoot,
$"train_{DateTime.UtcNow:yyyyMMdd_H
Hmss}_{Guid.NewGuid():N}");
    var datasetRoot =
Path.Combine(workDir, "dataset");
    var imagesOut =
Path.Combine(datasetRoot, "images");
    var labelsOut =
Path.Combine(datasetRoot, "labels");

    Directory.CreateDirectory(imagesOut);
    Directory.CreateDirectory(labelsOut);

    var copied = 0;

    var seedDatasetPath =
ResolveOptionalPath(_options.SeedDataset
Path);

```

```

if
(!string.IsNullOrEmpty(seedDatasetPa
th) && Directory.Exists(seedDatasetPath))
{
    copied +=
CopyPairs(seedDatasetPath, imagesOut,
labelsOut, "seed");
}

foreach (var report in
reports)
{
    if (report.Video is null ||
string.IsNullOrEmpty(report.Video.Vi
deoPath))
        continue;

    var extractedRoot =
Path.Combine(
_env.ContentRootPath,

report.Video.VideoPath.Replace("/",
Path.DirectorySeparatorChar.ToString()),
"extracted");

    if
(!Directory.Exists(extractedRoot))
        continue;

    copied +=
CopyPairs(extractedRoot, imagesOut,
labelsOut, $"report_{report.Id}");
}

if (copied == 0)
    throw new
InvalidOperationException(
    "Не найдено ни
одного размеченного кадра. Нужны
approved отчёты с images/labels или seed
dataset.");

    var yamlPath =
Path.Combine(datasetRoot, "dataset.yaml");
    await
File.WriteAllTextAsync(yamlPath,
BuildDatasetYaml(), ct);

```



```

        var zipPath =
Path.Combine(workDir, "dataset.zip");

ZipFile.CreateFromDirectory(datasetRoot,
zipPath);

        return new
TrainingDatasetPackage(workDir, zipPath,
copied);
    }

    private static int
CopyPairs(string sourceRoot, string
imagesOut, string labelsOut, string prefix)
    {
        var imagesDir =
Path.Combine(sourceRoot, "images");
        var labelsDir =
Path.Combine(sourceRoot, "labels");

        if
(!Directory.Exists(imagesDir) ||
!Directory.Exists(labelsDir))
            return 0;

        var copied = 0;

        foreach (var imagePath in
Directory.EnumerateFiles(imagesDir, "*.*",
SearchOption.TopDirectoryOnly)
            .Where(x =>
ImageExtensions.Contains(Path.GetExtensi
on(x))))
        {
            var labelPath =
Path.Combine(labelsDir,
Path.GetFileNameWithoutExtension(image
Path) + ".txt");

            if
(!File.Exists(labelPath))
                continue;

            copied++;
            var fileStem =
$"{{prefix}}_{{copied:D6}}";
            var imageExt =
Path.GetExtension(imagePath).ToLowerInv
ariant();

```

```

            File.Copy(imagePath,
Path.Combine(imagesOut, fileStem +
imageExt), overwrite: true);

            File.Copy(labelPath,
Path.Combine(labelsOut, fileStem + ".txt"),
overwrite: true);
        }

        return copied;
    }

    private string
ResolvePath(string path)
        => Path.IsPathRooted(path)
? path :
Path.Combine(_env.ContentRootPath,
path);

    private string?
ResolveOptionalPath(string? path)
    {
        if
(string.IsNullOrEmpty(path))
            return null;

        return ResolvePath(path);
    }

    private static string
BuildDatasetYaml() =>
        "path: \n" +
        "train: images\n" +
        "val: images\n\n" +
        "nc: 1\n" +
        "names: [expiry_date]\n";
    }

=====
=====
=====
=====
        Файл:
ProductsDateAPI/Services/TrainingFileStor
age.cs
=====
=====

```

```

=====
=====

using
Microsoft.Extensions.Options;
using ProductsDateAPI.Models;

namespace
ProductsDateAPI.Services;

public class TrainingFileStorage
{
    private readonly
IWebHostEnvironment _env;
    private readonly
TrainingServiceOptions _options;

    public
TrainingFileStorage(IWebHostEnvironment
env, IOptions<TrainingServiceOptions>
options)
    {
        _env = env;
        _options = options.Value;
    }

    public string RootPath
    {
        get
        {
            var root =
string.IsNullOrEmpty(_options.Storag
eRoot)
                ? "storage/training"
                :
_options.StorageRoot;

            return
Path.IsPathRooted(root)
                ? root
                :
Path.Combine(_env.ContentRootPath,
root);
        }
    }
}

```

```

        public string
SaveDatasetZip(string jobId, string
sourceZipPath)
    {
        var jobDir =
Path.Combine(RootPath, "jobs", jobId);

Directory.CreateDirectory(jobDir);

        var destination =
Path.Combine(jobDir, "dataset.zip");
        File.Copy(sourceZipPath,
destination, overwrite: true);
        return
ToRelativePath(destination);
    }

    public async Task<string>
SaveArtifactAsync(string jobId, IFormFile
file, string fileName, CancellationToken ct)
    {
        var safeName =
MakeSafeFileName(fileName);
        var artifactDir =
Path.Combine(RootPath, "models", jobId);

Directory.CreateDirectory(artifactDir);

        var destination =
Path.Combine(artifactDir, safeName);
        await using var stream =
File.Create(destination);

```

```

        await
file.CopyToAsync(stream, ct);
        return
ToRelativePath(destination);
    }

    public string
ToAbsolutePath(string relativePath)
    {
        var normalized =
relativePath.Replace('/',
Path.DirectorySeparatorChar);
        return
Path.IsPathRooted(normalized) ?
normalized : Path.Combine(RootPath,
normalized);
    }

    public FileStream
OpenRead(string relativePath)
    =>
File.OpenRead(ToAbsolutePath(relativePat
h));

    public void
TryDeleteFile(string? relativePath)
    {
        if
(string.IsNullOrEmpty(relativePath))
            return;

        try
        {

```

```

        var absolute =
ToAbsolutePath(relativePath);
        if (File.Exists(absolute))
            File.Delete(absolute);
    }
    catch
    {
        // best effort cleanup
    }

    private string
ToRelativePath(string absolutePath)
    =>
Path.GetRelativePath(RootPath,
absolutePath).Replace("\\", "/");

    private static string
MakeSafeFileName(string fileName)
    {
        var cleaned =
string.Join("_",
Path.GetFileName(fileName).Split(Path.Get
InvalidFileNameChars(),
StringSplitOptions.RemoveEmptyEntries)).
Trim();

        return
string.IsNullOrEmpty(cleaned) ?
$"artifact_{Guid.NewGuid():N}.bin" :
cleaned;
    }
}

```

Бакалаврская работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

Объем работы 70 листов.

Библиография 25 наименования.

Объем приложений 216 листов.

Отпечатано в 1 экземпляре.

Один экземпляр сдан на кафедру.

«__» _____ 2026 г.

_____/_____/_____
(подпись)